GOOGLE | CODE ONLY | HELPERS + MAIN CODE

Study path, code only

288 questions in 43 groups, in the same order and with the same numbers as the study path book. Each question shows its statement and examples, then only the code you write for it: the helper functions first, then the main answer, with the same syntax colours as the main book. The brute force, the follow-up code and the tests are not here; they are in the study path book, whose groups also start with the code template. The italic line under each title says what you change in the group's template for that question.

38 EASY	197 MEDIUM	53 HARD

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9)

1. Flood Fill
2. Number of Islands
3. Rotten Oranges / Multi-Source BFS (taxis)
4. Solve Grid and Counting Problems
5. Find shortest relationship path using BFS
6. Farthest City for a New Data Center
7. Compute distance to nearest taxi in grid
8. Infection Spread on a Grid (Cellular Automaton)
9. Shortest Distance From All Buildings

2. Sweep over events in time order (9)

10. Meeting Rooms / Interval Scheduling Variants
11. Train / Flight Schedule Reachability
12. Solve Rooms and Top-K Streams
13. Compute servers needed for daily recurring jobs
14. Gate Entry / Exit Time Resolution with Tie-Break Rules
15. Meeting Rooms III Variant
16. Count Overlapping Rectangle Updates on a Grid
17. Compute city skyline outline
18. Compute Turnstile Crossing Times

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward (9)

19. Given two trees, determine if the pre-order traversals of two trees are same or not.
20. Calculate the height of a binary tree.
21. Find right-side view of binary tree
22. Expression Tree Single-Leaf Mutation
23. Longest Consecutive Path in Binary Tree
24. Given the root of a binary tree, return the length of the diameter of the tree.
25. Implement zigzag level-order traversal
26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree
27. Delete an Entire Binary Tree

4. A heap of what is available: pop the best, put back the changed one (9)

28. Streaming Median (LC 295) with Follow-ups
29. Compute minimum number of rooms needed
30. Implement a Reusable Locker Allocation System
31. Simulate room assignments for scheduled meetings
32. Trip Expense Settlement
33. Playlist Shuffle No Adjacent Artists
34. Minimum Makespan on Identical CPUs
35. Design server allocation for multi-type nodes
36. Huffman Tree Construction

5. Recurrence on a position or a range (memo or table) (9)

37. Target Expressions from Ordered Digits
38. Count Word Decompositions into Chemical Element Symbols

39. Find the longest palindromic subsequence using dynamic programming.
40. Jump Game with Profit
41. Fence Painting Minimum Operations (Gemini Team)
42. Maximize 1D deliveries within distance and minimize total distance
43. Edit distance
44. Count Perfect Wake Numbers up to N
45. Word Break: All Sentences

6. Best-first search with a heap: Dijkstra and its minimax twin (9)

46. Path Minimizing the Maximum Cell (Swim in Rising Water)
47. Nearest Favorite City by Travel Time
48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.
49. Find the shortest path in a graph.
50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.
51. Minimize maximum height along a grid path
52. Find minimum threshold enabling grid path
53. Minimize travel time with optional meeting point
54. Compute minimax grid path and network delay

7. Backtracking: choose, recurse, undo (9)

55. Mahjong Winning Hand Detection (14 Tiles)
56. Count Prefix-Matching Simple Paths in a Labeled Graph
57. Generate Parentheses
58. American Football Score Combinations
59. Permutations
60. Determine if a 14-tile hand is winning
61. Count Passcode Patterns on a Keypad
62. Match people to questions using tags and priorities
63. Matrix Flower Placement with House Adjacency

8. Runs and groups: extend while it fits, restart or close when it does not (9)

64. Group Timestamps Into Close Intervals
65. First Out-of-Order Character
66. Longest Non-Decreasing Subarray with One Replacement
67. Piano Hand Movement Grouping
68. Find longest increasing contiguous subarray
69. Sum of Good Subarrays (Adjacent Diff ±1)
70. Sustained-Error Time Window
71. Longest Non-Decreasing Subarray From Two Arrays
72. Count Super Streaks

9. Union-find: merge the pairs, then ask who is together (8)

73. Count Islands in a Grid and Track the Count as Land Cells Are Added
74. 2D Points Connected Components by Radius
75. Count Clusters of 2D Points Within a Radius
76. Find A Threshold-Limited Path With Minimum Required Safety
77. Movie Recommendations via Transitive Similarity
78. Determine Reporting Relationships
79. Determine whether two nodes are related
80. Friend Graph with Timeline (Friend / Unfriend Events)

10. A dict of per-key state that you update, expire, or rank (8)

81. Robot Status Message Deduplication Window
82. Active Communication Top Users
83. Filter Repeated Robot Status Messages
84. Next-Word Predictor
85. Batch and Streaming Log Deduplication
86. Design a logger that returns top-K messages
87. Hit Counter
88. Website Activity Analytics: Unique Users + Session Time

11. Binary search on a yes/no condition (8)

89. First Index Reaching a Threshold (Binary Search)

90. Cutting Ribbons (LC 1891)
91. First Bad Version with Parallel Search Follow-up
92. Solve Banana Speed and Interval Merge
93. Given a list with response time for each server, calculate how long time it will take to process n requests.
94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant
95. Find the median of two sorted arrays.
96. Find longest duplicated substring

12. Greedy: scan once, carry the best choice so far, act only when forced (8)

97. Maximum Profit
98. Alarm for the Next Upcoming Birthday
99. Coins on the Table
100. Visible People in a Line (Last Person, Then Everyone)
101. Bounded Knapsack Variant
102. Find the Largest Monotone Increasing Number
103. Min boards to cover roof holes
104. Jump Game II (Minimum Jumps)

13. Sets and counters: remember the first collection, check the second against it (8)

105. Exclude First-K-Unique From Another List
106. Find the Duplicates
107. Positional Matches Between Two Lists
108. Contains Duplicate
109. Filter Common Elements Per Window (Two Lists)
110. Reconstruct original array from doubled shuffle
111. Min deletions to avoid overlap in first k
112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space

14. bisect on a sorted list: find a position, count a range, keep it sorted (8)

113. Count People in a Postal Code Range
114. Sliding-Window Mean After Removing the Largest K Values
115. Russian Doll Envelopes
116. Count Sorted Strings With a Given Prefix
117. Streaming Insert + Find K-th Largest
118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.
119. Find Maximum Envelope Nesting
120. Count of Smaller Numbers After Self

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8)

121. Parity Alternating Subarray Query
122. Cake Balance Line
123. Find the Pivot Index (Balanced Split) in O(n)
124. Range Duplicate Check with Constant-Time Queries
125. Find a Horizontal Cut That Bisects Rectangle Area
126. Check pivot after removing one element
127. Gold Chain Split into Two Equal Halves
128. Count the number of ways to split a string.

16. Sorted intervals: merge while they touch, then read the gaps or search (8)

129. Check an Interval Against Existing Intervals for Conflicts
130. Find Containing Range
131. Union of Sorted Interval Lists
132. Range Module (Insert and Point Query)
133. Meetings Minus a Do-Not-Schedule Block
134. Merge Intervals
135. Find safe travel intervals between planet influences
136. Implement a function that processes a stream of interval data.

17. Trie: nested dicts, one character per level (8)

137. URL Request Handler Router
138. Trie Prefix Search / Autocomplete
139. Build Prefix Lookup with a Trie
140. Greedy Longest-Match Dictionary Tokenizer
141. Check if all substrings are dictionary words

142. Return dictionary words matching a prefix
143. Implement Trie Insert and Exact-Word Search
144. Implement Longest-Match Text Replacement

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8)

145. Implement Batched Undo/Redo Layer
146. Design a restaurant waitlist system
147. Implement Python's itertools.tee
148. Store With Cheapest-Item Removal
149. Round-Robin Job Scheduler
150. Aggregate FizzBuzz Offer Matches
151. Restaurant Waitlist: join / delete / find_first_match
152. Implement LRU Cache.

19. Rolling state: the next state comes only from the previous one (7)

153. Climbing Stairs
154. Fibonacci Numbers
155. Maximize sum without choosing adjacent elements
156. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.
157. Paint House II
158. Write SQL code to publish the Fibonacci series.
159. Constrained Dice Roll Sequences

20. Two pointers: move i and j toward each other (or along two lists) (7)

160. Reverse a linked list.
161. Sort a binary array in linear time.
162. Is this a valid palindrome?
163. Reverse a Sentence
164. Find median of merged RLE arrays
165. Run-Length Vector Storage + Dot Product
166. Count Unique Triplets That Sum to Zero

21. Walk two strings at once with two indices (7)

167. Find the First Occurrence of a Substring
168. Determine if two words are sorted in lexicographic order.
169. Return all unique substrings of a given string.
170. Can board states be transformed?
171. Decide string transform with directional tokens and walls
172. Determine if two sentences are similar.
173. Find Dictionary Matches for a Jammed String

22. Simulate a machine: a class with state and one branch per kind of event (7)

174. Mario-Style Score Tracker (OOP)
175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win
176. Design Tic-Tac-Toe With K Players
177. Infinite Deck Card Duel
178. Implement iterator merging lists with filtering
179. Design an elevator control system
180. Simulate In-Place Cellular State Updates

23. Top-K by frequency: Counter plus nsmallest (6)

181. Top-K Frequent in Large Logs
182. Rearrange Tasks With Cooldown
183. Implement frequency + distance top■K queries
184. Solve chat, grid paths, and car rentals
185. Top-K Chatters by Word Count
186. Find top/bottom-k words in list or stream

24. Digits and characters: peel one at a time and carry (6)

187. Caesar Cipher: Recover Key, Encrypt, and Decrypt
188. Add two big integers from digit lists
189. Solve several coding interview problems
190. Check Whether Digits Are Monotone Increasing
191. Check Digits and Combine Ranges

192. Permutation Sequence

25. Look at the input once and keep only a small summary (and count your calls) (6)

193. Two-Day Log Intersection under Memory Limit
194. Find shared objects across two log files
195. Streaming Points: Emit K Within Distance
196. Minimize calls to find all bad test pairs
197. Find All Bad Pairs Using runTest Oracle
198. Find maximum value in huge text file

26. Grid DP: each cell from its neighbours above and to the left (5)

199. Grid Path Count with Diagonal Moves and Checkpoints
200. Longest Same-Color Grid Sequence
201. Count Same-Color Squares in Matrix
202. Find the Longest Straight Run of One Color
203. Count same-color squares in a character grid

27. Nested structures: a stack of contexts, or recursion with a cache (5)

204. Decode String
205. Template Variable Expansion with Follow-Ups
206. Expand Recursive Key-Value Placeholders in a Template
207. Nested List Weight Sum (LC 339) with Follow-Ups
208. Nested Function Expression Evaluator

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly (5)

209. Shortest Path Nodes and Escaping a Pursuer
210. Radio Broadcast Reachability
211. Minimum Direction Violations in a Directed Graph
212. Compute shortest paths with blocked nodes
213. Shortest Directed Cycle Through a Node

29. Topological sort (Kahn) and cycle detection (5)

214. Course Scheduling and Dependency Variants
215. Validate Race Results (Cycle Detection)
216. Solve tree, graph, sliding-window problems
217. Validate course catalog dependencies
218. Course Schedule

30. Counting pairs by looking up each partner in a Counter or a bucket (5)

219. Progressive Array Partitioning
220. K-diff Pairs in an Array
221. Count Axis-Aligned Squares Among Grid Points
222. Sum of Two Squares in Two Ways
223. Maximum-Area Rectangle from Planar Points

31. Tree DP: build a parent order once, then add up from the leaves (5)

224. Compute File System Total Size
225. Minimum-Cost Tree Disconnection
226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.
227. Compute distance-sum from every tree node
228. Tree Distance Sum (Re-root DP)

32. Counting or reaching an amount: the knapsack loop (5)

229. Maximize Shelf Space (Unbounded Knapsack)
230. Coin Change
231. Partition an array into two sub-arrays with equal sum.
232. Explain how to find a target sum in an array.
233. Reverse Coin Change

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5)

234. Maximum-Sum Subarray With Equal Endpoints
235. Maximum Subarray Sum With a Minimum Length Constraint
236. Solve array-sum and city-community problems
237. Find the maximum subarray sum.

238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

34. Group by a canonical key (5)

239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.
240. Check if all substrings are anagrams of words
241. Group anagrams
242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.
243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

35. K-way merge: a heap of the next item from each source (5)

244. Find k pairs with smallest sums
245. K Largest Elements Across Sorted Lists
246. Sort a nearly sorted (or K sorted) array.
247. Merge k sorted linked lists.
248. Return top-k subset sums in descending order

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5)

249. Next Birthday (BST In-Order Successor with Wraparound)
250. Lowest Common Ancestor
251. Determine if a given binary tree is a binary search tree (BST).
252. Find mode in a trinary search tree
253. Next Upcoming Birthday in a Family Tree

37. Random pick without repeats: swap the chosen item with the last one and pop (5)

254. Random Generator Without Repeats
255. Random Song Shuffler With a No-Repeat Window
256. Design set with O(1) random access
257. Unique Random Number Generator
258. Shuffle array using random integer API

38. Build and serialize: write a structure out, read it back with the same walk (5)

259. Construct a binary tree given its preorder and inorder traversals.
260. Write functions to serialize and deserialize a list of strings.
261. Print a Directory Tree
262. Implement a decode and encode function of a general N-ary tree.
263. Serialize and Deserialize a Graph

39. Games: win or lose with a cache, and guessing with feedback (5)

264. Choose optimal guesses for green-only Wordle
265. Wordle-Style 5-Letter Minimum-Guess Strategy
266. Determine winner in stack-merging game
267. Find secret word with match-count feedback
268. Babylon Tile-Stack Merge Game

40. Sliding window: grow on the right, shrink on the left while it is too bad (5)

269. Temperature Monitoring Data Structure
270. Shortest Subarray with K Distinct Integers
271. Find the minimum window substring.
272. Footpath Beauty After Repainting
273. Necklace Cut into Two Halves with Equal D/R Counts

41. Parent pointers: store who is above each node and climb (5)

274. Tree Node.
275. Employee Hierarchy Add / Delete
276. Validate parent array forms a tree
277. Lowest Common Ancestor of a Binary Tree III
278. Root a Tree So It Becomes a Binary Tree

42. Build or transform a grid with index arithmetic (5)

279. Filled ASCII Diamond
280. Connected Crop Arrangement on a Grid
281. Rotate Any m×n 2D Matrix

282. Solve matrix groups and recipe inventory
283. Range Sum Query 2D (Immutable Matrix)

43. Split into pieces, fix each piece, join (and pack words into lines) (5)

284. Implement string formatting and request tracking tasks
285. Snake Case to Camel Case with Repository-Wide Refactor
286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)
287. Implement a function to justify a list of words within a given width.
288. Convert Valid Snake Case to Lower Camel Case

GROUP 1 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 1 E 6 M 2 H

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

1 Flood Fill — EASY

Template fit: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour

QUESTION

You are given an m x n grid image where image[r][c] is the integer color of the pixel at row r , column c . You are also given a starting pixel (sr, sc) and a new_color . Perform a flood fill: starting from (sr, sc) , repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to new_color . Pixels connected only diagonally are not filled. Return the modified grid.

Function signature

```
from typing import List
def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
    ...
```

Follow-up (as reported): Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when new_color equals the original color.

EXAMPLES

Example 1	
Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Why	`Starting color is 1. All 1-pixels connected 4-directionally to (1,1) become 2. The bottom-right 1 is only diagonally reachable, so it stays.`

Example 2	
Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5`
Output	`[[5,5,5],[5,5,5]]`
Why	`The whole grid is one connected region of color 0.`

Example 3	
Input	`image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1`
Output	`[[0,0,0],[0,1,0]]`
Why	`The start pixel already equals new_color, so nothing changes.`

CONSTRAINTS

- 1 <= m, n <= 50
- 0 <= image[r][c], new_color < 2^16
- 0 <= sr < m , 0 <= sc < n
- If the starting pixel already has new_color , the grid is returned unchanged (guard against infinite recursion).

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
        if 0 <= nr < rows and 0 <= nc < cols:
```

```
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Idea: spread over neighbours that still have the old colour and
# repaint them. If the new colour equals the old one there is nothing
# to do (and the loop below would never finish), so return at once.
# Complexity: Time O(m * n), Space O(m * n)
def flood_fill(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    rows, cols = len(image), len(image[0])
    image[sr][sc] = new_color
    stack = [(sr, sc)]
    while stack:
        r, c = stack.pop()
        for nr, nc in neighbors(r, c, rows, cols):
            if image[nr][nc] == old:
                image[nr][nc] = new_color
                stack.append((nr, nc))
    return image
```

COMPLEXITY | Time O(mn) | Space O(mn)

2 Number of Islands — MEDIUM

Template fit: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start

QUESTION

Given a grid of 0s (water) and 1s (land), count the islands. An island is a group of 1s connected up, down, left or right (diagonal cells are not connected).

Function signature

def num_islands(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]`
Output	`2`
Why	`Two 2x2 blocks of land that only touch the water between them: 2 islands.`

Example 2	
Input	`[[1, 0, 1], [0, 1, 0], [1, 0, 1]]`
Output	`5`
Why	`Land that touches only diagonally is not connected, so every 1 is its own island.`

Example 3	
Input	`[[0, 0], [0, 0]]`
Output	`0`
Why	`No land, no islands.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
```

```
    # the cells beside (r, c) that are inside the grid
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def flood(grid, start, seen):
    # mark all land joined to `start` as seen and return how many cells it has;
    # a stack instead of recursion, so a huge island cannot overflow the call stack
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack = [start]
    size = 0
    while stack:
        r, c = stack.pop()
        size += 1
        for nr, nc in neighbors(r, c, rows, cols):
            if grid[nr][nc] == 1 and (nr, nc) not in seen:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return size


def island_sizes(grid):
    # one number per island: how many cells it has
    sizes = []
    if not grid:
        return sizes
    seen = set()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1 and (r, c) not in seen:     # land we have not visited starts a new island
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# ---- main question: main function ----
# Idea: scan every cell; each unvisited land cell starts a new
# island, and the flood marks all its connected land as visited.
# Complexity: Time O(m * n), Space O(m * n)
```

```
def num_islands(grid):
    return len(island_sizes(grid))
```

COMPLEXITY | Time O(mn) | Space O(mn)

3 Rotten Oranges / Multi-Source BFS (taxis) — MEDIUM

Template fit: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached

QUESTION

Each cell of a grid is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, each fresh orange that is next to (up, down, left or right) a rotten one becomes rotten. Return the number of minutes until no fresh orange is left, or -1 if some fresh orange can never rot.

Function signature

def rotting_minutes(grid: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	`[[2, 1, 1], [1, 1, 0], [0, 1, 1]]`
Output	`4`
Why	`The rot spreads outward from the corner and reaches the last orange after 4 minutes.`

Example 2	
Input	`[[2, 1, 1], [0, 1, 1], [1, 0, 1]]`
Output	`-1`
Why	`The orange at the bottom left is cut off by empty cells, so it never rots.`

Example 3	
Input	`[[0, 2]]`
Output	`0`
Why	`Nothing fresh to rot: 0 minutes.`

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def inside(r, c, rows, cols):
    # is (r, c) a real cell of the grid?
    return 0 <= r < rows and 0 <= c < cols


def find_cells(grid, value):
    # every (row, column) whose cell holds `value`
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == value:
                cells.append((r, c))
    return cells


def bfs_distances(grid, starts, wall):
    # distance from the nearest start to every cell it can reach;
    # a cell holding `wall` can never be entered
    rows, cols = len(grid), len(grid[0])
    dist = {}
    queue = deque()
    for start in starts:
        dist[start] = 0
        queue.append(start)
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if not inside(nr, nc, rows, cols):
                continue
            if grid[nr][nc] == wall or (nr, nc) in dist:
                continue
            dist[(nr, nc)] = dist[(r, c)] + 1
            queue.append((nr, nc))
    return dist
```

```
# ---- main question: main function ----
# Idea: every rotten orange starts at minute 0 in the same queue;
# each BFS level is one minute. A fresh orange that is never
# reached makes the answer -1.
```

```
# Complexity: Time O(m * n), Space O(m * n)
def rotting_minutes(grid):
    rotten = find_cells(grid, 2)
    fresh = find_cells(grid, 1)
    dist = bfs_distances(grid, rotten, 0)      # rot spreads through any non-empty cell
    minutes = 0
    for cell in fresh:
        if cell not in dist:
            return -1
        if dist[cell] > minutes:
            minutes = dist[cell]
    return minutes


# the same search answers "distance to the nearest taxi"
# Complexity: Time O(m * n), Space O(m * n)
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked; None where no taxi can reach
    taxis = find_cells(grid, "T")
    dist = bfs_distances(grid, taxis, "#")
    answer = []
    for r in range(len(grid)):
        row = []
        for c in range(len(grid[0])):
            row.append(dist.get((r, c)))
        answer.append(row)
    return answer
```

COMPLEXITY | Multi-source BFS by minute: Time O(mn) | Space O(mn)
Taxi variant: distance from every empty cell to the nearest taxi: Time O(mn) | Space O(mn)

4 Solve Grid and Counting Problems — MEDIUM

Template fit: the island count again (queue version); the second part of the question is a short counting formula, no search

QUESTION

A software engineering intern phone screen included two coding tasks: 1.
Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.
Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.
Constraints
0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:
Explanation: Empty grid contains no land, so there are no islands.

QUESTION

```
Input: [['1']]
```

Expected Output:
Explanation: A single land cell forms one island.

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []
Expected Output:
Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']
Expected Output:
Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def count_letters(letters):
    # how many times each letter appears
    counts = {}
    for ch in letters:
        counts[ch] = counts.get(ch, 0) + 1
    return counts


def choose(n, k):
    # the number of ways to pick k items out of n (n choose k)
    if k > n:
        return 0
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


# ---- main question: main function ----
# PART 1: number of islands ('1' is land)
# Idea: scan every cell; an unseen land cell starts a new island, and a
# BFS marks all of its land as seen so it is counted once.
# Complexity: Time O(m * n), Space O(m * n)
```

```python
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
```

```python
                for dx, dy in STEPS:
                    nx, ny = x + dx, y + dy
                    if not (0 <= nx < rows and 0 <= ny < cols):
                        continue
                    if grid[nx][ny] == "1" and (nx, ny) not in seen:
                        seen.add((nx, ny))
                        queue.append((nx, ny))
    return count


# PART 2: in how many ways can the letters of an array spell GOOGLE?
# Idea: for every letter of the word, pick that many of the copies
# available in the array; multiply the choices (follow-up 1 allows any word).
# Complexity: Time O(n + w) for n letters and a word of w letters, Space O(w)
def ways_to_spell(letters, word="GOOGLE"):
    have = count_letters(letters)
    need = count_letters(word)
    total = 1
    for ch in need:
        total = total * choose(have.get(ch, 0), need[ch])
    return total
```

COMPLEXITY | Islands: iterative BFS: Time O(m * n) | Space O(min(m, n)) queue
Counting: product of binomial coefficients: Time O(n) | Space O(1)

5 Find shortest relationship path using BFS — MEDIUM

Template fit: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path

QUESTION

People are connected by relationships, given as an adjacency dict that maps each person to the people they are directly related to. shortest_path(adj, s, t) returns the shortest chain of people from s to t (both ends included), or None when t cannot be reached.

Function signature

def shortest_path(adj: dict, s, t) -> list | None

EXAMPLES

QUESTION

Example 1

Input	`adj = {'ann': ['bob', 'cy'], 'bob': ['dee'], 'cy': ['dee', 'eli'], 'dee': ['fay'], 'eli': ['fay'], 'fay': []}`
	`shortest_path(adj, 'ann', 'fay')`
	`shortest_path(adj, 'ann', 'ann')`
	`shortest_path(adj, 'fay', 'ann')`

Output	`shortest_path(adj, 'ann', 'fay') -> ['ann', 'bob', 'dee', 'fay']`
	`shortest_path(adj, 'ann', 'ann') -> ['ann']`
	`shortest_path(adj, 'fay', 'ann') -> null`

Why	Several chains of 4 people exist; BFS finds the first. A person is trivially related to themselves, and fay has no outgoing relationships.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def walk_back(parent, target):
    # follow the parent links from target back to the start, then reverse
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    path.reverse()
    return path


# ---- main question: main function ----
# Idea: queue the source and mark it visited. Pop a person, queue
# each unvisited neighbour with parent = this person, and stop when
# the target is reached; the first time BFS reaches a node is by a
# shortest path.
# Complexity: Time O(V + E), Space O(V)
def shortest_path(adj, s, t):
    if s == t:
        return [s]
    parent = {s: None}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, []):
            if v not in parent:
                parent[v] = u
                if v == t:
                    return walk_back(parent, t)
                queue.append(v)
    return None
```

COMPLEXITY | Time O(V + E) | Space O(V)

6 Farthest City for a New Data Center

MEDIUM

Template fit: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance

QUESTION

Cities are nodes 0..num_cities-1 of an undirected, unweighted graph; edges[i] = [u, v] is a direct connection. You are given a non-empty list dcs of cities that already host a data center. You want to build a new data center in the city that is as far as possible from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized. Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Function signature

def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:

    ...

Follow-up (as reported): Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

EXAMPLES

Example 1	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]`
Output	`4`
Why	`Distances from city 0 are 0,1,2,3,4; city 4 is farthest.`

Example 2	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]`
Output	`2`
Why	`Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest at distance 2.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(edges) <= 2*10**5 .
- 1 <= len(dcs) <= num_cities , dcs are distinct valid city indices.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def index_of_largest(values):
    # the position of the biggest number (the first one on a tie)
    best = 0
    for i in range(len(values)):
        if values[i] > values[best]:
            best = i
    return best


# ---- main question: main function ----
# Idea: put every data center in the queue at distance 0 and run one
# BFS; the first time a city is reached is by its nearest data center.
# The answer is the city with the biggest distance.
# Complexity: Time O(V + E), Space O(V + E)
def farthest_city(num_cities, edges, dcs):
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v in edges:
        adj[u].append(v)
```

```
        adj[v].append(u)
    dist = [-1] * num_cities
    queue = deque()
    for d in dcs:
        dist[d] = 0
        queue.append(d)
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return index_of_largest(dist)
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

7 Compute distance to nearest taxi in grid — MEDIUM

Template fit: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1

QUESTION

A grid has 'T' for a taxi, '#' for a blocked cell and '.' for an open cell. For every cell return the number of steps (up, down, left or right, never through '#') to the nearest taxi: 0 for a taxi, and -1 for a blocked cell or a cell no taxi can reach.

Function signature

def taxi_distances(grid: list) -> list

EXAMPLES

Example 1	
Input	`['T.#', '...', '#.T']`
Output	`[[0, 1, -1], [1, 2, 1], [-1, 1, 0]]`
Why	The two taxis are in opposite corners; the middle cell is 2 steps from both; the blocked cells show -1.

Example 2	
Input	`['..', '..']`
Output	`[[-1, -1], [-1, -1]]`
Why	No taxi at all: every open cell is unreachable.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def spread(grid, dist, queue, allow_improve=False):
    # grow the distances outward from the cells in `queue`;
    # with allow_improve a cell may be lowered when a closer source reaches it
    rows, cols = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == "#":
```

```
                continue
            new = dist[r][c] + 1
            never_seen = dist[nr][nc] == -1
            if never_seen or (allow_improve and new < dist[nr][nc]):
                dist[nr][nc] = new
                queue.append((nr, nc))


def empty_distances(grid):
    # a table the size of the grid, -1 everywhere
    table = []
    for row in grid:
        table.append([-1] * len(row))
    return table


# ---- main question: main function ----
# Idea: 'T' taxi, '.' road, '#' wall. Start one BFS from every taxi
# at once; a cell gets its distance the first time it is reached,
# and cells no taxi can reach stay at -1.
# Complexity: Time O(m * n), Space O(m * n)
def taxi_distances(grid):
    dist = empty_distances(grid)
    queue = deque()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "T":
```

```
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist
```

COMPLEXITY | Time O(m * n) | Space O(m * n)

8 Infection Spread on a Grid (Cellular Automaton) HARD

Template fit: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)

QUESTION

A grid holds '.' for a healthy cell, 'I' for an infected cell and any other character (such as '#') for a barrier that is never infected. Every step, all infected cells infect their healthy neighbours (up, down, left, right) at the same time. infection_times(grid) returns the step at which each infected cell was infected (0 for the cells infected at the start); spread_steps(grid) returns the number of steps until nothing new gets infected.

Function signature

def infection_times(grid: list) -> dict
def spread_steps(grid: list) -> int

EXAMPLES

Example 1	
Input	`['I..', '...', '..#']`
Output	**3**
Why	**The infection travels from the corner: (0,1) and (1,0) at step 1, then three cells at step 2, and so on. The barrier at the bottom right is never infected.**

QUESTION

Example 2	
Input	['I...']
Output	{(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3): 3}
Why	A one-row grid: one cell per step.

Example 3	
Input	['I#.', 'I#.']
Output	0
Why	Both cells next to the barrier are already infected and the barrier blocks the right column, so nothing can spread: 0 steps.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def find_infected(grid):
    # the cells that start infected ('I')
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "I":
                cells.append((r, c))
    return cells


# ---- main question: main function ----
# Idea: only the cells infected in the last step can infect new
# cells, so keep a frontier. 'I' = infected at time 0, '.' =
# healthy, '#' = blocked. Returns the step at which each cell
# gets infected.
# Complexity: Time O(m * n), Space O(m * n)
def infection_times(grid):
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = find_infected(grid)
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        next_frontier = []
        for r, c in frontier:
            for dr, dc in STEPS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    if grid[nr][nc] == "." and (nr, nc) not in when:
                        when[(nr, nc)] = step       # every healthy neighbour is infected next step
                        next_frontier.append((nr, nc))
        frontier = next_frontier
    return when


# Complexity: Time O(m * n), Space O(m * n)
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
```

```
    last = 0
    for step in when.values():
        if step > last:
            last = step
    return last
```

COMPLEXITY | Time O(m * n) | Space O(m * n)

9 Shortest Distance From All Buildings — HARD

Template fit: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell

QUESTION

You are given a 2D grid where each cell is:
0 = empty land (a school can be built here), 1 = a house, 2 = a tree (an obstacle that cannot be entered or crossed). You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from every house. Return the coordinates (row, col) of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return (-1, -1) .

Function signature

def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...

Follow-up (as reported): (Reported) Minimize the average distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

EXAMPLES

Example 1	
Input	`grid = [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]`
Output	`(1, 2)`
Why	`Cell (1,2) reaches all three houses with total distance 3 + 3 + 1 = 7, the minimum over all reachable empty cells.`

Example 2	
Input	`grid = [[1, 0, 1]]`
Output	`(0, 1)`
Why	`The only empty cell is distance 1 from each house; total 2.`

Example 3	
Input	`grid = [[1, 2, 1]]`
Output	`(-1, -1)`
Why	`There is no empty cell, so no valid location exists.`

CONSTRAINTS

- 1 <= rows, cols <= 50
- Each cell is 0 , 1 , or 2 .
- There is at least one house.

MAIN CODE | helper functions first, then the answer

```
from collections import deque
```

```python
# ---- main question: helper functions ----
def bfs_from_house(grid, house, total, reached):
    # walk outward from one house over empty cells (0); add the distance to
    # `total` and count one more house in `reached` for every cell it reaches
    rows, cols = len(grid), len(grid[0])
    seen = {house}
    queue = deque([(house[0], house[1], 0)])
    while queue:
        r, c, d = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] != 0 or (nr, nc) in seen:
                continue
            seen.add((nr, nc))
            total[nr][nc] += d + 1
            reached[nr][nc] += 1
            queue.append((nr, nc, d + 1))


def school_totals(grid):
    # for every empty cell: the sum of the distances to all houses it can reach,
    # how many houses reach it, and the number of houses
    rows, cols = len(grid), len(grid[0])
    total = []
    reached = []
    for r in range(rows):
        total.append([0] * cols)
        reached.append([0] * cols)
    houses = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                houses += 1
                bfs_from_house(grid, (r, c), total, reached)
    return total, reached, houses


# ---- main question: main function ----
# Idea: one BFS per house adds its distances to the empty cells it
# reaches; a valid cell is one that every house reached. Pick the valid
# cell with the smallest total; (-1, -1) if there is none.
```

```python
# Complexity: Time O(houses * m * n), Space O(m * n)
def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best = None
    spot = (-1, -1)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 0 and reached[r][c] == houses:
                if best is None or total[r][c] < best:
                    best = total[r][c]
                    spot = (r, c)
    return spot
```

COMPLEXITY | Time O(houses * R * C) | Space O(R * C)

Sweep over events in time order

10 Meeting Rooms / Interval Scheduling Variants

MEDIUM

Template fit: the template as written: the largest running count is the number of rooms

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

QUESTION

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling).
Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example: [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40], D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap: intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main functions ----
# The minimum number of rooms for meetings (start, end).
# Idea (sweep line): a +1 event at every start and a -1 event at every end;
# sort them so an END at time t comes before a START at time t (a room is
# free the moment a meeting ends), and track the largest running count.
# Complexity: Time O(n log n), Space O(n)
def min_rooms(intervals):
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                                   # (-1) sorts before (+1) at equal times
    running = 0
    best = 0
    for time, delta in events:
        running += delta
        if running > best:
            best = running
    return best


# VARIANT: which room does each meeting get?
# Complexity: Time O(n log n), Space O(n)
```

```python
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()
    heap = []
    rooms = [0] * len(intervals)
    used = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)    # reuse the room that frees first, if it is free
        else:
            room = used
            used += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms
```

```python
# VARIANT: on-call consolidation. rotations are (name, start, end), half-open.
# Return the segments (start, end, set of people on call), merging neighbouring
# segments with the same people and leaving out the gaps.
# Complexity: Time O(T log T + T * p) for T change times and p people, Space O(T + p)
def oncall_segments(rotations):
    changes = {}                                    # time -> the (name, +1 or -1) changes at that time
    for name, start, end in rotations:
        if start not in changes:
            changes[start] = []
        changes[start].append((name, 1))
        if end not in changes:
            changes[end] = []
        changes[end].append((name, -1))
    active = {}
    segments = []
    times = sorted(changes)
    for i in range(len(times) - 1):
        t = times[i]
        following = times[i + 1]
        for name, change in changes[t]:
            active[name] = active.get(name, 0) + change
        people = set()
        for name in active:
            if active[name] != 0:
                people.add(name)
        who = frozenset(people)
        if not who:
            continue                                # nobody on call: a gap
        if segments and segments[len(segments) - 1][1] == t and segments[len(segments) - 1][2] == who:
            segments[len(segments) - 1] = (segments[len(segments) - 1][0], following, who)
        else:
            segments.append((t, following, who))
    return segments
```

COMPLEXITY | Sweep line over start and end events: Time O(n log n) | Space O(n)
Room assignment with a min-heap of end times: Time O(n log n) | Space O(n)

11 Train / Flight Schedule Reachability

MEDIUM

Template fit: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late for the Y->B connection, but the second one makes it
.
```

MAIN CODE | helper functions first, then the answer

```python
INF = float("inf")


# ---- main question: helper functions ----
def departure_of(leg):
    # a leg is (from station, to station, departure time, arrival time)
    return leg[2]


# ---- main question: main functions ----
# Which stations can you reach from `source`? Time is part of the state:
# you can only take a leg if you are already at its station BEFORE it leaves.
# Idea: sort the legs by departure time and keep, for every station, the
# EARLIEST time you can be there (the source: minus infinity, you are there
# from the start). Going through the legs in departure order, every leg that
# could feed the current one has already been handled. A leg is usable if we
# are at its station strictly before it departs (strict=False allows a
# connection at the same minute).
# Complexity: Time O(L log L) for L legs, Space O(S) for S stations
def earliest_arrivals(legs, source, strict=True):
    earliest = {source: -INF}
    for from_station, to_station, departs, arrives in sorted(legs, key=departure_of):
        have = earliest.get(from_station, INF)
        usable = have < departs or (not strict and have == departs)
        if usable and arrives < earliest.get(to_station, INF):
            earliest[to_station] = arrives
    return earliest


# Complexity: Time O(L log L), Space O(S)
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)
```

COMPLEXITY | Time O(L log L) | Space O(S)

12 Solve Rooms and Top-K Streams — MEDIUM

Template fit: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap

QUESTION

Two small tasks. (1) min_rooms(intervals): the minimum number of rooms needed to hold all meetings; a meeting may start exactly when another ends. (2) TopKStream(k): numbers arrive one at a time with add(x); top() returns the k largest numbers seen so far, largest first.

Function signature

```
def min_rooms(intervals: list) -> int
class TopKStream:
    def __init__(self, k: int)
    def add(self, x: int) -> None
    def top(self) -> list
```

EXAMPLES

Example 1	
Input	`[(0, 30), (5, 10), (15, 20)]`
Output	`2`
Why	`(5, 10) and (15, 20) both overlap the long meeting (0, 30): 2 rooms.`

Example 2	
Input	`[(7, 10), (2, 4)]`
Output	`1`
Why	`The second meeting ends before the first starts: one room is enough.`

Example 3	

Input	`s = TopKStream(3)`
	`s.add(5)`
	`s.add(1)`
	`s.add(9)`
	`s.add(7)`
	`s.top()`

Output	`s.top() -> [9, 7, 5]`
Why	`Of 5, 1, 9, 7 the three largest are 9, 7 and 5.`

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main functions ----
# PART 1: the minimum number of rooms for meetings (start, end).
# Idea: sort the starts and the ends SEPARATELY. For each start: if it is at
# or after the earliest end that is still unused, that room is reused (move the
# end pointer on); otherwise a new room is needed.
# Complexity: Time O(n log n), Space O(n)
def min_rooms(intervals):
    starts = []
    ends = []
    for start, end in intervals:
        starts.append(start)
        ends.append(end)
    starts.sort()
    ends.sort()
```

```
    rooms = 0
    j = 0                                           # the earliest end not yet used
    for start in starts:
        if start >= ends[j]:
            j += 1                                  # the earliest-ending meeting is over: reuse its room
        else:
            rooms += 1
    return rooms


# PART 2: the k largest values seen so far in a stream (fixed k).
# Idea: a min-heap of size k: its root is the k-th largest. A new value
# enters only if it beats the root.
# Complexity: Time add O(log k), top O(k log k), Space O(k)
class TopKStream:
    def __init__(self, k):
        self.k = k
        self.heap = []

    def add(self, x):
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
        return sorted(self.heap, reverse=True)
```

COMPLEXITY | Rooms: sorted starts and ends with two pointers: Time O(n log n) | Space O(n)
Stream: min-heap of size K: Time O(log k) per add | Space O(k)

13 Compute servers needed for daily recurring jobs

MEDIUM

Template fit: merge each job's own intervals first so one job counts once, then sweep all of them together

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

QUESTION

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def merge_by_job(intervals):
    # intervals of the SAME job may overlap but they run on one server, so
    # merge them first: (job, start, end) -> disjoint blocks (start, end, job)
    by_job = {}
    for job, start, end in intervals:
        if job not in by_job:
            by_job[job] = []
        by_job[job].append((start, end))
    blocks = []
    for job in by_job:
        current = None
        for start, end in sorted(by_job[job]):
            if current is not None and start <= current[1]:
                if end > current[1]:
                    current[1] = end
            else:
                if current is not None:
                    blocks.append((current[0], current[1], job))
                current = [start, end]
        blocks.append((current[0], current[1], job))
    return blocks
```

```python
# ---- main question: main function ----
# Daily jobs (job id, start, end), half-open [start, end). How many servers
# are needed if a job's own overlapping runs share one server?
# Idea: group the intervals by job and merge each job's overlapping or touching
# intervals; then sweep the start and end events of all the merged blocks: the
# answer is the largest number of blocks running at the same moment (at equal
# times an end comes first). `dedicated` jobs (follow-up 1) get a server of
# their own and are left out of the sweep.
# Complexity: Time O(n log n), Space O(n)
def servers_needed(intervals, dedicated=()):
    dedicated = set(dedicated)
    shared = []
    used_dedicated = set()
    for interval in intervals:
        if interval[0] in dedicated:
            used_dedicated.add(interval[0])
        else:
            shared.append(interval)
    events = []
    for start, end, job in merge_by_job(shared):
        events.append((start, 1))
```

```python
        events.append((end, -1))
    running = 0
    best = 0
    for time, delta in sorted(events):
        running += delta
        if running > best:
            best = running
    return best + len(used_dedicated)
```

COMPLEXITY | Time O(m log m) | Space O(m)

14 Gate Entry / Exit Time Resolution with Tie-Break Rules — MEDIUM

Template fit: the same two-queue simulation with the tie-break rules from the statement

QUESTION

A gate lets one person through per second. events[i] = (time, action) for person i, where action is 'enter' or 'exit'. When several people want the gate in the same second: if the gate was not used in the previous second, people who exit go first; if it was used, the direction used in the previous second goes first again. Within one direction, people go in order of arrival (person id). Return the second at which each person passes.

Function signature

def gate_pass_times(events: list) -> list

EXAMPLES

Example 1	
Input	`[(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]`
Output	`[1, 0, 2, 5]`
Why	`At second 0 the gate was idle, so the exit (person 1) goes first. Person 0 follows at 1, person 2 (arrived at 1) at 2, and person 3 arrives alone at 5.`

QUESTION

Example 2	
Input	`[(0, 'enter'), (1, 'enter'), (1, 'exit')]`
Output	`[0, 1, 2]`
Why	The gate was used for entering in the previous second, so the waiting enter keeps priority over the exit.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: main function ----
# People arrive at a gate wanting to enter or to exit (events[i] = (arrival
# time, "enter" or "exit")). The gate lets ONE person through per second.
# Rules when several wait: if the gate was used the second before, the same
# direction keeps going; if it was idle, one direction wins (exit, by default).
# Within a direction, people go in order of arrival, then by number.
# Return the second each person goes through.
# Idea: an event-driven simulation with one queue per direction. Sort the
# people by (arrival time, number). When nobody waits, jump the clock straight
# to the next arrival (so huge timestamps cost nothing); if that leaves at
# least one idle second, forget the previous direction. Then everyone who has
# arrived joins their queue, and one person goes through.
# (`idle_first` is follow-up 4: who wins when the gate was idle.)
# Complexity: Time O(n log n) (the sort of arrivals; the simulation is O(n)), Space O(n)
def gate_pass_times(events, idle_first="exit"):
    n = len(events)
    arrivals = []
    for i in range(n):
        arrivals.append((events[i][0], i))
    arrivals.sort()
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t = 0
    next_person = 0
    previous = None                                  # the direction used in the second before
    last_used = -10                                  # the last second the gate was used
    while next_person < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            if arrivals[next_person][0] > t:
                t = arrivals[next_person][0]         # nobody waits: jump to the next arrival
            if t > last_used + 1:
                previous = None                      # an idle second forgets the last direction
        while next_person < n and arrivals[next_person][0] <= t:
            person = arrivals[next_person][1]
            lines[events[person][1]].append(person)  # everyone who has arrived joins their line
            next_person += 1
        if lines["enter"] and lines["exit"]:
            if previous is not None:
                want = previous                      # keep the direction used last second
            else:
                want = idle_first                    # idle gate: the tie-break rule decides
        elif lines["enter"]:
            want = "enter"
        else:
            want = "exit"
        person = lines[want].popleft()
        result[person] = t
        previous = want
        last_used = t
        t += 1
```

```
    return result
```

COMPLEXITY | Time O(n log n) for the sort, O(n) simulation | Space O(n)

15 Meeting Rooms III Variant — MEDIUM

Template fit: split meetings that cross midnight into two pieces first, then the same sweep

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	intervals = [[0, 30], [5, 10], [15, 20]]
Output	2
Why	Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.

Example 2	
Input	intervals = [[7, 10], [2, 4]]
Output	1
Why	No overlap, 1 room is sufficient.

Example 3	
Input	intervals = [[22, 2], [23, 1], [0, 3]]
Output	3
Why	All three meetings overlap around midnight.

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

MAIN CODE | helper functions first, then the answer

```python
DAY = 24


# ---- main question: helper functions ----
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past midnight: split it
    # into the part before midnight and the part after it
    parts = []
    for start, end in meetings:
        if end < start:
            parts.append((start, day))
            parts.append((0, end))
        else:
            parts.append((start, end))
    return parts


# ---- main question: main function ----
# How many rooms are needed? Meetings are half-open [start, end), and a
# meeting may wrap past midnight (end < start).
```

```python
# Idea: turn every meeting into one or two ordinary intervals, make a +1
# event at every start and a -1 event at every end, sort the events (at the
# same time an END comes first: a meeting ending at 5 and one starting at 5
# can share a room) and take the biggest running count.
# Complexity: Time O(n log n), Space O(n)
def min_rooms_sweep(meetings, day=DAY):
    events = []
    for start, end in pieces(meetings, day):
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                                   # (-1) sorts before (+1) at equal times
    running = 0
    best = 0
    for time, step in events:
        running += step
        if running > best:
            best = running
    return best
```

COMPLEXITY | Time O(n log n) | Space O(n)

16 Count Overlapping Rectangle Updates on a Grid — HARD

Template fit: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement

`range_add_counts(n, rectangles)`. Start with an **`n x n`** matrix of zeroes. Each rectangle is represented as **`[row1, col1, row2, col2]`** with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For **`n = 3`** and **`rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`**, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- **`1 <= n <= 500`**
- **`0 <= len(rectangles) <= 100_000`**
- **`0 <= row1 <= row2 < n`**
- **`0 <= col1 <= col2 < n`**

EXAMPLES

Example 1	
Input	`3, [(0, 0, 1, 1), (1, 1, 2, 2)]`
Output	`[[1, 1, 0], [1, 2, 1], [0, 1, 1]]`
Why	`The cell (1, 1) is covered by both rectangles, so it is counted twice.`

Example 2	
Input	`2, []`
Output	`[[0, 0], [0, 0]]`
Why	`No rectangles: all zeros.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# An n x n grid; each rectangle [r1, c1, r2, c2] (corners inclusive) adds 1
# to every cell inside. Return the final count of every cell.
# Idea: a 2D DIFFERENCE array. For each rectangle mark only its 4 corners:
# +1 at (r1, c1), -1 at (r1, c2 + 1), -1 at (r2 + 1, c1), +1 at
# (r2 + 1, c2 + 1). Then two prefix-sum passes (along every row, then down
# every column) spread the marks over the cells. Cost O(R + n^2) for R
# rectangles instead of O(R * n^2).
# Complexity: Time O(n^2 + R) for R rectangles, Space O(n^2)
def range_add_counts(n, rects):
    d = []
    for i in range(n + 1):
        d.append([0] * (n + 1))
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    for r in range(n + 1):                          # prefix sums along every row ...
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):                          # ... then down every column
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    counts = []
    for r in range(n):
        counts.append(d[r][:n])
    return counts
```

COMPLEXITY | Time O(R + n^2) | Space O(n^2)

17 Compute city skyline outline — HARD

Template fit: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers **`[L, R, H]`**:

- **`L`**: the x-coordinate of the left edge of the building,
- **`R`**: the x-coordinate of the right edge of the building (**`L < R`**),
- **`H`**: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height **`H`**. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **`x`**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **`n`** — the number of buildings.
- A list of **`n`** buildings, where each building is given as a triple **`[L, R, H]`** of integers. You may assume:
- **`1 <= n <= 10^4`**
- **`0 <= L < R <= 10^9`**
- **`1 <= H <= 10^9`**

Output

QUESTION

Return a list of key points representing the skyline. Each key point is a pair **`[x, height]`** (or a 2-element array/tuple) of integers.

Example

Input:

- **`n = 3`** - buildings = **`[[2, 9, 10], [3, 7, 15], [5, 12, 12]]`** One valid output:
- **`[[2, 10], [3, 15], [7, 12], [12, 0]]`** Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

EXAMPLES

Example 1	
Input	**`[[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]`**
Output	**`[[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]`**
Why	**`Buildings are [left, right, height]; the outline lists the points where the height changes.`**

Example 2	
Input	**`[[0, 2, 3]]`**
Output	**`[[0, 3], [2, 0]]`**
Why	**`One building: a point at its left edge and one at its right edge, where the height returns to 0.`**

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main function ----
# Buildings are [left, right, height]. Return the skyline: the "key points"
# [x, height] where the height of the outline changes.
# Idea: a sweep line from left to right with a max-heap of the buildings
# that are active (entries (-height, right edge), push negatives to get a
# max-heap). Start events carry -height so that, at the same x, taller starts
# come first and starts come before ends; end events carry 0. Buildings that
# have ended are removed LAZILY: only when they reach the top of the heap. After
# each event, the top of the heap is the current height; write a key point
# only when it differs from the last one. The ground (height 0) is always in the heap.
# Complexity: Time O(n log n), Space O(n)
def skyline(buildings):
    events = []
    for left, right, height in buildings:
        events.append((left, -height, right))        # a start
        events.append((right, 0, 0))                 # an end
    events.sort()
    points = []
    heap = [(0, float("inf"))]                       # the ground, which never ends
    for x, minus_height, right in events:
        while heap[0][1] <= x:
            heapq.heappop(heap)                      # a building that ended at or before x
        if minus_height != 0:
            heapq.heappush(heap, (minus_height, right))
        top = -heap[0][0]
```

```
        if not points or points[len(points) - 1][1] != top:
            points.append([x, top])
    return points
```

COMPLEXITY | Time O(n log n) | Space O(n)

18 Compute Turnstile Crossing Times

HARD

Template fit: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second

QUESTION

A turnstile lets one person through per second. events[i] = (arrival time, direction) for person i, with direction 'enter' or 'exit'. When people in both directions wait in the same second: if the turnstile was not used in the previous second, people exiting go first; otherwise the direction used in the previous second goes first again. People of the same direction go in order of arrival (person id). Return the second at which each person crosses.

Function signature

def turnstile_times(events: list) -> list

EXAMPLES

Example 1	
Input	`[(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]`
Output	`[0, 1, 2, 3]`
Why	The idle turnstile lets the exit (person 0) go first at second 0, then the two entering people follow at 1 and 2; person 3 arrives alone at 3.

Example 2	
Input	`[(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]`
Output	`[2, 0, 1, 3]`
Why	Idle turnstile: the exits (persons 1 and 2) go first at seconds 0 and 1, then the entering people (persons 0 and 3) at seconds 2 and 3.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: main function ----
# events[i] = (arrival time, "enter" or "exit"). One person passes the
# turnstile per second. If it was used in the second before, the same
# direction keeps going when both lines wait; if it was idle, `idle_first`
# wins (exit, by default). Return the second each person goes through.
# Idea: an event-driven simulation with two queues. Sort the arrivals once;
# when both queues are empty jump straight to the next arrival time, and if
# the jump leaves at least one idle second, reset the previous direction to
# "none". Then everyone who has arrived joins a line, and one person goes.
# Complexity: Time O(n log n) (the sort of arrivals; the simulation is O(n)), Space O(n)
def turnstile_times(events, idle_first="exit"):
    n = len(events)
    arrivals = []
    for i in range(n):
        arrivals.append((events[i][0], i))
    arrivals.sort()                                    # by arrival time, then by number
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t = 0
    next_person = 0
    previous = None
    last_used = -10
```

```
    while next_person < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            if arrivals[next_person][0] > t:
                t = arrivals[next_person][0]           # nobody is waiting: jump to the next arrival
            if t > last_used + 1:
                previous = None                        # an idle second forgets the last direction
        while next_person < n and arrivals[next_person][0] <= t:
            person = arrivals[next_person][1]
            lines[events[person][1]].append(person)    # everyone who has arrived joins a line
            next_person += 1
        if lines["enter"] and lines["exit"]:
            if previous is not None:
                want = previous                        # both lines: keep the direction used last second
            else:
                want = idle_first                      # after an idle gap the tie-break decides
        elif lines["enter"]:
            want = "enter"
        else:
            want = "exit"
        person = lines[want].popleft()
```

```
        result[person] = t
        previous = want
        last_used = t
        t += 1
    return result
```

COMPLEXITY | Time O(n log n) | Space O(n)

GROUP 3 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 3 E 4 M 2 H

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

19 Given two trees, determine if the pre-order traversals of two trees are same or not. EASY

Template fit: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference

QUESTION

Given two binary trees, determine whether their pre-order traversals (node, then left subtree, then right subtree) produce the same sequence of values. The trees themselves may have different shapes.

Function signature

def same_preorder(a: TreeNode, b: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3], [1, 2, 3]`
Output	`true`
Why	`Identical trees.`

Example 2	
Input	`[1, 2, null, 3], [1, null, 2, null, 3]`
Output	`true`
Why	`Different shapes (a chain going left versus a chain going right) but both read 1, 2, 3.`

Example 3	
Input	`[1, 2, 3], [1, 3, 2]`
Output	`false`
Why	`1, 2, 3 versus 1, 3, 2.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def push_children(stack, node):
    # right goes in first so that the LEFT child comes out first (pre-order)
    if node.right:
        stack.append(node.right)
    if node.left:
        stack.append(node.left)


# ---- main question: main function ----
# Idea: walk both trees in pre-order at the same time with two stacks
# and compare the values one by one. The first mismatch stops the work
# and no list of values is built. If one walk ends before the other,
# the sequences differ.
```

```python
# Complexity: Time O(n), Space O(h)
def same_preorder(a, b):
    stack_a = []
    stack_b = []
    if a:
        stack_a.append(a)
    if b:
        stack_b.append(b)
    while stack_a and stack_b:
        x = stack_a.pop()
        y = stack_b.pop()
        if x.val != y.val:
            return False
        push_children(stack_a, x)
        push_children(stack_b, y)
    return not stack_a and not stack_b
```

COMPLEXITY | Time O(min(n, m)) with early exit, O(n + m) worst case | Space O(h)

20 Calculate the height of a binary tree. EASY

Template fit: no update needed; return 1 + max(left, right) and read the value at the root

QUESTION

Given the root of a binary tree, return its height: the number of nodes on the longest path from the root down to a leaf (an empty tree has height 0).

Function signature

def height(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	[3, 9, 20, null, null, 15, 7]
Output	3
Why	Levels: [3], [9, 20], [15, 7]: height 3.

Example 2	
Input	[1, 2]
Output	2
Why	A root with one child.

Example 3	
Input	[]
Output	0
Why	An empty tree.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```python
# ---- main question: main function ----
# Idea: process the tree one level at a time with a queue and count the
# levels. No recursion, so a deep tree cannot overflow the stack; the
# memory used is the widest level. Height is counted in NODES: an empty
# tree is 0 and a single node is 1 (follow-up 1 explains edges).
# Complexity: Time O(n), Space O(w) for the widest level w
def height(root):
    levels = 0
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        levels += 1
        for i in range(len(queue)):              # exactly the nodes of this level
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return levels
```

COMPLEXITY | Time O(n) | Space O(width)

21 Find right-side view of binary tree — EASY

Template fit: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

EXAMPLES

Example 1	
Input	`[1, 2, 3, null, 5, null, 4]`
Output	`[1, 3, 4]`
Why	`Looking from the right you see 1, then 3 (hiding 2), then 4 (hiding 5).`

QUESTION

Example 2	
Input	[1, 2]
Output	[1, 2]
Why	A left child is visible when there is no right child at that depth.

Example 3	
Input	[]
Output	[]
Why	An empty tree shows nothing.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: visit the RIGHT child before the left one and pass the depth
# down. The first node met at a depth that is not recorded yet is the
# rightmost node of that depth. Memory is O(height).
# Complexity: Time O(n), Space O(h)
def right_side_view(root):
    view = []

    def visit(node, depth):
        if node is None:
            return
        if depth == len(view):
            view.append(node.val)                # first node seen at this depth
        visit(node.right, depth + 1)
        visit(node.left, depth + 1)

    visit(root, 0)
    return view
```

COMPLEXITY | Time O(n) | Space O(h)

22 Expression Tree Single-Leaf Mutation — MEDIUM

Template fit: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed

QUESTION

A boolean expression tree has leaves holding True or False and internal nodes AND, OR, XOR (two children) or NOT (one child). For each leaf in turn, imagine flipping only that leaf and report the value of the whole tree after the flip (every flip is independent of the others). Return the leaves and the list of answers, in leaf order.

Function signature

def flip_answers(root) -> (list, list)

(the leaves, the root value

after flipping each one)

EXAMPLES

QUESTION

Example 1

Input	`t = Node('AND', Node(val=True), Node('OR', Node(val=False), Node(val=True)))`
	`evaluate(t)`
	`[leaf.val for leaf in flip_answers(t)[0]]`
	`flip_answers(t)[1]`

Output	`evaluate(t) -> true`
	`[leaf.val for leaf in flip_answers(t)[0]] -> [true, false, true]`
	`flip_answers(t)[1] -> [false, true, false]`

Why	The tree is True AND (False OR True) = True. Flipping the first leaf gives False, flipping the False leaf changes nothing (the OR is still True), flipping the last leaf makes the OR False.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class Node:
    def __init__(self, op=None, left=None, right=None, val=None):
        # op is "AND", "OR", "XOR", "NOT", or None for a leaf (which holds val)
        self.op = op
        self.val = val
        self.left = left
        self.right = right


def combine(op, a, b=None):
    # the value of one gate from its input values (NOT has only one input)
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    return not a


def right_value(value, node):
    # the value of the right child, or None when there is none (NOT)
    if node.right is None:
        return None
    return value[node.right]


def evaluate(node):
    # plain recursive evaluation (used as the checker in the tests)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = None
    if node.right is not None:
        b = evaluate(node.right)
    return combine(node.op, a, b)


def build_cache(root):
    # one pass that stores, for every node, its parent and its value,
    # and a list of the leaves in left-to-right order
    parent = {root: None}
    value = {}
    leaves = []
```

```python
    order = []
    stack = [root]
```

```
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.right, node.left):
            if child is not None:
                parent[child] = node
                stack.append(child)
    for node in reversed(order):                # children come before their parents
        if node.op is None:
            value[node] = node.val
        else:
            value[node] = combine(node.op, value[node.left], right_value(value, node))
    for node in order:                          # `order` is root, left subtree, right subtree
        if node.op is None:
            leaves.append(node)
    return parent, value, leaves


# ---- main question: main function ----
# Idea: store every node's value and parent once. To flip one leaf, walk
# UP from it recomputing only the nodes whose value really changes, and
# stop as soon as a value stays the same. Read the root's value, then put
# every changed value back for the next leaf.
# Complexity: Time O(n + L * h) for L leaves and height h (each flip walks up only while a value changes), Sp
ace O(n)
def flip_answers(root):
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo = []
        node = leaf
        new = not value[leaf]
        while node is not None and value[node] != new:
            undo.append((node, value[node]))
            value[node] = new
            node = parent[node]
            if node is not None:
                new = combine(node.op, value[node.left], right_value(value, node))
        answers.append(value[root])
        for node, old in reversed(undo):          # restore for the next leaf
            value[node] = old
    return leaves, answers
```

COMPLEXITY | Time O(n + L * depth) | Space O(n)

23 Longest Consecutive Path in Binary Tree — MEDIUM

Template fit: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1

QUESTION

Given a binary tree, return the length of the longest path that goes from a parent down to a child where every value is exactly 1 more than the previous one (counted in nodes).

Function signature

def longestConsecutive(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[1, null, 3, 2, 4, null, null, null, 5]`
Output	`3`
Why	`3 -> 4 -> 5 is the longest increasing-by-one downward path.`

QUESTION

Example 2	
Input	[2, null, 3, 2, null, 1]
Output	2
Why	2 -> 3 is the longest; 3 -> 2 and 2 -> 1 go the wrong way.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: go down the tree carrying the length of the run that ends at
# the current node. A child that is exactly parent + 1 continues the
# run; any other child starts a new run of 1. Keep the best run seen.
# The length is counted in NODES (say that choice out loud).
# This version uses a stack, so it is also the answer to follow-up 2
# (no recursion).
# Complexity: Time O(n), Space O(h) for the height h of the tree
def longestConsecutive(root):
    if root is None:
        return 0
    best = 0
    stack = [(root, 1)]
    while stack:
        node, run = stack.pop()
        if run > best:
            best = run
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                stack.append((child, run + 1))
            else:
                stack.append((child, 1))
    return best
```

COMPLEXITY | Time O(n) | Space O(h)

24 **Given the root of a binary tree, return the length of the diameter of the tree.** **MEDIUM**

Template fit: best = max(best, left + right); return 1 + max(left, right) (the diameter)

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4, 5]`
Output	`4`
Why	`The longest path 4 - 2 - 1 - 3 has 4 nodes.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single node.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: one post-order pass. depth(node) returns the height of the
# node's subtree (in nodes) and, on the way, notes the longest path that
# passes THROUGH this node: left height + right height + 1 nodes. Every
# node is visited once. The diameter is counted in NODES, as the
# question says (follow-up 3 shows edges).
# Complexity: Time O(n), Space O(h)
def diameter_nodes(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        if left + right + 1 > best:
            best = left + right + 1
        if left > right:
            return left + 1
        return right + 1
```

```
    depth(root)
    return best
```

COMPLEXITY | Time O(n) | Space O(h)

25 Implement zigzag level-order traversal

MEDIUM

Template fit: collect values by depth; reverse every odd level (or fill each level from the other end)

QUESTION

Given the root of a binary tree, return its values level by level, reading the first level from left to right, the next from right to left, and so on, alternating. The result is a list of lists, one per level. Aim for O(n) time.

Function signature

def zigzag_level_order(root: TreeNode) -> list

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`[[3], [20, 9], [15, 7]]`
Why	`Level 0 is [3], level 1 is read right to left [20, 9], level 2 left to right [15, 7].`

Example 2	
Input	`[1]`
Output	`[[1]]`
Why	`A single node.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree has no levels.`

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: go level by level with a queue. Read the first level left to
# right, the next one right to left, and so on: keep a flag and reverse
# the level list whenever the flag says "right to left".
# Complexity: Time O(n), Space O(w) for the widest level (O(n) with the answer)
def zigzag_level_order(root):
    result = []
    queue = deque()
    if root:
        queue.append(root)
    left_to_right = True
```

```
    while queue:
        level = []
        for i in range(len(queue)):               # exactly the nodes of this level
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result
```

COMPLEXITY | Time O(n) | Space O(width)

26 Repeatedly Delete Leaves Matching a Target Value From a Binary Tree — HARD

Template fit: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

`remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]`

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

QUESTION

EXAMPLES

Example 1	
Input	tree = [1, 2, 3, 2, null, 2, 4], target = 2 and tree = [1, 3, 3, 3, 2], target = 3
Output	values(remove_leaves(tree([1, 2, 3, 2, None, 2, 4]), 2)[0]) -> [1, null, 3, null, 4] values(remove_leaves(tree([1, 3, 3, 3, 2]), 3)[0]) -> [1, 3, null, null, 2]
Why	remove_leaves returns (the new root, how many leaves were removed). Deleting a leaf can turn its parent into a leaf, which is deleted too if it also equals the target: in the first tree the 2s go and the root's left branch disappears entirely.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(values):
    # level-order list with None for a missing child: [1, 2, 3, 2, None, 2, 4]
    if not values:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
    return root


def tree_to_list(root):
    # the same encoding, with the trailing Nones removed
    result = []
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            result.append(None)
        else:
            result.append(node.val)
            queue.append(node.left)
            queue.append(node.right)
    while result and result[-1] is None:
        result.pop()
```

```
    return result


def remove_leaves(root, target):
    # post-order: handle both children first, then look at the node itself.
```

```
    # Returns (the new root, how many nodes were deleted) -- follow-up 3.
    deleted = 0

    def visit(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = visit(node.left)
        node.right = visit(node.right)
        if node.left is None and node.right is None and node.val == target:
            deleted += 1                          # a leaf with the target value disappears
            return None
        return node

    return visit(root), deleted


# ---- main question: main function ----
# Idea: one post-order pass. Process both children first, then the node:
# if it has become a leaf and its value is the target, it is removed
# (its parent then sees None). That is exactly how a deleted leaf can
# turn its parent into a new leaf.
# Complexity: Time O(n), Space O(n) (the recursion is O(h))
def remove_target_leaves(tree, target):
    root, deleted = remove_leaves(build_tree(tree), target)
    return tree_to_list(root)
```

COMPLEXITY | Time O(n) | Space O(h)

27 Delete an Entire Binary Tree — HARD

Template fit: post-order: free the children first, then the node itself

QUESTION

You are given the root of a binary tree (not a BST); each node has only left and right child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a post-order traversal. Implement delete_tree(root) that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

Follow-up (as reported): Recursive DFS uses O(h) stack space. Achieve O(1) extra space by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

EXAMPLES

QUESTION

Example 1	
Input	`root = 1 / \ 2 3`
Output	`[2, 3, 1]`
Why	`Post-order frees the left child, then the right child, then the root.`

Example 2	
Input	`root = None`
Output	`[]`
Why	`An empty tree frees nothing.`

Example 3	
Input	`root = 1 / 2 / 3`
Output	`[3, 2, 1]`
Why	`A left-leaning chain frees deepest-first.`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 .
- Node values are integers.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(shape):
    # shape is None or (value, left shape, right shape)
    if shape is None:
        return None
    value, left, right = shape
    return TreeNode(value, build_tree(left), build_tree(right))


# ---- main question: main function ----
# Idea: a node may only be freed AFTER its children, so free the left
# subtree, then the right subtree, then the node itself (post-order).
# "Freeing" a node clears its two pointers; the values are returned in
# the order they were freed.
# Complexity: Time O(n), Space O(h) for the recursion (plus the O(n) list of freed values)
def delete_tree(root):
    freed = []

    def free(node):
        if node is None:
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = None
        node.right = None

    free(root)
    return freed
```

COMPLEXITY | Time O(n) | Space O(h)

A heap of what is available: pop the best, put back the changed one

28 Streaming Median (LC 295) with Follow-ups — MEDIUM

Template fit: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced

QUESTION

Design a structure that receives numbers one at a time and can report the median of everything received so far at any moment. addNum(x) adds a number; findMedian() returns the middle value, or the mean of the two middle values when the count is even.

Function signature

class MedianFinder:
  def addNum(self, x: float) -> None
  def findMedian(self) -> float

EXAMPLES

Example 1

Input	`m = MedianFinder()`
	`m.addNum(5)`
	`m.addNum(2)`
	`m.findMedian()`
	`m.addNum(8)`
	`m.findMedian()`
	`m.addNum(1)`
	`m.findMedian()`

Output	`m.findMedian() -> 3.5`
	`m.findMedian() -> 5`
	`m.findMedian() -> 3.5`

Why	`Sorted so far: [2, 5] -> 3.5; [2, 5, 8] -> 5; [1, 2, 5, 8] -> 3.5.`

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main class ----
# Idea: keep the numbers in two heaps. The LOWER half is in a max-heap
# (store the negatives, since heapq is a min-heap) and the UPPER half in a
# min-heap. Their sizes differ by at most one (the lower half may have one
# more) and every number in the lower half is at most every number in the
# upper half. The median is the top of the lower heap, or the average of the
# two tops when the sizes are equal. addNum is O(log n), findMedian O(1).
# Complexity: Time addNum O(log n), findMedian O(1), Space O(n)
class MedianFinder:
    def __init__(self):
        self.low = []                           # max-heap (negated numbers)
        self.high = []                          # min-heap

    def addNum(self, x):
        heapq.heappush(self.low, -x)            # go through the lower half so the halves stay ordered
        biggest_of_low = -heapq.heappop(self.low)
        heapq.heappush(self.high, biggest_of_low)
        if len(self.high) > len(self.low):      # keep the lower half the same size or one bigger
            smallest_of_high = heapq.heappop(self.high)
            heapq.heappush(self.low, -smallest_of_high)

    def findMedian(self):
```

```
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2
```

COMPLEXITY | Two heaps: Time O(log n) add, O(1) median | Space O(n)
Small value domain: counting array: Time O(1) add, O(V) median | Space O(V)
Any k-th smallest: Fenwick tree over the value range: Time O(log V) add and query | Space O(V)

29 Compute minimum number of rooms needed — MEDIUM

Template fit: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time **`t`** does not conflict with another starting at time **`t`**).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- **`intervals`**: an array of **`n`** pairs **`[start, end]`** where **`0 <= start <= end`**.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- **`1 <= n <= 200000`**
- Times are integers in a reasonable range (e.g., **`0 . 10^9`**).

Example

- Input: **`[[0,30],[5,10],[15,20]]`**
- Output: **`2`** (One room can host **`[0,30]`**, the other can host **`[5,10]`** then **`[15,20]`**.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.
Constraints
0 <= n <= 2000000 <= start <= end <= 10^9

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and [15,20], so at most 2 rooms are needed at the same
    time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main function ----
# Idea: handle the meetings in order of start time and keep a min-heap of
# the END times of the rooms in use. If the room that ends earliest is free
# by this meeting's start, reuse it (replace its end time); otherwise open a
# new room (push). The size of the heap at the end is the number of rooms
# needed. Meetings are half-open [start, end): a meeting ending at t does not
# clash with one starting at t, and an empty meeting needs no room.
# Complexity: Time O(n log n), Space O(n)
def min_meeting_rooms(intervals):
    meetings = []
    for start, end in intervals:
        if start < end:
            meetings.append((start, end))
    meetings.sort()
    ends = []
    for start, end in meetings:
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)           # the earliest room is free: reuse it
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

COMPLEXITY | Sort by start with a min-heap of end times: Time O(n log n) | Space O(n)
Sorted starts and ends with two pointers: Time O(n log n) | Space O(n)

30 Implement a Reusable Locker Allocation System

MEDIUM

Template fit: heap of freed locker numbers; take the smallest freed one before handing out a new number

QUESTION

Problem

A locker bank has lockers numbered from 1 through **`n`**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

QUESTION

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main class ----
# n lockers numbered 1 .. n. store(customer) gives the smallest free locker
# number (-1 if the customer already has one or none is free); pickup(customer)
# frees the customer's locker and returns its number (-1 if there is none);
# last() is the locker of the latest SUCCESSFUL store (-1 if none yet).
# Idea: a min-heap of RELEASED locker numbers plus a counter of the next
# never-used number. A released locker is always smaller than any never-used
# one, so: take from the heap if it is not empty, else take the counter if it
# is at most n. No list of 10^9 lockers is ever built (follow-up 1).
# Complexity: Time store and pickup are O(log n) (a heap of free lockers), last is O(1), Space O(n)
class Lockers:
    def __init__(self, n):
        self.n = n
        self.next_new = 1                    # the next locker number never used
        self.free = []                       # released lockers, smallest on top
        self.of = {}                         # customer -> locker number
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        if customer not in self.of:
            return -1
```

```
        locker = self.of.pop(customer)
        heapq.heappush(self.free, locker)      # it is available again at once
        return locker

    def last(self):
        return self.last_store                 # failed operations never change this
```

COMPLEXITY | Time O(log n) per operation | Space O(lockers released and not reused)

31 Simulate room assignments for scheduled meetings — MEDIUM

Template fit: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

QUESTION

There are n rooms numbered 0 to n - 1 and a list of meetings (start, end). Meetings are handled in order of start time. A meeting takes the lowest-numbered free room; if every room is busy it waits until the room that frees up first becomes free and then runs for its full original length, so it ends later. Return the room that hosted the most meetings (the lowest room number on a tie).

Function signature

def mostBookedRoom(n: int, meetings: list) -> int

EXAMPLES

Example 1	
Input	2, [[0, 10], [1, 5], [2, 7], [3, 4]]
Output	0
Why	Rooms 0 and 1 each host two meetings; the lower number wins the tie.

Example 2	
Input	3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]
Output	1
Why	With three rooms, room 1 hosts two of the five meetings and no room hosts more.

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: main function ----
# n rooms (0 .. n - 1) and meetings (start, end). Meetings are handled in
# order of start time; a meeting takes the lowest-numbered free room. If
# every room is busy it WAITS for the room that frees up first and then
# runs for its full original length (so it ends later). Which room hosts the
# most meetings (the lowest number on a tie)?
# Idea: two heaps. `free` holds the numbers of the free rooms (lowest on
# top); `busy` holds (time the room becomes free, room). For each meeting,
# first move the rooms that finished by its start from `busy` to `free`.
# If a room is free take the lowest one; otherwise take the room that
# frees up first and push the meeting back by the waiting time.
# (`by="time"` is follow-up 3: rank rooms by their total busy time.)
# Complexity: Time O(M log M + M log n) for M meetings and n rooms, Space O(n)
def mostBookedRoom(n, meetings, by="count"):
    free = list(range(n))
    heapq.heapify(free)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        while busy and busy[0][0] <= start:
            finished_at, room = heapq.heappop(busy)
```

```
            heapq.heappush(free, room)                      # this room is free again
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            finished_at, room = heapq.heappop(busy)         # wait for the room that frees up first
            heapq.heappush(busy, (finished_at + (end - start), room))
        if by == "count":
            used[room] += 1
        else:
            used[room] += end - start                       # the meeting's own length, not the delay
    best = 0
    for room in range(n):
        if used[room] > used[best]:
            best = room                                     # ties keep the lowest room number
    return best
```

COMPLEXITY | Time O(M log M + M log N) | Space O(N)

32 Trip Expense Settlement — MEDIUM

Template fit: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back

QUESTION

A group of friends go on a trip and each friend may pay for others. Every record gives the payer, the amount and the list of people who share it equally (the payer may be one of them). Compute a small set of payments that settles everyone's debts: each payment is {payer, amount, payee}.

Function signature

def settle_greedy(records: list) -> list

EXAMPLES

Example 1	
Input	`trip = [{'payer': 'alice', 'amount': 4000, 'payees': ['bob', 'jess', 'alice', 'sam']}, {'payer': 'jess', 'amount': 2000, 'payees': ['jess', 'alice']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'sam', 'amount': 1000, 'payee': 'alice'}]`
Why	Alice paid 4000 shared four ways and owes 1000 of Jess's 2000; Bob and Sam each owe 1000, and Jess is even. Two payments settle everything.

Example 2	
Input	`trip = [{'payer': 'alice', 'amount': 3000, 'payees': ['alice', 'bob', 'cy']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'cy', 'amount': 1000, 'payee': 'alice'}]`
Why	Bob and Cy each owe Alice their 1000 share.

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

MAIN CODE | helper functions first, then the answer

```
import heapq
```

```python
# ---- main question: helper functions ----
def split_equally(amount, people):
    # share an amount (in whole CENTS, never floats) equally; the cents that
    # do not divide go to the first people in sorted order, so the rule is
    # always the same (follow-up 2)
    people = sorted(people)
    base = amount // len(people)
    extra = amount % len(people)
    shares = {}
    for i in range(len(people)):
        shares[people[i]] = base
        if i < extra:
            shares[people[i]] += 1
    return shares


def shares_of(record):
    # who owes what for one record. `payees` is a list (equal split) or a
    # dictionary {person: amount} for different shares (follow-up 3).
    payees = record["payees"]
    if isinstance(payees, dict):
        total = 0
        for share in payees.values():
            total += share
        if total != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)


def balances(records):
    # the net balance of every person: what they paid minus what they owe.
    # Positive: the group owes them money. All balances add up to zero.
    net = {}
    for record in records:
        payer = record["payer"]
        net[payer] = net.get(payer, 0) + record["amount"]
        shares = shares_of(record)
        for person in shares:
            net[person] = net.get(person, 0) - shares[person]
    result = {}
    for person in net:
```

```python
        if net[person] != 0:
            result[person] = net[person]
    return result


# ---- main question: main function ----
# Idea: the people who are owed money (positive balance) go into one
# max-heap and the people who owe money into another (push negatives to get a
# max-heap with heapq). Repeatedly let the biggest debtor pay the biggest
# creditor as much as one of them can (the smaller of the two amounts), and
# put back whoever still has something left. At least one person is settled
# per payment, so there are at most n - 1 payments.
# Complexity: Time O(n log n) for n people, Space O(n)
def settle_greedy(records):
    net = balances(records)
    owed = []                                   # creditors: (minus what they are owed, person)
    debt = []                                   # debtors: (minus what they owe, person)
    for person in net:
        if net[person] > 0:
            owed.append((-net[person], person))
        else:
            debt.append((net[person], person))
    heapq.heapify(owed)
```

```
    heapq.heapify(debt)
    payments = []
    while owed and debt:
        owed_minus, creditor = heapq.heappop(owed)
        debt_minus, debtor = heapq.heappop(debt)
        pay = min(-owed_minus, -debt_minus)
        payments.append({"payer": debtor, "amount": pay, "payee": creditor})
        if -owed_minus > pay:
            heapq.heappush(owed, (owed_minus + pay, creditor))
        if -debt_minus > pay:
            heapq.heappush(debt, (debt_minus + pay, debtor))
    return payments
```

COMPLEXITY | Heaps: match the largest debtor with the largest creditor: Time O(n log n) | Space O(n)
Exact minimum with a bitmask DP over zero-sum groups: Time O(2^n * n) | Space O(2^n)

33 Playlist Shuffle No Adjacent Artists — MEDIUM

Template fit: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count

QUESTION

Given the artist of every song in a playlist, return an order of the artists in which as few neighbours as possible are by the same artist: with no repeats next to each other whenever that is possible.

Function signature

def shuffle_no_adjacent(artists: list) -> list

EXAMPLES

Example 1	
Input	`['a', 'a', 'b', 'b', 'c']`
Output	`['a', 'b', 'a', 'b', 'c']`
Why	Two a's and two b's alternate; c fits wherever it likes.

Example 2	
Input	`['a', 'a', 'a', 'b']`
Output	`['a', 'b', 'a', 'a']`
Why	Three songs by 'a' cannot all be separated by a single 'b': the best is to keep two a's together at the end.

Example 3	
Input	`['a', 'a']`
Output	`['a', 'a']`
Why	A single artist: nothing can be done.

CONSTRAINTS

- 1 <= songs.length <= 10^4

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: helper functions ----
def count_items(items):
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
```

```
    return counts


def adjacent_pairs(artists):
    # how many neighbouring songs are by the same artist?
    pairs = 0
    for i in range(1, len(artists)):
        if artists[i] == artists[i - 1]:
            pairs += 1
    return pairs


# ---- main question: main function ----
# Given the artist of every song, return an order with as few neighbours as
# possible by the same artist (none at all when that is possible).
# Idea (greedy with a max-heap): keep (songs left, artist) in a heap.
# At every step play the artist with the MOST songs left, but hold that
# artist back for one step so they cannot play twice in a row; put them back
# in the heap after the next artist has played. If only the artist just
# played is left, a repeat cannot be avoided.
# Complexity: Time O(N log A) for N songs and A artists, Space O(A)
def shuffle_no_adjacent(artists):
    counts = count_items(artists)
    heap = []
    for artist in counts:
        heap.append((-counts[artist], artist))         # the most songs left first; ties alphabetical
    heapq.heapify(heap)
    order = []
    held = None                                        # the artist who just played (held back one step)
    while heap or held is not None:
        if heap:
            minus_left, artist = heapq.heappop(heap)
        else:
            minus_left, artist = held                  # only that artist is left: a forced repeat
            held = None
        order.append(artist)
        if held is not None:
            heapq.heappush(heap, held)                 # the artist held back may play again now
            held = None
        if minus_left + 1 < 0:
            held = (minus_left + 1, artist)            # one song fewer left; hold this artist back
    return order
```

COMPLEXITY | Time O(n log A) | Space O(A)

34 Minimum Makespan on Identical CPUs — MEDIUM

Template fit: the template as written: heap of the times the CPUs become free

QUESTION

You have C identical CPUs and a list release_times , where release_times[i] is the earliest time job i may start. Every job runs non-preemptively for the same fixed duration S . When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time. Scheduling optimally, compute the minimum possible makespan: the time at which the last job finishes.

Function signature

def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:
    ...

Follow-up (as reported): (Reported) The naive approach tracked an O(n*C) table of CPU availability; reduce it to a min-heap of the C next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

EXAMPLES

QUESTION

Example 1	
Input	`release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3`
Output	`7`
Why	`CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6). CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7). Last finish = 7.`

Example 2	
Input	`release_times = [0, 0, 0], num_cpus = 1, duration = 2`
Output	`6`
Why	`One CPU runs them back to back: [0,2), [2,4), [4,6).`

Example 3	
Input	`release_times = [5], num_cpus = 3, duration = 4`
Output	`9`
Why	`The single job starts at its release time 5, finishes at 9.`

CONSTRAINTS

- 1 <= C <= 10^5
- 0 <= len(release_times) <= 10^5
- 0 <= release_times[i] <= 10^9
- 1 <= S <= 10^9
- If there are no jobs, the makespan is 0 .
- Target complexity: O(n log C) time, O(C) space.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main function ----
# Jobs become available at the times in `release_times`; every job takes
# `duration`; there are `num_cpus` identical CPUs. When is everything done
# (the makespan) if each job goes to the CPU that frees up first, but never
# starts before its release time?
# Idea: handle the jobs in release order and keep a min-heap of the times
# at which the CPUs become free (all 0 at the start).
# Complexity: Time O(n log n + n log c) for n jobs and c CPUs, Space O(c)
def min_makespan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    last = 0
    for release in sorted(release_times):
        cpu_free = heapq.heappop(free)            # the CPU that frees up first
        start = release
        if cpu_free > start:
            start = cpu_free
        heapq.heappush(free, start + duration)
        if start + duration > last:
            last = start + duration
    return last
```

COMPLEXITY | Time O(n log n) | Space O(C)

35 Design server allocation for multi-type nodes — MEDIUM

Template fit: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)

QUESTION

Design a service that manages a pool of server nodes. Each node supports several instance types. getServer(count, types) hands out up to count servers that together cover all the requested types, preferring the least flexible servers (the ones that support the fewest types) so that versatile servers stay free, and returns None (allocating nothing) if the request cannot be met. releaseServer(ids) returns servers to the pool.

Function signature

class Servers:

def __init__(self, servers: dict)

id -> set of instance types

def getServer(self, count: int, types: list) -> list | None

def releaseServer(self, ids: list) -> None

EXAMPLES

Example 1

Input	`s = Servers({1: {'a'}, 2: {'a', 'b'}, 3: {'b'}, 4: {'a', 'b', 'c'}})`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(1, ['a'])`
	`s.releaseServer([1])`
	`s.getServer(1, ['a'])`

Output	`s.getServer(2, ['a', 'b']) -> [1, 3]`
	`s.getServer(2, ['a', 'b']) -> [2, 4]`
	`s.getServer(1, ['a']) -> null`
	`s.getServer(1, ['a']) -> [1]`

Why	The first request takes the two least flexible servers (1 for a, 3 for b); the second takes 2 and 4; nothing is left for the third; after server 1 is released it can serve type a again.

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: main class ----
# Servers support several instance types. getServer(count, types) must
# return `count` free servers that TOGETHER cover all the requested types
# (None if it cannot). To keep the versatile servers free for later, use the
# LEAST flexible suitable server first.
# Idea: for every type keep a heap of (number of types the server supports,
# server id) for the servers that support it: the least flexible is on top.
# A server that gets allocated is not removed from the other heaps at once
# ("lazy deletion"): it is skipped and dropped when it reaches the top.
# Pick the servers first and commit only if the whole request works.
# Complexity: Time getServer is O(T log S) for T requested types and S servers (lazy deletion), releaseServer
 O(types of each server * log S), Space O(S * types per server)
class Servers:
    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {}
        for server in servers:
            self.types[server] = set(servers[server])
        self.allocated = set()
        self.heaps = {}                                 # type -> heap of (flexibility, server id)
        self.lease = {}                                 # allocated server -> time its lease runs out (follow-
up 4)
```

```
        for server in self.types:
            for t in self.types[server]:
                if t not in self.heaps:
                    self.heaps[t] = []
                heapq.heappush(self.heaps[t], (len(self.types[server]), server))

    def best_for(self, t, skip):
        # the best free server of type t that is not in `skip` (None if there is none)
        heap = self.heaps.get(t, [])
        looked_past = []
        while heap:
            flexibility, server = heap[0]
            if server in self.allocated:
                heapq.heappop(heap)                    # lazy deletion
            elif server in skip:
                looked_past.append(heapq.heappop(heap))
            else:
                break
        best = None
        if heap:
            best = heap[0]
        for item in looked_past:
```

```
            heapq.heappush(heap, item)                 # put back what we only looked past
        return best

    def covers(self, chosen, t):
        # does one of the chosen servers already support type t?
        for server in chosen:
            if t in self.types[server]:
                return True
        return False

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if self.covers(chosen, t):
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        while len(chosen) < count:                     # fill up with the least flexible free servers
            skip = set(chosen)
            options = []
            for t in types:
                found = self.best_for(t, skip)
                if found is not None:
                    options.append(found)
            if not options:
                return None
            chosen.append(min(options)[1])
        if len(chosen) > count:
            return None                                # more servers were needed to cover the types than all
owed
        for server in chosen:
            self.allocated.add(server)
        return chosen

    def releaseServer(self, ids):
        for server in ids:
            if server in self.allocated:
                self.allocated.discard(server)
                self.lease.pop(server, None)
                for t in self.types[server]:
                    heapq.heappush(self.heaps[t], (len(self.types[server]), server))
```

```
    # follow-up 4: a client that dies without releasing its servers
    def rent(self, count, types, now, ttl):
```

```
        got = self.getServer(count, types)
        if got:
            for server in got:
                self.lease[server] = now + ttl
        return got

    def expire(self, now):
        late = []
        for server in self.lease:
            if self.lease[server] <= now:
                late.append(server)
        self.releaseServer(late)
        return late
```

COMPLEXITY | Time O((count + types) log N) amortized per request | Space O(total type memberships)

36 Huffman Tree Construction — HARD

Template fit: pop the two smallest weights, push their sum as a new node; repeat until one node remains

QUESTION

Given symbol frequencies, build the Huffman code: repeatedly merge the two least frequent items into one until a single tree is left; the path from the root gives each symbol its bit string. Return the code of every symbol. To make the answer unique, the codes are assigned canonically: shorter codes first, then alphabetical order.

Function signature

def huffman_codes(freq: dict) -> dict

EXAMPLES

Example 1	
Input	{'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}
Output	{'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a': '1110', 'b': '1111'}
Why	The frequent symbol f gets a 1-bit code; the rare ones get longer codes.

Example 2	
Input	{'x': 3, 'y': 8}
Output	{'x': '0', 'y': '1'}
Why	Two symbols always get one bit each.

Example 3	
Input	{'z': 10}
Output	{'z': '0'}
Why	A single symbol still needs a code of one bit.

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: helper functions ----
def huffman_lengths(freq):
    # the code length of every symbol = its depth in the Huffman tree.
```

```
    # Idea: put every symbol in a min-heap by frequency; repeatedly take the
    # TWO lightest nodes and merge them under a new node whose weight is the
    # sum; every symbol below the merged node gets one level deeper.
    if not freq:
        return {}
    symbols = sorted(freq)
    if len(symbols) == 1:
        return {symbols[0]: 1}                      # a lone symbol still needs one bit
    heap = []
    for i in range(len(symbols)):
        heap.append((freq[symbols[i]], i, [symbols[i]]))   # (weight, tie-break, the symbols below)
    heapq.heapify(heap)
    depth = {}
    for symbol in symbols:
        depth[symbol] = 0
    tie = len(heap)
    while len(heap) > 1:
        weight1, t1, below1 = heapq.heappop(heap)
        weight2, t2, below2 = heapq.heappop(heap)
        for symbol in below1 + below2:
            depth[symbol] += 1
        heapq.heappush(heap, (weight1 + weight2, tie, below1 + below2))
        tie += 1
    return depth


def to_binary(number, length):
    # the number written in binary with exactly `length` digits
    digits = ""
    for i in range(length):
        digits = str(number % 2) + digits
        number = number // 2
    return digits


def canonical_codes(lengths):
    # CANONICAL codes (follow-up 2 and 4): sort the symbols by (length,
    # symbol) and give them consecutive binary numbers, shifting left when
    # the length grows. A decoder can rebuild all codes from the lengths alone
```

```
    # (this is how DEFLATE stores its tables).
    ordered = []
    for symbol in lengths:
        ordered.append((lengths[symbol], symbol))
    ordered.sort()
    code = 0
    previous_length = 0
    codes = {}
    for length, symbol in ordered:
        code = code * 2 ** (length - previous_length)   # a longer length shifts the code left
        codes[symbol] = to_binary(code, length)
        code += 1
        previous_length = length
    return codes


# ---- main question: main function ----
# Idea: the lengths come from the min-heap merging above; the actual
# codes are then assigned in the canonical way.
# Complexity: Time O(n log n) for n symbols, Space O(n)
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))
```

COMPLEXITY | Time O(n log n) | Space O(n)

GROUP 5 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 6 M 3 H

Recurrence on a position or a range (memo or table)

37 Target Expressions from Ordered Digits — MEDIUM

*Template fit: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and **

QUESTION

Given a list of digits in order and a target number, put +, - or * between the digits (never reorder them and never glue two digits into one number) and add parentheses wherever you like. Return every distinct expression that evaluates to the target.

Function signature

def expressions_for_target(digits: list, target: int) -> list

EXAMPLES

Example 1	
Input	[2, 3, 4], 14
Output	['2*(3+4)', '2+3*4']
Why	2 * (3 + 4) = 14 and 2 + 3 * 4 = 14 are the only two ways.

Example 2	
Input	[1, 2, 3], 6
Output	['1*2*3', '1+2+3']
Why	Three digits can reach 6 several ways.

Example 3	
Input	[1, 2], 100
Output	[]
Why	No way to reach 100 with these digits.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
PRECEDENCE = {"+": 1, "-": 1, "*": 2}
ATOM = 3                                    # a single digit never needs brackets


def needs_brackets(op, child_precedence, is_right_child):
    # a child that binds weaker than the operator needs brackets; on the RIGHT
    # of a minus even an equal one does: 5-(3-1) is not 5-3-1
    if child_precedence < PRECEDENCE[op]:
        return True
    if is_right_child and op == "-" and child_precedence == PRECEDENCE[op]:
        return True
    return False


def join(left_text, left_precedence, op, right_text, right_precedence):
    if needs_brackets(op, left_precedence, False):
        left_text = "(" + left_text + ")"
    if needs_brackets(op, right_precedence, True):
        right_text = "(" + right_text + ")"
    return left_text + op + right_text
```

```
def expressions_over(digits, left, right, memo):
    # every value that digits[left:right] can make, with every expression that makes it:
    # {value: {(text, precedence), ...}}
    if (left, right) in memo:
        return memo[(left, right)]
    result = {}
    if right - left == 1:
        result[digits[left]] = {(str(digits[left]), ATOM)}
    for middle in range(left + 1, right):                    # try every split point
        lefts = expressions_over(digits, left, middle, memo)
        rights = expressions_over(digits, middle, right, memo)
        for left_value in lefts:
            for right_value in rights:
                results_of = {"+": left_value + right_value,
                              "-": left_value - right_value,
                              "*": left_value * right_value}
                for op in results_of:
                    value = results_of[op]
                    if value not in result:
                        result[value] = set()
                    for left_text, left_precedence in lefts[left_value]:
                        for right_text, right_precedence in rights[right_value]:
                            text = join(left_text, left_precedence, op, right_text, right_precedence)
                            result[value].add((text, PRECEDENCE[op]))
    memo[(left, right)] = result
    return result


# ---- main question: main function ----
# Idea (interval DP): an expression over digits[l:r] is two smaller
# expressions over a split point joined by +, - or *. For every range
# remember, per value, the set of expressions that make it. Brackets are
# added only where precedence needs them.
# Complexity: Time exponential in the number of digits (every split and every operator), Space the same
def expressions_for_target(digits, target):
    table = expressions_over(digits, 0, len(digits), {})
    texts = set()
    if target in table:
        for text, precedence in table[target]:
            texts.add(text)
    return sorted(texts)
```

COMPLEXITY | Time exponential in the number of results; polynomial in the number of distinct values | Space O(distinct values per interval)

38 Count Word Decompositions into Chemical Element Symbols — MEDIUM

Template fit: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists

QUESTION

Chemical elements have symbols of one or two letters (H, He, Si, Cs ...). Given a word, count the ways to write it as a sequence of element symbols (ignore upper and lower case). Different splits count as different ways even if they spell the same word.

Function signature

def count_decompositions(word: str) -> int

EXAMPLES

QUESTION

Example 1	
Input	'Physics'
Output	4
Why	P-H-Y-Si-C-S, P-H-Y-Si-Cs, P-H-Y-S-I-C-S and P-H-Y-S-I-Cs: 4 ways.

Example 2	
Input	'Ho'
Output	2
Why	'Ho' is one symbol (holmium) and also H + O: 2 ways.

Example 3	
Input	'xqz'
Output	0
Why	No split works (x, y... cannot all be symbols).

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# ---- main question: main function ----
# Idea: ways[i] = the number of ways to split the END of the word, from
# position i on. A symbol has one or two letters, so
# ways[i] = ways[i + 1] (if word[i] is a symbol)
#         + ways[i + 2] (if word[i:i + 2] is a symbol).
# An empty ending has exactly one way. `mod` is for follow-up 5.
# Complexity: Time O(n), Space O(n)
def count_decompositions(word, symbols=ELEMENTS, mod=None):
    w = word.lower()
    n = len(w)
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] = ways[i] % mod
    return ways[0]
```

COMPLEXITY | Time O(n) | Space O(n)

39 Find the longest palindromic subsequence using dynamic programming. MEDIUM

Template fit: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end

QUESTION

Given a string s, return the length of its longest palindromic subsequence: the longest sequence of letters of s, taken in order but not necessarily next to each other, that reads the same forwards and backwards.

Function signature

def lps_length(s: str) -> int

EXAMPLES

Example 1	
Input	`'bbbab'`
Output	`4`
Why	`'bbbb' is a subsequence of 'bbbab' and a palindrome.`

Example 2	
Input	`'cbbd'`
Output	`2`
Why	`'bb' (the two b's) is the longest.`

Example 3	
Input	`'a'`
Output	`1`
Why	`A single letter.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def lps_table(s):
    # dp[i][j] = the length of the longest palindromic SUBSEQUENCE of s[i..j].
    # Equal end letters are both used (+2); otherwise drop one end.
    n = len(s)
    dp = []
    for i in range(n):
        dp.append([0] * n)
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            elif dp[i + 1][j] >= dp[i][j - 1]:
                dp[i][j] = dp[i + 1][j]
            else:
                dp[i][j] = dp[i][j - 1]
    return dp


# ---- main question: main function ----
# Idea (interval DP): dp[i][j] is the longest palindromic subsequence of
# s[i..j]. Fill i from the end of the string to the start and j from i + 1
# up. Answer: dp[0][n - 1].
# Complexity: Time O(n^2), Space O(n^2)
def lps_length(s):
    if not s:
        return 0
    return lps_table(s)[0][len(s) - 1]
```

COMPLEXITY | Time O(n^2) | Space O(n)

40 Jump Game with Profit

MEDIUM

Template fit: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]

QUESTION

You stand at index 0 of an array. From index i you may jump to any index from i + 1 up to i + arr[i]. Landing on index i earns profit[i] (the start earns profit[0] too). Return the largest total profit of a path that ends exactly at the last index, or -1 if the last index cannot be reached.

Function signature

def max_profit_fast(arr: list, profit: list) -> int

EXAMPLES

Example 1	
Input	[2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
Output	15
Why	Jumping one index at a time visits (and collects) everything: 1 + 5 + 2 + 3 + 4 = 15.

Example 2	
Input	[1, 0, 1], [1, 1, 1]
Output	-1
Why	From index 1 you cannot jump at all (arr[1] = 0), so the last index is unreachable.

MAIN CODE | helper functions first, then the answer

```
import heapq


NEG = float("-inf")


# ---- main question: main function ----
# Reading A: arr[i] is the farthest you may jump from cell i, profit[i]
# is gained when you land on i; maximise the total profit on the way to
# the last cell (-1 if it cannot be reached).
# Idea: best[i] = the largest profit possible when standing on i. You can
# land on i from any earlier cell j that reaches it (j + arr[j] >= i):
# best[i] = profit[i] + the largest best[j] among those. Keep the
# candidates in a MAX-heap (push negatives) and drop a candidate for good
# as soon as it can no longer reach the current cell.
# Complexity: Time O(n log n), Space O(n)
def max_profit_fast(arr, profit):
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]                 # (minus the profit, how far it reaches)
    for i in range(1, n):
        while heap and heap[0][1] < i:
            heapq.heappop(heap)                 # this cell can no longer reach i or anything after
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    if best[n - 1] == NEG:
        return -1
    return best[n - 1]
```

COMPLEXITY | Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index: Time O(n * maxJump) | Space O(n)
Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free: Time O(n) | Space O(n)

41 Fence Painting Minimum Operations (Gemini Team) MEDIUM

Template fit: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast

QUESTION

A fence has n planks, plank i is a[i] units tall. One stroke is either horizontal (paints one row of height h across a run of neighbouring planks that are all at least h tall) or vertical (paints one plank completely, bottom to top). Return the smallest number of strokes needed to paint the whole fence.

Function signature

def paint(a: list) -> int

EXAMPLES

Example 1	
Input	`[2, 2, 1, 2, 1]`
Output	`3`
Why	`Row 1 across all five planks, row 2 across the first two planks, and one vertical stroke on plank 4: 3 strokes.`

Example 2	
Input	`[1, 1, 1]`
Output	`1`
Why	`One horizontal stroke of height 1 across all three planks paints the whole fence.`

Example 3	
Input	`[5]`
Output	`1`
Why	`One tall plank: a single vertical stroke beats five horizontal ones.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def sparse_table(a):
    # table[j][i] = the index of the smallest plank among a[i : i + 2**j].
    # Built once, it answers "where is the smallest plank in a[l..r]?" in O(1).
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        previous = table[j - 1]
        row = []
        for i in range(len(a) - (1 << j) + 1):
            x = previous[i]
            y = previous[i + (1 << (j - 1))]
            if a[x] <= a[y]:
                row.append(x)
            else:
                row.append(y)
        table.append(row)
        j += 1
    return table


def smallest_index(table, a, left, right):
    # the index of the smallest plank in a[left..right]: two overlapping blocks cover the range
    k = 0
    while (1 << (k + 1)) <= right - left + 1:
        k += 1
    x = table[k][left]
```

```
    y = table[k][right - (1 << k) + 1]
    if a[x] <= a[y]:
        return x
    return y


def paint_strokes(a, width=None):
    # returns (the number of strokes, the list of strokes). `width` is the
    # width of the horizontal brush in planks (None = as wide as you like).
    if len(a) == 0:
        return 0, []
    table = sparse_table(a)

    def solve(left, right, base):
        # paint planks left..right, whose lowest `base` rows are already painted
        if left > right:
            return 0, []
        low = smallest_index(table, a, left, right)
```

```
        size = right - left + 1
        per_row = 1
        if width is not None:
            per_row = (size + width - 1) // width          # a narrow brush needs several strokes per row
        band = (a[low] - base) * per_row                   # horizontal strokes across the whole range
        left_count, left_strokes = solve(left, low - 1, a[low])
        right_count, right_strokes = solve(low + 1, right, a[low])
        horizontal = band + left_count + right_count
        if size <= horizontal:                             # painting every plank upright is cheaper
            strokes = []
            for i in range(left, right + 1):
                strokes.append(("vertical", i))
            return size, strokes
        strokes = []
        for row in range(base + 1, a[low] + 1):
            strokes.append(("horizontal", row, left, right))
        return horizontal, strokes + left_strokes + right_strokes

    return solve(0, len(a) - 1, 0)


# ---- main question: main function ----
# Idea: look at the SHORTEST plank in a range. Either paint the rows up
# to its height across the whole range (one stroke per row) and then
# solve the two sides on top of that, or paint every plank of the range
# upright (one stroke per plank): take the cheaper. The sparse table
# finds the shortest plank in O(1) and every plank is the split point at
# most once, so only n ranges are visited.
# Complexity: Time O(n log n), Space O(n log n)
def paint(a, width=None):
    return paint_strokes(a, width)[0]
```

COMPLEXITY | Time O(n log n) | Space O(n log n)

42 Maximize 1D deliveries within distance and minimize total distance — MEDIUM

Template fit: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)

QUESTION

Orders and drivers are points on a line. A driver can deliver an order that is at most D away; every driver takes at most one order and every order is delivered by at most one driver. First maximize the number of deliveries; among all plans with that many deliveries, minimize the total distance driven. Return (number of deliveries, total distance).

QUESTION

Function signature

def deliveries(orders: list, drivers: list, D: int) -> tuple

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 6, 20], 2`
Output	`(2, 2)`
Why	`1 is served by the driver at 2, and 5 by the driver at 6: 2 deliveries with total distance 1 + 1 = 2.`

Example 2	
Input	`[1, 5, 9], [2, 6, 20], 11`
Output	`(3, 13)`
Why	`With D = 11 the driver at 20 can also take the order at 9 (distance 11): 1 + 1 + 11 = 13 for 3 deliveries.`

Example 3	
Input	`[3, 4], [], 5`
Output	`(0, 0)`
Why	`No drivers, no deliveries.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def better(x, y):
    # the better of two (deliveries, minus distance) pairs: more deliveries
    # first, then less distance (a bigger "minus distance")
    if x >= y:
        return x
    return y


# ---- main question: main function ----
# Idea: sort the orders and the drivers (positions on a line). Let
# dp[i][j] be the best pair (number of deliveries, MINUS the total
# distance) using only the first i orders and the first j drivers. For
# the next order and driver you can skip the order, skip the driver, or
# match them if they are at most D apart. Pairs compare lexicographically,
# so more deliveries wins and then the smaller total distance.
# Complexity: Time O(m log m + k log k + m * k) for m orders and k drivers, Space O(m * k)
def deliveries(orders, drivers, D):
    o = sorted(orders)
    d = sorted(drivers)
    dp = []
    for i in range(len(o) + 1):
        dp.append([(0, 0)] * (len(d) + 1))
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            best = better(dp[i - 1][j], dp[i][j - 1])          # skip the order, or skip the driver
            gap = o[i - 1] - d[j - 1]
            if gap < 0:
                gap = -gap
            if gap <= D:                                       # or match them
                count, minus_distance = dp[i - 1][j - 1]
                best = better(best, (count + 1, minus_distance - gap))
            dp[i][j] = best
    count, minus_distance = dp[len(o)][len(d)]
    return count, -minus_distance
```

COMPLEXITY | Time O(n * m) | Space O(n * m)

43 Edit distance — HARD

Template fit: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

QUESTION

Given two words a and b, return the edit distance: the smallest number of single-letter operations (insert a letter, delete a letter, or replace a letter) that turn a into b.

Function signature

def edit_distance(a: str, b: str) -> (int, list)

(the distance, the whole DP

table)

EXAMPLES

Example 1	
Input	a = 'horse', b = 'ros'
Output	edit_distance('horse', 'ros')[0] -> 3
Why	horse -> rorse (replace h) -> rose (delete r) -> ros (delete e): 3 operations.

Example 2	
Input	a = 'intention', b = 'execution'
Output	edit_distance('intention', 'execution')[0] -> 5
Why	intention -> execution takes 5 operations.

Example 3	
Input	a = '', b = 'abc'
Output	edit_distance('', 'abc')[0] -> 3
Why	From the empty word, insert every letter.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
def smallest(x, y, z):
    result = x
    if y < result:
        result = y
    if z < result:
        result = z
    return result


# ---- main question: main function ----
# Idea: dp[i][j] = the fewest edits that turn the first i letters of a
# into the first j letters of b. With equal last letters it is
# dp[i-1][j-1]; otherwise 1 + the best of replace (dp[i-1][j-1]),
# delete (dp[i-1][j]) and insert (dp[i][j-1]). The first row and column
# are plain deletes / inserts. Returns (the distance, the whole table).
# Complexity: Time O(m * n), Space O(m * n)
def edit_distance(a, b):
    m = len(a)
    n = len(b)
    dp = []
```

```
    for i in range(m + 1):
        dp.append([0] * (n + 1))
    for i in range(m + 1):
        dp[i][0] = i                                # delete i letters
    for j in range(n + 1):
        dp[0][j] = j                                # insert j letters
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]         # same letter: no edit
            else:
                dp[i][j] = 1 + smallest(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])
    return dp[m][n], dp


# RELATED: are the words at most k edits apart?
# Complexity: Time O(m * n), Space O(m * n)
def within_k(a, b, k):
    if abs(len(a) - len(b)) > k:
        return False                                # lengths alone need more than k edits
    return edit_distance(a, b)[0] <= k
```

```
# RELATED: the word-ladder version: change one letter at a time and every
# word must be in the list; BFS over the words
# Complexity: Time O(N * L^2 * 26) for N words of length L, Space O(N * L)
def ladder_length(begin, end, words):
    words = set(words)
    if end not in words:
        return 0
    queue = deque([(begin, 1)])
    seen = {begin}
    while queue:
        word, steps = queue.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                following = word[:i] + c + word[i + 1:]
                if following in words and following not in seen:
                    seen.add(following)
                    queue.append((following, steps + 1))
    return 0
```

COMPLEXITY | Time O(m * n) | Space O(min(m, n))

44 Count Perfect Wake Numbers up to N — HARD

Template fit: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag

QUESTION

A number is perfect if its digits are all different, it contains no 0, and no digit is a valley (a digit strictly smaller than both of its neighbours). Part 1: decide whether a number is perfect. Part 2: count the perfect numbers from 1 to N.

Function signature

def is_perfect(n: int) -> bool

def count_up_to(N: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	196
Output	true
Why	Digits 1, 9, 6 are different, no zero, and 9 is a peak, not a valley.

Example 2	
Input	1546
Output	false
Why	The 4 in 1546 sits between 5 and 6, both larger: a valley.

Example 3	
Input	99
Output	81
Why	Every number 1-9 and every two-digit number with different non-zero digits: 9 + 72 = 81.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def digits_of(n):
    # the digits of n as a list of numbers: 1203 -> [1, 2, 0, 3]
    result = []
    for ch in str(n):
        result.append(int(ch))
    return result


def choose(n, k):
    # the number of ways to pick k things out of n
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


def with_k_digits(k):
    # how many perfect numbers have exactly k digits (closed form, follow-up 4):
    # pick k different digits out of 1..9; the biggest is the peak and every
    # other digit goes left or right of it in sorted order: choose(9, k) * 2^(k-1)
    if k < 1:
        return 0
    return choose(9, k) * 2 ** (k - 1)


# ---- main question: main functions ----
# Idea: a number is perfect when it has no zero, no repeated digit, and
# no "valley": no digit smaller than both of its neighbours.
# Complexity: Time O(d) for d digits, Space O(d)
def is_perfect(n):
    d = digits_of(n)
    seen = set()
    for digit in d:
        if digit == 0 or digit in seen:
            return False
        seen.add(digit)
    for i in range(1, len(d) - 1):
        if d[i - 1] > d[i] and d[i] < d[i + 1]:
            return False
```

```
    return True


# PART 2: how many perfect numbers are there from 1 to n?
# Idea (digit DP): numbers with fewer digits come from the closed form.
```

```
# For numbers with the SAME number of digits, build them digit by digit
# from the left, remembering: which digits are used (a bitmask), the last
# digit, whether we already went DOWN (`falling`: going up after that
# would be a valley), and whether we are still equal to the limit
# (`tight`: then the next digit may not exceed n's digit).
# Complexity: Time O(d * 2^10 * 10 * 10) states (a fixed amount for decimal digits), Space the same
def count_up_to(n):
    if n < 1:
        return 0
    digits = digits_of(n)
    size = len(digits)
    total = 0
    for k in range(1, size):
        total += with_k_digits(k)
    memo = {}

    def count_from(position, used, last, falling, tight):
        if position == size:
            return 1
        key = (position, used, last, falling, tight)
        if key in memo:
            return memo[key]
        limit = 9
        if tight:
            limit = digits[position]
        count = 0
        for d in range(1, limit + 1):
            if (used >> d) % 2 == 1:
                continue                            # digit already used
            now_falling = falling
            if position > 0:
                if d > last and falling:
                    continue                        # going up after going down: a valley
                if d < last:
                    now_falling = True
            count += count_from(position + 1, used + (1 << d), d, now_falling, tight and d == digits[position
])
        memo[key] = count
        return count

    return total + count_from(0, 0, 0, False, True)
```

COMPLEXITY | Part 1: test a single number: Time O(d) | Space O(d)
Part 2: digit DP with tight flag, used-digit mask, last digit and direction: Time O(d * 2^9 * 9 * 9) | Space O(2^9 * 9)

45 Word Break: All Sentences — HARD

Template fit: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end)

QUESTION

You are given a string s and a dictionary word_dict of distinct words. Return ALL possible ways to segment s into a sequence of space-separated words such that every word in the sequence appears in word_dict . A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

QUESTION

Function signature

from typing import List

def word_break_all(s: str, word_dict: List[str]) -> List[str]:

    ...

Follow-up (as reported): Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid exponential re-work.

EXAMPLES

Example 1	
Input	`s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]`
Output	`["cats and dog", "cat sand dog"]`
Why	`Two segmentations use only dictionary words.`

Example 2	
Input	`s = "pineapplepenapple", word_dict = ["apple","pen","applepen","pine","pineapple"]`
Output	`["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]`
Why	`All three segmentations consist entirely of dictionary words.`

Example 3	
Input	`s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]`
Output	`[]`
Why	`No segmentation covers the whole string with dictionary words.`

CONSTRAINTS

- 1 <= len(s) <= 20
- 1 <= len(word_dict) <= 1000
- 1 <= len(word) <= 10 for each word; words consist of lowercase letters and are distinct.
- If no valid segmentation exists, return an empty list [] .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea (memoised search on the start index): the sentences that can be
# made from the ending of s that starts at `start` do not depend on how
# we got there, so compute them once per start. Try every word that
# begins at `start` and put it in front of every sentence of the rest.
# Complexity: Time O(n^3 + size of the output) (n^2 substrings, each hashed in O(n)), Space O(size of the out
put)
def word_break_all(s, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(s):
            return [""]                                  # nothing left: one empty sentence
        if start in memo:
            return memo[start]
        result = []
        for end in range(start + 1, len(s) + 1):
            word = s[start:end]
            if word in words:
                for rest in sentences(end):
                    if rest == "":
                        result.append(word)
```

```
                else:
                    result.append(word + " " + rest)
        memo[start] = result
        return result

    return sentences(0)
```

COMPLEXITY | Time O(n^2 + output) | Space O(n + output)

GROUP 6 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 5 M 4 H

Best-first search with a heap: Dijkstra and its minimax twin

46 Path Minimizing the Maximum Cell (Swim in Rising Water) MEDIUM

Template fit: the same minimax loop with a given start and end cell (Swim in Rising Water)

QUESTION

You get a grid of heights, a start cell and an end cell, and you may move to any of the four neighbouring cells. The cost of a path is the largest height on it (start and end included). Return the smallest cost over all paths from start to end.

Function signature

def min_max_path(g: list, start: tuple, end: tuple) -> int

EXAMPLES

Example 1	
Input	`[[0, 2], [1, 3]], (0, 0), (1, 1)`
Output	`3`
Why	`Both routes pass the cell of height 3 at the end, so the best cost is 3.`

Example 2	
Input	`[[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0, 0), (4, 4)`
Output	`16`
Why	`A 5x5 board: the best route has to climb to 16 somewhere, and no route stays lower.`

MAIN CODE | helper functions first, then the answer

```
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Idea: Dijkstra on the running maximum. Heap entries are (cost so
# far, cell); taking a cell out fixes its cost; a neighbour gets
# max(cost, its own value). Stop when the end cell comes out; skip
# outdated heap entries.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_max_path(g, start, end):
    rows, cols = len(g), len(g[0])
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, cell = heapq.heappop(heap)
        if cost > best[cell]:
```

```
            continue                                # an outdated entry
        if cell == end:
            return cost
        for nr, nc in neighbors(cell[0], cell[1], rows, cols):
            new_cost = cost
            if g[nr][nc] > new_cost:
                new_cost = g[nr][nc]                # highest cell so far, or this one
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, (nr, nc)))
    return None


# the same idea on a weighted graph: every edge must be below `limit`,
# and the cost of a path is its heaviest edge
```

```
# Complexity: Time O(E log V), Space O(V)
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, weight), ...]
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            if w >= limit:
                continue                            # edges at or above the limit are not allowed
            new_cost = cost
            if w > new_cost:
                new_cost = w
            if new_cost < best.get(v, INF):
                best[v] = new_cost
                heapq.heappush(heap, (new_cost, v))
    return -1                                       # no admissible path


# the related plain Dijkstra: when does a signal reach every node?
# Complexity: Time O((V + E) log V), Space O(V + E)
def network_delay(n, edges, src):
    graph = {}
    for u, v, w in edges:
        if u not in graph:
            graph[u] = []
        graph[u].append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    if len(dist) < n:
        return -1                                   # some node is never reached
    worst = 0
    for d in dist.values():
```

```
        if d > worst:
            worst = d
    return worst
```

COMPLEXITY | Time O(V log V) | Space O(V)

47 Nearest Favorite City by Travel Time — MEDIUM

Template fit: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

QUESTION

You are given a network of num_cities cities (0..num_cities-1) connected by weighted, undirected roads roads[i] = [u, v, w] where w > 0 is the travel time. Given a start city and a non-empty list favorites of favorite cities, return the favorite city reachable from start in the shortest total travel time. Break ties by smallest city index. If none of the favorite cities is reachable, return -1 .

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	`num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites = [3, 4]`
Output	`3`
Why	`Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.`

Example 2	
Input	`num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]`
Output	`-1`
Why	`City 2 is disconnected from start, so no favorite is reachable.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(roads) <= 2*10**5 , 1 <= w <= 10**9 .
- The interviewer required the optimal solution (Dijkstra with a priority queue).

MAIN CODE | helper functions first, then the answer

```python
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def build_roads(num_cities, roads):
    # roads are two-way: [city, city, travel time]
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


# ---- main question: main function ----
# Idea: Dijkstra from the start finalises cities in order of distance,
# so the first favourite taken out is the nearest. Cities at the same
# distance: the smaller number wins, so keep going while d does not
# exceed the best distance found. Returns -1 if none is reachable.
# Complexity: Time O((V + E) log V), Space O(V + E)
def nearest_favorite(num_cities, roads, start, favorites):
```

```
    adj = build_roads(num_cities, roads)
    dist = [INF] * num_cities
    dist[start] = 0
    heap = [(0, start)]
    best_distance = None
    best_city = -1
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if best_distance is not None and d > best_distance:
            break                                   # everything left is farther
        if u in favorites and (best_distance is None or u < best_city):
            best_distance = d
            best_city = u
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return best_city
```

COMPLEXITY | Time O((V + E) log V) | Space O(V + E)

48 Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time. MEDIUM

Template fit: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time

QUESTION

A schedule lists trains as (from, to, departure time, arrival time). Starting in city src at time start, find the sequence of trains that gets you to dst at the earliest possible time. You can only board a train that leaves at or after the time you arrive in its departure city. Return (arrival time, the trains you take), or None if dst cannot be reached.

Function signature

def earliest_arrival(trains: list, src, dst, start=0) -> tuple | None

EXAMPLES

Example 1	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9), ('B', 'C', 2, 5)], 'A', 'C'`
Output	`(6, [('A', 'B', 1, 3), ('B', 'C', 4, 6)])`
Why	A -> B arrives at 3; the train B -> C that leaves at 2 is already gone, so take the one leaving at 4 and arrive at 6 (the direct A -> C train arrives later, at 9).

Example 2	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'`
Output	`null`
Why	No train leaves C, so A cannot be reached from C.

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: helper functions ----
# A train is a tuple (from_city, to_city, departure_time, arrival_time).
def trains_by_city(trains):
    # city -> the trains that leave from it
```

```
    table = {}
    for train in trains:
        if train[0] not in table:
            table[train[0]] = []
        table[train[0]].append(train)
    return table


# ---- main question: main function ----
# Idea: Dijkstra on arrival time. best[city] is the earliest moment you
# can be there. From a city you may board any train that leaves at or
# after your arrival (plus `layover`, follow-up 1; no layover is needed
# in the city you start in). Remember which train improved each city to
# rebuild the sequence. Returns (arrival time, trains taken) or None.
# Complexity: Time O(T log T) for T trains, Space O(T + C) for C cities
def earliest_arrival(trains, src, dst, start=0, layover=0):
    leaving = trains_by_city(trains)
    best = {src: start}
    took = {}                                   # city -> the train that got us there
    heap = [(start, src)]
    while heap:
        time, city = heapq.heappop(heap)
        if time > best[city]:
            continue                            # an outdated entry
        if city == dst:
            break
        wait = layover
        if city == src:
            wait = 0
        for train in leaving.get(city, []):
            _from, to_city, departs, arrives = train
            if departs >= time + wait and (to_city not in best or arrives < best[to_city]):
                best[to_city] = arrives
                took[to_city] = train
                heapq.heappush(heap, (arrives, to_city))
    if dst not in best:
        return None
    route = []
```

```
    city = dst
    while city != src:                          # walk back from the destination
        route.append(took[city])
        city = took[city][0]
    route.reverse()
    return best[dst], route
```

COMPLEXITY | Time O((T + C) log C) with trains sorted by departure per city | Space O(T + C)

49 Find the shortest path in a graph. MEDIUM

Template fit: combine = cost + w; also store parent[v]; stop at the target and rebuild the path

QUESTION

A directed graph is given as a list of weighted edges (u, v, w) with non-negative weights. Return the total length and the nodes of a shortest path from s to t, or None if t cannot be reached.

Function signature

def dijkstra_path(edges: list, s, t) -> tuple | None

(distance, path)

EXAMPLES

QUESTION

Example 1	
Input	`[('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D', 1)], 'A', 'D'`
Output	`(4, ['A', 'B', 'C', 'D'])`
Why	`A -> B -> C -> D costs 1 + 2 + 1 = 4; the direct edge A -> C costs 4, which is longer overall.`

Example 2	
Input	`[('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'`
Output	`null`
Why	`There is no edge leading from D back to A.`

MAIN CODE | helper functions first, then the answer

```python
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def make_graph(edges):
    # node -> list of (neighbour, weight); an edge is (u, v) or (u, v, weight)
    graph = {}
    for edge in edges:
        u, v = edge[0], edge[1]
        weight = 1
        if len(edge) == 3:
            weight = edge[2]
        if u not in graph:
            graph[u] = []
        graph[u].append((v, weight))
    return graph


def walk_back(parent, node):
    # follow the parent links back to the start, then reverse
    path = []
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path


# ---- main question: main function ----
# Idea: keep the best known distance to every node and a min-heap of
# (distance, node). Take the closest unsettled node, relax its edges
# (try to improve each neighbour), and remember who improved whom.
# Works for weights that are not negative. Returns (distance, path).
# Complexity: Time O((V + E) log V), Space O(V + E)
def dijkstra_path(edges, s, t):
    graph = make_graph(edges)
    dist = {s: 0}
    parent = {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
```

```
            continue                        # an outdated heap entry
        if u == t:
            return d, walk_back(parent, u)  # the first time t comes out of the heap it is final
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (d + w, v))
    return None
```

COMPLEXITY | Time O((V + E) log V) | Space O(V)

50 Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat. MEDIUM

Template fit: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)

QUESTION

A grid contains a rat R, a piece of bread B, a cat C, walls

and open cells '.'. Move

the rat from R to B through open cells (up, down, left, right). The safety of a path is the smallest distance to the cat of any cell on it (distances count steps around the walls). Return the largest safety you can achieve, or -1 if the bread cannot be reached.

Function signature

def safest_path(grid: list) -> int

EXAMPLES

Example 1	
Input	`['R..', '...', 'C.B']`
Output	`2`
Why	`The path along the top and right side keeps at least 2 steps from the cat at the bottom left.`

Example 2	
Input	`['R#B']`
Output	`-1`
Why	`The wall blocks the only route to the bread.`

MAIN CODE | helper functions first, then the answer

```
from collections import deque
import heapq


INF = 10 ** 9


# ---- main question: helper functions ----
def open_neighbors(r, c, grid):
    # the cells beside (r, c) that exist and are not an obstacle ("#")
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                result.append((nr, nc))
    return result
```

```python
def find(grid, letter):
    # where is the letter ("R" rat, "B" bread, "C" cat)?
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == letter:
                return (r, c)
    return None


def cat_distances(grid):
    # one BFS from the cat gives every cell's distance to the cat
    dist = []
    for row in grid:
        dist.append([INF] * len(row))
    cat = find(grid, "C")
    if cat is None:
        return dist
    dist[cat[0]][cat[1]] = 0
    queue = deque([cat])
    while queue:
        r, c = queue.popleft()
        for nr, nc in open_neighbors(r, c, grid):
            if dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist
```

```python
# ---- main question: main function ----
# Idea: a path is only as safe as its cell closest to the cat. Always
# extend the safest frontier cell (like Dijkstra, but with "max of the
# minimum" instead of "min of the sum"; the heap is a max-heap, so we
# push negative values). Returns that safety, or None if no path exists.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def safest_path(grid):
    dist = cat_distances(grid)
    start = find(grid, "R")
    goal = find(grid, "B")
    best = {start: dist[start[0]][start[1]]}
    heap = [(-best[start], start)]
    while heap:
        negative, cell = heapq.heappop(heap)
        safety = -negative
        if cell == goal:
            return safety
        if safety < best[cell]:
            continue                                    # an outdated entry
        for nr, nc in open_neighbors(cell[0], cell[1], grid):
            new_safety = safety
            if dist[nr][nc] < new_safety:
                new_safety = dist[nr][nc]               # the path is only as safe as its closest cell
            if new_safety > best.get((nr, nc), -1):
                best[(nr, nc)] = new_safety
                heapq.heappush(heap, (-new_safety, (nr, nc)))
    return None
```

COMPLEXITY | Time O(m * n log(m * n)) | Space O(m * n)

51 Minimize maximum height along a grid path

HARD

Template fit: the same minimax loop on a height grid

QUESTION

You are given a grid of heights and walk from the top-left cell to the bottom-right cell, moving up, down, left or right. The cost of a path is the largest height of any cell on it. Return the smallest possible cost (and, with with_path=True, a path that achieves it).

Function signature

def min_max_path(H: list, with_path=False)

EXAMPLES

Example 1	
Input	`[[1, 9], [2, 3]]`
Output	`3`
Why	`Down then right passes cells 1, 2, 3: the largest is 3; going right first would have to cross the 9.`

Example 2	
Input	`[[1, 2], [4, 3]], with_path = true`
Output	`(3, [(0, 0), (0, 1), (1, 1)])`
Why	`With with_path=True you also get the route.`

MAIN CODE | helper functions first, then the answer

```
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def reachable(H, limit):
    # can we go top-left to bottom-right using only cells of height <= limit?
    rows, cols = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen = {(0, 0)}
    stack = [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (rows - 1, cols - 1):
            return True
        for nr, nc in neighbors(r, c, rows, cols):
            if (nr, nc) not in seen and H[nr][nc] <= limit:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return False
```

```python
# ---- main question: main function ----
# Idea: Dijkstra where a path costs its highest cell. Heap of (highest
# cell so far, cell): take the smallest, extend to each neighbour with
# cost max(current, neighbour), keep only improvements.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_max_path(H, with_path=False):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    parent = {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            path.reverse()
            return cost, path
        for nr, nc in neighbors(r, c, rows, cols):
            new_cost = cost
            if H[nr][nc] > new_cost:
                new_cost = H[nr][nc]
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                parent[(nr, nc)] = (r, c)
                heapq.heappush(heap, (new_cost, nr, nc))
    return None


# the same answer by binary search over the sorted heights
# Complexity: Time O(m * n log(m * n)) (a search for each of about log(m * n) heights), Space O(m * n)
def min_max_path_bsearch(H):
    seen = set()
    for row in H:
        for h in row:
            seen.add(h)
    heights = sorted(seen)
    low, high = 0, len(heights) - 1
    while low < high:
        mid = (low + high) // 2
        if reachable(H, heights[mid]):
            high = mid                  # this height is enough: try lower
        else:
            low = mid + 1               # not enough: go higher
    return heights[low]
```

COMPLEXITY | Dijkstra with cost = maximum cell on the path: Time O(m * n log(m * n)) | Space O(m * n)
Alternative: binary search the limit with a flood fill: Time O(m * n log(m * n)) | Space O(m * n)

52 Find minimum threshold enabling grid path

HARD

Template fit: the same minimax loop: the smallest threshold that lets you walk from corner to corner

QUESTION

You are given a grid of heights. Find the smallest threshold T such that you can walk from the top-left cell to the bottom-right cell, moving up, down, left or right, using only cells of height at most T. Return T together with one path that achieves it.

Function signature

def min_max_path(H: list) -> (int, list)

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 2]]`
Output	`(2, [(0, 0), (1, 0), (1, 1)])`
Why	`Going down then right uses cells 1, 2, 2: the largest is 2, and no path stays below that.`

Example 2	
Input	`[[7]]`
Output	`(7, [(0, 0)])`
Why	`A single cell: the threshold is its own height.`

MAIN CODE | helper functions first, then the answer

```
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def bottleneck_path(H, use_difference):
    # Dijkstra where a path costs the MAX of its steps (not their sum). A
    # step costs the height of the cell entered, or (use_difference) the
    # height difference between the two cells. Returns (cost, path).
    rows, cols = len(H), len(H[0])
    start_cost = 0
    if not use_difference:
        start_cost = H[0][0]
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue                                    # an outdated entry
        if (r, c) == (rows - 1, cols - 1):
            path = [(r, c)]
            while (r, c) in parent:
```

```
                r, c = parent[(r, c)]
                path.append((r, c))
            path.reverse()
            return cost, path
        for nr, nc in neighbors(r, c, rows, cols):
            if use_difference:
                step = H[nr][nc] - H[r][c]
                if step < 0:
                    step = -step
            else:
```

```
                step = H[nr][nc]
            new_cost = cost
            if step > new_cost:
                new_cost = step
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                parent[(nr, nc)] = (r, c)
                heapq.heappush(heap, (new_cost, nr, nc))
    return None, []


# ---- main question: main function ----
# Idea: Dijkstra with cost = the highest cell on the path. The first
# time a cell leaves the heap its cost is final. (Water rising over
# time is the same problem: a cell is usable once the water reaches
# its height.) Returns (threshold, path).
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_max_path(H):
    return bottleneck_path(H, False)
```

COMPLEXITY | Dijkstra with cost = maximum cell on the path: Time O(m * n log(m * n)) | Space O(m * n)
Alternative: binary search the limit with a flood fill: Time O(m * n log(m * n)) | Space O(m * n)

53 Minimize travel time with optional meeting point — HARD

Template fit: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd

QUESTION

A weighted graph has n nodes and edges (u, v, w) (undirected unless directed=True). Two people start at nodes a and b and both must end at dest. They may meet at some node m and walk the rest of the way together, so the total time is max(time a to m, time b to m) + time m to dest. Return [the minimum total time, the meeting node] (or [-1, -1] when dest cannot be reached). Meeting at dest itself, or at a or b, is allowed.

Function signature

def meeting_point(n: int, edges: list, a: int, b: int, dest: int, directed=False) -> list

EXAMPLES

Example 1	
Input	`5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4`
Output	`[7, 1]`
Why	`They meet at node 1: a needs 2, b needs 1, then 5 more to node 4: 2 + 5 = 7. Meeting at node 4 itself also takes 7, and the smaller node number is reported.`

Example 2	
Input	`3, [(0, 1, 1)], 0, 1, 2`
Output	`[-1, -1]`
Why	`Node 2 is not connected to anything.`

MAIN CODE | helper functions first, then the answer

```python
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def dijkstra(graph, source):
    # shortest distance from source to every node (INF if unreachable)
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # the forward graph, and the reversed graph (used to get distances TO the destination)
    forward = []
    reverse = []
    for i in range(n):
        forward.append([])
        reverse.append([])
    for u, v, w in edges:
        forward[u].append((v, w))
        reverse[v].append((u, w))
        if not directed:
            forward[v].append((u, w))
            reverse[u].append((v, w))
    return forward, reverse


# ---- main question: main function ----
# Idea: both travellers start together, meet at some node m (the one
# who arrives first waits), then travel together to dest. Run Dijkstra
# from A, from B, and from dest on the reversed graph (distance from
# every node TO dest). The best meeting node minimises
# max(distA, distB) + distance to dest. Meeting AT dest is the same as
```

```python
# "no meeting". Returns [total time, meeting node] or [-1, -1].
# Complexity: Time O((V + E) log V) (three Dijkstra runs), Space O(V + E)
def meeting_point(n, edges, a, b, dest, directed=False):
    forward, reverse = build(n, edges, directed)
    dist_a = dijkstra(forward, a)
    dist_b = dijkstra(forward, b)
    to_dest = dijkstra(reverse, dest)
    best_total = INF
    best_node = -1
    for m in range(n):
        wait_for_both = dist_a[m]
        if dist_b[m] > wait_for_both:
            wait_for_both = dist_b[m]
        total = wait_for_both + to_dest[m]
        if total < best_total:
            best_total = total
            best_node = m
    if best_total == INF:
        return [-1, -1]
    return [best_total, best_node]
```

COMPLEXITY | Time O((V + E) log V) | Space O(V + E)

54 Compute minimax grid path and network delay — HARD

Template fit: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

QUESTION

Two tasks. (1) minimax_path(H): given a grid of heights, find a path from the top-left to the bottom-right cell (moving up, down, left or right) whose largest height is as small as possible, and return that largest height. (2) network_delay(n, edges, source, node_delay=None): a signal starts at source and travels along directed weighted edges (u, v, w); return the time until every node has received it, or -1 if some node never does. node_delay[u], when given, is a pause at node u before it forwards the signal.

Function signature

def minimax_path(H: list) -> int

def network_delay(n: int, edges: list, source: int, node_delay=None) -> int

EXAMPLES

Example 1	
Input	`[[1, 5, 1], [2, 2, 5]]`
Output	`5`
Why	`Both ways around pass a cell of height 5, so the answer is 5.`

Example 2	
Input	`4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0`
Output	`10`
Why	`The signal reaches node 1 at 2, node 2 at 5, node 3 at 10: everyone has it after 10.`

Example 3	
Input	`3, [(0, 1, 1)], 0`
Output	`-1`
Why	`Node 2 has no incoming edge: it is never reached.`

Example 4	
Input	`4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0, node_delay = [1, 0, 0, 0]`
Output	`11`
Why	`With a pause of 1 at node 0, every arrival is 1 later: node 3 is reached at 11.`

MAIN CODE | helper functions first, then the answer

```python
import heapq


INF = float("inf")


# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result
```

```python
# ---- main question: main function ----
# PART 1: walk from the top-left to the bottom-right so that the
# HIGHEST cell on the path is as low as possible.
# Idea: Dijkstra where the cost of a path is its maximum (not a sum).
# Heap of (highest cell so far, cell); pop the smallest.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def minimax_path(H):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            return cost
        for nr, nc in neighbors(r, c, rows, cols):
            new_cost = cost
            if H[nr][nc] > new_cost:
                new_cost = H[nr][nc]                  # the cost of a path is its maximum, never a sum
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, nr, nc))
    return -1


# PART 2: when does a signal sent from `source` reach every node?
# Plain Dijkstra; -1 if some node is never reached. `node_delay` is
```

```python
# an optional processing delay paid when leaving each node (follow-up 3).
# Complexity: Time O((V + E) log V), Space O(V + E)
def network_delay(n, edges, source, node_delay=None):
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        pause = 0
        if node_delay:
            pause = node_delay[u]
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
    worst = 0
    for d in dist:
        if d > worst:
            worst = d
    if worst == INF:
        return -1
    return worst
```

COMPLEXITY | Part 1: Dijkstra where the cost of a path is its maximum cell: Time O(m * n log(m * n)) | Space O(m * n)
Part 2: Dijkstra from the source, answer is the largest distance: Time O((V + E) log V) | Space O(V + E)

Backtracking: choose, recurse, undo

55 Mahjong Winning Hand Detection (14 Tiles) — MEDIUM

Template fit: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 31]`
Output	`true`
Why	`Tiles are numbers: 1-9 one suit, 11-19 a second suit, 21-29 a third, 31 and up honours. Pair 31-31, triplets 1-1-1 and 21-21-21, and sequences 2-3-4 and 11-12-13.`

Example 2	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 32]`
Output	`false`
Why	`31 and 32 are different honours, so there is no pair to finish the hand.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# A tile is a number: suit * 10 + rank for the numbered suits (suits 0 to
# 2, ranks 1 to 9), and 30 or more for honour tiles, which never form
# runs. A winning hand is ONE pair plus `melds` melds (4 melds: 14 tiles).
# A meld is a triplet (three equal tiles) or a run (three consecutive
# ranks of one suit).
def is_run_start(tile, count):
    if tile >= 30 or tile % 10 > 7:
        return False                                # honours never run; a run cannot start at rank 8 or 9
    return count.get(tile + 1, 0) > 0 and count.get(tile + 2, 0) > 0


def smallest_tile(count):
    smallest = None
    for tile in count:
        if count[tile] > 0 and (smallest is None or tile < smallest):
            smallest = tile
    return smallest
```

```python
def melds_left(count, need):
    # can the tiles in `count` (tile -> how many) form exactly `need` melds?
    tile = smallest_tile(count)
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3                            # undo
        if ok:
            return True
    # ... or the start of a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1                           # undo
        return ok
    return False
```

```python
# ---- main question: main function ----
# Idea: count the tiles. Try every tile value that has at least two tiles
# as the PAIR; then the rest must form `melds` melds: always take the
# SMALLEST remaining tile and try the (at most two) ways to use it.
# melds=2 is the smaller variant with a pair and two melds, 8 tiles
# (follow-up 1).
# Complexity: Time O(1) for a fixed hand size, Space O(1)
def is_winning_hand(hand, melds=4):
    if len(hand) != 2 + 3 * melds:
        return False                                # one pair plus the melds must use every tile
    count = {}
    for tile in hand:
        count[tile] = count.get(tile, 0) + 1
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False
```

COMPLEXITY | Time O(1) for 14 tiles (about 2^4 branches per pair) | Space O(distinct tiles)

56 Count Prefix-Matching Simple Paths in a Labeled Graph — MEDIUM

Template fit: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth

QUESTION

A directed graph has labelled vertices (labels[i] is the label of vertex i) and edges (u, v). A simple path never visits a vertex twice. For a query word, count for every prefix length L (1 up to the length of the word) the number of simple paths with L vertices whose labels spell the first L letters of the word. Return the list of counts.

Function signature

def prefix_paths(labels: list, edges: list, query: str) -> list

EXAMPLES

QUESTION

Example 1	
Input	`['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'`
Output	`[2, 2, 2]`
Why	Two paths spell 'a' (vertices 0 and 3), and both continue to b (vertex 1) and then c (vertex 2).

Example 2	
Input	`['a', 'a'], [(0, 1), (1, 0)], 'aaa'`
Output	`[2, 2, 0]`
Why	Two paths spell 'a' and two spell 'aa' (0 -> 1 and 1 -> 0); none spells 'aaa' because that would reuse a vertex.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Every vertex has a label (a letter). A SIMPLE path never visits a vertex
# twice. counts[d] is the number of simple paths whose first d + 1
# labels spell the first d + 1 letters of the query.
# Idea: DFS with pruning. Start from every vertex whose label is the first
# query letter. From a vertex go only to an UNVISITED neighbour whose
# label is the NEXT query letter; add one to the count for the current
# length at each step, and undo the visit when coming back.
# `undirected=True` lets every edge be used in both directions (follow-up 1).
# Complexity: Time exponential in the query length in the worst case (every simple path is walked), Space O(L
) for the path
def prefix_paths(labels, edges, query, undirected=False):
    adj = {}
    for v in range(len(labels)):
        adj[v] = []
    for u, v in edges:
        adj[u].append(v)
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)
    visited = set()

    def go(u, depth):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            if w in visited:
                continue                            # a simple path never revisits a vertex
            if labels[w] == query[depth + 1]:
                visited.add(w)
                go(w, depth + 1)
                visited.remove(w)                   # undo

    for start in range(len(labels)):
        if len(query) > 0 and labels[start] == query[0]:
            visited.add(start)
            go(start, 0)
            visited.remove(start)
    return counts
```

COMPLEXITY | Time O(number of matching paths * degree) | Space O(path)

57 Generate Parentheses

MEDIUM

Template fit: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also

EXAMPLES

Example 1

Input	`3`
Output	`['((()))', '(()())', '(())()', '()(())', '()()()']`
Why	`Five well-formed combinations of three pairs.`

Example 2

Input	`1`
Output	`['()']`
Why	`One pair.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea (backtracking): build the string one character at a time. Add "(" as
# long as fewer than n have been placed; add ")" only while the number
# of ")" is smaller than the number of "(" (so it closes something). That
# rule keeps every string valid. At length 2n the string is complete.
# Complexity: Time O(4^n / sqrt(n)) (the Catalan number of strings, each O(n) long), Space O(n) working plus
the answer
def generate(n):
    result = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return result
```

COMPLEXITY | Time O(4^n / sqrt(n)) | Space O(n) beyond the output

58 American Football Score Combinations

MEDIUM

Template fit: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0

QUESTION

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: 2 (safety), 3 (field goal), 6 (touchdown), 7 (touchdown + extra point), or 8 (touchdown + 2-point conversion). Given a target total , return every distinct combination of scoring plays whose values sum to total . Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

Function signature

def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted ascending."""
    ...

Follow-up (as reported): If you only need the count of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

EXAMPLES

Example 1	
Input	`total = 6`
Output	`[[2, 2, 2], [3, 3], [6]]`
Why	`2+2+2 = 3+3 = 6 = 6.`

Example 2	
Input	`total = 7`
Output	`[[2, 2, 3], [7]]`
Why	`2+2+3 = 7 and the single play 7.`

Example 3	
Input	`total = 0`
Output	`[[]]`
Why	`The empty combination is the only way to score 0.`

CONSTRAINTS

- 0 <= total <= 60
- Allowed play values are exactly [2, 3, 6, 7, 8] .
- If total == 0 , the only combination is the empty one, so return [[]] .

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# A team can score 2, 3, 6, 7 or 8 points at a time. List every
# combination of plays that adds up to `total`.
# Idea (backtracking): choose the plays in NON-DECREASING order, so every
# combination is listed exactly once (2 + 3 and 3 + 2 are the same
# combination). `start` is the index of the smallest play we may still use;
# the same play may be used again, so the index stays the same after a
# choice. Undo the choice (pop) after the recursive call.
# Complexity: Time O(number of combinations * length), Space O(total / 2) for the path plus the answer
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    result = []
```

```
    def dfs(start, left, path):
        if left == 0:
            result.append(path[:])
            return
        for i in range(start, len(plays)):
            if plays[i] <= left:
                path.append(plays[i])
                dfs(i, left - plays[i], path)
                path.pop()

    dfs(0, total, [])
    return result
```

COMPLEXITY | Time O(number of combinations * length) | Space O(total) recursion

59 Permutations — MEDIUM

Template fit: the template as written

QUESTION

Given an array of distinct integers, return all possible permutations of the array (every rearrangement of its elements), in any order.

Function signature

def permute(nums: list) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3]`
Output	`[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]`
Why	Six orders of three numbers.

Example 2	
Input	`[0, 1]`
Output	`[[0, 1], [1, 0]]`
Why	Two numbers have two orders.

Example 3	
Input	`[1]`
Output	`[[1]]`
Why	One number has a single permutation.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea (backtracking): fill the positions from left to right. At each
# position try every value that is not used yet: choose it (mark it used,
# put it in the current list), recurse for the next position, then UNDO
# the choice (remove it, unmark it). Only valid prefixes are ever
# extended, so every complete list is a permutation.
# Complexity: Time O(n * n!), Space O(n * n!) for the answer (O(n) working space)
def permute(nums):
    result = []
    current = []
    used = [False] * len(nums)
```

```
    def go():
        if len(current) == len(nums):
            result.append(current[:])               # a copy, because `current` keeps changing
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                current.append(nums[i])
                go()
                current.pop()
                used[i] = False

    go()
    return result
```

COMPLEXITY | Time O(n * n!) | Space O(n) beyond the output

60 Determine if a 14-tile hand is winning

MEDIUM

Template fit: the same try-triple-then-run recursion over a count array, after choosing the pair

QUESTION

A hand is 14 tiles with values 1 to 9 of one suit (you may reorder them). It wins if it splits into one pair (two identical tiles) and four melds of three tiles, where a meld is either a triplet (three identical tiles) or a sequence (three consecutive values). Return whether the hand is winning.

Function signature

def is_winning_hand(tiles: list) -> bool

EXAMPLES

Example 1	
Input	[1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
Output	true
Why	Pair 9-9, triplets 1-1-1 and 5-5-5, sequences 2-3-4 and 6-7-8.

Example 2	
Input	[1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]
Output	false
Why	These tiles cannot be split into a pair and four melds.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# A hand is made of tiles with values 1 to 9 in one suit. It WINS when its
# 14 tiles split into ONE pair (two equal tiles) and FOUR melds. A meld is
# a triplet (three equal tiles) or a run (three consecutive values).
# Complexity: Time O(n), Space O(1)
def count_tiles(tiles):
    # counts[v] = how many tiles with value v (index 0 is unused)
    counts = [0] * 12
    for tile in tiles:
        counts[tile] += 1
    return counts


def melds_ok(counts, v=1):
    # can the tiles in counts be split into melds only?
    while v <= 9 and counts[v] == 0:
        v += 1                                      # the smallest value that still has tiles
```

```
    if v > 9:
        return True                                  # no tiles left
    # The smallest tile must be in a triplet or at the START of a run
    # (nothing smaller is left to take it), so only these two choices exist.
    if counts[v] >= 3:
        counts[v] -= 3
        ok = melds_ok(counts, v)
        counts[v] += 3                               # undo
        if ok:
            return True
    if v <= 7 and counts[v + 1] > 0 and counts[v + 2] > 0:
        counts[v] -= 1
        counts[v + 1] -= 1
        counts[v + 2] -= 1
        ok = melds_ok(counts, v)
        counts[v] += 1
        counts[v + 1] += 1
        counts[v + 2] += 1                           # undo
        return ok
    return False


def is_winning_counts(counts):
    # try every value that has two tiles as the PAIR; the rest must be melds
    for pair in range(1, 10):
        if counts[pair] >= 2:
            counts[pair] -= 2
            ok = melds_ok(counts)
```

```
            counts[pair] += 2
            if ok:
                return True
    return False


# ---- main question: main function ----
# Complexity: Time O(1) for a fixed hand size (at most 9 pairs, each followed by a search with 2 choices per
step), Space O(1)
def is_winning_hand(tiles):
    return is_winning_counts(count_tiles(tiles))
```

COMPLEXITY | Time O(9 * small branching) = constant for 14 tiles | Space O(1)

61 Count Passcode Patterns on a Keypad

MEDIUM

Template fit: the choice is the next adjacent unused key; complete at k keys; count instead of storing

QUESTION

A keypad is a rows x cols grid of keys (the classic phone dial is 3x3):
1 2 3 4 5 6 7 8 9 A passcode pattern is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are different patterns. Given the grid dimensions and an integer k , return the number of distinct passcode patterns that use exactly k keys.

Function signature

def count_patterns(rows: int, cols: int, k: int) -> int:
    """Return the number of distinct passcode patterns using exactly k keys,
    moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""
    ...

Follow-up (as reported): Count patterns of length at least k instead of exactly k . Hint: sum the DFS counts over all lengths from k to rows*cols .

QUESTION

EXAMPLES

Example 1	
Input	rows = 3, cols = 3, k = 1
Output	9
Why	Each of the 9 keys alone is one pattern.

Example 2	
Input	rows = 3, cols = 3, k = 2
Output	24
Why	There are 12 adjacent key pairs; each can be traversed in 2 directions, giving 24 patterns.

Example 3	
Input	rows = 3, cols = 3, k = 3
Output	44
Why	Summing over each possible middle key b, deg(b)*(deg(b)-1) directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.

CONSTRAINTS

- 1 <= rows, cols <= 4 .
- 1 <= k <= rows * cols .
- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def neighbours(r, c, rows, cols):
    # the keys right next to key (r, c) on the keypad
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr = r + dr
        nc = c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Count the patterns of exactly k keys: start on any key and keep moving to
# an ADJACENT key that is not used yet.
# Idea (DFS with backtracking): from the current key try every unused
# neighbour: mark it used, recurse with one more key, then UNMARK it.
# A pattern is complete when it has k keys.
# Complexity: Time O(rows * cols * 4^k) in the worst case, Space O(k)
def count_patterns(rows, cols, k):
    def dfs(r, c, used, seen):
        if used == k:
            return 1
        total = 0
        for nr, nc in neighbours(r, c, rows, cols):
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))                # undo
        return total
```

```
    total = 0
    for r in range(rows):
        for c in range(cols):
            total += dfs(r, c, 1, {(r, c)})
    return total
```

COMPLEXITY | Time O(number of simple paths) | Space O(k)

62 Match people to questions using tags and priorities — MEDIUM

Template fit: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure

QUESTION

A set of people and a set of interview questions: adj[p] lists the questions person p is allowed to answer (their tags match). Each person is given at most one question and each question goes to at most one person. Return the largest number of people that can be matched.

Function signature

def max_matching(adj: list) -> int

EXAMPLES

Example 1	
Input	`[[0, 1], [0], [1, 2]]`
Output	`3`
Why	`Person 0 takes question 0, person 1 has only question 0 so person 0 moves to question 1, person 2 takes question 2: all three are matched.`

Example 2	
Input	`[[0], [0], [0]]`
Output	`1`
Why	`Three people can only answer the same single question: just one match.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def build_graph(people, questions):
    # a person can take a question when they share at least one tag.
    # people: [(name, priority, tags)]   questions: [(id, tags, capacity)]
    adj = {}
    for name, priority, tags in people:
        adj[name] = []
        for question_id, question_tags, capacity in questions:
            if len(set(tags) & set(question_tags)) > 0:
                adj[name].append(question_id)
    return adj


# ---- main question: main function ----
# Idea (Kuhn's augmenting paths): adj[p] is the list of questions person p
# accepts. For each person run a DFS over the questions they accept: if a
# question is free, take it; if it belongs to someone else, try to move
# that owner to ANOTHER question (a recursive try). A `seen` set makes sure
# no question is tried twice in one attempt. The number of people who
# found a question is the size of the maximum matching.
# Complexity: Time O(V * E) (one search per person), Space O(V + E)
def max_matching(adj):
    owner = {}                                      # question -> the person who holds it

    def try_assign(person, seen):
```

```
        for question in adj[person]:
            if question in seen:
                continue
            seen.add(question)
            if question not in owner or try_assign(owner[question], seen):
                owner[question] = person
                return True
        return False

    matched = 0
    for person in range(len(adj)):
        if try_assign(person, set()):
            matched += 1
    return matched
```

COMPLEXITY | Time O(P * E) | Space O(P + Q)

63 Matrix Flower Placement with House Adjacency — HARD

Template fit: place row by row; reject a row early when the houses two rows above are still not next to a flower

QUESTION

A grid has houses (H) and empty cells (0). Place flowers (F) on empty cells so that every row and every column holds at most one flower (exactly one of each when exactly_one is True) and every house has a flower directly beside it (up, down, left or right). Return one valid grid as a list of strings, or None if there is none.

Function signature

def place_flowers(grid: list, exactly_one=False) -> list | None

EXAMPLES

Example 1	
Input	['H000', 'H000', '000H', '0000']
Output	['HF00', 'H0F0', 'F00H', '000F']
Why	Flowers at (0,1), (2,0) and (1,3): one per row and column, and each house touches a flower.

Example 2	
Input	['H000', 'H000', '000H', '0000'], exactly_one = true
Output	['HF00', 'H0F0', 'F00H', '000F']
Why	Exactly one flower in every row and column: a fourth flower at (3,2) completes it.

Example 3	
Input	['H', 'H']
Output	null
Why	The two houses can only be served by the same column, so no valid placement exists.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# ---- main question: main function ----
# The grid has houses "H" and empty cells "0". Put at most one flower "F"
# in every row and every column, only on empty cells, so that every house
# has a flower right next to it (above, below, left or right).
# Idea: backtracking row by row. For each row either put a flower in a
```

```python
# free column (remember the used columns) or leave the row empty; undo the
# choice afterwards. Houses are checked LATE: when we start row r, the
# rows before r are decided, so the houses in row r - 2 can no longer
# get a new neighbour and must already be satisfied. That prunes early.
# exactly_one=True: a flower in EVERY row and EVERY column (follow-up 3);
# find_all=True: return every solution (follow-up 4).
# Complexity: Time O(C^R) in the worst case (a flower in some free column or none, for every row), Space O(R
* C)
def place_flowers(grid, exactly_one=False, find_all=False):
    rows = len(grid)
    cols = len(grid[0])
    cells = []
    for row in grid:
        cells.append(list(row))
    houses = []
    for r in range(rows):
        for c in range(cols):
            if cells[r][c] == "H":
                houses.append((r, c))
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a = r + dr
            b = c + dc
            if 0 <= a < rows and 0 <= b < cols and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        for hr, hc in houses:
            if hr == r and not house_ok(hr, hc):
                return False
        return True

    def solve(r, used):
```

```python
        if len(found) > 0 and not find_all:
            return                                  # one solution is enough
        if r >= 2 and not row_houses_ok(r - 2):
            return                                  # a house in row r - 2 can no longer be served
        if r == rows:
            full = len(used) == cols
            if row_houses_ok(rows - 1) and (full or not exactly_one):
                found.append(["".join(line) for line in cells])
            return
        for c in range(cols):                       # try a flower in every free column of this row
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                used.add(c)
                solve(r + 1, used)
                used.remove(c)                      # undo
                cells[r][c] = "0"
        if not exactly_one:
            solve(r + 1, used)                      # or leave the row empty

    solve(0, set())
    if find_all:
        return found
    if len(found) > 0:
        return found[0]
    return None
```

COMPLEXITY | Time exponential worst case, heavily pruned | Space O(rows + cols)

Runs and groups: extend while it fits, restart or close when it does not

64 Group Timestamps Into Close Intervals — EASY

Template fit: close the group when the gap to the next timestamp is at least k; otherwise extend it

QUESTION

You are given an array of positive integer timestamps in ascending order and an integer k . Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than k . A run breaks wherever an adjacent difference is >= k . Return each run as an interval [start, end] (its first and last value), in order.

Function signature

def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:

    ...

Follow-up (as reported): One pass, O(n) time and O(1) extra space (besides the output).

EXAMPLES

Example 1	
Input	`timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2`
Output	`[[1,4],[11,11],[14,15],[17,17],[21,23]]`
Why	`Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15- >17, 17->21) starts a new run.`

Example 2	
Input	`timestamps = [5], k = 100`
Output	`[[5,5]]`
Why	`A single timestamp forms a run by itself.`

Example 3	
Input	`timestamps = [1,2,3], k = 1`
Output	`[[1,1],[2,2],[3,3]]`
Why	`A diff of 1 is not strictly less than k=1, so every element stands alone.`

CONSTRAINTS

- 1 <= len(timestamps) <= 10^5 , values strictly increasing.
- k >= 1 .

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: one pass. Remember where the current run started; whenever the
# gap to the previous timestamp reaches k, close the run [start, previous]
# and start a new one at this timestamp.
# Complexity: Time O(n), Space O(n) for the output
def group_into_intervals(timestamps, k):
    if not timestamps:
        return []
    runs = []
    start = timestamps[0]
    for i in range(1, len(timestamps)):
        previous = timestamps[i - 1]
        current = timestamps[i]
```

```
        if current - previous >= k:
            runs.append([start, previous])
            start = current
    runs.append([start, timestamps[len(timestamps) - 1]])
    return runs
```

COMPLEXITY | Time O(n) | Space O(1) extra

65 First Out-of-Order Character

EASY

Template fit: fits = s[i] >= s[i-1]; return i at the first item that does not fit

QUESTION

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed). Return the index of the FIRST character that breaks this rule — that is, the first index i >= 1 where s[i] < s[i-1] . If the entire string is non-decreasing, return -1 .

Function signature

def first_out_of_order(s: str) -> int:

...

Follow-up (as reported): Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

EXAMPLES

Example 1	
Input	"abcda"
Output	4
Why	a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.

Example 2	
Input	"aabbz"
Output	-1
Why	Every character is >= the previous one; fully non-decreasing.

Example 3	
Input	"cba"
Output	1
Why	At index 1, 'b' < 'c', the first violation.

Example 4	
Input	""
Output	-1
Why	An empty string is trivially in order.

CONSTRAINTS

- 0 <= len(s) <= 10^5 s consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order -> return -1 .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: one pass, comparing each letter with the one before it. The
```

```python
# order is broken exactly where a letter is SMALLER than the previous
# letter; return that index, or -1 if the string is in order.
# Complexity: Time O(n), Space O(1)
def first_out_of_order(s):
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            return i
    return -1
```

COMPLEXITY | Time O(n) | Space O(1)

66 Longest Non-Decreasing Subarray with One Replacement — MEDIUM

Template fit: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of length 5.
```

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def in_order(x, y, strict):
    # may y follow x? non-decreasing: x <= y; strictly increasing: x < y
    if strict:
        return x < y
    return x <= y


def run_lengths(a, strict):
    # left[i] = length of the run that ENDS at i, right[i] = the run that STARTS at i
    n = len(a)
    left = [1] * n
    right = [1] * n
    for i in range(1, n):
        if in_order(a[i - 1], a[i], strict):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if in_order(a[i], a[i + 1], strict):
            right[i] = right[i + 1] + 1
    return left, right


def bigger(x, y):
    if x > y:
```

```
        return x
    return y


# ---- main question: main function ----
# Idea: changing a[i] can join the run on its left and the run on its
# right: left[i-1] + 1 + right[i+1], but only if some value fits between
# the neighbours (a[i-1] <= a[i+1]). Otherwise it extends just the longer
# of the two runs. At the ends there is only one side to join.
# (strict=True is follow-up 1: strictly increasing runs, and then the
# neighbours must differ by at least 2 so that a whole number fits between.)
# Complexity: Time O(n), Space O(n)
def longest_with_one_change(a, strict=False):
    n = len(a)
    if n <= 2:
        return n                                    # nothing to fix
    gap = 0
    if strict:
        gap = 2
    left, right = run_lengths(a, strict)
    best = 1
```

```
    for i in range(n):
        left_run = 0
        if i > 0:
            left_run = left[i - 1]
        right_run = 0
        if i + 1 < n:
            right_run = right[i + 1]
        if left_run == 0 or right_run == 0 or a[i + 1] - a[i - 1] >= gap:
            best = bigger(best, left_run + 1 + right_run)      # the change joins both runs
        else:
            best = bigger(best, bigger(left_run, right_run) + 1)   # it joins only the longer one
    return best


# the warm-up: no replacement allowed
# Complexity: Time O(n), Space O(1)
def longest_without_change(a):
    best = 0
    run = 0
    for i in range(len(a)):
        if i > 0 and a[i - 1] < a[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best
```

COMPLEXITY | Time O(n) | Space O(n)

67 Piano Hand Movement Grouping — MEDIUM

Template fit: start a new group when adding the note would make max - min exceed k

QUESTION

A pianist plays a sequence of notes in order (a bigger number is a higher key). The hand can stay in one position while every note it plays lies within a span of k (highest minus lowest at most k). Starting a new hand position costs a move. Return the smallest number of hand positions needed to play all the notes in order.

Function signature

def groups(notes: list, k: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	[1, 3, 5, 7, 9, 11], 4
Output	2
Why	[1, 3, 5] fits in a span of 4, then [7, 9, 11]: 2 positions.

Example 2	
Input	[1, 2, 3, 4, 5], 4
Output	1
Why	All five notes fit in a span of 4: one position.

Example 3	
Input	[5, 9, 1], 4
Output	2
Why	5 and 9 fit together, then 1 is too far from them: 2 positions.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def hand_positions(notes, k):
    # split the notes into groups that one hand position (a span of at most
    # k) can play, as early as possible. Each group is (start index,
    # lowest note, highest note).
    groups_found = []
    for i in range(len(notes)):
        x = notes[i]
        if groups_found:
            start, low, high = groups_found[-1]
            new_low = low
            if x < new_low:
                new_low = x
            new_high = high
            if x > new_high:
                new_high = x
            if new_high - new_low <= k:                # the note still fits the current span
                groups_found[-1] = (start, new_low, new_high)
                continue
        groups_found.append((i, x, x))                 # otherwise the hand must move: a new group starts
    return groups_found


# ---- main question: main function ----
# Idea: a greedy scan. Keep the lowest and highest note of the current
# group; extend the group while high - low stays within k, otherwise
# close it and start a new group at this note. The answer is the number
# of hand placements.
# Complexity: Time O(n), Space O(g) for g hand positions
def groups(notes, k):
    return len(hand_positions(notes, k))
```

COMPLEXITY | Time O(n) | Space O(1)

68 Find longest increasing contiguous subarray

MEDIUM

Template fit: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs

QUESTION

Given an array you may change at most one element to any value (an integer when integers=True, any real number otherwise). Return the length of the longest strictly increasing contiguous subarray you can obtain.

Function signature

def longest_one_change(a: list, integers: bool = True) -> int

EXAMPLES

Example 1	
Input	[1, 2, 3, 2, 5, 6]
Output	6
Why	Change the second 2 to 4 and the whole array 1, 2, 3, 4, 5, 6 increases.

Example 2	
Input	[1, 3, 2, 4]
Output	3
Why	Between 1 and 2 there is no integer, so the array 1, 3, 2, 4 only improves to a run of 3.

Example 3	
Input	[1, 3, 2, 4], integers = false
Output	4
Why	With real numbers, 3 can become 1.5 and the whole array increases.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea: left[i] is the length of the strictly increasing run that ENDS
# at i, right[i] the length of the one that STARTS at i. Try every
# position as the one changed value: it can join the run on its left, the
# run on its right, or both if a value fits between its neighbours (for
# whole numbers the neighbours must differ by at least 2; for real
# numbers by more than 0).
# Complexity: Time O(n), Space O(n)
def longest_one_change(a, integers=True):
    n = len(a)
    if n == 0:
        return 0
    left = [1] * n
    right = [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    best = 1
    for i in range(n):
```

```
        best = bigger(best, left[i])                    # no change at all
        if i > 0:
            best = bigger(best, left[i - 1] + 1)        # change a[i] to continue the run on its left
        if i < n - 1:
            best = bigger(best, right[i + 1] + 1)       # or to start the run on its right
        if 0 < i < n - 1:                               # or to join both, if a value fits between
            room = a[i + 1] - a[i - 1]
            if integers:
                fits = room >= 2
            else:
                fits = room > 0
            if fits:
                best = bigger(best, left[i - 1] + 1 + right[i + 1])
    if best > n:
        return n
    return best
```

COMPLEXITY | Time O(n) | Space O(n)

69 Sum of Good Subarrays (Adjacent Diff ±1) — MEDIUM

Template fit: keep the running length and running sum of the current up-run and down-run; add both to the total at every index

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.

A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).

Singleton subarrays are always good.

Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).

Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then -1).
```

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# A subarray is GOOD when every step between neighbours is +1, or every
# step is -1 (a single element is good). Return the sum of the elements
# of all good subarrays.
# Idea: go left to right and, for each direction, keep the total of the
# sums of all good subarrays that END at the current element, and how
# many there are. When the next element x continues a +1 run, every old
# subarray grows by x and the new one [x] appears: the total grows by
# (count + 1) * x. A break restarts the total at x. The single [x] is in
# both the "up" and the "down" total, so subtract it once.
# Complexity: Time O(n), Space O(1)
def good_subarray_sum(a):
    total = 0
    up_sum = 0
    up_count = 0
```

```
    down_sum = 0
    down_count = 0
    for i in range(len(a)):
        x = a[i]
        step = None
        if i > 0:
            step = x - a[i - 1]
        if step == 1:
            up_sum += (up_count + 1) * x
            up_count += 1
        else:
            up_sum = x
            up_count = 1
        if step == -1:
            down_sum += (down_count + 1) * x
            down_count += 1
        else:
            down_sum = x
            down_count = 1
        total += up_sum + down_sum - x
    return total
```

COMPLEXITY | Time O(n) | Space O(1)

70 Sustained-Error Time Window

MEDIUM

Template fit: from the given second extend left and right while errors >= baseline; compare the time span with the interval

QUESTION

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: times (timestamps in seconds, strictly increasing) and errors (the error reading recorded at each timestamp), where errors[i] was recorded at times[i] . You are also given three integers: second , interval , and baseline . Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp second (guaranteed to exist exactly once in times), 2. Has every reading in the run satisfy error >= baseline , and 3. Spans a duration of at least interval seconds — that is, times[right] - times[left] >= interval , where left and right are the endpoints of the run. Return True if such a run exists, otherwise False . Note that if the reading at second is itself below baseline , the answer is immediately False .

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes second and spans >=
interval seconds."""
    ...
```

EXAMPLES

Example 1	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 1, interval = 1, baseline = 8`
Output	`True`
Why	`Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8) but stops at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval 1.`

QUESTION

Example 2	
Input	times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 4, interval = 2, baseline = 8
Output	False
Why	Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3 (err 5) blocks going left. The run covers times 4..5, span 1 < interval 2.

Example 3	
Input	times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8
Output	False
Why	The reading at second=2 is 4 < baseline 8, so no valid run includes it.

CONSTRAINTS

- 1 <= len(times) == len(errors) <= 10^5 times is strictly increasing; 0 <= times[i] <= 10^9
- -10^9 <= errors[i] <= 10^9 , baseline in the same range second is one of the values in times ; 0 <= interval <= 10^9

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# The readings are (time, error count) pairs in time order. Is the error
# count AT `second` part of a stretch of readings, all at or above the
# `baseline`, that lasts at least `interval` seconds?
# Idea: the longest stretch around that reading is the best one, so only
# its two ends matter: expand to the left and right while the readings
# stay at or above the baseline.
# Complexity: Time O(n) per query, Space O(1)
def has_sustained_error_window(times, errors, second, interval, baseline):
    at = times.index(second)
    if errors[at] < baseline:
        return False
    left = at
    right = at
    while left > 0 and errors[left - 1] >= baseline:
        left -= 1
    while right + 1 < len(times) and errors[right + 1] >= baseline:
        right += 1
    return times[right] - times[left] >= interval
```

COMPLEXITY | Time O(n) | Space O(1)

71 Longest Non-Decreasing Subarray From Two Arrays — MEDIUM

Template fit: two run lengths per index, one for each array you may take the value from

QUESTION

You are given two integer arrays nums1 and nums2 , both of length n . You must construct a third array nums3 of length n where, for each index i , nums3[i] is chosen to be either nums1[i] or nums2[i] (chosen independently per index). Return the length of the longest non-decreasing subarray (a contiguous run) achievable in nums3 over all valid choices.

Function signature

def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:

    ...

Follow-up (as reported): (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to write test cases.

EXAMPLES

QUESTION

Example 1	
Input	`nums1 = [2, 3, 1], nums2 = [1, 2, 1]`
Output	`2`
Why	`Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2. No choice yields a non-decreasing run of length 3.`

Example 2	
Input	`nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]`
Output	`4`
Why	`nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3]) is non-decreasing over all 4 positions.`

Example 3	
Input	`nums1 = [1, 1], nums2 = [2, 2]`
Output	`2`
Why	`Any choice, e.g. [1, 1], is non-decreasing across both.`

CONSTRAINTS

- 1 <= n <= 10^5
- nums1 and nums2 have the same length n .
- 1 <= nums1[i], nums2[i] <= 10^9
- Target complexity: O(n) time, O(1) extra space.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# At every index you may take the value from nums1 or from nums2. Find
# the longest non-decreasing subarray you can build.
# Idea (two-state DP): dp1 = the best run ending here if we take nums1[i],
# dp2 = the same if we take nums2[i]. Each can extend the run that ended
# at the previous index with either choice, if the order holds.
# Complexity: Time O(n), Space O(1)
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = 1
    dp2 = 1
    best = 1
    for i in range(1, len(nums1)):
        new1 = 1
        new2 = 1
        if nums1[i - 1] <= nums1[i]:
            new1 = bigger(new1, dp1 + 1)
        if nums2[i - 1] <= nums1[i]:
            new1 = bigger(new1, dp2 + 1)
        if nums1[i - 1] <= nums2[i]:
            new2 = bigger(new2, dp1 + 1)
        if nums2[i - 1] <= nums2[i]:
            new2 = bigger(new2, dp2 + 1)
        dp1 = new1
        dp2 = new2
        best = bigger(best, bigger(dp1, dp2))
```

```
    return best
```

COMPLEXITY | Time O(n) | Space O(1)

72 Count Super Streaks — MEDIUM

Template fit: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

QUESTION

A player produces a list of events, each (event_id, event_type, event_time) . Sort events by event_time . A streak is a maximal run of events, consecutive in time order, in which every event has the same event_type AND the gap between each event and the previous one in the run is at most T seconds. A streak ends when the next event has a different event_type or its gap from the previous event exceeds T . A streak is a Super Streak if it contains at least N events AND spans at least X seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Function signature

```
def count_super_streaks(events: list[tuple[int, str, int]],
                        N: int, T: int, X: int) -> int:
    ...
```

Follow-up (as reported): Follow-up 1: events are tagged with user_id ; return a per-user count of Super Streaks. Follow-up 2: after processing, support count_streaks(user_id, time_A, time_B) returning how many of that user's Super Streaks completed entirely within [time_A, time_B] . Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

EXAMPLES

Example 1	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10, X=30`
Output	`1`
Why	`One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4). Super.`

Example 2	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40), (6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30`
Output	`1`
Why	`The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is Super; the second [55,56,57] has only 3 events and span 2, so not Super.`

Example 3	
Input	`events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1`
Output	`0`
Why	`Type changes each step, so every streak has length 1 — none reaches N=2.`

CONSTRAINTS

- 0 <= len(events) <= 10^5 .
- N >= 1 , T >= 0 , X >= 0 ; times are non-negative integers, possibly with duplicate timestamps.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def event_time(event):
    # an event is (id, type, time) or (id, type, time, user)
    return event[2]
```

```python
def find_streaks(events, N, T, X):
    # A STREAK is a run of events of the same type where each event comes at
    # most T after the previous one. It is a SUPER streak if it has at least N
    # events and lasts at least X from its first to its last event.
    # Returns the (start time, end time) of every super streak.
    spans = []
    start = None
    previous = None
    kind = None
    size = 0
    for event in sorted(events, key=event_time):
        event_type = event[1]
        time = event[2]
        if kind == event_type and time - previous <= T:
            size += 1                                   # the open streak continues
        else:
            if size > 0 and size >= N and previous - start >= X:
                spans.append((start, previous))         # the open streak was a super streak
            start = time
            kind = event_type
            size = 1
        previous = time
    if size > 0 and size >= N and previous - start >= X:
        spans.append((start, previous))
    return spans


# ---- main question: main function ----
# Idea: one pass over the events sorted by time. Keep only the open
# streak (when it started, the time of its last event, its size and its
# type); when an event does not continue it, check whether it was a
# super streak and start a new one.
# Complexity: Time O(n log n) for n events (they are sorted by time), Space O(n)
def count_super_streaks(events, N, T, X):
    return len(find_streaks(events, N, T, X))
```

COMPLEXITY | Time O(n log n) | Space O(1) extra

Union-find: merge the pairs, then ask who is together

73 Count Islands in a Grid and Track the Count as Land Cells Are Added

MEDIUM

Template fit: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition

QUESTION

A map is a grid with **`m`** rows and **`n`** columns in which every cell is land (**`1`**) or water (**`0`**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **`grid[r][c]`** is **`1`** for land and **`0`** for water, where **`r`** is the row index and **`c`** is the column index.
- An addition **`(r, c)`** turns cell **`(r, c)`** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **`len(additions) + 1`**. Element **`0`** is the number of islands in the initial grid. For **`i >= 1`**, element **`i`** is the number of islands after the first **`i`** additions have been applied.

Constraints

- **`1 <= m <= 1000`**, **`1 <= n <= 1000`** and **`m * n <= 200000`**
- Every row of **`grid`** has exactly **`n`** entries, each **`0`** or **`1`**.
- **`0 <= len(additions) <= 200000`**
- Every addition **`(r, c)`** satisfies **`0 <= r < m`** and **`0 <= c < n`**. The same cell may appear more than once.
- Every count is at most **`m * n`**, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

QUESTION

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


class Islands:
    # land cells are added one at a time; the island count is kept up to date
    def __init__(self):
        self.parent = {}       # land cell -> its parent cell (union-find)
        self.size = {}         # size of the island, stored at its leader
        self.count = 0
        self.largest = 0       # used by follow-up 3

    def find(self, cell):
        while self.parent[cell] != cell:
            self.parent[cell] = self.parent[self.parent[cell]]
            cell = self.parent[cell]
        return cell

    def add(self, r, c):
        if (r, c) in self.parent:
            return                                  # already land
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1                             # a new island ...
        if self.largest < 1:
            self.largest = 1
        for dr, dc in STEPS:
            neighbour = (r + dr, c + dc)
            if neighbour in self.parent:
                self.union((r, c), neighbour)       # ... that may join others

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        if self.size[a] < self.size[b]:
            a, b = b, a                             # the small island joins the big one
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1                             # two islands became one
        if self.size[a] > self.largest:
            self.largest = self.size[a]


def build_islands(grid):
```

```
    # put every land cell (1) of the starting grid into the structure
    islands = Islands()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1:
                islands.add(r, c)
    return islands


# ---- main question: main function ----
# Idea: build the starting islands with unions, then for every added
# cell: ignore it if it is already land; otherwise it is a new island
# (+1) and every time it is joined to a different island the count
```

```
# goes down by one. Returns the count before any addition and after each one.
# Complexity: Time O(m * n + K) alpha for the starting grid and K additions, Space O(land cells)
def island_counts(grid, additions):
    islands = build_islands(grid)
    counts = [islands.count]
    for r, c in additions:
        islands.add(r, c)
        counts.append(islands.count)
    return counts
```

COMPLEXITY | Time O(m * n + A * alpha(m * n)) | Space O(m * n)

74 2D Points Connected Components by Radius — MEDIUM

Template fit: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Points within distance 2 of each other form a cluster: two near the origin, two near (10, 10), one alone.`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0), (3, 0)], 1`
Output	`1`
Why	`A chain is one cluster even though its ends are far apart.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def near(p, q, r):
    # compare SQUARED distances: no square root, and exact for integer points
    dx = p[0] - q[0]
```

```
    dy = p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# ---- main question: main function ----
# Idea: put each point in the cell (x // r, y // r). Two points within
# distance r are in the same or in neighbouring cells, so only those 9
# cells are compared; union every close pair and count the groups.
# Complexity: Time about O(n) for evenly spread points (9 grid cells per point), O(n^2) worst case; Space O(n
)
def count_clusters(points, r):
    size = r
    if size <= 0:
        size = 1
    cells = {}
    for i in range(len(points)):
        key = (points[i][0] // size, points[i][1] // size)
        if key not in cells:
            cells[key] = []
        cells[key].append(i)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        cx, cy = points[i][0] // size, points[i][1] // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                for j in cells.get((cx + dx, cy + dy), []):
                    if j > i and near(points[i], points[j], r):
                        uf.union(i, j)
```

```
    groups = 0
    for i in range(len(points)):
        if uf.find(i) == i:
            groups += 1
    return groups
```

COMPLEXITY | Time about O(n) when points are spread out (O(n^2) if they all share a cell) | Space O(n)

75 Count Clusters of 2D Points Within a Radius

MEDIUM

Template fit: identical to the line above: cell buckets give the candidate pairs, answer = groups

QUESTION

Given 2D points and a radius r, two points are connected when their distance is at most r. A cluster is a group of points connected directly or through chains of other points. Return the number of clusters.

Function signature

def count_clusters(points: list, r: float) -> int

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Three groups: the two points near the origin, the two near (10, 10), and the lone point at (20, 20).`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0)], 1`
Output	`1`
Why	`A chain: (0,0)-(1,0)-(2,0) are linked neighbour to neighbour, so it is one cluster.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Clusters:
    # points arrive one by one; the cluster count is kept up to date.
    # Two points are in one cluster when a chain of points, each within
    # distance r of the next, joins them.
    def __init__(self, r):
        self.r = r
        self.cells = {}          # grid cell (side r) -> indices of the points inside it
        self.points = []
        self.parent = []         # union-find over the point indices
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx = p[0] - q[0]
        dy = p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r      # squared distances: no square root

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        side = self.r
        if side == 0:
            side = 1
        cx, cy = x // side, y // side
        if (cx, cy) not in self.cells:
            self.cells[(cx, cy)] = []
        self.cells[(cx, cy)].append(i)
        # a point within r can only sit in this cell or one of the 8 around it
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                for j in self.cells.get((cx + dx, cy + dy), []):
                    if j != i and self.near((x, y), self.points[j]):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components
```

```
    def largest(self):
        # size of the biggest cluster
        sizes = {}
        for i in range(len(self.points)):
            root = self.find(i)
            sizes[root] = sizes.get(root, 0) + 1
        biggest = 0
        for size in sizes.values():
            if size > biggest:
                biggest = size
        return biggest


# ---- main question: main function ----
# Idea: put each point in the cell (x // r, y // r). A point within
# distance r of another is in the same cell or one of the 8 cells
# around it, so only those are compared; union the close pairs.
# Complexity: Time about O(n) for evenly spread points (each point looks at 9 grid cells), O(n^2) worst case;
 Space O(n)
```

```
def count_clusters(points, r):
    clusters = Clusters(r)
    for x, y in points:
        clusters.add(x, y)
    return clusters.components
```

COMPLEXITY | Time O(n) expected for evenly spread points | Space O(n)

76 Find A Threshold-Limited Path With Minimum Required Safety

MEDIUM

Template fit: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge

QUESTION

A graph has n nodes and edges (u, v, safety). The safety of a path is the smallest safety among its edges. Using only edges whose safety is at least threshold (no limit when threshold is None), return the best (largest) safety you can reach going from s to t, or -1 if no such path exists. A path from a node to itself has infinite safety.

Function signature

def best_path_safety(n: int, edges: list, s: int, t: int, threshold=None) -> int

EXAMPLES

Example 1	
Input	3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2
Output	3
Why	Direct edge 0-2 has safety 1; the route 0-1-2 has safety min(5, 3) = 3, which is better.

Example 2	
Input	3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold = 4
Output	-1
Why	Only edges with safety at least 4 may be used: 0-1 is allowed but nothing leads to node 2.

Example 3	
Input	3, [(0, 1, 5)], 1, 1
Output	infinity
Why	Source and target are the same node.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def safety_of(edge):
    # an edge is (u, v, safety)
```

```
    return edge[2]


# ---- main question: main function ----
# Idea: the safety of a path is its weakest edge. Add edges from the
# safest to the least safe; the edge that finally joins s and t is the
# weakest edge of the best path. -1 if s and t never join (or only
# edges below `threshold` are allowed and that is not enough).
# Complexity: Time O(E log E) for the sort, then O(E * alpha(n)); Space O(n)
def best_path_safety(n, edges, s, t, threshold=None):
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    for u, v, w in sorted(edges, key=safety_of, reverse=True):
        if threshold is not None and w < threshold:
            break                                   # follow-up 3: weaker edges are forbidden
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1
```

COMPLEXITY | Time O(E log E) | Space O(V)

77 Movie Recommendations via Transitive Similarity — MEDIUM

Template fit: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating

QUESTION

You are given a target movie name, a dictionary ratings mapping each movie to its numeric rating, an integer n , and a list similarities of [movieX, movieY] pairs meaning the two movies are directly similar. Similarity is transitive: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group. Return up to n recommended movies from the target's similarity group, excluding the target itself, ordered by rating descending (break ties alphabetically by name). If n == 1 , this returns the single highest- rated similar movie. If the group has fewer than n other movies, return all of them.

Function signature

def recommend(target: str, ratings: dict[str, float], n: int,
similarities: list[list[str]]) -> list[str]:
...

EXAMPLES

Example 1	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C"]`
Why	`A, B, C form one group. Excluding A, the highest rated is C (9).`

Example 2	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C", "B"]`
Why	`Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].`

QUESTION

Example 3	
Input	`target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3, similarities = [["A", "B"], ["B", "C"]]`
Output	`[]`
Why	`D is in no similarity pair, so its group is just {D}; excluding D leaves nothing.`

CONSTRAINTS

- Every movie mentioned appears as a key in ratings .
- 1 <= n ; ratings are numbers, names are distinct strings.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class UnionFind:
    # union-find over names (any hashable value); unknown names join on first use
    def __init__(self):
        self.parent = {}

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def best_first(movies, ratings):
    # the movies ordered by rating, highest first (ties by name)
    pairs = []
    for movie in movies:
        pairs.append((-ratings[movie], movie))
    pairs.sort()
    ordered = []
    for pair in pairs:
        ordered.append(pair[1])
    return ordered


# ---- main question: main function ----
# Idea: movies joined by "similar" pairs (directly or through other
# movies) form one group. Take the target's group and rank it by rating.
# Complexity: Time O(P alpha + M log M) for P similar pairs and M movies in the group, Space O(M)
def recommend(target, ratings, n, similarities):
    uf = UnionFind()
    for x, y in similarities:
        uf.union(x, y)
    root = uf.find(target)
    group = []
    for movie in list(uf.parent):
        if movie != target and uf.find(movie) == root:
            group.append(movie)
    return best_first(group, ratings)[:n]
```

COMPLEXITY | Time O(P alpha + g log g) | Space O(P)

78 Determine Reporting Relationships

MEDIUM

Template fit: pairs = peer links; keep the boss at the root and carry it over when two groups merge

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints
0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: union-find joins peers (people with the same manager); the
# leader of every peer group stores that group's manager.
# add_peer(a, b)    a and b have the same manager
# add_manager(a, b) a is the direct manager of b (and of b's peers)
# Complexity: Time add_peer, add_manager and manager_of are O(alpha(n)); is_manager is O(depth of the chain);
 Space O(n)
class Org:
    def __init__(self):
```

```
        self.parent = {}       # union-find over people
        self.boss = {}         # leader of a peer group -> the manager of the group

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        if ra in self.boss and rb in self.boss and self.boss[ra] != self.boss[rb]:
            raise ValueError("conflicting managers")          # follow-up 2
        self.parent[rb] = ra
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]                     # the merged group keeps the known manager

    def add_manager(self, a, b):
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")          # follow-up 2
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))

    def is_manager(self, a, b):
        # climb b's chain of managers; `seen` stops a cycle
        seen = set()
        person = b
        while person is not None and person not in seen:
            seen.add(person)
```

```
            person = self.manager_of(person)
            if person == a:
                return True
        return False
```

COMPLEXITY | Time O(alpha) per update, O(depth) per query | Space O(employees)

79 Determine whether two nodes are related — MEDIUM

Template fit: pairs = the given edges; the query is find(a) == find(b)

QUESTION

Nodes 0 to n - 1 are linked by edges, and two nodes are related when a chain of edges joins them (links work in both directions; the graph may be disconnected and may contain cycles). Support union(a, b) to add a link and related(a, b) to answer the query.

Function signature

class Relations:
  def __init__(self, n: int, edges=())
  def union(self, a: int, b: int) -> None
  def related(self, a: int, b: int) -> bool

EXAMPLES

QUESTION

Example 1

Input	`r = Relations(5, [(0, 1), (1, 2)])`
	`r.related(0, 2)`
	`r.related(0, 3)`
	`r.union(3, 4)`
	`r.related(3, 4)`
	`r.related(2, 4)`

Output	`r.related(0, 2) -> true`
	`r.related(0, 3) -> false`
	`r.related(3, 4) -> true`
	`r.related(2, 4) -> false`

Why	`0, 1, 2 form one group and 3, 4 another until they are joined; the two groups are never linked to each other.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: union the two ends of every relationship; two nodes are
# related exactly when they have the same leader (root).
# Complexity: Time union and related are O(alpha(n)) (about O(1)) each, building from E edges is O(E * alpha(
n)), Space O(n)
class Relations:
    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        # walk up to the leader; path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant time each
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra                # the smaller group joins the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)
```

COMPLEXITY | Time O(E alpha(V)) to build, O(alpha(V)) per query | Space O(V)

80 Friend Graph with Timeline (Friend / Unfriend Events) HARD

Template fit: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

QUESTION

There are n people numbered 0 to n - 1. Each event is (time, a, b, kind). A 'friend' event makes a and b friends from that time on, and two people are connected when a chain of friends links them. Return the earliest time at which all n people are connected, or -1 if that never happens.

Function signature

def earliest_all_connected(n: int, events: list) -> int

EXAMPLES

Example 1	
Input	`3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]`
Output	`5`
Why	`After time 1, {0, 1} are connected; at time 5 person 2 joins, so everyone is connected at 5.`

Example 2	
Input	`4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]`
Output	`-1`
Why	`Person 3 is never befriended, so the group never becomes one.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def event_time(event):
    # events are (time, a, b, "friend" or "unfriend"); sort by the time
    return event[0]


class UnionFind:
    # union by size WITHOUT path compression, so every link keeps the time it was made
    # (follow-up 4 needs those times)
    def __init__(self, n):
        self.parent = list(range(n))      # everybody starts as their own leader
        self.size = [1] * n               # group sizes decide who hangs under whom
        self.link_time = [0] * n          # the time a person was linked under its parent
        self.groups = n                   # how many separate groups exist now

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, time):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False                  # already together: nothing changes
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra               # ra is the bigger group
        self.parent[rb] = ra              # the smaller group hangs under the bigger one
        self.size[ra] += self.size[rb]
        self.link_time[rb] = time
        self.groups -= 1                  # two groups became one
        return True


# ---- main question: main function ----
```

```
# Idea: sort the events by time and union the two people of every
# friend event. The first time only one group is left is the answer;
# -1 if that never happens.
# Complexity: Time O(E log E + E log n) (the sort, then each union walks up a tree of height log n), Space O(
n)
def earliest_all_connected(n, events):
    if n <= 1:
        return 0
    uf = UnionFind(n)
    for time, a, b, kind in sorted(events, key=event_time):
        if kind == "friend":
```

```
            uf.union(a, b, time)
            if uf.groups == 1:
                return time
    return -1
```

COMPLEXITY | Time O(E log E) | Space O(N)

A dict of per-key state that you update, expire, or rank

81 Robot Status Message Deduplication Window — EASY

Template fit: the template as written: show a message only if it was not shown in the last 10 seconds

QUESTION

Status events arrive from a robot as (time, message). Show a message only if the same message was not shown during the previous 10 seconds; a hidden message does not restart the timer, and exactly 10 seconds later it is shown again. show(t, msg) returns True when the message should be displayed.

Function signature

class Dedup:
    def show(self, t: int, msg: str) -> bool

EXAMPLES

Example 1

Input	`d = Dedup()`
	`d.show(0, 'low battery')`
	`d.show(4, 'low battery')`
	`d.show(6, 'door open')`
	`d.show(9, 'low battery')`
	`d.show(10, 'low battery')`

Output	`d.show(0, 'low battery') -> true`
	`d.show(4, 'low battery') -> false`
	`d.show(6, 'door open') -> true`
	`d.show(9, 'low battery') -> false`
	`d.show(10, 'low battery') -> true`

Why	The repeats at 4 and 9 are hidden and do not move the timer; at time 10 it is exactly 10 seconds after the last display, so it shows again.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(items, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(items, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
```

```
            high = middle
    return low


# ---- main question: main class ----
# show(t, msg) says whether the message may be displayed now: not if it was
# displayed less than `window` seconds ago. Events arrive in time order.
# Idea: a dictionary with the time each message was last DISPLAYED. If the
# gap is still inside the window, hide the message WITHOUT changing the stored
# time (only a displayed event starts a new window); otherwise display it and
# store the new time. boundary_allowed says whether exactly `window` seconds later
# may show.
# Complexity: Time show O(1), Space O(distinct messages)
class Dedup:
    def __init__(self, window=10, boundary_allowed=True):
        self.window = window
        self.last = {}
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
```

```
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True
```

COMPLEXITY | Time O(1) per event | Space O(distinct messages)

82 Active Communication Top Users

MEDIUM

Template fit: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)

QUESTION

A stream of messages like 'A-B' means users A and B talked. Each distinct pair counts once (A-B and B-A are the same pair) and a pair of a user with themselves is ignored. A user's count is how many different partners they have. Support add(pair), most_active() (the set of users with the highest count) and top_k(k) (k users by count, ties broken by name).

Function signature

class Active:
    def add(self, pair: str) -> None
    def most_active(self) -> set
    def top_k(self, k: int) -> list

EXAMPLES

QUESTION

Example 1

Input	a = Active() a.add('A-B') a.add('B-C') a.add('B-A') a.most_active() a.top_k(2)
Output	a.most_active() -> {'B'} a.top_k(2) -> ['B', 'A']
Why	B talked to A and C (2 partners). The repeated pair B-A adds nothing. A and C have 1 each, so the top 2 are B, then A (ties by name).

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: main class ----
# A stream of messages like "A-B" means users A and B talked. Each DISTINCT
# pair counts once (A-B and B-A are the same pair) and a pair of a user with
# themselves is ignored. A user's count is how many different partners they have.
# add(pair), most_active() = the set of users with the highest count,
# top_k(k) = k users by count, ties broken by name.
# Idea: keep the set of edges seen (to drop repeats), a count per user, and
# BUCKETS: count -> the set of users with exactly that count. A new edge moves
# both users up one bucket, and most_active just reads the top bucket: O(1).
# (count_self=True is follow-up 1: a user talking to themselves counts, once.)
# Complexity: Time add O(1), most_active O(size of the top bucket), top_k O(u log k) for u users, Space O(num
ber of different pairs)
class Active:
    def __init__(self, count_self=False):
        self.edges = set()                      # the distinct pairs seen so far
        self.count = {}                         # user -> how many different partners
        self.bucket = {}                        # count -> the users with exactly that count
        self.top = 0                            # the highest count right now (it never drops)
        self.count_self = count_self

    def bump(self, user):
        old = self.count.get(user, 0)
        if old > 0:
            self.bucket[old].discard(user)      # leave bucket `old` ...
        self.count[user] = old + 1
        if old + 1 not in self.bucket:
            self.bucket[old + 1] = set()
        self.bucket[old + 1].add(user)          # ... and join the next bucket
        if old + 1 > self.top:
            self.top = old + 1

    def add(self, pair):
        a, b = pair.split("-")
        if a == b and not self.count_self:
            return                              # a user talking to themselves is ignored
        if a < b:
            edge = (a, b)                       # A-B and B-A are the same edge
        else:
            edge = (b, a)
        if edge in self.edges:
            return                              # a repeated pair counts only once
        self.edges.add(edge)
        self.bump(a)
```

```
        if a != b:
            self.bump(b)                        # a self pair counts once, not twice

    def most_active(self):
        if self.top == 0:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        rows = []
        for user in self.count:
            rows.append((-self.count[user], user))
        best = heapq.nsmallest(k, rows)
        users = []
        for minus_count, user in best:
            users.append(user)
        return users
```

COMPLEXITY | Time O(1) per pair, O(U log k) for top-K, O(1) for the most active user | Space O(E)

83 Filter Repeated Robot Status Messages — MEDIUM

Template fit: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted

QUESTION

A robot status stream is a list of (timestamp, message). Show a message when it has never been shown, or when strictly more than gap seconds (default 10) have passed since it was last shown. Messages are compared exactly (case-sensitive), a hidden repeat does not restart the interval, and the shown messages keep their input order. Return the shown (timestamp, message) pairs.

Function signature

def filter_status(stream: list, gap: int = 10) -> list

EXAMPLES

Example 1	
Input	`[(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11, 'OK')]`
Output	`[(0, 'ok'), (11, 'ok'), (11, 'OK')]`
Why	`'ok' at 5 and 10 are hidden (not more than 10 seconds after 0); at 11 it is 11 seconds later, so it shows; 'OK' is a different message.`

Example 2	
Input	`[(0, 'low'), (3, 'low'), (7, 'low')], gap = 2`
Output	`[(0, 'low'), (3, 'low'), (7, 'low')]`
Why	`A smaller gap lets repeats through sooner.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# The stream is a list of (timestamp, message). Show a message only if MORE
# than `gap` seconds have passed since it was last SHOWN; a hidden repeat
# changes nothing (it does not move the last-shown time).
# Idea: a dictionary from message to the time it was last shown. One lookup
# per message.
# Complexity: Time O(n), Space O(distinct messages)
def filter_status(stream, gap=10):
    last_shown = {}
    shown = []
    for ts, msg in stream:
        if msg not in last_shown or ts - last_shown[msg] > gap:
            last_shown[msg] = ts
```

```
            shown.append((ts, msg))
    return shown
```

COMPLEXITY | Time O(n) | Space O(distinct messages)

84 Next-Word Predictor — MEDIUM

Template fit: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word

QUESTION

Train on sentences, then predict the word that most often follows a given word. train(sentence) learns every pair of neighbouring words; predict(word) returns the word that followed it most often (ties go to the alphabetically smaller word) or None if nothing ever followed it. predict must run in O(1).

Function signature

class Predictor:
  def train(self, sentence: str) -> None
  def predict(self, word: str) -> str | None

EXAMPLES

Example 1

Input	`p = Predictor()`
	`p.train('I love apples')`
	`p.train('I love pears')`
	`p.train('you love pears too')`
	`p.predict('love')`
	`p.predict('I')`
	`p.predict('too')`

Output	`p.predict('love') -> 'pears'`
	`p.predict('I') -> 'love'`
	`p.predict('too') -> null`

Why	`After 'love' came 'apples' once and 'pears' twice, so 'pears' wins. Nothing ever followed 'too'.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main class ----
# train(sentence) learns every pair of neighbouring words; predict(word)
# returns the word that followed it most often (ties: the alphabetically
# smaller word), or None. predict must be O(1).
# Idea: counts[word][next] is the number of times `next` followed `word`.
# After every increment compare that one count with the best successor kept for
# the word (a higher count wins, then the smaller word). predict then only has to
# read the stored best word.
# Complexity: Time train O(W) for W words, predict O(1), Space O(number of different word pairs)
class Predictor:
    def __init__(self):
        self.counts = {}                        # word -> {next word: how often}
        self.best = {}                          # word -> the most likely next word so far

    def train(self, sentence):
        words = sentence.split()
        for i in range(len(words) - 1):
            self.update(words[i], words[i + 1])

    def update(self, word, following):
        if word not in self.counts:
            self.counts[word] = {}
        table = self.counts[word]
        table[following] = table.get(following, 0) + 1
```

```
        current = self.best.get(word)
        if current is None:
            self.best[word] = following
        elif table[following] > table[current]:
            self.best[word] = following
        elif table[following] == table[current] and following < current:
            self.best[word] = following            # a tie goes to the smaller word

    def predict(self, word):
        return self.best.get(word)
```

COMPLEXITY | Time O(1) per pair when training and per prediction | Space O(number of distinct pairs)

85 Batch and Streaming Log Deduplication — MEDIUM

Template fit: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end

QUESTION

Given log entries (timestamp, message), keep one entry per message and return the kept entries sorted by timestamp. With policy 'first' keep the entry seen first for each message; with policy 'latest' keep the one with the latest timestamp.

Function signature

def dedupe(logs: list, policy: str = 'first') -> list

EXAMPLES

Example 1	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]`
Output	`[(1, 'b'), (3, 'c'), (5, 'a')]`
Why	`Messages 'a' and 'b' each appear twice; the first one seen is kept.`

Example 2	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')], policy = 'latest'`
Output	`[(2, 'b'), (3, 'c'), (9, 'a')]`
Why	`With 'latest', the later timestamp of each repeated message wins.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# logs are (timestamp, message) pairs. Keep ONE entry per message: the
# first one seen (policy "first") or the newest (policy "latest").
# Idea: one pass with a dictionary from message to the kept timestamp; then
# sort the kept entries by time for the output.
# Complexity: Time O(n log n) (the final sort), Space O(n)
def dedupe(logs, policy="first"):
    kept = {}
    for ts, msg in logs:
        if msg not in kept:
            kept[msg] = ts
        elif policy == "latest" and ts > kept[msg]:
            kept[msg] = ts
    entries = []
    for msg in kept:
        entries.append((kept[msg], msg))
    entries.sort()
    return entries
```

COMPLEXITY | Dictionary on the message, then sort: Time O(n log n) | Space O(n)
Streaming: map + min-heap with lazy deletion: Time O(log n) amortized per add and get_next | Space O(n)

86 Design a logger that returns top-K messages

MEDIUM

Template fit: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest

QUESTION

Design a logger. log(ts, msg) records a message at time ts (timestamps arrive in non-decreasing order). top_k(k, start, end) returns the k most frequent messages whose timestamps fall in [start, end]; leave start and end out to use the whole history. Ties are broken alphabetically.

Function signature

class Logger:
    def log(self, ts: int, msg: str) -> None
    def top_k(self, k: int, start=None, end=None) -> list

EXAMPLES

Example 1

Input	`lg = Logger()`
	`lg.log(1, 'disk full')`
	`lg.log(2, 'timeout')`
	`lg.log(3, 'timeout')`
	`lg.log(4, 'disk full')`
	`lg.log(5, 'timeout')`
	`lg.top_k(1)`
	`lg.top_k(2, 4, 5)`
	`lg.top_k(2, 1, 1)`

Output	`lg.top_k(1) -> ['timeout']`
	`lg.top_k(2, 4, 5) -> ['disk full', 'timeout']`
	`lg.top_k(2, 1, 1) -> ['disk full']`

Why	`Overall 'timeout' occurred 3 times. In the window [4, 5] 'disk full' and 'timeout' occur once each (tie, alphabetical). The window [1, 1] only holds 'disk full'.`

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(items, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(items, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
```

```
            high = middle
    return low


# ---- main question: main class ----
# log(ts, msg) records a message at time ts (timestamps never go backwards).
# top_k(k, start, end) returns the k most frequent messages with a timestamp in
# [start, end] (the whole history if they are left out); ties alphabetically.
# Idea: keep, per message, the list of its timestamps. They arrive in
# order, so every list stays sorted. The number of times a message occurs in
# [start, end] is two binary searches: (first index > end) - (first index >= start).
# A heap then picks the k best (count, message) rows.
# Complexity: Time log O(1), top_k O(M log n + M log k) for M messages, expire O(total timestamps), Space O(n
)
class Logger:
    def __init__(self):
        self.times = {}                             # message -> its timestamps, in order of arrival
```

```
    def log(self, ts, msg):
        if msg not in self.times:
            self.times[msg] = []
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        rows = []
        for msg in self.times:
            stamps = self.times[msg]
            low = 0
            high = len(stamps)
            if start is not None:
                low = first_at_least(stamps, start)
            if end is not None:
                high = first_greater(stamps, end)
            if high > low:
                rows.append((-(high - low), msg))      # minus the count: the most frequent sorts first
        best = heapq.nsmallest(k, rows)
        messages = []
        for minus_count, msg in best:
            messages.append(msg)
        return messages

    def expire(self, before):
        # bounded memory (follow-up 2): forget timestamps older than `before`
        for msg in list(self.times):
            stamps = self.times[msg]
            self.times[msg] = stamps[first_at_least(stamps, before):]
            if not self.times[msg]:
                del self.times[msg]
```

COMPLEXITY | Time O(1) per log, O(d log n + d log k) per query | Space O(n)

87 Hit Counter — MEDIUM

Template fit: one deque of hit times; drop the ones older than 300 seconds; the answer is the length

QUESTION

Design a hit counter. hit(timestamp) records a hit at that second, and get_hits(timestamp) returns the number of hits in the past 300 seconds (timestamps never go backwards).

Function signature

class QueueCounter:
    def hit(self, ts: int) -> None
    def get_hits(self, ts: int) -> int

QUESTION

EXAMPLES

Example 1

Input	**h = QueueCounter()** **h.hit(1)** **h.hit(2)** **h.hit(3)** **h.get_hits(4)** **h.hit(300)** **h.get_hits(300)** **h.get_hits(301)**
Output	**h.get_hits(4) -> 3** **h.get_hits(300) -> 4** **h.get_hits(301) -> 3**
Why	**At time 301 the hit at time 1 is exactly 300 seconds old and no longer counts.**

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

MAIN CODE | helper functions first, then the answer

```
from collections import deque


WINDOW = 300                                    # seconds


# ---- main question: main class ----
# hit(ts) records a hit at second ts; get_hits(ts) returns the number of
# hits in the past 300 seconds (timestamps never go backwards).
# Idea: keep a queue of the hit times in order of arrival. get_hits removes
# the hits from the front while the oldest is outside the window; what is left
# is the answer. Every hit is added once and removed once: O(1) amortised.
# Complexity: Time hit O(1), get_hits O(1) amortized, Space O(hits in the last 300 seconds)
class QueueCounter:
    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        while self.hits and self.hits[0] <= ts - WINDOW:     # keep only the hits in (ts - 300, ts]
            self.hits.popleft()
        return len(self.hits)
```

COMPLEXITY | Time O(1) amortized | Space O(hits in the window)

88 Website Activity Analytics: Unique Users + Session Time — MEDIUM

Template fit: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time

QUESTION

Each record has a user_id, a timestamp ('YYYY-MM-DD HH:MM:SS') and an activity. Return (1) the number of distinct users per activity and (2) the average number of seconds a user spends on the site. A user's time is the sum of the gaps from each 'login' to the next 'logout'; a login without a logout counts until the user's last event.

Function signature

def activity_stats(records: list) -> (dict, float)

QUESTION

EXAMPLES

Example 1

Input	`rec = lambda u, t, a: {'user_id': u, 'timestamp': '2026-01-01 ' + t, 'activity': a}`
	`records = [rec(1, '10:00:00', 'login'), rec(1, '10:10:00', 'click'), rec(1, '10:30:00', 'logout'), rec(2, '10:05:00', 'login'), rec(2, '10:20:00', 'logout')]`
	`activity_stats(records)`

Output	`activity_stats(records) -> ({'click': 1, 'login': 2, 'logout': 2}, 1350)`
Why	`User 1 spent 30 minutes (1800 s), user 2 spent 15 minutes (900 s): the average is 1350 s. Both users logged in and out; only user 1 clicked.`

MAIN CODE | helper functions first, then the answer

```
from datetime import datetime


FORMAT = "%Y-%m-%d %H:%M:%S"


# ---- main question: helper functions ----
def when(record):
    # the timestamp text of a record as a date and time
    return datetime.strptime(record["timestamp"], FORMAT)


def unique_users_by_activity(records):
    # per activity: how many DIFFERENT users did it (a set counts each user once)
    users = {}
    for record in records:
        if record["activity"] not in users:
            users[record["activity"]] = set()
        users[record["activity"]].add(record["user_id"])
    counts = {}
    for activity in sorted(users):
        counts[activity] = len(users[activity])
    return counts


def session_seconds(events):
    # one user's (time, activity) rows, in any order: the seconds spent logged in.
    # Sort by time first (follow-up 1: rows may arrive out of order). A second
    # login before a logout is ignored; a login with no logout ends at the
    # user's last event.
    total = 0
    start = None
    for t, activity in sorted(events):
        if activity == "login" and start is None:
            start = t
        elif activity == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    if start is not None:
        last = events[0][0]
        for t, activity in events:
            if t > last:
                last = t
        total += (last - start).total_seconds()
    return total
```

```


def average_time_per_user(records):
    # the average session time over EVERY user who appears (state the denominator)
```

```
    events = {}
    for record in records:
        if record["user_id"] not in events:
            events[record["user_id"]] = []
        events[record["user_id"]].append((when(record), record["activity"]))
    if not events:
        return 0.0
    total = 0
    for user in events:
        total += session_seconds(events[user])
    return total / len(events)


# ---- main question: main function ----
# records are {"user_id", "activity", "timestamp"}. Return the number of
# unique users per activity and the average session time per user.
# Idea: one pass groups the records, then each user's rows are paired up
# login -> logout.
# Complexity: Time O(n) for n records, Space O(users + activities)
def activity_stats(records):
    return unique_users_by_activity(records), average_time_per_user(records)
```

COMPLEXITY | Time O(N log N) | Space O(N)

Binary search on a yes/no condition

89 First Index Reaching a Threshold (Binary Search) EASY

Template fit: x = an index; ok(x) = arr[x] >= k on a sorted array

QUESTION

Given a non-decreasing (sorted ascending, duplicates allowed) array arr and a target value k , return the smallest index i such that arr[i] >= k — the first position where the array "reaches" the value k . If no element is >= k , return -1 . Solve in O(log n) time and O(1) extra space.

Function signature

def first_index_at_least(arr: list[int], k: int) -> int:

    ...

Follow-up (as reported): The interviewer probed space complexity; note the solution is already O(1) space. If arr is a huge sorted stream/file, describe reading via random access rather than loading it all.

EXAMPLES

Example 1	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 2`
Output	`1`
Why	`arr[1] = 2 is the first element >= 2.`

Example 2	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 4`
Output	`4`
Why	`The first element >= 4 is 5 at index 4.`

Example 3	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 8`
Output	`-1`
Why	`No element is >= 8.`

CONSTRAINTS

- 0 <= len(arr) <= 10**7
- Values and k are integers.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the array is sorted, so the values below k come first and the
# values at least k come after. Halve the range while keeping that
# boundary inside it; when the range is one spot, that spot is the first
# value >= k. -1 if every value is smaller.
# Complexity: Time O(log n), Space O(1)
def first_index_at_least(arr, k):
    lo = 0
    hi = len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] >= k:
            hi = mid                        # mid could be the answer: keep it
        else:
```

```
            lo = mid + 1                    # mid is too small: the answer is after it
    if lo < len(arr):
        return lo
    return -1
```

COMPLEXITY | Time O(log n) | Space O(1)

90 Cutting Ribbons (LC 1891) MEDIUM

Template fit: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1

QUESTION

You have ribbons of given lengths and an integer k. Each ribbon may be cut into pieces of one common integer length L (leftover bits are thrown away). Return the largest L that gives at least k pieces in total, or 0 if even length 1 cannot give k pieces.

Function signature

def max_length(ribbons: list, k: int) -> int

EXAMPLES

Example 1	
Input	[9, 7, 5], 3
Output	5
Why	Length 5 gives 1 + 1 + 1 = 3 pieces, length 6 only 2.

Example 2	
Input	[5, 7], 12
Output	1
Why	Length 1 already gives 12 pieces.

Example 3	
Input	[1, 2], 10
Output	0
Why	Total length is 3 but 10 pieces are needed: impossible, so 0.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def pieces(ribbons, length):
    # how many pieces of this length can we cut? (the check for a GIVEN length)
    total = 0
    for r in ribbons:
        total += r // length
    return total


def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and k pieces of length L
    # need k * L in total, so L <= sum // k
    longest = 0
    total = 0
    for r in ribbons:
        total += r
        if r > longest:
            longest = r
    if total // k < longest:
```

```
        return total // k
    return longest


# ---- main question: main function ----
# Idea: if k pieces of length L can be cut, k pieces of any SHORTER
# length can too, so "can we cut k pieces?" goes from yes to no as L
# grows: binary search the largest L that is still yes (whole numbers).
# The answer is 0 when it is impossible.
# Complexity: Time O(n * log(longest ribbon)), Space O(1)
def max_length(ribbons, k):
    low = 1
    high = upper_bound(ribbons, k)
    best = 0
    while low <= high:
        mid = (low + high) // 2
        if pieces(ribbons, mid) >= k:
            best = mid                    # mid works: remember it and try longer
            low = mid + 1
        else:
            high = mid - 1
    return best


# the same idea when lengths may be fractions: a fixed number of halvings
# Complexity: Time O(n * rounds), Space O(1)
```

```
def max_length_float(ribbons, k, rounds=50):
    low = 0.0
    high = float(max(ribbons))
    for i in range(rounds):
        mid = (low + high) / 2
        count = 0
        for r in ribbons:
            count += int(r // mid)
        if count >= k:
            low = mid                     # enough pieces: the answer is at least mid
        else:
            high = mid
    return low
```

COMPLEXITY | Binary search the answer (integer lengths): Time O(n log(max length)) | Space O(1)
Floating-point lengths with a fixed iteration count: Time O(n * iterations) | Space O(1)

91 First Bad Version with Parallel Search Follow-up

MEDIUM

Template fit: x = the version number; ok(x) = is_bad(x)

QUESTION

Versions are numbered 1 to n and from some version on every version is bad. The function is_bad(v) tells whether version v is bad. Find the first bad version with as few calls to is_bad as possible.

Function signature

def first_bad_version(n: int, is_bad) -> int

EXAMPLES

Example 1	
Input	`bad_from = 4` `first_bad_version(10, lambda v: v >= bad_from)`
Output	`first_bad_version(10, lambda v: v >= bad_from) -> 4`
Why	`Versions 4 to 10 are bad, so 4 is the first; binary search needs about log2(10) calls.`

QUESTION

Example 2	
Input	`bad_from = 1` `first_bad_version(1, lambda v: v >= bad_from)`
Output	`first_bad_version(1, lambda v: v >= bad_from) -> 1`
Why	`With a single version that is bad, the answer is 1.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: is_bad is monotone (all good versions, then all bad ones), so
# keep the invariant "the first bad version is in [lo, hi]" and halve the
# interval. Use lo + (hi - lo) // 2 in languages with fixed-size integers.
# Complexity: Time O(log n) calls, Space O(1)
def first_bad_version(n, is_bad):
    lo = 1
    hi = n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            hi = mid                        # the first bad version is mid or earlier
        else:
            lo = mid + 1
    return lo
```

COMPLEXITY | Base problem: binary search: Time O(log n) calls | Space O(1)
Parallel: probe k points at once, then recurse into the bucket that flips: Time O(log_{k+1} n) rounds | Space O(k)

92 Solve Banana Speed and Interval Merge — MEDIUM

Template fit: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.

QUESTION

- Complexity and edge cases.

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`** Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4: ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) = 1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 =
    10 > 8, so 4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile takes a rounded-up number of hours
    total = 0
    for p in piles:
        total += (p + speed - 1) // speed
    return total


# ---- main question: main functions ----
# PART 1 (bananas): the smallest eating speed that finishes all piles
# within h hours. Hours needed never go UP when the speed goes up, so
# binary search the speed between 1 and the biggest pile. -1 if h is
# smaller than the number of piles (each pile takes at least one hour).
# Complexity: Time O(n * log(biggest pile)), Space O(1)
def min_eating_speed(piles, h):
    if h < len(piles):
        return -1
    low = 1
    high = max(piles)
    while low < high:
        mid = (low + high) // 2
        if hours_needed(piles, mid) <= h:
            high = mid                    # fast enough: try slower
        else:
            low = mid + 1
    return low


# PART 2 (intervals): merge overlapping intervals. Sort by start; an
# interval that starts before (or when) the last merged one ends is
# absorbed by it. Sharing one point counts as overlapping.
# Complexity: Time O(n log n), Space O(n)
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged
```

COMPLEXITY | Bananas: binary search the speed: Time O(n log max) | Space O(1)
Intervals: sort and sweep: Time O(n log n) | Space O(n)

93 Given a list with response time for each server, calculate how long time it will take to process n requests.

MEDIUM

Template fit: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests

QUESTION

Servers work in parallel and server i needs times[i] time units to process one request (optionally server i only starts at starts[i]). Return the earliest time at which n requests have been processed in total.

Function signature

def time_for(times: list, n: int, starts=None) -> int

EXAMPLES

QUESTION

Example 1	
Input	[1, 2], 3
Output	2
Why	By time 2 the first server has done 2 requests and the second one: 3 in total.

Example 2	
Input	[3], 4
Output	12
Why	One server needs 3 units per request: 4 requests take 12.

Example 3	
Input	[2, 1], 6, starts = [0, 5]
Output	8
Why	The second server only starts at time 5, so the first one does most of the work.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def start_of(starts, i):
    # the startup delay of server i (0 when no delays are given)
    if starts:
        return starts[i]
    return 0


def done_by(times, t, starts=None):
    # how many requests do all servers finish within time t? Server i needs
    # times[i] per request and only begins after its startup delay (follow-up 1).
    total = 0
    for i in range(len(times)):
        working_time = t - start_of(starts, i)
        if working_time > 0:
            total += working_time // times[i]
    return total


# ---- main question: main function ----
# Idea: binary search on the TOTAL TIME. In time t, server i finishes
# t // times[i] requests, and done_by(t) never decreases as t grows, so the
# answer is the smallest t with done_by(t) >= n.
# starts[i] is an optional startup delay (follow-up 1).
# Complexity: Time O(s * log(range)) for s servers, Space O(1)
def time_for(times, n, starts=None):
    if n <= 0:
        return 0
    low = 0
    high = start_of(starts, 0) + times[0] * n      # the first server alone is always enough
    for i in range(len(times)):
        if start_of(starts, i) + times[i] * n < high:
            high = start_of(starts, i) + times[i] * n
    while low < high:
        mid = (low + high) // 2
        if done_by(times, mid, starts) >= n:
            high = mid
        else:
            low = mid + 1
    return low
```

COMPLEXITY | Time O(S log(min(times) * n)) | Space O(1)

94 Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — HARD

Template fit: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest

QUESTION

Given two sorted arrays of numbers, return the median of all their elements together, without merging them into one array. The median is the middle element when the total count is odd, and the average of the two middle elements when it is even. Aim for O(log(min(m, n))) time.

Function signature

def median_two(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	[1, 3], [2]
Output	2
Why	Combined [1, 2, 3]: the middle element is 2.

Example 2	
Input	[1, 2], [3, 4]
Output	2.5
Why	Combined [1, 2, 3, 4]: the two middle elements are 2 and 3, so the median is 2.5.

Example 3	
Input	[], [7, 9]
Output	8
Why	One array may be empty: the median is just the other array's median.

MAIN CODE | helper functions first, then the answer

```
INF = float("inf")


# ---- main question: helper functions ----
def left_border(arr, i):
    # the last of the first i items of arr (minus infinity when i is 0)
    if i > 0:
        return arr[i - 1]
    return -INF


def right_border(arr, i):
    # the first item after the first i items (plus infinity when there is none)
    if i < len(arr):
        return arr[i]
    return INF


def bigger(a, b):
    if a > b:
        return a
    return b


def smaller(a, b):
    if a < b:
        return a
```

```
    return b


# ---- main question: main function ----
# Idea: make a the shorter array. Binary search i, the number of items of
# a in the left half, and take half - i items from b, so that the left
# half holds (m + n + 1) // 2 items. The cut is right when every left
# item is <= every right item; then the median is read off the four items
# next to the cuts.
# Complexity: Time O(log(min(m, n))), Space O(1)
def median_two(a, b):
    if len(a) > len(b):
        a, b = b, a
    m = len(a)
    n = len(b)
    if m + n == 0:
        raise ValueError("no elements")
    low = 0
```

```
    high = m
    half = (m + n + 1) // 2
    while low <= high:
        i = (low + high) // 2
        j = half - i
        a_left, a_right = left_border(a, i), right_border(a, i)
        b_left, b_right = left_border(b, j), right_border(b, j)
        if a_left <= b_right and b_left <= a_right:        # a valid cut
            if (m + n) % 2 == 1:
                return bigger(a_left, b_left)
            return (bigger(a_left, b_left) + smaller(a_right, b_right)) / 2
        if a_left > b_right:
            high = i - 1
        else:
            low = i + 1
```

COMPLEXITY | Time O(log min(m, n)) | Space O(1)

95 Find the median of two sorted arrays. HARD

Template fit: the same partition search on the shorter array

QUESTION

Given two sorted arrays, return the median of all their elements together. The median is the middle value when the total count is odd and the average of the two middle values when it is even. The best solutions run in O(log(min(m, n))).

Function signature

def findMedianSortedArrays(nums1: list, nums2: list) -> float

EXAMPLES

Example 1	
Input	`[1, 3], [2]`
Output	`2`
Why	`Combined: 1, 2, 3.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`2.5`
Why	`Combined: 1, 2, 3, 4: the middle two are 2 and 3.`

QUESTION

Example 3	
Input	[], [5]
Output	5
Why	One empty array.

MAIN CODE | helper functions first, then the answer

```
INF = float("inf")


# ---- main question: helper functions ----
def left_border(arr, i):
    # the last of the first i items of arr (minus infinity when i is 0)
    if i > 0:
        return arr[i - 1]
    return -INF


def right_border(arr, i):
    # the first item after the first i items (plus infinity when there is none)
    if i < len(arr):
        return arr[i]
    return INF


def bigger(a, b):
    if a > b:
        return a
    return b


# Complexity: Time O(log(min(m, n))), Space O(1)
def kth_of_two(a, b, k):
    # the k-th smallest (k starts at 1) of two sorted arrays. Take i items
    # from a and k - i items from b as the "left part". The cut is right when
    # every left item is <= every right item, and only the four items next to
    # the two cuts have to be compared. Binary search on i: O(log n).
    # (This also answers follow-up 3: any k-th smallest, not only the median.)
    if len(a) > len(b):
        a, b = b, a
    low = k - len(b)
    if low < 0:
        low = 0
    high = k
    if len(a) < high:
        high = len(a)
    while low <= high:
        i = (low + high) // 2
        j = k - i
        if left_border(a, i) > right_border(b, j):
            high = i - 1                    # took too many from a
        elif left_border(b, j) > right_border(a, i):
            low = i + 1                     # took too few from a
        else:
            return bigger(left_border(a, i), left_border(b, j))
    raise ValueError("k is out of range")


# ---- main question: main function ----
# Idea: binary search how many items of the shorter array go into the
# left half. The median is the middle item (odd total) or the mean of
```

```python
# the two middle items (even total).
# Complexity: Time O(log(min(m, n))), Space O(1)
def findMedianSortedArrays(nums1, nums2):
    total = len(nums1) + len(nums2)
    if total == 0:
        raise ValueError("no data")
    if total % 2 == 1:
        return kth_of_two(nums1, nums2, total // 2 + 1)
    low = kth_of_two(nums1, nums2, total // 2)
    high = kth_of_two(nums1, nums2, total // 2 + 1)
    return (low + high) / 2
```

COMPLEXITY | Time O(log(min(m, n))) | Space O(1)

96 Find longest duplicated substring — HARD

Template fit: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., **`s[i..j)`** for **`0 <= i < j <= len(s)`**. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., **`1 <= n <= 2 × 10^5`**).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive **`O(n^2)`** substring comparison approach.

Examples

- **`s = "banana"`**
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include **`"ana"`**.
- **`s = "abcd"`**
- No duplicated substring, so output should be **`""`** (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

EXAMPLES

Example 1	
Input	`'banana'`
Output	`'ana'`
Why	`'ana' occurs twice (overlapping is allowed).`

Example 2	
Input	`'abcd'`
Output	`''`
Why	`No substring repeats.`

Example 3	
Input	`'aaaaa'`
Output	`'aaaa'`
Why	`'aaaa' appears at positions 0 and 1.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
MOD = (1 << 61) - 1          # a big prime: hash values stay small
BASE = 131


def find_length(s, length):
    # is there a substring of this length that appears twice? Return one, or None.
    # Rolling hash: the hash of every window is updated in O(1) when the
    # window slides by one character.
    codes = []
    for c in s:
        codes.append(ord(c))
    power = pow(BASE, length, MOD)                    # BASE ** length, used to remove the left character
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}                                   # hash -> start positions of windows with it
    for i in range(1, len(s) - length + 1):
        h = (h * BASE - codes[i - 1] * power + codes[i + length - 1]) % MOD
        if h in seen:
            for j in seen[h]:
                if s[j:j + length] == s[i:i + length]:    # equal hashes may be a collision: compare the text
                    return s[i:i + length]
        else:
            seen[h] = []
        seen[h].append(i)
    return None


# ---- main question: main function ----
# Idea: if a substring of length L appears twice, so does every shorter
# one, so binary search the length. For each length find out whether a
# repeat exists with the rolling hash (Rabin-Karp) above.
# Complexity: Time O(n log n) expected (a rolling hash for each of log n lengths), Space O(n)
def longest_dup(s):
    low = 1
    high = len(s) - 1
    best = ""
    while low <= high:
        mid = (low + high) // 2
        found = find_length(s, mid)
        if found:
            best = found
            low = mid + 1                 # a repeat exists: try longer
        else:
            high = mid - 1
    return best
```

COMPLEXITY | Time O(n log n) expected | Space O(n)

GROUP 12 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 3 E 5 M

Greedy: scan once, carry the best choice so far, act only when forced

97 Maximum Profit — EASY

Template fit: pick the cheapest and the dearest city with a fixed tie-break (min and max once)

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

EXAMPLES

Example 1	
Input	`{'A': 3, 'B': 10, 'C': 1}`
Output	`9`
Why	`Buy in the cheapest city (C, price 1) and sell in the dearest (B, price 10): profit 9.`

Example 2	
Input	`{'A': 5}`
Output	`0`
Why	`With a single city there is nothing to trade.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def bigger(a, b):
    if a > b:
        return a
    return b


def smaller(a, b):
    if a < b:
        return a
    return b


# ---- main question: main function ----
# Idea: prices is a dictionary city -> price. Buy in the cheapest city
# and sell in the dearest one: the order of the cities does not matter,
# so the best profit is highest price - lowest price (0 with one city).
# Complexity: Time O(n), Space O(1)
```

```python
def max_profit_prices(prices):
    if len(prices) < 2:
        return 0
    lowest = None
    highest = None
    for city in prices:
        price = prices[city]
        if lowest is None or price < lowest:
            lowest = price
        if highest is None or price > highest:
            highest = price
    return highest - lowest
```

COMPLEXITY | Time O(n) | Space O(1)

98 Alarm for the Next Upcoming Birthday

EASY

Template fit: minimum over people of the circular gap (their day - today) modulo 365, ties by name

QUESTION

You maintain a list of people and their birthdays. Given today's date and a list of (name, month, day) records, return the name of the person whose birthday comes soonest starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as 0 days away (soonest). Ignore leap years; treat every date simply as a (month, day) pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Function signature

```
def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:
    ...
```

EXAMPLES

Example 1	
Input	`today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]`
Output	`"Alice"`
Why	`Alice's birthday is 2 days away; Carol's is later this year; Bob's is next January. Alice is soonest.`

Example 2	
Input	`today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]`
Output	`"Dan"`
Why	`Dan's birthday is today (0 days away), which beats Alice's tomorrow.`

Example 3	
Input	`today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]`
Output	`"Eve"`
Why	`Both birthdays already passed this year, so both are ~next-year and tie; alphabetical order picks "Eve".`

CONSTRAINTS

- 1 <= len(people) <= 10**4
- 1 <= month <= 12 , 1 <= day <= 31 (assume valid calendar dates). today is a valid (month, day) pair.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
MONTH_DAYS = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]      # a non-leap year


# Complexity: Time O(1) (at most 12 months), Space O(1)
def day_of_year(month, day):
    # 1 January is day 1
    total = day
    for m in range(month - 1):
        total += MONTH_DAYS[m]
    return total


# ---- main question: main function ----
# Idea: the number of days from today until a birthday is
# (its day of the year - today's day of the year) mod 365, so a birthday
# that already passed this year wraps around to next year. Take the
# smallest number of days; on a tie the name that comes first.
# today is (month, day); people are (name, month, day).
# Complexity: Time O(p) for p people, Space O(1)
def next_birthday(today, people):
    now = day_of_year(today[0], today[1])
    best_name = None
    best_days = None
    for name, month, day in people:
        days_away = (day_of_year(month, day) - now) % 365
        if best_days is None or days_away < best_days or (days_away == best_days and name < best_name):
            best_days = days_away
            best_name = name
    return best_name
```

COMPLEXITY | Time O(p) | Space O(1)

99 Coins on the Table

EASY

Template fit: scan the events with one counter: add on '+', subtract on '-' but never below 0

QUESTION

There is a table that starts with 0 coins on it. You are given a string events , where each character is a single event applied left to right: '+' places one coin on the table. '-' removes one coin from the table. The table can never hold a negative number of coins: a '-' event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Function signature

def coins_on_table(events: str) -> int:

...

Follow-up (as reported): Generalize so each event is an integer delta (e.g. "+3" , "-2"), still clamping the running total at 0. Hint: parse tokens instead of single characters.

EXAMPLES

Example 1	
Input	events = "+++-"
Output	2
Why	+1, +1, +1, then -1 -> 2 coins remain.

Example 2	
Input	events = "----"
Output	0
Why	Every removal happens on an empty table and is ignored.

QUESTION

Example 3	
Input	`events = "+-+-+"`
Output	`1`
Why	`1, 0, 1, 0, 1 -> 1 coin remains.`

CONSTRAINTS

- 0 <= len(events) <= 10^5
- Every character of events is either '+' or '-' .
- An empty string returns 0 .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: one counter that can never go below zero. "+" puts a coin on the
# table; "-" takes one, but only if there is one (otherwise it stays 0):
# total = max(0, total + delta) after every event.
# Complexity: Time O(n), Space O(1)
def coins_on_table(events):
    total = 0
    for event in events:
        if event == "+":
            total += 1
        elif total > 0:
            total -= 1
    return total
```

COMPLEXITY | Time O(n) | Space O(1)

100 Visible People in a Line (Last Person, Then Everyone) — MEDIUM

Template fit: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9, 10).
```

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def bigger(a, b):
    if a > b:
        return a
    return b


# ---- main question: main function ----
# Part 1: how many people can the LAST person see? Person k is seen by
# the last person when nobody standing strictly between them is taller
# than both of them. Walk from the second-last person to the front and
# keep the tallest height seen so far between us.
```

```python
# Complexity: Time O(n), Space O(1)
def last_sees(a):
    if len(a) == 0:
        return 0
    last = a[-1]
    between = float("-inf")                    # the tallest person strictly between k and the last person
    count = 0
    for k in range(len(a) - 2, -1, -1):
        if between <= bigger(last, a[k]):
            count += 1
        between = bigger(between, a[k])
    return count
```

COMPLEXITY | One pass with a running maximum (last person): Time O(n) | Space O(1)
Monotonic stack for everyone: Time O(n) | Space O(n)

101 Bounded Knapsack Variant — MEDIUM

Template fit: sort items by value, highest first; take min(available count, remaining capacity) of each

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

EXAMPLES

Example 1	
Input	`[(10, 2), (6, 3), (3, 5)], 4`
Output	`32`
Why	`Items are (value, available count), each weighs 1. Take both 10s and two 6s: 20 + 12 = 32.`

Example 2	
Input	`[(10, 2), (6, 3), (3, 5)], 7`
Output	`44`
Why	`A bigger capacity takes more items.`

MAIN CODE | helper functions first, then the answer

```python
import random


# ---- main question: main function ----
# Idea: every item weighs 1, so every slot of the bag is worth the same
# and the dearest copies should go in first. items are (value, count)
# pairs and W is the number of slots. Sort by value, highest first; for
# each type take as many copies as still fit and add value * taken.
# Complexity: Time O(t log t) for t item types, Space O(t)
def max_value_sorted(items, W):
    total = 0
    for value, count in sorted(items, reverse=True):
        take = count
        if W < take:
            take = W                           # only W slots are left
        total += value * take
        W -= take
```

```
        if W == 0:
            break
    return total


# the same answer in O(n) expected time: quickselect finds where the W-th
# slot falls without sorting everything
# Complexity: Time O(t) expected (quickselect), Space O(t)
def max_value_select(items, W):
    total = 0
    pool = list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = []
        middle = []
        low = []
        for value, count in pool:
            if value > pivot:
                high.append((value, count))
            elif value == pivot:
                middle.append((value, count))
            else:
                low.append((value, count))
        high_count = 0
        for value, count in high:
            high_count += count
        if high_count >= W:
            pool = high                          # the bag fills up inside the dearer types
```

```
            continue
        for value, count in high:
            total += value * count               # every dearer copy fits
        W -= high_count
        middle_count = 0
        for value, count in middle:
            middle_count += count
        take = middle_count
        if W < take:
            take = W
        total += pivot * take
        W -= take
        pool = low
    return total
```

COMPLEXITY | Sort by value and take greedily: Time O(n log n) | Space O(1) extra
Quickselect on the value: Time O(n) expected | Space O(n)

102 Find the Largest Monotone Increasing Number

MEDIUM

Template fit: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it

QUESTION

Given an integer n, return the largest number not greater than n whose digits never decrease from left to right (for example 1223 qualifies, 1232 does not).

Function signature

def largest_monotone(n: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	`10`
Output	`9`
Why	`10 itself has a drop (1 then 0); the largest valid number below it is 9.`

Example 2	
Input	`1234`
Output	`1234`
Why	`1234 is already monotone.`

Example 3	
Input	`332`
Output	`299`
Why	`332: lower the first 3 to 2 and fill the rest with 9s: 299.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the largest number <= n whose digits never decrease. Scan the
# digits from RIGHT to LEFT. When a digit is smaller than the one before
# it (a drop), the left digit must go down by one, and every digit after
# it can become 9 (the largest choice). Remember where the nines start.
# Complexity: Time O(d) for d digits, Space O(d)
def largest_monotone(n):
    digits = list(str(n))
    nines_from = len(digits)
    for i in range(len(digits) - 1, 0, -1):
        if digits[i - 1] > digits[i]:
            digits[i - 1] = str(int(digits[i - 1]) - 1)
            nines_from = i
    for i in range(nines_from, len(digits)):
        digits[i] = "9"
    return int("".join(digits))
```

COMPLEXITY | Time O(d) | Space O(d)

103 Min boards to cover roof holes — MEDIUM

Template fit: the template as written

QUESTION

Holes in a roof are at integer positions along a line (the same position may be listed twice). A board of the given length covers that many consecutive positions. Return the fewest boards that cover every hole.

Function signature

def min_boards(holes: list, length: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 4, 7, 8], 3`
Output	`3`
Why	`Boards cover [1..3], [4..6] and [7..9].`

QUESTION

Example 2	
Input	`[1, 10], 5`
Output	`2`
Why	`The holes are far apart: one board each.`

Example 3	
Input	`[5, 5, 5], 1`
Output	`1`
Why	`Repeated positions need only one board.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea (greedy): sort the holes. The leftmost hole that is not covered
# yet must start a new board, and the best place for it is exactly at
# that hole (starting earlier covers nothing more). That board covers
# every hole below start + length. Count the boards.
# Complexity: Time O(n log n) (the sort), Space O(n)
def min_boards(holes, length):
    count = 0
    covered_to = float("-inf")              # the last position the boards so far cover
    for position in sorted(set(holes)):
        if position > covered_to:
            count += 1
            covered_to = position + length - 1
    return count
```

COMPLEXITY | Time O(n log n) | Space O(1) extra

104 Jump Game II (Minimum Jumps) — MEDIUM

Template fit: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

QUESTION

You are given a 0-indexed array of non-negative integers nums . You start at index 0. From any index i , you can jump to any index in the range [i + 1, i + nums[i]] (i.e., nums[i] is the maximum forward jump length from i). Return the minimum number of jumps required to reach the last index len(nums)

- 1 . The test cases guarantee that the last index is always reachable.

Function signature

def jump(nums: list[int]) -> int:
"""Return the minimum number of jumps to reach the last index."""
...

Follow-up (as reported): Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

EXAMPLES

Example 1	
Input	`nums = [2, 3, 1, 1, 4]`
Output	`2`
Why	`Jump 1 step from index 0 to 1, then 3 steps to the last index.`

QUESTION

Example 2	
Input	`nums = [2, 3, 0, 1, 4]`
Output	`2`
Why	`0 -> 1 -> 4.`

Example 3	
Input	`nums = [0]`
Output	`0`
Why	`Already at the last index.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- 0 <= nums[i] <= 1000
- If the array has length 1 you are already at the last index, so the answer is 0.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea (greedy by ranges): each jump lets you reach a whole range of
# indexes. Walk through the array and remember the farthest index you
# can reach from anything seen so far. When you pass the end of the
# range of the current jump, you must take another jump, and the new
# range ends at that farthest index.
# Complexity: Time O(n), Space O(1)
def jump(nums):
    jumps = 0
    current_end = 0                     # the last index reachable with `jumps` jumps
    farthest = 0
    for i in range(len(nums) - 1):
        if i + nums[i] > farthest:
            farthest = i + nums[i]
        if i == current_end:            # the range of this jump is used up: take another
            jumps += 1
            current_end = farthest
    return jumps
```

COMPLEXITY | Time O(n) | Space O(1)

GROUP 13 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 4 E 3 M 1 H

Sets and counters: remember the first collection, check the second against it

105 Exclude First-K-Unique From Another List — EASY

Template fit: the set holds the first k distinct values of list 1; keep list 2's values that are not in it

QUESTION

You are given two lists list1 and list2 and an integer k . Scan list1 left to right and collect the first k distinct values it contains (in order of first appearance; if list1 has fewer than k distinct values, take all of them). Then return list2 with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Function signature

def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:

...

EXAMPLES

Example 1	
Input	`list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2`
Output	`[2, 4, 5]`
Why	`First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from list2 leaves [2, 4, 5].`

Example 2	
Input	`list1 = [7], list2 = [7, 8, 9], k = 3`
Output	`[8, 9]`
Why	`list1 has only one distinct value (7); remove all 7s from list2.`

CONSTRAINTS

- 0 <= len(list1), len(list2) <= 10**5 , 1 <= k .
- Values are integers.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def first_k_unique(list1, k):
    # the first k DISTINCT values of list1, as a set
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return blocked


# ---- main question: main function ----
# Take the first k distinct values of list1 and remove them from list2.
# Idea: put those values in a set; a set lookup is O(1), so filtering
# list2 takes one pass.
# Complexity: Time O(n + m), Space O(k)
def exclude_first_k_unique(list1, list2, k):
    blocked = first_k_unique(list1, k)
    result = []
    for y in list2:
        if y not in blocked:
            result.append(y)
```

```
    return result
```

COMPLEXITY | Time O(n + m) | Space O(k)

106 Find the Duplicates — EASY

Template fit: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

EXAMPLES

Example 1	
Input	[1, 3, 5, 7], [3, 4, 5, 8]
Output	[3, 5]
Why	Two sorted lists of passport numbers: 3 and 5 appear in both.

Example 2	
Input	[1, 2], [3, 4]
Output	[]
Why	Nothing in common.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def first_at_least(values, x, start):
    # binary search in a sorted list, looking only from index `start` on:
    # the first index whose value is >= x
    low = start
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def bit_length(n):
    # how many binary digits n has (roughly log2 of n)
    bits = 0
```

```
    while n > 0:
        bits += 1
        n = n // 2
    return bits


# ---- main question: main functions ----
# Find the values that appear in BOTH sorted arrays (each result once).
# CASE 1: the arrays have similar sizes: two pointers. Walk both arrays
# together and move on the pointer that points at the smaller value; when
# they are equal, record the value once and move both. O(N + M) time.
# Complexity: Time O(n + m), Space O(1) plus the answer
def common_two_pointers(a, b):
    result = []
    i = 0
    j = 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not result or result[len(result) - 1] != a[i]:
                result.append(a[i])
            i += 1
            j += 1
    return result
```

```
# CASE 2: one array is much shorter: binary-search every item of the short
# array in the long one. O(N log M); each search starts where the previous
# one stopped.
# Complexity: Time O(s log b) for the smaller list of size s and the bigger of size b, Space O(1) plus the an
swer
def common_binary_search(small, big):
    result = []
    position = 0
    for x in small:
        position = first_at_least(big, x, position)
        if position == len(big):
            break
        if big[position] == x and (not result or result[len(result) - 1] != x):
            result.append(x)
    return result


# Choose by cost: N log M against N + M.
# Complexity: Time O(min(n + m, s log b)), Space O(1) plus the answer
def common(a, b):
    if len(a) <= len(b):
        small = a
        big = b
    else:
        small = b
        big = a
    if len(small) * bit_length(len(big)) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)
```

COMPLEXITY | Case 1 (M close to N): two pointers: Time O(N + M) | Space O(1) extra
Case 2 (M much bigger than N): binary search the big array: Time O(N log M) | Space O(1) extra

107 Positional Matches Between Two Lists

EASY

Template fit: no set needed: zip both lists and keep a where a == b

QUESTION

You are given two lists of equal length, first and second . Return, in order, the values at every position where the two lists agree — that is, all first[i] such that first[i] == second[i] .

Function signature

def positional_matches(first: list[int], second: list[int]) -> list[int]:

...

EXAMPLES

Example 1	
Input	`first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]`
Output	`[5, 7]`
Why	`Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches" between a digits-of-pi list and an index list.)`

Example 2	
Input	`first = [1, 2, 3], second = [4, 5, 6]`
Output	`[]`
Why	`No position matches.`

CONSTRAINTS

- len(first) == len(second) , 0 <= length <= 10**6 .
- Values are integers.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: walk both lists together and keep the value wherever they agree at
# the same position. The shorter list decides how far to go.
# Complexity: Time O(min(n, m)), Space O(number of matches)
def positional_matches(first, second):
    matches = []
    for i in range(min(len(first), len(second))):
        if first[i] == second[i]:
            matches.append(first[i])
    return matches
```

COMPLEXITY | Time O(n) | Space O(matches)

108 Contains Duplicate

EASY

Template fit: add values to a set as you go; the first value already in the set means a duplicate

QUESTION

Given an integer array nums , return True if any value appears at least twice in the array, and False if every element is distinct.

Function signature

def contains_duplicate(nums: list[int]) -> bool:

"""Return True if any value appears more than once."""

...

EXAMPLES

QUESTION

Example 1	
Input	`nums = [1, 2, 3, 1]`
Output	`True`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`False`

Example 3	
Input	`nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
Output	`True`

CONSTRAINTS

- 1 <= len(nums) <= 10^5
- -10^9 <= nums[i] <= 10^9

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: remember the values seen so far in a set; meeting a value that is
# already in the set means a duplicate. O(n) time, O(n) memory.
# Complexity: Time O(n), Space O(n)
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

COMPLEXITY | Time O(n) | Space O(n)

109 Filter Common Elements Per Window (Two Lists) — MEDIUM

Template fit: build a set per window of the first list; keep the matching window of the second list minus that set

QUESTION

You are given two lists a and b of equal length n and a window size k that divides n . Split both lists into consecutive, non-overlapping windows of size k (window 0 is indices 0..k-1 , window 1 is k..2k-1 , and so on). For each window, remove from b 's window every value that also appears in the corresponding window of a . Concatenate the surviving elements of b , preserving their original order, and return that list.

Function signature

def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:

    ...

Follow-up (as reported): The interviewer asked to do the removal in place / O(1) extra space. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

EXAMPLES

Example 1	
Input	`a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3`
Output	`[7, 8, 9]`
Why	`Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8]. Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].`

QUESTION

Example 2	
Input	`a = [1, 1], b = [1, 2], k = 2`
Output	`[2]`
Why	`Single window: a={1}, b=[1,2] -> remove 1 -> [2].`

CONSTRAINTS

- 0 <= n <= 10**5 , n % k == 0 , 1 <= k <= n (for n > 0).
- Values are integers.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Cut both lists into windows of k elements. In every window keep only the
# values of b that do NOT appear in the same window of a.
# Idea: put the window of a into a set (a set lookup is O(1)), then keep the
# elements of the window of b that are not in it.
# Complexity: Time O(n + m), Space O(k) for one window plus the answer
def filter_windows(a, b, k):
    result = []
    for start in range(0, len(a), k):
        seen = set(a[start:start + k])
        for x in b[start:start + k]:
            if x not in seen:
                result.append(x)
    return result
```

COMPLEXITY | Time O(n) | Space O(k)

110 Reconstruct original array from doubled shuffle — MEDIUM

Template fit: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up

QUESTION

An array of numbers was doubled: for every element x of the original, the value 2x was added, and then all the numbers were shuffled. Given the shuffled array changed, return an original array (in any order), or an empty list if changed cannot be a doubled array. Zero and negative numbers are possible.

Function signature

def find_original(changed: list) -> list

EXAMPLES

Example 1	
Input	`[1, 3, 4, 2, 6, 8]`
Output	`[1, 3, 4]`
Why	`Original [1, 3, 4] gives doubles [2, 6, 8].`

Example 2	
Input	`[1, 3, 2]`
Output	`[]`
Why	`An odd number of values cannot be doubled.`

QUESTION

Example 3	
Input	`[0, 0, 0, 0]`
Output	`[0, 0]`
Why	Zeros pair with zeros.

Example 4	
Input	`[-2, -1, -4, -2]`
Output	`[-1, -2]`
Why	Negatives work too: -2 is the double of -1.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def count_values(values):
    counts = {}
    for v in values:
        counts[v] = counts.get(v, 0) + 1
    return counts


def size_then_value(v):
    # the sort key: smaller absolute value first (a negative number sorts by
    # its size, so -1 comes before 2)
    return (abs(v), v)


# ---- main question: main function ----
# `changed` holds every original x and also its double 2x, shuffled. Find
# the originals (or return [] if `changed` is not such an array).
# Idea: count the values and go through them from the SMALLEST absolute
# value: the smallest remaining value can only be an original, never someone's
# double. Every copy of x needs its own copy of 2x: if there are fewer copies of
# 2x, fail; otherwise use them up and write x down that many times. Zero is its
# own double, so zeros pair with each other. (Going by absolute value also makes
# negatives work: -2 is the double of -1, and |-1| < |-2|: follow-up 3.)
# Complexity: Time O(n log n) (the sort of distinct values), Space O(n)
def find_original(changed):
    if len(changed) % 2 == 1:
        return []
    count = count_values(changed)
    originals = []
    for x in sorted(count, key=size_then_value):
        if count[x] == 0:
            continue                                # all copies of x were used as doubles
        if x == 0:
            if count[0] % 2 == 1:
                return []
            for i in range(count[0] // 2):
                originals.append(0)
            count[0] = 0
            continue
        if count.get(2 * x, 0) < count[x]:
            return []                               # not enough doubles for every x
        for i in range(count[x]):
            originals.append(x)
        count[2 * x] -= count[x]
        count[x] = 0
    return originals
```

COMPLEXITY | Time O(n log n) | Space O(n)

111 Min deletions to avoid overlap in first k

MEDIUM

Template fit: Counter of the left part and of the right part; for each value in both, delete min(left count, right count)

QUESTION

Split an array into its first k elements and the rest. Return the minimum number of elements to delete so that no value appears on both sides (for every shared value delete all copies from the side that has fewer).

Function signature

def min_deletions(a: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 2, 3, 2, 1], 3`
Output	`2`
Why	`Left [1, 2, 2], right [3, 2, 1]: value 1 appears once on each side (delete 1), value 2 appears twice on the left and once on the right (delete 1): 2 deletions.`

Example 2	
Input	`[1, 2, 3, 4], 2`
Output	`0`
Why	`No value is shared between [1, 2] and [3, 4].`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def count_values(values):
    counts = {}
    for v in values:
        counts[v] = counts.get(v, 0) + 1
    return counts


# ---- main question: main function ----
# The values in a[:k] and in a[k:] must not share any value. Delete as few
# elements as possible.
# Idea: count every value in the first part and in the rest. For a value
# that occurs on BOTH sides, one side has to lose ALL of its copies, so delete
# the copies on the side where there are fewer. Add that up over the shared values.
# Complexity: Time O(n), Space O(n)
def min_deletions(a, k):
    left = count_values(a[:k])
    right = count_values(a[k:])
    total = 0
    for v in left:
        if v in right:
            if left[v] < right[v]:
                total += left[v]
            else:
                total += right[v]
    return total


# which indices to delete (either side may lose elements)
# Complexity: Time O(n log n), Space O(n)
def deletions_plan(a, k):
    left = count_values(a[:k])
    right = count_values(a[k:])
    indices = []
```

```
    for v in left:
        if v in right:
            if left[v] <= right[v]:
                side = range(k)                    # the first part has fewer copies: delete those
            else:
                side = range(k, len(a))
            for i in side:
                if a[i] == v:
                    indices.append(i)
    indices.sort()
    return indices
```

COMPLEXITY | Time O(n) | Space O(n)

112 Smallest Missing Positive Integer in Linear Time and Constant Extra Space HARD

Template fit: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main functions ----
# Find the smallest positive integer that is missing from the array.
# Idea (cyclic placement): the answer is between 1 and n + 1, so only
# values in 1..n matter. Put each value v at index v - 1 by swapping: for
# each position, while the value there is in 1..n and not yet at its home
# index, swap it home. Every swap puts one value in its final place, so there
# are at most n swaps in total: O(n) time, O(1) extra space. Then the first
# index that does not hold index + 1 gives the answer.
# Complexity: Time O(n) (every value moves at most once to its own place), Space O(1) extra (plus the copy)
def first_missing_positive(nums):
    a = list(nums)                                  # work on a copy: the input stays untouched
    n = len(a)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            home = a[i] - 1
            a[i], a[home] = a[home], a[i]
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# the easy version (say it first): O(n) extra memory
# Complexity: Time O(n), Space O(n)
def first_missing_set(nums):
    seen = set(nums)
    v = 1
    while v in seen:
        v += 1
    return v
```

COMPLEXITY | Time O(n) | Space O(1)

bisect on a sorted list: find a position, count a range, keep it sorted

113 Count People in a Postal Code Range

EASY

Template fit: sort the codes once; the answer is the template as written, never below 0

QUESTION

You are given a list of people, each as a tuple (name, postal_code) . Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range [low, high] ?" The interviewer explicitly wanted the optimal query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in O(log n) . Implement a class with: a constructor that accepts the list of people, and count_in_range(low, high) returning the number of people whose postal code is in [low, high] inclusive.

Function signature

```
from typing import List, Tuple
class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...
    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Follow-up (as reported): Also return the list of matching names. Hint: sort by (postal_code, name) and slice between bisect_left(low) and bisect_right(high) .

EXAMPLES

Example 1

Input	`people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]` `count_in_range(90000, 99999)`
Output	`2`
Why	`Alice (90210) and Cara (90211) fall in the range.`

Example 2

Input	`same people; count_in_range(10001, 30301)`
Output	`2`
Why	`Bob (10001) and Dan (30301); range is inclusive on both ends.`

Example 3

Input	`same people; count_in_range(50000, 60000)`
Output	`0`
Why	`No postal codes in that range.`

CONSTRAINTS

- 0 <= number of people <= 10^5 ; the same postal code may appear for multiple people (duplicates allowed).
- Postal codes are integers, 0 <= postal_code <= 10^9 .
- 0 <= low <= high <= 10^9 .
- Each count_in_range call must run in O(log n) after an O(n log n) preprocessing step.

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left, bisect_right


# ---- main question: main function ----
# Idea: keep only the postal codes, sorted. The codes in the range
# [low, high] form one block of the sorted list: bisect_left finds where
# it starts, bisect_right where it ends. The count is the block's size.
# people is a list of (name, postal code).
# Complexity: Time build O(n log n), each query O(log n), Space O(n)
class PostalRegistry:
    def __init__(self, people):
        self.codes = []
        for name, code in people:
            self.codes.append(code)
        self.codes.sort()

    def count_in_range(self, low, high):
        count = bisect_right(self.codes, high) - bisect_left(self.codes, low)
        if count < 0:
            return 0                        # low is bigger than high
        return count
```

COMPLEXITY | Time O(log n) per query | Space O(n)

114 Sliding-Window Mean After Removing the Largest K Values

MEDIUM

Template fit: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region

QUESTION

For every window of a fixed size over the array, remove the k largest values of the window (repeated values count separately) and compute the mean of the rest. Return the means in window order as reduced fractions written 'numerator/denominator'. Update the windows incrementally instead of sorting each one again.

Function signature

def trimmed_window_means(nums: list, window: int, k: int) -> list

EXAMPLES

Example 1	
Input	[1, 2, 3, 4, 5], 3, 1
Output	['3/2', '5/2', '7/2']
Why	Window [1,2,3] without its largest value 3 averages (1+2)/2 = 3/2; then 5/2 and 7/2.

Example 2	
Input	[2, 4, 6, 8], 2, 0
Output	['3/1', '5/1', '7/1']
Why	With k = 0 nothing is removed: plain window means.

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left, bisect_right
from fractions import Fraction


# ---- main question: helper functions ----
def as_text(fraction):
    # a reduced fraction as text: Fraction(5, 2) -> "5/2"
    return str(fraction.numerator) + "/" + str(fraction.denominator)
```

```
# Complexity: Time insert and remove O(w) (a binary search plus a list shift), Space O(w)
class Window:
    # the values of the current window in a SORTED list, with three sums kept
    # up to date while values come and go: the whole sum (total), the sum of
    # the k largest values (top) and the sum of the k smallest values (bottom)
    def __init__(self, k):
        self.k = k
        self.values = []
        self.total = 0
        self.top = 0
        self.bottom = 0

    def insert(self, x):
        k = self.k
        values = self.values
        old_length = len(values)
        place = bisect_right(values, x)
        values.insert(place, x)
        self.total += x
        # the top region is the last k places of the new list
        if old_length < k:
            self.top += x                                   # the list is still short: everything is "top"
        elif place >= old_length + 1 - k:
            self.top += x - values[old_length - k]          # x entered it, the value just below it left
        # the bottom region is the first k places
        if old_length < k:
            self.bottom += x
        elif place < k:
            self.bottom += x - values[k]                    # x entered it, the value now at index k left

    def remove(self, x):
        k = self.k
        values = self.values
        old_length = len(values)
        place = bisect_left(values, x)
```

```
        self.total -= x
        if old_length <= k:
            self.top -= x
            self.bottom -= x
        else:
            if place >= old_length - k:
                self.top += values[old_length - k - 1] - x     # the value just below the region joins it
            if place < k:
                self.bottom += values[k] - x                   # the value at index k joins the region
        del values[place]


# ---- main question: main function ----
# Idea: for every window of the given size, drop its k largest values
# and take the mean of the rest. Keep the window in a sorted list and keep
# the sum of its k largest values up to date while one value enters and
# one leaves, so no window is sorted again. With both=True the k smallest
# are dropped as well (a trimmed mean, follow-up 4). Each mean is
# returned as an exact reduced fraction in text, like "5/2".
# Complexity: Time O(n * w) for windows of size w (each step is one insert and one remove), Space O(w)
def trimmed_window_means(nums, window, k, both=False):
    w = Window(k)
    means = []
    for i in range(len(nums)):
        w.insert(nums[i])
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - k
            dropped = w.top
```

```
            if both:
                kept = window - 2 * k
                dropped = w.top + w.bottom
            means.append(as_text(Fraction(w.total - dropped, kept)))
    return means
```

COMPLEXITY | Time O(n * (log w + w) in memmove, O(n log w) with a balanced tree) | Space O(w)

115 Russian Doll Envelopes — MEDIUM

Template fit: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left


# ---- main question: helper functions ----
def width_then_taller_first(envelope):
    # sort key: width ascending; for EQUAL widths the taller one first
    return (envelope[0], -envelope[1])


# ---- main question: main function ----
# Idea: sort by (width ascending, height descending), then find the
# longest strictly increasing run of heights with a binary search.
# tails[k] is the smallest possible last height of a chain of k + 1
# envelopes; each new height either extends the longest chain or lowers
# one of the tails.
# Complexity: Time O(n log n), Space O(n)
def max_envelopes(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    for width, height in ordered:
        i = bisect_left(tails, height)
        if i == len(tails):
            tails.append(height)
        else:
            tails[i] = height
    return len(tails)
```

COMPLEXITY | Time O(n log n) | Space O(n)

116 Count Sorted Strings With a Given Prefix

MEDIUM

Template fit: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo

QUESTION

Given a lexicographically sorted array of strings words and a query string prefix , return the number of strings in words that start with prefix . Because words is sorted, all matching strings occupy a contiguous range; find that range with binary search in O(log n * L) time (where L is the prefix length), not by scanning.

Function signature

def count_with_prefix(words: list[str], prefix: str) -> int:

    ...

Follow-up (as reported): Many repeated queries: cache counts in a hash map keyed by prefix. All queries unique: build a trie of all words storing subtree sizes, so each query is O(L) .

EXAMPLES

Example 1	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "app"`
Output	`3`
Why	`"app", "apple", "application" all start with "app".`

Example 2	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"`
Output	`2`
Why	`"banana" and "bat".`

Example 3	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"`
Output	`0`
Why	`No string starts with "cat".`

CONSTRAINTS

- 0 <= len(words) <= 10**6 , sorted in non-decreasing lexicographic order.
- 0 <= len(prefix) <= 100 . An empty prefix matches every string.

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left


# ---- main question: main function ----
# Idea: in a sorted list the words that start with the prefix form ONE
# block. It starts at the first word >= prefix and ends just before the
# first word >= prefix followed by the biggest possible character. Two
# binary searches find both ends; the answer is the size of the block.
# Complexity: Time O(L log n) for a prefix of length L (each comparison costs up to L), Space O(1)
def count_with_prefix(words, prefix):
    start = bisect_left(words, prefix)
    end = bisect_left(words, prefix + "\uffff")
    return end - start
```

COMPLEXITY | Time O(L log n) | Space O(1)

117 Streaming Insert + Find K-th Largest

MEDIUM

Template fit: insort every new value; the k-th largest is sorted_list[-1 - k]

QUESTION

Design a structure with insert(x), which adds a number, and findLargest(k), which returns the k-th largest number inserted so far (k = 0 is the largest), counting repeated values separately, or None when fewer than k + 1 numbers exist.

Function signature

class StreamKth:
  def insert(self, x: float) -> None
  def findLargest(self, k: int) -> float | None

EXAMPLES

Example 1

Input	**s = StreamKth()** **s.insert(3)** **s.insert(3)** **s.insert(2)** **s.findLargest(0)** **s.findLargest(1)** **s.findLargest(2)** **s.findLargest(3)**
Output	**s.findLargest(0) -> 3** **s.findLargest(1) -> 3** **s.findLargest(2) -> 2** **s.findLargest(3) -> null**
Why	**Sorted descending the numbers are 3, 3, 2, so the 0th, 1st and 2nd largest are 3, 3 and 2, and there is no 3rd.**

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left, insort


# ---- main question: main function ----
# Idea: keep every value in an ascending list. insort puts a new value
# in its place with a binary search (O(log n) to find the spot, O(n) to
# shift the others). The k-th largest (k = 0 is the largest, as in the
# examples) is then simply a[-1 - k].
# Complexity: Time insert O(n) (a binary search plus a list shift), delete O(n), findLargest O(1), Space O(n)
class StreamKth:
    def __init__(self):
        self.a = []

    def insert(self, x):
        insort(self.a, x)

    def findLargest(self, k):
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None

    # follow-ups 1 and 3: values can also be deleted. A sorted list can
    # delete: find a copy of x with a binary search and remove it.
    def delete(self, x):
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False
```

COMPLEXITY | Sorted list with bisect (k chosen per query): Time O(log n) search + O(n) shift per insert, O(1) query | Space O(n)
Fixed k: min-heap of the k largest: Time O(log k) per insert, O(1) query | Space O(k)

118 Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

MEDIUM

Template fit: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

QUESTION

Kids and ice cream sellers stand at positions on a line. Find the smallest distance E such that every kid is within E of at least one seller (the kid at position K is served by a seller at P when P - E <= K <= P + E).

Function signature

def smallest_e(kids: list, sellers: list) -> int

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 8]`
Output	`3`
Why	`The kid at 5 is 3 away from both sellers (at 2 and 8); the others are 1 away.`

Example 2	
Input	`[4], [4]`
Output	`0`
Why	`A kid standing on a seller needs distance 0.`

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left


# ---- main question: helper functions ----
def distance_to_nearest(sorted_sellers, kid):
    # a binary search finds where the kid would stand among the sellers; the
    # nearest seller is the one right at or after that spot, or the one before it
    i = bisect_left(sorted_sellers, kid)
    nearest = float("inf")
    if i < len(sorted_sellers):
        nearest = sorted_sellers[i] - kid
    if i > 0 and kid - sorted_sellers[i - 1] < nearest:
        nearest = kid - sorted_sellers[i - 1]
    return nearest


# ---- main question: main function ----
# Idea: every kid should go to the nearest seller, so E is the WORST such
# distance over all kids. Sort the sellers once and binary search each kid.
# Complexity: Time O((n + m) log m), Space O(m)
def smallest_e(kids, sellers):
    sellers = sorted(sellers)
    worst = 0
    for kid in kids:
        d = distance_to_nearest(sellers, kid)
        if d > worst:
            worst = d
    return worst
```

COMPLEXITY | Time O((N + M) log M) | Space O(1) extra

119 Find Maximum Envelope Nesting

HARD

Template fit: exactly the same sort and tails list

QUESTION

Envelopes are given as (width, height). One envelope fits inside another only when both its width and its height are strictly smaller. Return the largest number of envelopes that can be nested one inside the next (Russian dolls).

Function signature

def max_envelopes(envelopes: list) -> int

EXAMPLES

Example 1	
Input	`[(5, 4), (6, 4), (6, 7), (2, 3)]`
Output	`3`
Why	`(2, 3) fits in (5, 4), which fits in (6, 7): 3 nested.`

Example 2	
Input	`[(1, 1), (1, 1), (1, 1)]`
Output	`1`
Why	`Equal envelopes cannot go inside each other.`

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left


# ---- main question: helper functions ----
def width_then_taller_first(envelope):
    # sort key: width ascending; for EQUAL widths the taller one first
    return (envelope[0], -envelope[1])


# ---- main question: main function ----
# Idea: sort by width ascending and, for equal widths, height descending.
# Now a strictly increasing run of heights is a valid nesting chain: two
# envelopes of the same width are in decreasing height order, so they can
# never both be in an increasing run. Find the longest strictly increasing
# run of heights (LIS) with a binary search in O(n log n): tails[k] is the
# smallest possible last height of a run of length k + 1.
# Complexity: Time O(n log n), Space O(n)
def max_envelopes(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    for width, height in ordered:
        i = bisect_left(tails, height)
        if i == len(tails):
            tails.append(height)           # longer than any run so far
        else:
            tails[i] = height              # a smaller ending for runs of length i + 1
    return len(tails)
```

COMPLEXITY | Time O(n log n) | Space O(n)

120 Count of Smaller Numbers After Self

HARD

Template fit: go from the right, insert each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version)

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of elements to the right of nums[i] that are strictly smaller than nums[i].

Function signature

def count_smaller_after(nums: list) -> list

EXAMPLES

Example 1	
Input	`[5, 2, 6, 1]`
Output	`[2, 1, 1, 0]`
Why	`5 has two smaller numbers to its right (2 and 1), 2 has one (1), 6 has one (1), 1 has none.`

Example 2	
Input	`[-1]`
Output	`[0]`
Why	`A single number has nothing to its right.`

Example 3	
Input	`[-1, -1]`
Output	`[0, 0]`
Why	`Equal numbers are not smaller.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Fenwick:
    # a Fenwick tree (binary indexed tree): add(i) counts one more value with
    # rank i, prefix(i) tells how many counted values have a rank below i.
    # Both take O(log size).
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i                  # jump to the next block that covers i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i                  # jump to the previous block
        return total


def ranks(nums):
```

```
    # coordinate compression: give each distinct value a rank 0, 1, 2, ... by size
    rank = {}
    for i, v in enumerate(sorted(set(nums))):
        rank[v] = i
    return rank


# ---- main question: main function ----
# Idea: compress the values to ranks and scan from the RIGHT. The answer
# for x is how many already-seen values have a smaller rank (one prefix
# query on the Fenwick tree); then x itself is added to the tree.
# Complexity: Time O(n log n), Space O(n)
def count_smaller_after(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    for x in reversed(nums):
        answers.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    answers.reverse()
    return answers
```

COMPLEXITY | Time O(n log n) | Space O(n)

GROUP 15 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 3 E 5 M

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

121 Parity Alternating Subarray Query

EASY

Template fit: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0

QUESTION

Given an array nums, answer many queries alternates(l, r): do the elements nums[l], ..., nums[r] (both ends included) alternate in parity (odd, even, odd, ... or even, odd, even, ...)? Preprocess the array once so that every query takes O(1).

Function signature

class ParityQueries:
  def __init__(self, nums: list)
  def alternates(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`p = ParityQueries([1, 2, 3, 5, 4])`
	`p.alternates(0, 2)`
	`p.alternates(0, 3)`
	`p.alternates(3, 4)`

Output	`p.alternates(0, 2) -> true`
	`p.alternates(0, 3) -> false`
	`p.alternates(3, 4) -> true`

Why	`1, 2, 3 alternates; the neighbours 3 and 5 are both odd, so any range containing both fails; 5, 4 alternates.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main class ----
# Does the subarray nums[l..r] alternate between even and odd numbers?
# Idea: write down, for the first i neighbouring pairs, how many pairs have
# the SAME parity (bad[i]). The pairs inside nums[l..r] are the pairs
# l, l+1, ..., r-1, so the subarray alternates when bad[r] - bad[l] == 0.
# Building is O(n); every query is O(1).
# Complexity: Time build O(n), each query O(1), Space O(n)
class ParityQueries:
    def __init__(self, nums):
        self.bad = [0]
        for i in range(len(nums) - 1):
            same = 0
            if nums[i] % 2 == nums[i + 1] % 2:
                same = 1
            self.bad.append(self.bad[len(self.bad) - 1] + same)

    def alternates(self, l, r):
        return self.bad[r] - self.bad[l] == 0
```

COMPLEXITY | Time O(n) to build, O(1) per query | Space O(n)

122 Cake Balance Line

EASY

Template fit: the same balance test on the cake heights

QUESTION

A cake is described by a list heights , where heights[i] is the amount of cake sitting at position i (positions are left to right). You want to place a single vertical cut exactly at some position i so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position i itself is consumed by the cut and counts for neither side). Return the smallest index i where sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no such position exists.

Function signature

def cake_balance_index(heights: list[int]) -> int:

...

Follow-up (as reported): Do it in one pass and O(1) extra space by tracking a running left sum against total - left - heights[i] .

EXAMPLES

Example 1	
Input	`heights = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.`

Example 2	
Input	`heights = [1, 2, 3]`
Output	`-1`
Why	`No cut position balances the two sides.`

Example 3	
Input	`heights = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is empty (0); right is 1+(-1)=0.`

CONSTRAINTS

- 1 <= len(heights) <= 10^5 .
- -10^4 <= heights[i] <= 10^4 (heights may be negative in this abstracted version).

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Find a column of the cake whose left side weighs as much as its right
# side. Idea: the right side is total - left - heights[i], so keep a
# running left sum and compare in one pass.
# Complexity: Time O(n), Space O(1)
def cake_balance_index(heights):
    total = 0
    for h in heights:
        total += h
    left = 0
    for i in range(len(heights)):
        if left == total - left - heights[i]:
            return i
        left += heights[i]
    return -1
```

COMPLEXITY | Time O(n) | Space O(1)

123 Find the Pivot Index (Balanced Split) in O(n) EASY

Template fit: the template as written: left total equals total - left - x

QUESTION

Given an integer array nums , return the leftmost index i such that the sum of all elements strictly to the left of i equals the sum of all elements strictly to the right of i . The element at i itself is not counted on either side. For the first index, the left sum is 0 ; for the last index, the right sum is 0 . If no such index exists, return -1 . Your solution must run in O(n) time and O(1) extra space.

Function signature

def pivot_index(nums: list[int]) -> int:

...

Follow-up (as reported): If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

EXAMPLES

Example 1	
Input	`nums = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.`

Example 2	
Input	`nums = [1, 2, 3]`
Output	`-1`
Why	`No index balances the two sides.`

Example 3	
Input	`nums = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is 0; right is 1 + (-1) = 0.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -1000 <= nums[i] <= 1000

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: the sum on the right of index i is total - left - nums[i], so one
# pass with a running left sum is enough.
# Complexity: Time O(n), Space O(1)
def pivot_index(nums):
    total = 0
    for x in nums:
        total += x
    left = 0
    for i in range(len(nums)):
        if left == total - left - nums[i]:
            return i
        left += nums[i]
    return -1
```

COMPLEXITY | Time O(n) | Space O(1)

124 Range Duplicate Check with Constant-Time Queries

MEDIUM

Template fit: next[i] = next occurrence of the same value; reach[i] = smallest next over i..end; the range l..r is all distinct when reach[l] > r

QUESTION

Preprocess an array once so that each query unique(l, r) answers in O(1) whether all elements from index l to index r (inclusive) are different.

Function signature

class RangeUnique:
  def __init__(self, a: list)
  def unique(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`ru = RangeUnique([1, 2, 3, 1, 4])`
	`ru.unique(0, 2)`
	`ru.unique(0, 3)`
	`ru.unique(1, 4)`

Output	`ru.unique(0, 2) -> true`
	`ru.unique(0, 3) -> false`
	`ru.unique(1, 4) -> true`

Why	`Indexes 0..3 hold 1, 2, 3, 1: the value 1 repeats. Ranges 0..2 and 1..4 have no repeats.`

MAIN CODE | helper functions first, then the answer

```
INF = float("inf")


# ---- main question: helper functions ----
def unique_with_set(a, l, r):
    # the easy version (say it first): put the range in a set
    seen = set()
    for i in range(l, r + 1):
        seen.add(a[i])
    return len(seen) == r - l + 1


def longest_unique_window(a):
    # stage 2: a sliding window with a count of every value in it
    count = {}
    left = 0
    best = 0
    for right in range(len(a)):
        count[a[right]] = count.get(a[right], 0) + 1
        while count[a[right]] > 1:              # shrink until the new value occurs only once
            count[a[left]] -= 1
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: main function ----
# Idea: for every index i find next[i], the next index holding the same
# value (infinity if none): scan from the right with a dictionary of the
# latest index of every value. Then reach[i] = the earliest "next repeat"
# among indices i, i+1, ... (a minimum from the right). The range [l, r]
# has no duplicate exactly when reach[l] > r. Build O(n), query O(1).
# The array can also change (follow-up 1): the simple answer is to rebuild
# in O(n) when reads dominate.
```

```python
# Complexity: Time build O(n), each query O(1), update O(n) (it rebuilds), Space O(n)
class RangeUnique:
    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a = self.a
        n = len(a)
        next_same = [INF] * n
```

```python
        latest = {}
        for i in range(n - 1, -1, -1):
            if a[i] in latest:
                next_same[i] = latest[a[i]]
            latest[a[i]] = i
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            if next_same[i] < self.reach[i + 1]:
                self.reach[i] = next_same[i]
            else:
                self.reach[i] = self.reach[i + 1]

    def unique(self, l, r):
        return self.reach[l] > r

    def update(self, i, value):
        # follow-up 1: reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()
```

COMPLEXITY | Time O(n) preprocessing, O(1) per query | Space O(n)

125 Find a Horizontal Cut That Bisects Rectangle Area

MEDIUM

Template fit: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip

QUESTION

Given many disjoint axis-aligned rectangles (x1, y1, x2, y2), return the y of the horizontal line that splits their combined area in half, so the area below the line equals the area above it. Return an exact fraction (no floating-point error).

Function signature

def bisecting_cut(rects: list) -> Fraction

EXAMPLES

Example 1	
Input	`[(0, 0, 2, 2)]`
Output	`1`
Why	`One 2x2 square: the middle line is y = 1.`

Example 2	
Input	`[(0, 0, 1, 1), (0, 2, 1, 4)]`
Output	`5/2`
Why	`Areas 1 and 2, total 3, so half is 1.5: all of the first rectangle (area 1) plus 0.5 of the second, which starts at y = 2: y = 5/2.`

MAIN CODE | helper functions first, then the answer

```python
from fractions import Fraction


# ---- main question: helper functions ----
# Rectangles are disjoint and written (x1, y1, x2, y2).
def total_area(rects):
    total = 0
    for x1, y1, x2, y2 in rects:
        total += (x2 - x1) * (y2 - y1)
    return total


def area_below(rects, y):
    # the exact area under the horizontal line at height y (used to check the answer)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
            total += (x2 - x1) * (top - y1)
    return total


# ---- main question: main function ----
# Find the height of a horizontal line with exactly half of the total
# area below it.
# Idea: sweep upward through the heights where a rectangle starts (its
# width is added) or ends (its width is removed). Between two neighbouring
# event heights the covered width is constant, so the area grows linearly
# there. Find the strip where the running area reaches the wanted amount
# and interpolate inside it. `share` is the fraction of the area that
# must be below the cut (follow-up 4); exact Fractions are used, never floats.
# Complexity: Time O(R log R) for R rectangles (the sort of heights), Space O(R)
def cut_height(rects, share=Fraction(1, 2)):
    wanted = share * total_area(rects)
    width_change = {}                                  # height -> width gained when the sweep passes it
    for x1, y1, x2, y2 in rects:
        width_change[y1] = width_change.get(y1, 0) + (x2 - x1)
        width_change[y2] = width_change.get(y2, 0) - (x2 - x1)
    heights = sorted(width_change)
    width = 0
    so_far = Fraction(0)
    for i in range(len(heights) - 1):
        low = heights[i]
        high = heights[i + 1]
        width += width_change[low]                     # inside this strip the covered width is constant
        gain = width * (high - low)
        if width > 0 and so_far + gain >= wanted:
            return low + (wanted - so_far) / width     # the cut is inside this strip: interpolate
        so_far += gain
    return Fraction(heights[len(heights) - 1])
```

COMPLEXITY | Time O(R log R) | Space O(R)

126 Check pivot after removing one element

MEDIUM

Template fit: prefix sums plus first/last position maps: look up the value whose removal would balance both sides

QUESTION

A pivot index of an array is an index whose left-side sum equals its right-side sum (the pivot element itself is excluded). Remove exactly one element from the array, then ask whether the remaining array has a pivot index. Return one pair (removed index, pivot index), both as indexes in the original array, or None if no removal creates a pivot. Negative numbers and duplicates are allowed.

QUESTION

Function signature

def pivot_after_removal(a: list) -> tuple | None

EXAMPLES

Example 1	
Input	`[2, 1, -1, 3]`
Output	`(0, 3)`
Why	Removing the 2 at index 0 leaves [1, -1, 3]; the 3 (original index 3) has left sum 1 + (-1) = 0 and nothing on its right.

Example 2	
Input	`[1, 2, 3]`
Output	`null`
Why	Whichever element is removed, no pivot appears.

Example 3	
Input	`[0, 0]`
Output	`(0, 1)`
Why	Remove either 0: the other one is a pivot of an array with one element.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Remove ONE element of the array, then is there a pivot: an element
# whose left sum equals its right sum? Return (removed index, pivot index)
# or None.
# Idea: let `prefix[j]` be the sum before index j and key[j] =
# 2 * prefix[j] + a[j].
#  - Pivot j AFTER the removed index i: the left sum is prefix[j] - a[i]
#    and the right sum is total - prefix[j] - a[j]; equal means
#    a[i] = key[j] - total. Scan j from the left, remembering the earlier values.
#  - Pivot j BEFORE the removed index i: left = prefix[j], right = total -
#    prefix[j] - a[j] - a[i]; equal means a[i] = total - key[j]. Scan j
#    from the right, remembering the later values.
# Two scans with a dictionary (value -> index): O(n).
# Complexity: Time O(n), Space O(n)
def pivot_after_removal(a):
    n = len(a)
    total = 0
    for x in a:
        total += x
    prefix = [0]
    for x in a:
        prefix.append(prefix[len(prefix) - 1] + x)
    key = []
    for j in range(n):
        key.append(2 * prefix[j] + a[j])
    index_of = {}                                   # a value seen EARLIER -> its index
    for j in range(n):
        i = index_of.get(key[j] - total)
        if i is not None:
            return i, j
        index_of[a[j]] = j
    index_of = {}                                   # a value seen LATER -> its index
    for j in range(n - 1, -1, -1):
        i = index_of.get(total - key[j])
        if i is not None:
            return i, j
```

```
            index_of[a[j]] = j
    return None
```

COMPLEXITY | Time O(n) | Space O(n)

127 Gold Chain Split into Two Equal Halves

MEDIUM

Template fit: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link

QUESTION

A gold chain is an array of segment weights. Remove exactly one segment, reconnect the rest, then cut the chain once so that the two parts have equal total weight. Return whether this is possible for some choice of the removed segment and the cut.

Function signature

def gold_chain(a: list) -> bool

EXAMPLES

Example 1	
Input	[1, 2, 3, 4]
Output	true
Why	Remove the 4: [1, 2, 3] splits as [1, 2] and [3], both 3.

Example 2	
Input	[1, 1, 5]
Output	true
Why	Remove the 5: [1, 1] splits into 1 and 1.

Example 3	
Input	[1, 2, 4]
Output	false
Why	Removing 1 leaves 6 (halves of 3 need [2] | [4]: no), removing 2 leaves 5, removing 4 leaves 3: nothing works.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def prefix_sums(a):
    # P[j] = the sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[len(P) - 1] + x)
    return P


# ---- main question: main function ----
# A chain of gold segments with weights a. Remove exactly ONE segment,
# then cut the rest once into two non-empty parts of equal weight. Can it
# be done?
# Idea: with prefix sums P (P[j] = the first j segments) removing segment r
# leaves total - a[r]; each half must weigh half of that. A cut BEFORE the
# removed segment (position j <= r) needs P[j] == half. A cut AFTER it
# (j > r) needs P[j] - a[r] == half. Instead of trying every cut, look up the
# FIRST position with a given prefix sum (for cuts before r) and the LAST
# position (for cuts after r) in two dictionaries: O(n) in total.
# Complexity: Time O(n), Space O(n)
def gold_chain(a):
```

```
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[n]
    first = {}                              # prefix sum -> its first cut position
    last = {}                               # prefix sum -> its last cut position
    for j in range(1, n):                   # cuts sit between segments: positions 1 .. n - 1
        if P[j] not in first:
            first[P[j]] = j
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2 == 1:
            continue
        half = left_over // 2
        latest_cut_before = r               # the cut must leave a piece on the right, so j <= n - 2
        if n - 2 < latest_cut_before:
            latest_cut_before = n - 2
        if half in first and first[half] <= latest_cut_before:
            return True                     # cut before the removed segment
        if half + a[r] in last and last[half + a[r]] > r:
            return True                     # cut after it
    return False
```

COMPLEXITY | Time O(n) | Space O(n)

128 Count the number of ways to split a string. MEDIUM

Template fit: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

QUESTION

A binary string s is split into three non-empty parts by two cuts. Count the ways in which the three parts contain the same number of 1s, and return the count modulo 1,000,000,007.

Function signature

def numWays(s: str) -> int

EXAMPLES

Example 1	
Input	'10101'
Output	4
Why	Four ways to split '10101' so each part has one 1 (for example 1|010|1 and 10|10|1).

Example 2	
Input	'1001'
Output	0
Why	Two 1s cannot be divided into three equal groups.

Example 3	
Input	'0000'
Output	3
Why	No 1s: any two cuts work: choose 2 of the 3 gaps = 3 ways.

MAIN CODE | helper functions first, then the answer

```python
MOD = 10 ** 9 + 7


# ---- main question: helper functions ----
def choose(n, k):
    # the number of ways to pick k things out of n
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


# ---- main question: main function ----
# Cut a binary string into `parts` non-empty pieces (3 in the classic
# question, LeetCode 1573) so that every piece has the same number of 1s.
# Count the ways.
# Idea: let `ones` be the positions of the 1s. If their number is not
# divisible by `parts` the answer is 0. With no 1s at all, any choice of
# parts - 1 cut positions among the n - 1 gaps works: choose(n - 1, parts - 1).
# Otherwise each piece holds `each` ones, and the cut after piece k can go
# anywhere in the run of zeros between the last 1 of that piece and the first
# 1 of the next: (the distance between those two 1s) choices. Multiply the
# choices of all cuts. (parts = m is follow-up 1.)
# For huge answers the result is kept modulo 10^9 + 7 after every
# multiplication (follow-up 3); mod=None gives the exact number.
# Complexity: Time O(n), Space O(n)
def numWays(s, parts=3, mod=MOD):
    n = len(s)
    if n < parts:
        return 0
    ones = []
    for i in range(n):
        if s[i] == "1":
            ones.append(i)
    if len(ones) % parts != 0:
        return 0
    if len(ones) == 0:
        total = choose(n - 1, parts - 1)
        if mod:
            return total % mod
        return total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        gap = ones[cut * each] - ones[cut * each - 1]      # where the cut may go
        answer *= gap
        if mod:
            answer %= mod
    return answer
```

COMPLEXITY | Time O(n) | Space O(n)

Sorted intervals: merge while they touch, then read the gaps or search

129 Check an Interval Against Existing Intervals for Conflicts — EASY

Template fit: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end

QUESTION

Design a calendar of half-open intervals [start, end). book(start, end) books the interval and returns True when it does not overlap any interval booked so far, and returns False (booking nothing) when it does. Intervals that only touch, one ending exactly where the next starts, do not conflict.

Function signature

class Calendar:
    def book(self, start: int, end: int) -> bool

EXAMPLES

Example 1

Input	`c = Calendar()`
	`c.book(10, 20)`
	`c.book(15, 25)`
	`c.book(20, 30)`
	`c.book(5, 10)`
	`c.book(12, 14)`

Output	`c.book(10, 20) -> true`
	`c.book(15, 25) -> false`
	`c.book(20, 30) -> true`
	`c.book(5, 10) -> true`
	`c.book(12, 14) -> false`

Why	`[15, 25) overlaps [10, 20) and [12, 14) lies inside it, so both fail. [20, 30) and [5, 10) only touch the booked interval at an endpoint, so they succeed.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(n), Space O(1)
def conflicts(candidate, intervals):
    # the simple check against every booking: half-open [start, end), so
    # touching ends do NOT conflict
    start, end = candidate
    for a, b in intervals:
        if a < end and start < b:
            return True
    return False
```

```python
# ---- main question: main class ----
# book(start, end) says whether the interval can be booked without
# overlapping an existing booking (and books it if so). Half-open intervals
# [start, end): a booking may start exactly when another ends.
# Idea: keep the bookings sorted by start. Then only the TWO neighbours of
# the new interval can conflict: the booking just before (it must end by the
# new start) and the booking just after (it must start at or after the new
# end). One binary search finds the place: O(log n) per check.
# Complexity: Time book is O(log n) to search plus O(n) for the list insert, Space O(n)
class Calendar:
    def __init__(self):
        self.starts = []
        self.ends = []

    def book(self, start, end):
        i = first_greater(self.starts, start)            # the new booking would go at position i
        if i > 0 and self.ends[i - 1] > start:
            return False                                 # the booking before runs into it
        if i < len(self.starts) and self.starts[i] < end:
            return False                                 # the booking after starts too early
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True
```

COMPLEXITY | Time O(log n) search, O(n) list insert | Space O(n)

130 Find Containing Range

MEDIUM

Template fit: merge once, keep the starts in a list, and bisect to find the one interval that could contain x

QUESTION

Given a collection of inclusive ranges [start, end] (overlapping or touching ranges are merged), find(x) returns the merged range that contains the number x, or None when no range contains it.

Function signature

class RangeIndex:
  def __init__(self, ranges: list)
  def find(self, x: int) -> tuple | None

EXAMPLES

Example 1

Input	`r = RangeIndex([(1, 3), (2, 6), (8, 10)])`
	`r.find(5)`
	`r.find(7)`
	`r.find(10)`

Output	`r.find(5) -> (1, 6)`
	`r.find(7) -> null`
	`r.find(10) -> (8, 10)`

Why	`[1, 3] and [2, 6] overlap and merge into [1, 6]; 7 falls in the gap between the merged ranges.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
```

```
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Given ranges [start, end] (closed), find(x) returns the MERGED range that
# contains x, or None.
# Idea: merge the ranges once (insert them one by one; an insert replaces
# every range it touches by one merged range). After that the merged ranges
# are sorted and disjoint, so the only candidate for x is the range with the
# largest start <= x (one binary search), and it contains x if its end is at
# least x. The merging is paid once.
# Complexity: Time insert O(log n) to search plus O(n) to splice, find O(log n), find_original O(n), find_man
y O(Q log Q + n), Space O(n)
class RangeIndex:
    def __init__(self, ranges=()):
        self.starts = []
        self.ends = []
        self.original = sorted(ranges)
        for start, end in self.original:
            self.insert(start, end)
```

```
    def insert(self, start, end):
        # follow-up 2: ranges can be inserted over time
        i = first_at_least(self.ends, start)              # the first range ending at or after the new start
 ...
        j = first_greater(self.starts, end)               # ... up to the last range starting at or before t
he new end
        if i < j:
            if self.starts[i] < start:
                start = self.starts[i]
            if self.ends[j - 1] > end:
                end = self.ends[j - 1]
        self.starts[i:j] = [start]                        # replace everything the new range touches
        self.ends[i:j] = [end]

    def find(self, x):
        i = first_greater(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return (self.starts[i], self.ends[i])
        return None

    def find_original(self, x):
        # follow-up 1: every ORIGINAL range that contains x (only those starting at or before x can)
        found = []
```

```
        for start, end in self.original:
            if start > x:
                break
            if end >= x:
                found.append((start, end))
        return found

    def find_many(self, queries):
        # follow-up 3: queries in bulk: sort the queries and sweep once along the ranges
        order = []
        for i in range(len(queries)):
            order.append((queries[i], i))
        order.sort()
        answers = [None] * len(queries)
        j = 0
        for x, i in order:
            while j < len(self.ends) and self.ends[j] < x:
                j += 1                                  # skip ranges that end before x
            if j < len(self.starts) and self.starts[j] <= x:
                answers[i] = (self.starts[j], self.ends[j])
        return answers
```

COMPLEXITY | Time O(n log n) to build, O(log n) per query | Space O(n)

131 Union of Sorted Interval Lists — MEDIUM

Template fit: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test

QUESTION

Two lists of intervals are each sorted and non-overlapping on their own. Return the union of both lists as sorted, non-overlapping intervals. Intervals that touch, such as (1, 3) and (3, 5), are merged unless touching=False.

Function signature

def union_two(a: list, b: list, touching=True) -> list

EXAMPLES

Example 1	
Input	[(1, 3), (6, 9)], [(2, 5), (10, 12)]
Output	[(1, 5), (6, 9), (10, 12)]
Why	(1, 3) and (2, 5) overlap and become (1, 5); (6, 9) and (10, 12) stay.

Example 2	
Input	[(1, 3)], [(3, 5)]
Output	[(1, 5)]
Why	(1, 3) and (3, 5) touch at 3, so they merge.

Example 3	
Input	[(1, 3)], [(3, 5)], touching = false
Output	[(1, 3), (3, 5)]
Why	With touching=False they only merge when they truly overlap.

MAIN CODE | helper functions first, then the answer

```
import heapq


END = object()                                  # marks "this list has no more intervals"
```

```python
# ---- main question: helper functions ----
def merge_sorted(lists):
    # merge several sorted lists (or streams) into one sorted stream, one item
    # at a time: a heap holds the next item of every list, so memory is O(K)
    iterators = []
    for items in lists:
        iterators.append(iter(items))
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    while heap:
        item, i = heapq.heappop(heap)
        yield item
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))


def union_stream(lists, touching=True):
    # the union of sorted lists of (start, end), as a stream (follow-up 1).
    # Take the interval with the smaller start next; if it touches or overlaps
    # the block being built, extend that block, otherwise finish the block.
    current = None
    for start, end in merge_sorted(lists):
        if current is None:
            current = [start, end]
            continue
        if touching:
            joined = start <= current[1]           # closed intervals: [1, 3] and [3, 5] share the point 3
        else:
            joined = start < current[1]
        if joined:
            if end > current[1]:
                current[1] = end
        else:
            yield tuple(current)                   # finished one block: give it out now
            current = [start, end]
    if current is not None:
        yield tuple(current)


# ---- main question: main function ----
# Idea: a two-pointer merge: always take the interval with the smaller
# start next. If it touches or overlaps the last output interval (start <=
# its end), extend that one, otherwise start a new one.
# Complexity: Time O(n + m) for two sorted lists, Space O(n + m) for the answer
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# the same code with any number of lists: K lists, O(K) memory
# Complexity: Time O(N log k) for N intervals in k lists, Space O(k) plus the answer
def union_k(lists, touching=True):
    return list(union_stream(lists, touching))
```

COMPLEXITY | Two-pointer merge: Time O(m + n) | Space O(m + n) for the output
K lists with a heap (streaming, O(K) memory): Time O(N log K) | Space O(K) beyond the output

132 Range Module (Insert and Point Query) MEDIUM

Template fit: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval

QUESTION

Design a data structure that tracks a set of half-open ranges [left, right) over the number line. add_range(left, right) adds the interval; overlapping or touching intervals are merged so that only disjoint, non-adjacent intervals remain. query_point(x) returns True when x lies inside a tracked interval; because the intervals are half-open, a point equal to a right endpoint is not inside.

Function signature

class RangeModule:
    def add_range(self, left: float, right: float) -> None
    def query_point(self, x: float) -> bool

EXAMPLES

Example 1

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.query_point(15)`
	`rm.query_point(20)`
	`rm.query_point(10)`

Output	`rm.query_point(15) -> true`
	`rm.query_point(20) -> false`
	`rm.query_point(10) -> true`

Why	`[10, 20) contains 10 and 15 but not 20.`

Example 2

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.add_range(20, 30)`
	`rm.query_point(20)`
	`rm.query_point(25)`
	`rm.query_point(30)`

Output	`rm.query_point(20) -> true`
	`rm.query_point(25) -> true`
	`rm.query_point(30) -> false`

Why	`The touching ranges merge into [10, 30), so 20 is now covered; 30 still is not.`

Example 3

Input	`rm = RangeModule()` `rm.query_point(5)`
Output	`rm.query_point(5) -> false`
Why	`Nothing has been added yet.`

CONSTRAINTS

- Endpoints and query points are integers or floats with -10^9 <= value <= 10^9 .
- For every add_range call, left < right .
- Up to 10^5 total operations.
- Touching intervals merge: add_range(1, 5) followed by add_range(5, 10) yields one interval [1,
- 10) .

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Ranges are half-open [left, right): add_range(left, right) adds one,
# query_point(x) says whether x is inside any added range.
# Idea: keep the ranges merged, sorted and disjoint (two lists, lefts and
# rights). Adding replaces every range the new one touches by one merged
# range (found with two binary searches). A query is one binary search for the
# only candidate: the range with the largest left <= x.
# Complexity: Time add_range O(log n) to search plus O(n) to splice, query_point O(log n), Space O(n)
class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []

    def add_range(self, left, right):
        i = first_at_least(self.rights, left)              # the first range ending at or after the new left
        j = first_greater(self.lefts, right)               # the first range starting after the new right
        if i < j:
            if self.lefts[i] < left:
                left = self.lefts[i]
            if self.rights[j - 1] > right:
                right = self.rights[j - 1]
        self.lefts[i:j] = [left]
        self.rights[i:j] = [right]

    def query_point(self, x):
        i = first_greater(self.lefts, x) - 1
        return i >= 0 and x < self.rights[i]
```

COMPLEXITY | Time O(log n) query | Space O(n)

133 Meetings Minus a Do-Not-Schedule Block

MEDIUM

Template fit: merge first, then for each merged meeting keep the part before the blocked window and the part after it

QUESTION

You are given a list of meetings , each a (start, end) interval with start < end , and one "do-not- schedule" (DNS) block (dns_start, dns_end) . Consider the set of all time points covered by at least one meeting. Return the smallest list of non-overlapping, sorted intervals that covers exactly those points except any point lying inside the DNS block (treat the DNS block as removing the open coverage on (dns_start, dns_end) ; equivalently split coverage at those bounds). Meetings may overlap and are given in any order. Return the result sorted by start time.

Function signature

```
def subtract_dns(meetings: list[tuple[int, int]],
                 dns: tuple[int, int]) -> list[tuple[int, int]]:
    ...
```

Follow-up (as reported): Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

EXAMPLES

Example 1	
Input	`meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)`
Output	`[(1,10),(12,20),(35,40),(50,65)]`
Why	`Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40] into [12,20] and [35,40].`

Example 2	
Input	`meetings = [(0,10)], dns = (0, 10)`
Output	`[]`
Why	`The DNS block removes the entire covered range.`

Example 3	
Input	`meetings = [(0,5),(6,9)], dns = (100, 200)`
Output	`[(0,5),(6,9)]`
Why	`The DNS block covers nothing that was scheduled.`

CONSTRAINTS

- 1 <= len(meetings) <= 10^5 .
- Coordinates are integers; 0 <= start < end <= 10^9 .

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def merge_intervals(meetings):
    # sort by start, then join the ones that overlap or touch
    merged = []
    for start, end in sorted(meetings):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1] = (merged[-1][0], end)
        else:
            merged.append((start, end))
    return merged


# ---- main question: main function ----
# Remove a do-not-schedule block (low, high) from the meetings.
```

```
# Idea: merge the meetings first; then every merged meeting keeps at most
# two pieces: the part to the LEFT of the block and the part to the RIGHT of it.
# Complexity: Time O(n log n), Space O(n)
def subtract_dns(meetings, dns):
    low, high = dns
    pieces = []
    for start, end in merge_intervals(meetings):
        if start < low:
            pieces.append((start, min(end, low)))          # the part before the block
        if end > high:
            pieces.append((max(start, high), end))         # the part after the block
    return pieces
```

COMPLEXITY | Time O(n log n) | Space O(n)

134 Merge Intervals — MEDIUM

Template fit: the template as written

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in th

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 6], [8, 10], [15, 18]]`
Output	`[[1, 6], [8, 10], [15, 18]]`
Why	`[1, 3] and [2, 6] overlap and merge into [1, 6].`

Example 2	
Input	`[[1, 4], [4, 5]]`
Output	`[[1, 5]]`
Why	`Intervals that touch at an endpoint are merged as well.`

Example 3	
Input	`[[8, 10], [1, 3], [2, 4]]`
Output	`[[1, 4], [8, 10]]`
Why	`Unsorted input is fine.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: sort by start, then sweep. If an interval starts at or before the
# end of the last output interval, extend that end (to the larger end);
```

```python
# otherwise append it as a new interval. Because of the sort, nothing earlier
# can overlap the new interval. (touching_ints is follow-up 2.)
# Complexity: Time O(n log n), Space O(n)
def merge_intervals(intervals, touching_ints=False):
    gap = 0
    if touching_ints:
        gap = 1                                  # [1, 2] and [3, 4] cover 1..4 for whole numbers
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1] + gap:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged
```

COMPLEXITY | Time O(n log n) | Space O(n)

135 Find safe travel intervals between planet influences — HARD

Template fit: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

QUESTION

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8, 12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

MAIN CODE | helper functions first, then the answer

```python
INF = float("inf")


# ---- main question: helper functions ----
def unsafe_blocks(planets):
    # planet (c, r) makes the stretch [c - r, c + r] unsafe; merge overlapping stretches
    spans = []
    for c, r in planets:
        spans.append((c - r, c + r))
    spans.sort()
    blocks = []
    for start, end in spans:
        if blocks and start <= blocks[-1][1]:             # sharing even one point joins two blocks
            if end > blocks[-1][1]:
                blocks[-1][1] = end
        else:
            blocks.append([start, end])
    return blocks


def gaps_between(blocks):
    # the open intervals between neighbouring blocks, and the two infinite ends
    if not blocks:
        return [(-INF, INF)]
    safe = [(-INF, blocks[0][0])]
    for i in range(len(blocks) - 1):
        safe.append((blocks[i][1], blocks[i + 1][0]))
    safe.append((blocks[len(blocks) - 1][1], INF))
    return safe


# ---- main question: main function ----
# Idea: sort the unsafe stretches by start and merge the ones that overlap or
# touch (they share a point, and that point is unsafe). The safe
# intervals are the gaps between the merged blocks, plus the two infinite ends.
# Complexity: Time O(n log n) for n planets, Space O(n)
def safe_intervals(planets):
    return gaps_between(unsafe_blocks(planets))
```

COMPLEXITY | Time O(n log n) | Space O(n)

136 Implement a function that processes a stream of interval data. HARD

Template fit: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

QUESTION

Intervals arrive one at a time. add(start, end) receives a new interval, and intervals() returns the current list of disjoint intervals, sorted by start; intervals that overlap or touch are merged.

Function signature

class IntervalStream:
    def add(self, start: int, end: int) -> None
    def intervals(self) -> list

EXAMPLES

Example 1

Input	`s = IntervalStream()`
	`s.add(1, 3)`
	`s.add(6, 9)`
	`s.intervals()`
	`s.add(2, 5)`
	`s.intervals()`
	`s.add(5, 6)`
	`s.intervals()`

Output	`s.intervals() -> [(1, 3), (6, 9)]`
	`s.intervals() -> [(1, 5), (6, 9)]`
	`s.intervals() -> [(1, 9)]`

Why	`(2, 5) overlaps (1, 3) and extends it to (1, 5); (5, 6) touches (1, 5) at 5 and (6, 9) at 6, so everything merges into (1, 9).`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Intervals arrive one at a time; after every add(start, end), intervals()
```

```
# must return the merged, sorted list.
# Idea: keep the merged intervals sorted by start (two lists: starts and
# ends). A new interval can only interact with the intervals found by two
# binary searches: the first interval that touches it and the first one
# completely to its right. Replace that slice by ONE merged interval.
# integer=True: [1, 3] and [4, 5] cover 1..5, so they merge (follow-up 2);
# integer=False: only overlapping intervals or a shared end merge.
# The sorted disjoint intervals also answer follow-up 3: is a point
# covered (contains) and how much is covered in total (total).
# Complexity: Time add is O(log n) to search plus O(n) to splice the lists, contains O(log n), total O(1), Sp
ace O(n)
class IntervalStream:
    def __init__(self, integer=True):
        self.starts = []
        self.ends = []
```

```
        self.gap = 0                                # how far apart two intervals may be and still merge
        self.extra = 0                              # the length of [a, b] is b - a + extra
        if integer:
            self.gap = 1
            self.extra = 1
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        i = first_at_least(self.ends, start - self.gap)       # the first interval that touches the new one
        j = first_greater(self.starts, end + self.gap)        # the first one completely to its right
        for k in range(i, j):
            self.covered -= self.length(self.starts[k], self.ends[k])    # the swallowed intervals leave the
total
        if i < j:
            if self.starts[i] < start:
                start = self.starts[i]
            if self.ends[j - 1] > end:
                end = self.ends[j - 1]
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]
        self.covered += self.length(start, end)               # and the merged one joins it

    def intervals(self):
        result = []
        for i in range(len(self.starts)):
            result.append((self.starts[i], self.ends[i]))
        return result

    def contains(self, point):
        # the interval with the largest start <= point must still reach the point
        i = first_greater(self.starts, point) - 1
        return i >= 0 and self.ends[i] >= point

    def total(self):
        return self.covered


# Complexity: Time O(S * n) for S steps (it copies the interval list after every step), Space O(S * n)
def process(stream, integer=True):
    # feed a stream of (start, end) pairs; give the merged list after each add
    cover = IntervalStream(integer)
```

```
    steps = []
    for start, end in stream:
        cover.add(start, end)
```

```
        steps.append(cover.intervals())
    return steps
```

COMPLEXITY | Time O(log n) to find the spot, O(n) worst-case list shift | Space O(n)

Trie: nested dicts, one character per level

137 URL Request Handler Router

MEDIUM

Template fit: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first

QUESTION

Design a request router. add(pattern) registers a URL pattern whose segments are either literal text or a placeholder written {name}; match(url) returns the registered pattern that handles the URL, or None. When a URL matches both a literal segment and a placeholder, the literal wins.

Function signature

class Router:
  def add(self, pattern: str) -> None
  def match(self, url: str) -> str | None

EXAMPLES

Example 1

Input	`r = Router()`
	`r.add('/users/{userId}/photo/{photoId}')`
	`r.add('/users/me/photo/{photoId}')`
	`r.match('/users/123/photo/456')`
	`r.match('/users/me/photo/9')`
	`r.match('/orders/1')`

Output	`r.match('/users/123/photo/456') -> '/users/{userId}/photo/{photoId}'`
	`r.match('/users/me/photo/9') -> '/users/me/photo/{photoId}'`
	`r.match('/orders/1') -> null`

Why	The placeholder pattern handles any user id, but for /users/me/... the literal 'me' pattern wins; an unknown path matches nothing.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    parts = []
    for piece in path.split("/"):
        if piece != "":
            parts.append(piece)
    return parts


class Node:
    # one node of the trie: one path segment per level
    def __init__(self):
        self.literal = {}          # fixed segment -> child node
        self.wild = None           # the child for a {placeholder} segment
        self.wild_name = None      # the placeholder's name, e.g. "userId"
        self.pattern = None        # the registered pattern that ends at this node
        self.rest = None           # the pattern with a trailing "*" (matches the rest of the path)


# ---- main question: main function ----
# Idea: store the patterns in a trie with one path SEGMENT per level.
# Every node has literal children plus at most one placeholder child, and
# the pattern is stored at its last node. To look up a URL, try the
# literal child first; if that whole branch fails, back up and try the
# placeholder child. So a literal segment wins over a placeholder.
```

```
# Complexity: Time add O(s) for s path segments, match O(s) usually (backtracking from a literal to a placeho
lder can revisit a branch), Space O(total segments)
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern):
        node = self.root
        for segment in segments(pattern):
            if segment == "*":                          # follow-up 2: a trailing * matches the rest of the p
ath
                node.rest = pattern
                return
            if segment.startswith("{") and segment.endswith("}"):
                if node.wild is None:
                    node.wild = Node()
                node.wild_name = segment[1:-1]
                node = node.wild
            else:
                if segment not in node.literal:
                    node.literal[segment] = Node()
```

```
                node = node.literal[segment]
        node.pattern = pattern

    def lookup(self, url):
        # returns (the matching pattern or None, the values of the placeholders)
        parts = segments(url)
        values = {}

        def walk(node, i):
            if i == len(parts):
                return node.pattern
            child = node.literal.get(parts[i])            # 1) the exact segment first
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            if node.wild is not None:                     # 2) then a placeholder, remembering its value
                values[node.wild_name] = parts[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                del values[node.wild_name]                # that branch failed: forget the value
            return node.rest                              # 3) finally a trailing wildcard

        pattern = walk(self.root, 0)
        return pattern, values

    def match(self, url):
        pattern, values = self.lookup(url)
        return pattern
```

COMPLEXITY | Time O(L * branching) per lookup, O(total pattern length) to build | Space O(total pattern length)

138 Trie Prefix Search / Autocomplete

MEDIUM

Template fit: walk to the prefix node; return the words under it (autocomplete)

QUESTION

Build a trie from a list of words. search_prefix(prefix) returns every word that starts with the prefix, sorted alphabetically (empty list when none).

Function signature

class Trie:

def insert(self, word: str) -> None

def search_prefix(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = Trie(['car', 'card', 'care', 'cat', 'dog'])`
	`t.search_prefix('car')`
	`t.search_prefix('ca')`
	`t.search_prefix('x')`

Output	`t.search_prefix('car') -> ['car', 'card', 'care']`
	`t.search_prefix('ca') -> ['car', 'card', 'care', 'cat']`
	`t.search_prefix('x') -> []`

Why	`'car' is itself a word that starts with 'car'; the prefix 'x' matches nothing.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}         # letter -> child node
        self.freq = 0              # how often the word ending here was inserted
        self.top = []              # the best (negative frequency, word) pairs below this node


# ---- main question: main function ----
# Idea: insert each word into the trie and mark its last node. For a
# query, walk the prefix (a missing step means no words), then collect
# every word below that node and sort them alphabetically.
# Complexity: Time insert O(L * k log k) (the best-k list of every node on the path is refreshed), search_pre
fix O(L + size of the subtree), Space O(total characters * k)
class Trie:
    def __init__(self, k=3):
        self.root = Node()
        self.k = k                 # how many suggestions to keep per node (follow-up 1)

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
            path.append(node)
        node.freq += freq
        # follow-up 1: refresh the best-k list on every node of the path
        for n in path:
            kept = []
            for item in n.top:
                if item[1] != word:
                    kept.append(item)
            kept.append((-node.freq, word))
            kept.sort()                                  # most frequent first, then alphabetical
```

```
            n.top = kept[:self.k]

    def find(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

```
    def words_below(self, node, text):
        # every word at or below node (spelled `text` so far), in no particular order
        found = []
        stack = [(node, text)]
        while stack:
            current, spelled = stack.pop()
            if current.freq > 0:
                found.append(spelled)
            for ch in current.children:
                stack.append((current.children[ch], spelled + ch))
        return found

    def search_prefix(self, prefix):
        node = self.find(prefix)
        if node is None:
            return []
        return sorted(self.words_below(node, prefix))
```

COMPLEXITY | Time O(L + output) per query after O(total characters) to build | Space O(total characters)

139 Build Prefix Lookup with a Trie — MEDIUM

Template fit: build_trie, then walk to the prefix node and collect every word below it

QUESTION

Build a trie from a list of words. completions(prefix) returns every inserted word that starts with the prefix (including the prefix itself when it is a word), sorted alphabetically, or an empty list when none does.

Function signature

class PrefixTrie:
  def insert(self, word: str) -> None
  def completions(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = PrefixTrie(['apple', 'app', 'apply', 'apt', 'bat'])`
	`t.completions('app')`
	`t.completions('ap')`
	`t.completions('z')`

Output	`t.completions('app') -> ['app', 'apple', 'apply']`
	`t.completions('ap') -> ['app', 'apple', 'apply', 'apt']`
	`t.completions('z') -> []`

Why	`'app' is itself a word and a prefix of two more; the prefix 'z' matches nothing.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}        # letter -> child node
```

```
        self.is_word = False       # does a word end here?
        self.count = 0             # how many different words pass through this node (follow-up 1)


# ---- main question: main function ----
# Idea: a trie stores the words in nested nodes, one letter per level.
# insert walks down creating missing nodes and marks the last one as a
# word end. For a prefix, walk down one letter at a time (a missing step
# means no completions), then go through everything below that node
# collecting the words.
# Complexity: Time insert O(L), completions O(L + size of the subtree), Space O(total characters)
class PrefixTrie:
    def __init__(self):
        self.root = Node()
        self.size = 0                                      # how many different words

    def find(self, text):
        # the node reached by following `text`, or None
        node = self.root
        for ch in text:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def insert(self, word):
        existing = self.find(word)
        if existing is not None and existing.is_word:
            return                                         # a repeated word counts once
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
            node.count += 1
        node.is_word = True
        self.size += 1

    def words_below(self, node, text, limit):
        # the words at or below `node` in alphabetical order, at most `limit` of them.
        # A stack is used; the children are pushed in reverse order so the
```

```
        # smallest letter is taken first.
        found = []
        stack = [(node, text)]
        while stack and len(found) < limit:
            current, spelled = stack.pop()
            if current.is_word:
                found.append(spelled)
            for ch in sorted(current.children, reverse=True):
                stack.append((current.children[ch], spelled + ch))
        return found

    def completions(self, prefix):
        # every word that starts with the prefix, in alphabetical order
        node = self.find(prefix)
        if node is None:
            return []
        return self.words_below(node, prefix, self.size + 1)
```

COMPLEXITY | Time O(L) to walk the prefix + O(size of the answer) | Space O(total characters)

140 Greedy Longest-Match Dictionary Tokenizer — MEDIUM

Template fit: the same longest-match walk, with a priority rule to choose between equal-length matches

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

EXAMPLES

Example 1

Input	`t = Tokenizer({'ab': 1, 'abc': 2, 'c': 3})`
	`t.tokenize('abcab')`
	`t.tokenize('abx')`

Output	`t.tokenize('abcab') -> [2, 1]` `t.tokenize('abx') -> [1, 'x']`
Why	`At each position the longest dictionary key wins: 'abc' (not 'ab' followed by 'c'), then 'ab'. A character that starts no key is emitted as it is.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: put every key in a trie with its id at the last node. From each
# text position walk down the trie while the next letter exists and
# remember the LAST (deepest) key that ended on the way: that is the
# longest match. Output its id and skip its length; if no key matches,
# output the character itself and move one position.
# priority(key) (follow-up 1) is an optional rule: a LOWER number wins over
# a longer key.
# Complexity: Time build O(total key characters), tokenize O(n * longest key), Space O(total key characters)
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id
        self.priority = priority
        self.root = {}
        for key in table:
            node = self.root
            for ch in key:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = (key, table[key])              # the end of a key stores the key and its id

    def better(self, new, old):
        # should the match `new` replace the match `old` found earlier?
        if old is None:
            return True
        if self.priority is None:
            return True                                # deeper (longer) matches come later: keep them
        return self.priority(new[0]) < self.priority(old[0])
```

```
    def best_at(self, text, i):
        node = self.root
        best = None
        j = i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node and self.better(node["$"], best):
                best = node["$"]
        return best

    def tokenize(self, text):
        output = []
        i = 0
        while i < len(text):
```

```
            match = self.best_at(text, i)
            if match is None:
                output.append(text[i])                # no key starts here: keep the character
                i += 1
            else:
                key, ident = match
                output.append(ident)
                i += len(key)
        return output


# a simpler version: one dictionary lookup for each possible length,
# longest first
# Complexity: Time O(n * longest key^2) (a slice and a hash for every length), Space O(keys)
def tokenize_map(text, table):
    longest = 0
    for key in table:
        if len(key) > longest:
            longest = len(key)
    output = []
    i = 0
    while i < len(text):
        size = longest
        if len(text) - i < size:
            size = len(text) - i
        matched = False
        while size > 0 and not matched:
            if text[i:i + size] in table:
                output.append(table[text[i:i + size]])
                i += size
                matched = True
            size -= 1
        if not matched:
            output.append(text[i])
            i += 1
    return output
```

COMPLEXITY | Time O(n * L) | Space O(total key length)

141 Check if all substrings are dictionary words

MEDIUM

Template fit: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

EXAMPLES

Example 1	
Input	`'abcd', {'abc', 'abcd', 'bcd'}`
Output	`true`
Why	`Every substring of length 3 or more (abc, bcd, abcd) is in the dictionary.`

Example 2	
Input	`'abcd', {'abc', 'abcd'}`
Output	`false`
Why	`'bcd' is missing from the dictionary.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
# Complexity: Time O(total characters), Space O(total characters)
def build_trie(words):
    # nested dictionaries, one letter per level; the key "$" marks a word end
    root = {}
    for word in words:
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node["$"] = True
    return root


# ---- main question: main function ----
# Idea: every substring of length >= min_len (3) must be a word. For each
# start position i, walk down the trie along s[i:]. For the first
# min_len - 1 letters only walk; from depth min_len on, the node we are
# at must be marked as a word end. If the walk breaks (no word has this
# prefix) the answer is False when at least min_len letters remain from i,
# because then a substring of that length exists and is not a word.
# Complexity: Time O(n * longest word), Space O(total dictionary characters)
def all_substrings_words(s, words, min_len=3):
    root = build_trie(words)
    for i in range(len(s)):
        node = root
        depth = 0
        for j in range(i, len(s)):
            ch = s[j]
            depth += 1
            if ch not in node:
                return len(s) - i < min_len
```

```
            node = node[ch]
            if depth >= min_len and "$" not in node:
                return False
    return True
```

COMPLEXITY | Time O(n^2) | Space O(dictionary size)

142 Return dictionary words matching a prefix — MEDIUM

Template fit: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)

QUESTION

You are given a dictionary of words. Return all words that start with a given prefix. Matching is case-sensitive, duplicates in the dictionary are returned as often as they occur, and the result is an empty list when nothing matches. The index returns the matches in alphabetical order; limit (optional) stops after that many words.

Function signature

class PrefixIndex:
  def __init__(self, words: list)
  def matches(self, prefix: str, limit=None) -> list

EXAMPLES

Example 1

Input	`idx = PrefixIndex(['apple', 'app', 'Apple', 'apply', 'apple', 'bat'])`
	`idx.matches('app')`
	`idx.matches('app', limit=2)`
	`idx.matches('cat')`

Output	`idx.matches('app') -> ['app', 'apple', 'apple', 'apply']`
	`idx.matches('app', limit=2) -> ['app', 'apple']`
	`idx.matches('cat') -> []`

Why	`'Apple' starts with a capital letter so it does not match 'app'; 'apple' is listed twice because it is in the dictionary twice.`

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left, bisect_right, insort


# ---- main question: main function ----
# Idea: sort the words once. All words with a given prefix then sit
# together in ONE block. Binary search the start of the block (the first
# word >= prefix) and the end (just after the last word that starts with
# the prefix, found by searching for the prefix followed by the biggest
# possible character). `limit` stops after that many words (follow-up 1).
# Complexity: Time build O(n log n * L), matches O(L log n + size of the answer), add O(n), Space O(n)
class PrefixIndex:
    def __init__(self, words):
        self.words = sorted(words)

    def matches(self, prefix, limit=None):
        start = bisect_left(self.words, prefix)
        end = bisect_right(self.words, prefix + "\uffff")
        found = self.words[start:end]
        if limit is None:
            return found
        return found[:limit]

    # follow-up 2: words can be added while queries run. Keep the list
    # sorted by inserting at the right place (O(n) shifting, fast in practice).
    def add(self, word):
```

```
        insort(self.words, word)
```

COMPLEXITY | Time O(log W + m) per query after O(W log W) sorting | Space O(W)

143 Implement Trie Insert and Exact-Word Search — MEDIUM

Template fit: insert is build_trie; search is walk and then END in node

QUESTION

Implement a trie with insert(word), search(word) and startsWith(prefix). search is true only when the complete word was inserted earlier (a longer stored word that merely starts with it does not count); startsWith is true when some inserted word begins with the prefix.

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search(self, word: str) -> bool
  def startsWith(self, prefix: str) -> bool

EXAMPLES

Example 1

Input	`t = Trie()`
	`t.insert('apple')`
	`t.search('apple')`
	`t.search('app')`
	`t.startsWith('app')`
	`t.insert('app')`
	`t.search('app')`

Output	`t.search('apple') -> true`
	`t.search('app') -> false`
	`t.startsWith('app') -> true`
	`t.search('app') -> true`

Why	`'app' is only a prefix of 'apple' until it is inserted on its own.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}          # letter -> child node
        self.end = False            # does a word end here?


# ---- main question: main function ----
# Idea: each node has children by letter and an "a word ends here" flag.
# insert creates the missing nodes and marks the last one. search walks
# the word and also needs the flag at the end; startsWith only needs the
# walk to succeed.
# Complexity: Time insert, search and startsWith are O(L) each, Space O(total characters)
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters of text; None if the path breaks
        node = self.root
        for ch in text:
            if ch not in node.children:
                return None
```

```
            node = node.children[ch]
        return node

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
        node.end = True

    def search(self, word):
        node = self.walk(word)
        return node is not None and node.end        # the whole word must have been inserted

    def startsWith(self, prefix):
        return self.walk(prefix) is not None
```

COMPLEXITY | Time O(L) per operation | Space O(total characters)

144 Implement Longest-Match Text Replacement — HARD

Template fit: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed. Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2", "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

QUESTION

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: put the tokens in a trie (a tree with one letter per level); the
# last node of a token stores the token and its id. At a text position,
# walk down the trie along the text; every node on the way that stores
# an id is a token that matches here. Take the LONGEST one, output its id,
# and move on by its length. If nothing matches, output the character
# itself and move on by one.
# Complexity: Time build O(total token characters), replace O(n * longest token), Space O(total token charact
ers)
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id
        self.root = {}
        for token in mapping:
            node = self.root
            for ch in token:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = (token, mapping[token])

    def candidates(self, text, i):
        # every token that matches at position i, shortest first (all the word
        # ends along the trie path, not only the last one)
        found = []
        node = self.root
        j = i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
```

```
            j += 1
            if "$" in node:
                found.append(node["$"])
        return found

    def replace(self, text, choose=None, word_boundary=False):
        # `choose(candidates)` picks the winner (follow-up 1); by default
        # the longest token. word_boundary=True accepts only whole-word
        # matches (follow-up 3).
        output = []
        i = 0
        while i < len(text):
            found = self.candidates(text, i)
            if word_boundary:
                whole = []
                for token, ident in found:
                    if is_whole_word(text, i, len(token)):
                        whole.append((token, ident))
                found = whole
            if found:
                if choose is None:
                    token, ident = found[-1]
                else:
                    token, ident = choose(found)
                output.append(ident)
                i += len(token)
            else:
                output.append(text[i])                # nothing matches here: keep the character
                i += 1
        return output


# Complexity: Time O(1), Space O(1)
def is_whole_word(text, start, size):
    # no letter or digit touches the match on either side
    before_ok = start == 0 or not text[start - 1].isalnum()
    end = start + size
    after_ok = end == len(text) or not text[end].isalnum()
    return before_ok and after_ok
```

COMPLEXITY | Time O(len(text) * longest token) | Space O(total token length)

GROUP 18 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 1 E 6 M 1 H

A dict for lookup plus an ordered container for who is next (lazy deletion)

145 Implement Batched Undo/Redo Layer

EASY

Template fit: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value after the batch string old_value; // value before the ba
tch bool has_old_value; // whether the key existed before the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.
Constraints
0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',),
    ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color': 'red'}]
```

Explanation: A single batch is committed, undone, and redone.

QUESTION

```
Input: [('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
class Change:
    # one key changed inside one batch: its value before and after
    def __init__(self, key, old, had_old, new):
        self.key = key
        self.old = old              # the value before the batch (if it had one)
        self.had_old = had_old      # did the key exist before the batch?
        self.new = new              # the value at the end of the batch


def merge_batch(parent, child):
    # a nested batch folds into its parent: keep the parent's OLD value
    # when the key was already touched there, but take the newest value
    for key in child:
        if key in parent:
            parent[key].new = child[key].new
        else:
            parent[key] = child[key]


# ---- main question: main function ----
# Idea: while a batch is open, remember for every key touched the value
# it had BEFORE the batch (recorded only the first time the key is
# touched) and its latest value. end_batch puts the batch on the undo
# stack as ONE action and clears the redo stack. Undo restores the old
# values; redo applies the new ones.
# `history` limits how many batches are remembered (follow-up 1): a deque
# with a maximum length drops the oldest batch by itself.
# `open_batches` is a stack, so batches can be nested (follow-up 2).
# Complexity: Time set_property O(1), end_batch O(keys in the batch), undo and redo O(keys in the batch), Spa
ce O(total changes kept)
class Layer:
    def __init__(self, history=None):
        self.props = {}
        self.undo_stack = deque(maxlen=history)       # committed batches, newest last
        self.redo_stack = []
        self.open_batches = []                        # each open batch: key -> Change

    def begin_batch(self):
        self.open_batches.append({})

    def set_property(self, key, value):
        if len(self.open_batches) == 0:               # outside a batch: it is a batch of one change
```

```
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open_batches[-1]
        if key in batch:
            batch[key].new = value                    # touched again: keep the ORIGINAL old value
        else:
            batch[key] = Change(key, self.props.get(key), key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open_batches.pop()
        if len(self.open_batches) > 0:
            merge_batch(self.open_batches[-1], batch)     # a nested batch folds into its parent
        elif len(batch) > 0:
            self.undo_stack.append(list(batch.values()))  # only the OUTERMOST batch is one undoable action
            self.redo_stack.clear()                       # a new action clears the redo history

    def undo(self):
        if len(self.undo_stack) == 0:
            return False
        changes = self.undo_stack.pop()
        for change in changes:
            if change.had_old:
                self.props[change.key] = change.old
            else:
                del self.props[change.key]                # the key did not exist before: remove it
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if len(self.redo_stack) == 0:
            return False
        changes = self.redo_stack.pop()
        for change in changes:
            self.props[change.key] = change.new
        self.undo_stack.append(changes)
        return True
```

COMPLEXITY | Time O(keys changed in the batch) per operation | Space O(total changed keys)

146 Design a restaurant waitlist system — MEDIUM

Template fit: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **c** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**

- Adds a new party to the end of the waitlist and returns its id.

2. **`cancelParty(partyId) -> bool`**

QUESTION

- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.

3. **`seatTable(capacity) -> partyId or null`**

- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.

4. **`getPosition(partyId) -> int`**

- Returns how many parties are ahead of this party in the current waitlist order (0-based).
- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.add(1, 'ann', 4)`
	`w.add(2, 'bob', 2)`
	`w.add(3, 'cy', 3)`
	`w.seat(3)`
	`w.cancel(3)`
	`w.seat(3)`
	`w.seat(4)`

Output	`w.seat(3) -> 2`
	`w.cancel(3) -> true`
	`w.seat(3) -> null`
	`w.seat(4) -> 1`

Why	`A table for 3 takes the earliest party that fits: bob (2), not ann (4). After cy cancels, nobody left fits a table for 3; a table for 4 seats ann.`

MAIN CODE | helper functions first, then the answer

```
INF = float("inf")


# ---- main question: main function ----
# Idea: seat the EARLIEST party that fits the table. Each arriving party
# gets the next position (its arrival order). A segment tree over these
# positions stores, for each range, the SMALLEST party size waiting in it
# (infinity for empty or already seated positions). To seat a table of
# capacity c: if the smallest size overall is above c nobody fits;
# otherwise go down the tree, always LEFT first when the left half holds a
# party that fits, so we reach the earliest one.
# Complexity: Time add, cancel and seat are O(log capacity), Space O(capacity)
class Waitlist:
    def __init__(self, capacity=1024):
        self.cap = capacity                          # the most parties that can ever join
        self.tree = [INF] * (2 * capacity)           # leaves are tree[cap .. 2 * cap - 1]
        self.count = 0                               # how many parties have joined so far
        self.slot = {}                               # party id -> its position
        self.who = {}                                # position -> party id
```

```
    def update(self, position, value):
        i = position + self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            left = self.tree[2 * i]
            right = self.tree[2 * i + 1]
            if left < right:
                self.tree[i] = left
            else:
                self.tree[i] = right

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        if party_id not in self.slot:
            return False
        position = self.slot[party_id]
        if self.tree[position + self.cap] == INF:
            return False                              # already seated or cancelled
        self.update(position, INF)
```

```
        return True

    def seat(self, capacity):
        if self.tree[1] > capacity:
            return None                               # no waiting party is small enough
        i = 1
        while i < self.cap:
            if self.tree[2 * i] <= capacity:
                i = 2 * i                             # the left half holds a party that fits
            else:
                i = 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]
```

COMPLEXITY | Time O(log n) per operation | Space O(n)

147 Implement Python's itertools.tee — MEDIUM

Template fit: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue

QUESTION

Implement itertools.tee: given an iterable and n, return n independent iterators that each produce the same values from the one source. The source must be read only once, and a value must be kept only until every iterator has taken it.

Function signature

def tee_queues(iterable, n=2) -> tuple

EXAMPLES

QUESTION

Example 1

Input	`a, b = tee_queues([1, 2, 3])`
	`next(a)`
	`next(a)`
	`next(b)`
	`list(b)`
	`list(a)`

Output	`next(a) -> 1`
	`next(a) -> 2`
	`next(b) -> 1`
	`list(b) -> [2, 3]`
	`list(a) -> [3]`

Why	Iterator a runs ahead by two values; b still sees every value from the start, and neither disturbs the other.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: main function ----
# Idea: one waiting line (deque) of values per consumer. When a
# consumer's line is empty it pulls the next value from the source,
# gives it to EVERY consumer's line, and then takes its own copy. Each
# consumer is a generator, so values are read from the source only when
# somebody asks for them (it works on endless sources too).
# Complexity: Time O(1) per value per consumer, Space O(values the slowest consumer has not read yet)
def tee_queues(iterable, n=2):
    source = iter(iterable)
    queues = []
    for i in range(n):
        queues.append(deque())

    def consumer(mine):
        while True:
            if len(mine) == 0:
                try:
                    value = next(source)
                except StopIteration:
                    return
                for q in queues:
                    q.append(value)              # give the new value to everybody
            yield mine.popleft()

    consumers = []
    for q in queues:
        consumers.append(consumer(q))
    return tuple(consumers)
```

COMPLEXITY | Time O(n) total, O(1) amortized per item per consumer | Space O(lag between fastest and slowest consumer)

148 Store With Cheapest-Item Removal

MEDIUM

Template fit: the order is a heap keyed by (price, timestamp); remove_cheapest pops it

QUESTION

Design a Store that holds items. Each item has a string value , an integer price in cents, and an integer timestamp . Support: add(value, price, timestamp) : insert an item. remove_cheapest() : remove and return the value of the item with the smallest price. If several items share the smallest price, remove the one with the smallest timestamp . Return None if the store is empty. A heap keyed by (price, timestamp) gives O(log n) per operation.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...
    def add(self, value: str, price: int, timestamp: int) -> None:
        ...
    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item (ties broken by
earliest timestamp)."""
        ...
```

Follow-up (as reported): Improve remove_cheapest from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

EXAMPLES

Example 1	
Input	`add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()`
Output	`"b"`
Why	`"b" has the smallest price (2 cents).`

Example 2	
Input	`add("x",7,10); add("y",7,4); remove_cheapest()`
Output	`"y"`
Why	`Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).`

Example 3	
Input	`remove_cheapest() # on an empty store`
Output	`None`

CONSTRAINTS

- Prices are integers with 1 <= price <= 10000 (i.e. 1 cent up to $100.00).
- Up to 10^6 operations.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main function ----
# Idea: a min-heap of (price, timestamp, value). The smallest tuple is
# always on top: the cheapest item, and among equal prices the one with
# the earliest timestamp.
# Complexity: Time add and remove_cheapest are O(log n), Space O(n)
class Store:
    def __init__(self):
```

```
        self.heap = []

    def add(self, value, price, timestamp):
        heapq.heappush(self.heap, (price, timestamp, value))

    def remove_cheapest(self):
        if len(self.heap) == 0:
            return None
        price, timestamp, value = heapq.heappop(self.heap)
        return value
```

COMPLEXITY | Time O(log n) per operation | Space O(n)

149 Round-Robin Job Scheduler — MEDIUM

Template fit: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time

QUESTION

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round- robin approach. Reconstruct it as follows. You are given a list of jobs, each with a unique string id and a positive integer burst (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time quantum : it takes the job at the front of the queue, runs it for min(quantum, remaining) time units, and then — if the job still has work left — puts it at the back of the queue; otherwise the job completes at that moment. Return the list of (id, completion_time) pairs in the order the jobs complete.

Function signature

def round_robin(jobs: list[tuple[str, int]], quantum: int) -> list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id, completion_time) pairs in completion order."""
    ...

Follow-up (as reported): Report note ("a constraint was changed"): add per-job arrival times so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

EXAMPLES

Example 1	
Input	`jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2`
Output	`[("C", 6), ("A", 8), ("B", 9)]`
Why	`t=0-2 A (rem 2), t=2-4 B (rem 1), t=4-6 C completes at 6, t=6-8 A completes at 8, t=8-9 B completes at 9.`

Example 2	
Input	`jobs = [("X", 5)], quantum = 2`
Output	`[("X", 5)]`
Why	`One job, run in slices 2+2+1, finishing at time 5.`

Example 3	
Input	`jobs = [("P", 1), ("Q", 1)], quantum = 3`
Output	`[("P", 1), ("Q", 2)]`
Why	`Each finishes within a single quantum: P at 1, Q at 2.`

CONSTRAINTS

- 1 <= len(jobs) <= 10^4
- 1 <= burst <= 10^6 , 1 <= quantum <= 10^6

QUESTION

- Job ids are unique, non-empty strings.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: main function ----
# Idea: a queue of jobs. Take the job at the front, run it for one time
# slice (at most `quantum`), and if it is not finished put it at the
# BACK of the queue. Record the time each job finishes.
# jobs is a list of (id, burst time); the result is a list of
# (id, finish time) in the order the jobs finish.
# Complexity: Time O(n + T / quantum) time slices for total burst time T, Space O(n)
def round_robin(jobs, quantum):
    queue = deque()
    for job_id, burst in jobs:
        queue.append([job_id, burst])
    clock = 0
    finished = []
    while queue:
        job = queue.popleft()
        run = quantum
        if job[1] < run:
            run = job[1]
        clock += run
        job[1] -= run
        if job[1] > 0:
            queue.append(job)                       # not finished: back of the line
        else:
            finished.append((job[0], clock))
    return finished
```

COMPLEXITY | Time O(total burst / quantum + n) | Space O(n)

150 Aggregate FizzBuzz Offer Matches

MEDIUM

Template fit: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front

QUESTION

You are given a list of offers in priority order (earliest = highest priority). Each offer is a dict with keys type (either "Fizz" or "Buzz"), name (a string), and quantity (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be partially fulfilled. Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming min(remaining quantities) and recording a match (fizz_name, buzz_name, matched_quantity) ; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue. Aggregate matches so the same (fizz_name, buzz_name) pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: "FizzBuzz! <quantity> <FizzName>:<BuzzName>" .

Function signature

def fizzbuzz_matches(offers: list[dict]) -> list[str]:

...

EXAMPLES

QUESTION

Example 1	
Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 5}, {"type": "Buzz", "name": "X", "quantity": 3}, {"type": "Buzz", "name": "Y", "quantity": 4}, ]`
Output	`["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]`
Why	`A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining 2 -> (A,Y,2); Y's leftover 2 waits with no Fizz to match.`

Example 2	
Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 3}, {"type": "Buzz", "name": "X", "quantity": 1}, {"type": "Buzz", "name": "X", "quantity": 2}, ]`
Output	`["FizzBuzz! 3 A:X"]`
Why	`A matches the first X for 1 and the second X for 2. Both are the pair (A, X), so they aggregate to a single "FizzBuzz! 3 A:X".`

CONSTRAINTS

- 0 <= len(offers) <= 10**4 ; quantities are positive integers.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def other_side(kind):
    if kind == "Fizz":
        return "Buzz"
    return "Fizz"


def match_offers(offers):
    # Each offer is {"type": "Fizz" or "Buzz", "name": ..., "quantity": ...}.
    # Offers are handled in order. A new offer is matched against the
    # OPPOSITE side's waiting line, front first, for as long as it has
    # quantity left and the line is not empty; the rest of the offer
    # waits in its own line. Returns the matched totals per
    # (fizz name, buzz name) pair and the two waiting lines.
    queues = {"Fizz": deque(), "Buzz": deque()}       # each entry is [name, quantity left]
    totals = {}
    for offer in offers:
        kind = offer["type"]
        mine = queues[kind]
        other = queues[other_side(kind)]
        left = offer["quantity"]
        while left > 0 and len(other) > 0:
            front = other[0]
            matched = left
            if front[1] < matched:
                matched = front[1]
            if kind == "Fizz":
                pair = (offer["name"], front[0])
            else:
                pair = (front[0], offer["name"])
            totals[pair] = totals.get(pair, 0) + matched
            left -= matched
            front[1] -= matched
            if front[1] == 0:
                other.popleft()
        if left > 0:
            mine.append([offer["name"], left])
    return totals, queues
```

```
# ---- main question: main function ----
# Idea: two FIFO queues (one per side) and an ordered total per pair of
```

```
# names. Matching an offer against the front of the opposite queue until
# the offer or the queue is used up is match_offers above; this
# function turns the totals into the lines "FizzBuzz! quantity fizz:buzz".
# Complexity: Time O(n + matches), Space O(n)
def fizzbuzz_matches(offers):
    totals, queues = match_offers(offers)
    lines = []
    for fizz_name, buzz_name in totals:
        lines.append("FizzBuzz! " + str(totals[(fizz_name, buzz_name)]) + " " + fizz_name + ":" + buzz_name)
    return lines
```

COMPLEXITY | Time O(offers + matches) | Space O(offers)

151 Restaurant Waitlist: join / delete / find_first_match — MEDIUM

Template fit: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets

QUESTION

Design a restaurant waitlist. join(name, size) adds a party with its size at the back. delete(name) removes a party (it may be anywhere in the queue). find_first_match(table) returns the name of the party that joined earliest among those whose size fits the table (size at most table), or None.

Function signature

class Waitlist:
  def join(self, name: str, size: int) -> None
  def delete(self, name: str) -> None
  def find_first_match(self, table: int) -> str | None

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.join('ann', 4)`
	`w.join('bob', 2)`
	`w.join('cy', 3)`
	`w.find_first_match(3)`
	`w.delete('bob')`
	`w.find_first_match(3)`
	`w.find_first_match(1)`

Output	`w.find_first_match(3) -> 'bob'`
	`w.find_first_match(3) -> 'cy'`
	`w.find_first_match(1) -> null`

Why	A table for 3 skips ann (4 is too big) and takes bob, who joined before cy. After bob leaves it is cy. No party fits a table for 1.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: main function ----
# Idea: one queue per party size, plus a dictionary of the people who
# are still waiting. join puts (arrival number, name) in the queue of its
# size. delete only removes the person from the dictionary ("lazy
# deletion": the old entry stays in its queue and is skipped later).
```

```python
# find_first_match looks at the front of every queue whose size fits the
# table (after removing deleted people from the front) and returns the
# earliest arrival among them. It does not seat anybody.
# Complexity: Time join O(1), delete O(1), find_first_match O(S + deleted entries skipped) for S party sizes,
 Space O(n)
class Waitlist:
    def __init__(self):
        self.counter = 0                            # arrival number: lower means earlier
        self.waiting = {}                           # name -> arrival number (people still waiting)
        self.queues = {}                            # party size -> deque of (arrival number, name)

    def join(self, name, size):
        self.counter += 1
        self.waiting[name] = self.counter
        if size not in self.queues:
            self.queues[size] = deque()
        self.queues[size].append((self.counter, name))

    def delete(self, name):
        if name in self.waiting:
            del self.waiting[name]

    def front(self, queue):
        # drop deleted (or re-joined) people from the front, then look at the first one
        while queue:
            number, name = queue[0]
            if self.waiting.get(name) == number:
                return queue[0]
            queue.popleft()
        return None

    def find_first_match(self, table):
        best = None
        for size in self.queues:
            if size > table:
                continue                            # this party is too big for the table
            head = self.front(self.queues[size])
            if head is not None and (best is None or head[0] < best[0]):
                best = head
        if best is None:
            return None
        return best[1]
```

COMPLEXITY | Time O(1) join and delete, O(S) find (S distinct sizes up to the table size) | Space O(n)

152 Implement LRU Cache. HARD

Template fit: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end)

QUESTION

Your program keeps re-reading the same data from disk, so you want an in-memory cache that holds only the n most recently used items (a least-recently-used cache). Write a class LRUCache(n) with get(key), which returns the stored value or -1, and put(key, value), which stores a value and, when the cache is full, first removes the item that was used least recently. Both get and put count as a use. Both must run in O(1).

Function signature

class LRUCache:
  def __init__(self, n: int)
  def get(self, key) -> int
  def put(self, key, value) -> None

QUESTION

EXAMPLES

Example 1

Input	`c = LRUCache(2)`
	`c.put(1, 1)`
	`c.put(2, 2)`
	`c.get(1)`
	`c.put(3, 3)`
	`c.get(2)`
	`c.put(4, 4)`
	`c.get(1)`
	`c.get(3)`
	`c.get(4)`

Output	`c.get(1) -> 1`
	`c.get(2) -> -1`
	`c.get(1) -> -1`
	`c.get(3) -> 3`
	`c.get(4) -> 4`

Why	`get(1) makes key 1 the most recent, so put(3, 3) evicts key 2; then put(4, 4) evicts key 1 (older than 3).`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class Node:
    # one entry of the doubly linked list
    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


# ---- main question: main function ----
# Idea: a dictionary maps each key to its node in a doubly linked list
# that is ordered by recency (most recently used at the FRONT). Two
# dummy nodes, `head` and `tail`, sit at the ends so no operation has
# to treat an empty list or the ends specially. get and put are O(1).
# Complexity: Time get and put are O(1), Space O(capacity)
class LRUCache:
    def __init__(self, n):
        self.capacity = n
        self.nodes = {}                              # key -> node
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.nodes:
            return -1
        node = self.nodes[key]
```

```
        self.unlink(node)                           # reading makes it the most recently used
        self.push_front(node)
        return node.value

    def put(self, key, value):
        if key in self.nodes:
```

```
            node = self.nodes[key]
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.nodes) == self.capacity:
            oldest = self.tail.prev                 # full: evict the least recently used (at the back)
            self.unlink(oldest)
            del self.nodes[oldest.key]
        node = Node(key, value)
        self.nodes[key] = node
        self.push_front(node)
```

COMPLEXITY | Hash map plus doubly linked list (built by hand): Time O(1) per operation | Space O(capacity)
OrderedDict version: Time O(1) per operation | Space O(capacity)

GROUP 19 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE | 2 E 4 M 1 H

Rolling state: the next state comes only from the previous one

153 Climbing Stairs

EASY

Template fit: state = the last two counts: a, b = b, a + b, repeated n - 1 times

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.

Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:

1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:

1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

EXAMPLES

Example 1	
Input	2
Output	2
Why	Two ways to climb 2 steps: 1+1 or 2.

Example 2	
Input	3
Output	3
Why	Three ways to climb 3 steps: 1+1+1, 1+2, 2+1.

Example 3	
Input	5
Output	8
Why	Eight ways for 5 steps.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the ways to reach step n are the ways to reach step n - 1 (then
# one step) plus the ways to reach step n - 2 (then two steps). Keep just
# the last two numbers: (a, b) becomes (b, a + b). Start with
# ways(0) = ways(1) = 1. O(n) time, O(1) space, no table, no recursion.
# Complexity: Time O(n), Space O(1)
def climb(n):
    a = 1
```

```
    b = 1
    for i in range(n - 1):
        a, b = b, a + b
    if n >= 1:
        return b
    return 1
```

COMPLEXITY | Time O(n) | Space O(1)

154 Fibonacci Numbers

EASY

Template fit: the same pair update starting from (0, 1)

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

EXAMPLES

Example 1	
Input	`0`
Output	`1`
Why	`F(0) = 1 and F(1) = 1, as defined in this question.`

Example 2	
Input	`5`
Output	`8`
Why	`1, 1, 2, 3, 5, 8: F(5) = 8.`

Example 3	
Input	`10`
Output	`89`
Why	`F(10) = 89.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main functions ----
# This question numbers the sequence 1, 1, 2, 3, 5, 8 ...: fib(0) = fib(1) = 1.
# The definition (slow: about 1.6 ** n calls; say it first):
# Complexity: Time O(2^n) (the same values are computed again and again), Space O(n) for the recursion
def fib_slow(n):
    if n < 2:
        return 1
    return fib_slow(n - 1) + fib_slow(n - 2)


# Idea: keep only the previous two values and move forward n times.
# O(n) time and O(1) memory (memoising the recursion gives the same time
# with O(n) memory).
# Complexity: Time O(n), Space O(1)
def fib(n):
    a = 1
    b = 1
    for i in range(n):
        a, b = b, a + b
    return a
```

COMPLEXITY | Iteration with two variables: Time O(n) | Space O(1)
Fast doubling: Time O(log n) big-integer operations | Space O(log n) recursion depth

155 Maximize sum without choosing adjacent elements — MEDIUM

Template fit: exactly the house robber update on the array

QUESTION

Given a list of non-negative numbers, choose some of them so that no two chosen numbers are neighbours in the list, and return the largest possible sum of the chosen numbers.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	[2, 7, 9, 3, 1]
Output	12
Why	Choose 2, 9 and 1 (positions 0, 2, 4): 12.

Example 2	
Input	[5]
Output	5
Why	One number: take it.

Example 3	
Input	[]
Output	0
Why	Nothing to choose: 0.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y
```

```python
# ---- main question: main function ----
# Idea: keep two running numbers: `take` = the best total whose LAST
# chosen element is the current one, `skip` = the best total that does
# not use the current one. For the next number x:
#   new take = skip + x   (the previous element must have been skipped)
#   new skip = the bigger of take and skip.
# Complexity: Time O(n), Space O(1)
def rob_line(nums):
    take = 0
    skip = 0
    for x in nums:
        new_take = skip + x
        new_skip = bigger(take, skip)
        take = new_take
        skip = new_skip
    return bigger(take, skip)
```

COMPLEXITY | Time O(n) | Space O(1)

156 Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses. MEDIUM

Template fit: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)

QUESTION

Houses stand in a line and house i holds nums[i] money. A robber cannot rob two houses that are next to each other. Return the largest total amount he can steal.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 3, 1]`
Output	`4`
Why	`Rob houses 0 and 2: 1 + 3 = 4.`

Example 2	
Input	`[2, 7, 9, 3, 1]`
Output	`12`
Why	`Rob houses 0, 2 and 4: 2 + 9 + 1 = 12.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea: two running numbers. `take` = the best total when the current
# house IS robbed, `skip` = the best total when it is not. For the next
# house with money x: new take = skip + x (the house before must have
# been skipped), new skip = the bigger of take and skip.
# Complexity: Time O(n), Space O(1)
```

```python
def rob_line(nums):
    take = 0
    skip = 0
    for x in nums:
        new_take = skip + x
        new_skip = bigger(take, skip)
        take = new_take
        skip = new_skip
    return bigger(take, skip)
```

COMPLEXITY | Time O(n) | Space O(1)

157 Paint House II — MEDIUM

Template fit: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)

QUESTION

There are n houses in a row, each to be painted one of k colors. costs[i][j] is the cost to paint house i with color j . No two adjacent houses may share the same color. Return the minimum total cost to paint all houses.

Function signature

def min_cost_paint_houses(costs: list[list[int]]) -> int:

...

Follow-up (as reported): Reduce the per-row transition from O(k) per cell to O(1) amortized by tracking the two smallest values of the previous row.

EXAMPLES

Example 1	
Input	costs = [[1, 5, 3], [2, 9, 4]]
Output	5
Why	Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2 (3) and house 1 color 0 (2) -> 5.

Example 2	
Input	costs = [[1, 3], [2, 4]]
Output	5
Why	1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.

Example 3	
Input	costs = [[7]]
Output	7
Why	A single house, one color.

CONSTRAINTS

- costs is an n x k grid, 0 <= n , k >= 1 (and k >= 2 whenever n >= 2 , so a valid painting exists).
- Costs are non-negative integers. Return 0 for n == 0 .
- Aim for O(n*k) time.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def two_smallest(values):
    # (index of the smallest, the smallest value, the second smallest value)
    first = 0
    for i in range(len(values)):
```

```
        if values[i] < values[first]:
            first = i
    second_value = values[first]                    # with one colour there is no second choice
    found_second = False
    for i in range(len(values)):
        if i != first and (not found_second or values[i] < second_value):
            second_value = values[i]
            found_second = True
    return first, values[first], second_value


# ---- main question: main function ----
# Every house is painted one of k colours; neighbouring houses must
# differ; costs[i][j] is the cost of colour j for house i. Find the
# cheapest total cost.
# Idea: the best total so far for a house painted with colour j is its
# cost plus the cheapest total of the previous house in a DIFFERENT
# colour. That is the smallest previous total, unless the smallest is
# colour j itself: then take the second smallest. So only the two
# smallest values of the previous row are needed: O(n * k).
# Complexity: Time O(n * k) for n houses and k colours, Space O(k)
def min_cost_paint_houses(costs):
    previous = [0] * len(costs[0])
    for row in costs:
        first, smallest, second_smallest = two_smallest(previous)
        current = []
        for j in range(len(row)):
            if j == first:
                current.append(row[j] + second_smallest)
            else:
                current.append(row[j] + smallest)
        previous = current
    return min(previous)
```

COMPLEXITY | Time O(n k) | Space O(1) extra

158 Write SQL code to publish the Fibonacci series. MEDIUM

Template fit: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query

QUESTION

Write a SQL query that publishes the first N Fibonacci numbers (0, 1, 1, 2, 3, 5, ...), one per row and in order, where N is given as the parameter :limit. Use a recursive common table expression.

Function signature

-- result: one column named fibonacci, N rows

EXAMPLES

Example 1	
Input	`:limit = 8`
Output	`0, 1, 1, 2, 3, 5, 8, 13`
Why	`Eight rows: each number is the sum of the two before it.`

Example 2	
Input	`:limit = 1`
Output	`0`
Why	`A single row.`

MAIN CODE | helper functions first, then the answer

```python
import sqlite3


# ---- main question: helper functions ----
# A recursive query: every new row is computed from the previous row.
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""


def run_query(sql, parameters):
    # run one query on a fresh in-memory database and return the first column of every row
    db = sqlite3.connect(":memory:")
    rows = db.execute(sql, parameters).fetchall()
    db.close()
    column = []
    for row in rows:
        column.append(row[0])
    return column


def fibonacci_python(count):
    # the same sequence in Python, used to check the SQL
    a = 0
    b = 1
    terms = []
    for i in range(count):
        terms.append(a)
        a, b = b, a + b
    return terms


# ---- main question: main function ----
# Idea: a recursive CTE that carries the three values (n, a, b). The
# first row is (1, 0, 1): the first term is 0 and the next pair is (0, 1).
# Every recursive step makes (n + 1, b, a + b) while n is below the limit,
# so column a holds the Fibonacci numbers.
# The limit is a QUERY PARAMETER (the ? above), never glued into the SQL
# text, and the recursion runs inside the database: one round trip for any n
# (this is also follow-up 4).
```

```python
# Complexity: Time O(n) rows for n terms, Space O(n)
def fibonacci(count):
    return run_query(FIB_SQL, (count,))
```

COMPLEXITY | Time O(n) | Space O(1) working set (O(n) result)

159 Constrained Dice Roll Sequences — HARD

Template fit: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit)

QUESTION

You roll a single 6-faced die exactly n times. You are given an array roll_max of length 6, where roll_max[i] is the maximum number of times the value i + 1 may appear consecutively. That is, you may never roll the same value more than roll_max[i] times in a row. Return the number of distinct sequences of n rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo 10**9 + 7 .

QUESTION

Function signature

def count_dice_sequences(n: int, roll_max: list[int]) -> int:

...

Follow-up (as reported): The interviewer's actual twist used two dice per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

EXAMPLES

Example 1	
Input	`n = 2, roll_max = [1, 1, 2, 2, 2, 3]`
Output	`34`
Why	`There are 6*6 = 36 sequences of length 2. Two of them are forbidden: rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.`

Example 2	
Input	`n = 2, roll_max = [1, 1, 1, 1, 1, 1]`
Output	`30`
Why	`No value may repeat consecutively, so the 6 sequences "value v then v" are removed: 36 - 6 = 30.`

Example 3	
Input	`n = 3, roll_max = [1, 1, 1, 2, 2, 3]`
Output	`181`
Why	`Hand-verified against the memoized DP over (rolls_used, last_value, run_length).`

CONSTRAINTS

- 1 <= n <= 5000 roll_max has exactly 6 entries, each 1 <= roll_max[i] <= 15 .
- A naive brute force over all 6**n sequences is far too slow; you must memoize over (rolls_used, last_value, consecutive_count_of_last_value) .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
MOD = 10 ** 9 + 7


def total_of(dp):
    # the sum of every number in the table
    total = 0
    for row in dp:
        for count in row:
            total += count
    return total


# ---- main question: main function ----
# n rolls of a six-sided die; the value v+1 may appear at most roll_max[v]
# times IN A ROW. Count the valid sequences (modulo 10^9 + 7).
# Idea (DP over the last value and how long it has been repeated):
# dp[v][r] = the number of valid sequences so far that end with value v
# repeated r times in a row. For the next roll:
#   - a DIFFERENT value than before starts a run of length 1:
#     (all sequences) - (those that already end in v);
#   - the SAME value extends its run by one (only while r < roll_max[v]).
# Complexity: Time O(n * S) where S is the sum of the roll limits (at most 90), Space O(S)
def count_dice_sequences(n, roll_max):
```

```
    dp = []
    for v in range(6):
        row = [0] * (roll_max[v] + 1)
        row[1] = 1                                          # one roll: every value once
        dp.append(row)
    for roll in range(n - 1):
        everything = total_of(dp) % MOD
        new = []
        for v in range(6):
            new.append([0] * (roll_max[v] + 1))
        for v in range(6):
            ending_in_v = 0
            for count in dp[v]:
                ending_in_v += count
            new[v][1] = (everything - ending_in_v) % MOD
            for r in range(1, roll_max[v]):
                new[v][r + 1] = dp[v][r]
        dp = new
    return total_of(dp) % MOD
```

COMPLEXITY | Time O(n * 6 * R) | Space O(6 * R)

GROUP 20 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE | 4 E 3 M

Two pointers: move i and j toward each other (or along two lists)

160 Reverse a linked list. EASY

Template fit: three pointers prev, current, next: point current.next back at prev, then move all three one step

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

EXAMPLES

Example 1	
Input	`head = 1 -> 2 -> 3`
Output	`vals(reverse_list(head)) -> [3, 2, 1]`
Why	`1 -> 2 -> 3 becomes 3 -> 2 -> 1.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make_list(values):
    # [1, 2, 3] -> 1 -> 2 -> 3
    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


# ---- main question: main function ----
# Idea: walk with `prev` and `node`. Remember node.next FIRST (the step
# people forget), point node back at prev, then move both forward. When
# node is None, prev is the new head.
# Complexity: Time O(n), Space O(1)
def reverse_list(head):
    prev = None
    node = head
    while node:
        following = node.next
        node.next = prev
        prev = node
        node = following
    return prev
```

COMPLEXITY | Time O(n) | Space O(1)

161 Sort a binary array in linear time. EASY

Template fit: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them

QUESTION

Sort an array that contains only 0s and 1s in place, in one pass over the array (O(n) time) and with O(1) extra space.

Function signature

def sort_binary(arr: list) -> list

EXAMPLES

Example 1	
Input	`[1, 0, 1, 0, 0, 1]`
Output	`[0, 0, 0, 1, 1, 1]`
Why	`All the 0s move to the front.`

Example 2	
Input	`[0, 0, 1, 1]`
Output	`[0, 0, 1, 1]`
Why	`Already sorted arrays stay as they are.`

Example 3	
Input	`[1]`
Output	`[1]`
Why	`A single value.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: two pointers. Zeros collect on the left, ones on the right.
# Move lo past a 0, move hi back past a 1; when lo sits on a 1 and hi on
# a 0, swap them. Every step moves a pointer: O(n), in place.
# Complexity: Time O(n), Space O(1)
def sort_binary(arr):
    lo = 0
    hi = len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            arr[lo], arr[hi] = arr[hi], arr[lo]       # a 1 on the left and a 0 on the right
            lo += 1
            hi -= 1
    return arr


# the same idea for records that only have a 0/1 key: swapping keeps the
# objects themselves (counting would lose them). Not stable.
# Complexity: Time O(n), Space O(1)
def sort_records(items, key):
    lo = 0
    hi = len(items) - 1
    while lo < hi:
        if key(items[lo]) == 0:
            lo += 1
```

```
        elif key(items[hi]) == 1:
            hi -= 1
        else:
            items[lo], items[hi] = items[hi], items[lo]
            lo += 1
            hi -= 1
    return items


# the same idea for 0, 1 and 2 (Dutch national flag)
# Complexity: Time O(n), Space O(1)
def sort_three(arr):
    low = 0              # arr[:low] are all 0
    mid = 0              # arr[low:mid] are all 1
    high = len(arr)      # arr[high:] are all 2; arr[mid:high] is not looked at yet
    while mid < high:
```

```
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 2:
            high -= 1
            arr[mid], arr[high] = arr[high], arr[mid]
        else:
            mid += 1
    return arr
```

COMPLEXITY | Two pointers swapping from both ends: Time O(n) | Space O(1)
Three values (Dutch national flag): Time O(n) | Space O(1)

162 Is this a valid palindrome? EASY

Template fit: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

EXAMPLES

Example 1	
Input	`'A man, a plan, a canal: Panama'`
Output	`true`
Why	`Ignoring punctuation, spaces and case it reads 'amanaplanacanalpanama'.`

QUESTION

Example 2	
Input	`'race a car'`
Output	`false`
Why	`'raceacar' is not a palindrome.`

Example 3	
Input	`' '`
Output	`true`
Why	`Nothing but a space: nothing to compare, so it is a palindrome.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: two pointers, one at each end, moving toward each other. Skip
# characters that are not letters or digits and compare the lowercase
# forms. The first mismatch ends it; nothing is copied.
# Complexity: Time O(n), Space O(1)
def is_palindrome(s):
    lo = 0
    hi = len(s) - 1
    while lo < hi:
        if not s[lo].isalnum():
            lo += 1
        elif not s[hi].isalnum():
            hi -= 1
        elif s[lo].lower() != s[hi].lower():
            return False
        else:
            lo += 1
            hi -= 1
    return True
```

COMPLEXITY | Time O(n) | Space O(1)

163 Reverse a Sentence — EASY

Template fit: swap from both ends to reverse the whole array, then reverse every word the same way

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

EXAMPLES

QUESTION

Example 1	
Input	`arr = ['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']`
Output	`''.join(reverse_words(list('the sky is blue'))) -> 'blue is sky the'`
Why	The words come out in reverse order while each word keeps its letters in order (the array is reversed in place).

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1


def find_words(arr):
    # the (start, end) of every word: a run of characters without a space
    words = []
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        words.append((i, j))
        i = j + 1
    return words


# ---- main question: main function ----
# Idea: reverse the WHOLE array first (the words are now in the right
# order but spelled backwards), then reverse each word back. Everything
# happens inside the same array: O(n) time, O(1) extra space. Spaces stay
# where the reversal puts them.
# Complexity: Time O(n), Space O(1) extra
def reverse_words(arr):
    reverse_range(arr, 0, len(arr) - 1)
    for start, end in find_words(arr):
        reverse_range(arr, start, end)
    return arr
```

COMPLEXITY | Time O(n) | Space O(1) extra on the character array

164 Find median of merged RLE arrays — MEDIUM

Template fit: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position

QUESTION

Two sorted arrays are stored run-length encoded as (value, count) pairs in increasing order of value. Return the median of all their elements together, without expanding the runs (a count can be huge).

Function signature

def median_rle(a: list, b: list) -> float

EXAMPLES

QUESTION

Example 1	
Input	`[(1, 2), (5, 1)], [(3, 3)]`
Output	`3`
Why	`Combined sequence 1, 1, 3, 3, 3, 5: the two middle elements are 3 and 3.`

Example 2	
Input	`[(1, 1)], [(2, 1), (9, 1)]`
Output	`2`
Why	`Combined sequence 1, 2, 9: the middle one is 2.`

Example 3	
Input	`[(1, 1000000)], [(5, 1000000)]`
Output	`3`
Why	`Counts may be enormous: a million 1s and a million 5s have median 3.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def total_count(runs):
    # how many elements the run-length list stands for
    total = 0
    for value, count in runs:
        total += count
    return total


def kth_smallest(runs_a, runs_b, k):
    # the element of (zero-based) rank k in the merged order, found by
    # walking the runs of the two sorted lists; nothing is expanded
    i = 0
    j = 0
    seen = 0
    while True:
        take_a = j >= len(runs_b)                    # b is used up: take from a
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        if take_a:
            value, count = runs_a[i]
        else:
            value, count = runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1


# ---- main question: main function ----
# Idea: let N be the total number of elements. For odd N the median has
# rank N // 2; for even N it is the average of the elements of rank
# N // 2 - 1 and N // 2. Walk the two run lists in sorted order and stop
# at those ranks. a and b are [(value, count), ...] sorted by value.
# Complexity: Time O(a + b) for a and b runs, Space O(1)
def median_rle(a, b):
    total = total_count(a) + total_count(b)
    if total % 2 == 1:
```

```
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2
```

COMPLEXITY | Time O(r1 + r2) | Space O(1)

165 Run-Length Vector Storage + Dot Product — MEDIUM

Template fit: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs

QUESTION

Two vectors of the same length are stored run-length encoded as (value, count) pairs. Compute their dot product without expanding them to full arrays.

Function signature

def rle_dot(a: list, b: list) -> int

EXAMPLES

Example 1	
Input	`[(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]`
Output	`25`
Why	`A = [3,3,3,0,0,0,4,4], B = [1,1,1,1,2,2,2,2]: 3*1*3 + 0 + 4*2*2 = 9 + 16 = 25.`

Example 2	
Input	`[(2, 1)], [(5, 1)]`
Output	`10`
Why	`One element each: 2 * 5.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def rle_encode(values):
    # [3, 3, 3, 0, 0] -> [(3, 3), (0, 2)]: each run is (value, how many in a row)
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1] = (v, runs[-1][1] + 1)
        else:
            runs.append((v, 1))
    return runs


def rle_decode(runs):
    values = []
    for v, count in runs:
        for i in range(count):
            values.append(v)
    return values


# ---- main question: main function ----
# Idea: two pointers over the RUNS (never expand them). Look at what is
# left of the current run of a and of b, multiply the two values by the
# overlap, then move on in whichever run is used up (or in both).
# Complexity: Time O(a + b) for a and b runs, Space O(1)
def rle_dot(a, b):
    i = 0
    j = 0
```

```
    left_a = 0
    left_b = 0
    if a:
        left_a = a[0][1]
    if b:
        left_b = b[0][1]
    total = 0
    while i < len(a) and j < len(b):
        take = left_a                       # both runs overlap for min(left_a, left_b) positions
        if left_b < take:
            take = left_b
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            if i < len(a):
```

```
                left_a = a[i][1]
        if left_b == 0:
            j += 1
            if j < len(b):
                left_b = b[j][1]
    return total
```

COMPLEXITY | Time O(r1 + r2) | Space O(1)

166 Count Unique Triplets That Sum to Zero — MEDIUM

Template fit: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

QUESTION

Given an integer array nums , count how many unique triplets (nums[i], nums[j], nums[k]) with three distinct indices i < j < k satisfy nums[i] + nums[j] + nums[k] == 0 . Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Function signature

def count_zero_sum_triplets(nums: list[int]) -> int:

...

Follow-up (as reported): Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

EXAMPLES

Example 1	
Input	`nums = [-1, 0, 1, 2, -1, -4]`
Output	`2`
Why	`The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1]. Even though [-1, 0, 1] can be formed with different index sets, it is counted once.`

Example 2	
Input	`nums = [0, 0, 0, 0]`
Output	`1`
Why	`The only triplet is [0, 0, 0].`

QUESTION

Example 3	
Input	`nums = [1, 2, 3]`
Output	`0`
Why	`No three elements sum to 0.`

CONSTRAINTS

- 0 <= len(nums) <= 3000
- -10^5 <= nums[i] <= 10^5
- The array may contain duplicates.
- If len(nums) < 3 , the answer is 0 .

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: sort. Fix the smallest value nums[i] and sweep two pointers
# (left, right) inward over the rest: a sum below 0 needs a bigger left
# value, a sum above 0 needs a smaller right value. Skip repeated values
# so each triplet is counted once.
# Complexity: Time O(n^2), Space O(n) for the sorted copy
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
    count = 0
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue                                # the same smallest value again
        left = i + 1
        right = len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1                       # skip a repeated middle value
    return count
```

COMPLEXITY | Time O(n^2) | Space O(1) extra

Walk two strings at once with two indices

167 Find the First Occurrence of a Substring — EASY

Template fit: try every start i; inside, j walks the needle; a full match returns i

QUESTION

Reconstructing the "find substring in array" item as substring search: given two strings haystack and needle , return the index of the first occurrence of needle in haystack , or -1 if needle is not part of haystack .

Function signature

def str_str(haystack: str, needle: str) -> int:
    """Return the index of the first occurrence of needle in haystack, or -1."""
    ...

Follow-up (as reported): Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

EXAMPLES

Example 1	
Input	`haystack = "sadbutsad", needle = "sad"`
Output	`0`
Why	`"sad" occurs at indices 0 and 6; the first is index 0.`

Example 2	
Input	`haystack = "leetcode", needle = "leeto"`
Output	`-1`
Why	`"leeto" does not occur in "leetcode".`

Example 3	
Input	`haystack = "abc", needle = "c"`
Output	`2`

CONSTRAINTS

- 1 <= len(haystack), len(needle) <= 10^4 haystack and needle consist of only lowercase English letters.
- If needle is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: try every start position i of the haystack; compare the needle
# letter by letter and stop at the first mismatch. The first start where
# all letters match is the answer; -1 if there is none.
# Complexity: Time O(n * m) in the worst case, Space O(1)
def str_str(haystack, needle):
    n = len(haystack)
    m = len(needle)
    for i in range(n - m + 1):
        j = 0
        while j < m and haystack[i + j] == needle[j]:
            j += 1
        if j == m:
            return i
    return -1
```

COMPLEXITY | Time O(n * m) worst case | Space O(1)

168 Determine if two words are sorted in lexicographic order. EASY

Template fit: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter

QUESTION

Words are ordered using a custom alphabet: rank maps each letter to its position in that alphabet. Return whether word a comes before word b (or is equal to it). The first letter where the words differ decides; if one word is a prefix of the other, the shorter one comes first.

Function signature

def in_order(a: str, b: str, rank: dict) -> bool

EXAMPLES

Example 1

Input	`alphabet = 'hlabcdefgijkmnopqrstuvwxyz' # 'h' comes first, then 'l', then a, b, c ...`
	`rank = {c: i for i, c in enumerate(alphabet)}`
	`in_order('hello', 'leetcode', rank)`
	`in_order('word', 'world', rank)`
	`in_order('app', 'apple', rank)`

Output	`in_order('hello', 'leetcode', rank) -> true`
	`in_order('word', 'world', rank) -> false`
	`in_order('app', 'apple', rank) -> true`

Why	`'h' ranks before 'l', so hello comes first. word vs world differ at d versus l, and d ranks after l, so the order is wrong. A prefix always comes first.`

MAIN CODE | helper functions first, then the answer

```
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# ---- main question: helper functions ----
def make_rank(order=ALPHABET):
    # the position of every letter in the alphabet: {"a": 0, "b": 1, ...}
    rank = {}
    for i in range(len(order)):
        rank[order[i]] = i
    return rank


def rank_of(rank, ch):
    # a character that is not in the alphabet is ranked after all the known ones
    if ch in rank:
        return rank[ch]
    return len(rank) + ord(ch)


# ---- main question: main function ----
# Idea: walk both words with one index. The first position where the
# letters differ decides: compare the two letters by their rank. If one
# word is a prefix of the other, the SHORTER one comes first ("app" is
# before "apple", not the other way round).
# Complexity: Time O(min(len(a), len(b))), Space O(1)
def in_order(a, b, rank):
    i = 0
    while i < len(a) and i < len(b):
        if a[i] != b[i]:
            return rank_of(rank, a[i]) < rank_of(rank, b[i])
        i += 1
    return len(a) <= len(b)                      # the prefix rule (and equal words are in order)
```

COMPLEXITY | Time O(min(len a, len b)) | Space O(1)

169 Return all unique substrings of a given string. EASY

Template fit: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes

QUESTION

Return every distinct substring of a string, sorted alphabetically (a substring is a contiguous piece, the whole string and single letters included).

Function signature

def unique_substrings(s: str) -> list

EXAMPLES

Example 1	
Input	'abc'
Output	['a', 'ab', 'abc', 'b', 'bc', 'c']
Why	Six different substrings of 'abc'.

Example 2	
Input	'aaa'
Output	['a', 'aa', 'aaa']
Why	'a' appears three times but is listed once.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea (the simplest): cut out every slice s[i:j] and put it in a set (a
# set keeps each text only once), then sort. That is O(n^3): fine for a
# short string; the follow-ups make it faster.
# Complexity: Time O(n^3) (n^2 substrings, each copied in O(n)), Space O(n^3) for the stored text
def unique_substrings(s):
    seen = set()
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
            seen.add(s[i:j])
    return sorted(seen)
```

COMPLEXITY | Time O(n^2 log n) with naive sorting (O(n log n) with a real suffix array) | Space O(n)

170 Can board states be transformed? MEDIUM

Template fit: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right

QUESTION

A board is a string of 'L', 'R', '#' and 'X'. 'X' is an empty square. An 'L' piece may move left into an empty square, an 'R' piece may move right into an empty square, pieces never jump over each other, and '#' (a wall piece) never moves. Return whether start can be turned into end by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

QUESTION

Example 1	
Input	'RXXLRXRXL', 'XRLXXRRLX'
Output	true
Why	L moves left and R moves right while keeping their order: this classic board is reachable.

Example 2	
Input	'XL', 'LX'
Output	true
Why	L can move left.

Example 3	
Input	'LX', 'XL'
Output	false
Why	L cannot move to the right.

Example 4	
Input	'#X', 'X#'
Output	false
Why	A wall never moves.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def pieces(board):
    # the symbols that are not blank ("X"), each with its position
    result = []
    for i in range(len(board)):
        if board[i] != "X":
            result.append((board[i], i))
    return result


# ---- main question: main function ----
# Idea: "L" can only move left, "R" only right, "X" is a blank cell and
# "#" is a wall that never moves. Pieces never pass each other, so the
# ORDER of the non-blank symbols is fixed. Take the pieces of both boards
# and compare them one by one (two pointers): the same symbol; an L may
# only end up further left (or in place), an R further right; a wall must
# stay exactly where it is. (Walls are also never jumped over, because
# the order of the pieces cannot change.)
# Complexity: Time O(n), Space O(n)
def can_transform(start, end):
    if len(start) != len(end):
        return False
    a = pieces(start)
    b = pieces(end)
    if len(a) != len(b):
        return False
    for k in range(len(a)):
        symbol_a, i = a[k]
        symbol_b, j = b[k]
        if symbol_a != symbol_b:
            return False
        if symbol_a == "L" and i < j:
            return False                    # an L cannot move right
```

```
        if symbol_a == "R" and i > j:
            return False                    # an R cannot move left
        if symbol_a == "#" and i != j:
            return False                    # a wall cannot move
    return True
```

COMPLEXITY | Time O(n) | Space O(1)

171 Decide string transform with directional tokens and walls

MEDIUM

Template fit: the same walk with dots as blanks and the movement rule from the statement (walls block)

QUESTION

Two equal-length strings start and target use the characters 'L', 'R', '.' (an empty slot) and 'X' (an immovable wall). An 'L' token can move left into empty slots, an 'R' token can move right into empty slots, tokens can never pass through each other or through a wall, and 'X' never moves. Return whether start can be turned into target by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	'R..L', '.RL.'
Output	true
Why	R moves one step right and L one step left: both stay in order.

Example 2	
Input	'L.', '.L'
Output	false
Why	L can only move left, never right.

Example 3	
Input	'X.', '.X'
Output	false
Why	A wall cannot move.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def pieces(board):
    # everything except the empty "." slots, each with its position
    result = []
    for i in range(len(board)):
        if board[i] != ".":
            result.append((board[i], i))
    return result


# ---- main question: main function ----
# Idea: "L" moves left, "R" moves right, "X" is a wall that never
# moves, "." is an empty slot. Pieces never pass each other, so the order
# of the non-dot symbols is fixed. Walk both strings skipping the dots:
# the next symbol must be equal; a wall must be at the same index; an L
# must have start index >= end index; an R must have start index <= end index.
# Complexity: Time O(n), Space O(n)
def can_transform(start, end):
    if len(start) != len(end):
```

```
        return False
    a = pieces(start)
    b = pieces(end)
    if len(a) != len(b):
        return False
    for k in range(len(a)):
        symbol_a, i = a[k]
        symbol_b, j = b[k]
        if symbol_a != symbol_b:
            return False
        if symbol_a == "X" and i != j:
            return False
        if symbol_a == "L" and i < j:
            return False
        if symbol_a == "R" and i > j:
            return False
    return True
```

COMPLEXITY | Time O(n) | Space O(1)

172 Determine if two sentences are similar. MEDIUM

Template fit: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

EXAMPLES

Example 1	
Input	`['great', 'acting', 'skills'], ['fine', 'drama', 'talent'], [['great', 'fine'], ['acting', 'drama'], ['skills', 'talent']]`
Output	`true`
Why	`Each word is paired with a similar word.`

Example 2	
Input	`['a'], ['a', 'b'], []`
Output	`false`
Why	`Sentences of different lengths are never similar.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: put every similar pair in a set, in both directions. Two sentences
# are similar when they have the same number of words and every word
# pair is equal or listed. This is NOT transitive: good ~ fine and
# fine ~ okay do not make good ~ okay.
# Complexity: Time O(n + p) for n words and p pairs, Space O(p)
def similar_direct(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    allowed = set()
    for a, b in pairs:
        allowed.add((a, b))
        allowed.add((b, a))
    for i in range(len(s1)):
        if s1[i] != s2[i] and (s1[i], s2[i]) not in allowed:
            return False
    return True
```

COMPLEXITY | Time O(n + P) | Space O(P)

173 Find Dictionary Matches for a Jammed String — HARD

Template fit: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3

QUESTION

A word was typed on a keyboard where some keys got stuck, so a letter may be repeated. Given a dictionary of words, matches(query) returns the dictionary words the query could be a jammed version of: the runs of equal letters line up with the word's runs, and each run of the query is either exactly as long as in the word or longer and at least min_stretch (default 3) letters long.

Function signature

class JammedIndex:
  def __init__(self, words: list)
  def matches(self, query: str, min_stretch: int = 3) -> list

EXAMPLES

Example 1

Input	`idx = JammedIndex(['hello', 'help', 'hel'])`
	`idx.matches('heeello')`
	`idx.matches('hello')`
	`idx.matches('heello')`
	`idx.matches('helllo')`

Output	`idx.matches('heeello') -> ['hello']`
	`idx.matches('hello') -> ['hello']`
	`idx.matches('heello') -> []`
	`idx.matches('helllo') -> ['hello']`

Why	'heeello' stretches the e to 3 letters, which is allowed. 'heello' only stretches it to 2 (shorter than min_stretch), and 'helllo' stretches the l run from 2 to 3 letters, which is allowed.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]: each letter with how many times in a row
    result = []
    for ch in word:
        if result and result[-1][0] == ch:
            result[-1] = (ch, result[-1][1] + 1)
        else:
```

```
            result.append((ch, 1))
    return result


def run_letters(word):
    # "hello" -> "helo": the letters of the runs, repeats collapsed
    letters = ""
    for ch, count in runs(word):
        letters += ch
    return letters


def run_counts(word):
    counts = []
    for ch, count in runs(word):
        counts.append(count)
    return counts


def stretch_fits(query_count, word_count, min_stretch):
    # a run matches when it is the same length, or when the query stretched
    # it to at least min_stretch letters
    return query_count == word_count or (query_count > word_count and query_count >= min_stretch)


# ---- main question: main function ----
# Idea: group the dictionary words by their run letters ("hello" -> "helo").
# A query can only match words in its own group, so compare the run
# lengths only there. Build the index once and reuse it for many queries.
# min_stretch is the shortest stretch that is allowed (follow-up 1: use 1
# for "any run may be longer").
# Complexity: Time build O(total length of the words), matches O(words in the group * runs), Space O(total le
ngth)
class JammedIndex:
    def __init__(self, words):
        self.groups = {}
        for word in words:
            letters = run_letters(word)
```

```
            if letters not in self.groups:
                self.groups[letters] = []
            self.groups[letters].append((word, run_counts(word)))

    def matches(self, query, min_stretch=3):
        letters = run_letters(query)
        counts = run_counts(query)
        found = []
        for word, word_counts in self.groups.get(letters, []):
            all_fit = True
            for i in range(len(counts)):
                if not stretch_fits(counts[i], word_counts[i], min_stretch):
                    all_fit = False
            if all_fit:
                found.append(word)
        return sorted(found)
```

COMPLEXITY | Time O(L) per query plus the size of the matching bucket | Space O(total dictionary length)

Simulate a machine: a class with state and one branch per kind of event

174 Mario-Style Score Tracker (OOP) EASY

Template fit: the template as written: an event name decides how score, lives and size change

QUESTION

Design a class that tracks the state of a platformer character across a sequence of events. The character has a score (starts at 0), a lives count (starts at 3), and a big flag (starts False). Process events with these rules: "coin" : add 100 to the score. "mushroom" : add 1000 to the score and set big = True . "enemy" : add 200 to the score (the character stomps an enemy). "damage" : if big is True , set big = False (no life lost); otherwise decrease lives by 1. Score is unchanged. The character is alive while lives > 0 . Once lives reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...
    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...
    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...
    def get_score(self) -> int:
        ...
    def is_alive(self) -> bool:
        ...
```

EXAMPLES

Example 1	
Input	`g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]); (g.get_score(), g.is_alive())`
Output	`(1300, True)`
Why	`coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False (no life lost); damage -> small, lives 3->2. Score 1300, still alive.`

Example 2	
Input	`g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(), g.is_alive())`
Output	`(0, False)`
Why	`Three small-state hits drop lives 3->0 (game over). The trailing "coin" is ignored.`

CONSTRAINTS

- Event strings are always one of the four listed above.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: a small class with the state (score, lives, big or small) and ONE
```

```
# method that owns the rules, one branch per kind of event. The game ends
# when the lives reach zero; events after that are ignored.
# Complexity: Time O(1) per event, Space O(1)
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return                                  # the game is over
        if event == "coin":
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False                    # damage first costs the mushroom power-up
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score

    def is_alive(self):
        return self.lives > 0
```

COMPLEXITY | Time O(events) | Space O(1)

175 Tic-Tac-Toe System with K Players and 3-in-a-Row Win — MEDIUM

Template fit: store the board; after each move count the run through the new cell in four directions and compare with the target length

QUESTION

Design Tic-Tac-Toe for k players on an n x n board. Players move in turn (1, 2, ..., k, then 1 again). move(player, r, c) places a mark and returns 'player p wins' as soon as that player has 3 marks in a straight line (row, column or diagonal), 'draw' when the board is full, and 'continue' otherwise. A move out of turn, off the board or onto a taken cell raises an error.

Function signature

class TicTacToe:
  def __init__(self, n: int, k: int, target: int = 3)
  def move(self, player: int, r: int, c: int) -> str

EXAMPLES

QUESTION

Example 1

Input	g = TicTacToe(3, 2) g.move(1, 0, 0) g.move(2, 1, 0) g.move(1, 0, 1) g.move(2, 1, 1) g.move(1, 0, 2)
Output	g.move(1, 0, 0) -> 'continue' g.move(2, 1, 0) -> 'continue' g.move(1, 0, 1) -> 'continue' g.move(2, 1, 1) -> 'continue' g.move(1, 0, 2) -> 'player 1 wins'
Why	Player 1 takes the whole top row while player 2 has two in the middle row: player 1 wins.

Example 2

Input	g = TicTacToe(3, 3) g.move(1, 0, 0) g.move(2, 0, 1) g.move(3, 0, 2) g.move(1, 1, 1)
Output	g.move(1, 0, 0) -> 'continue' g.move(2, 0, 1) -> 'continue' g.move(3, 0, 2) -> 'continue' g.move(1, 1, 1) -> 'continue'
Why	With three players the turns go 1, 2, 3, 1; nobody has three in a line yet.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, and the two diagonals


# ---- main question: main function ----
# Idea: place the mark, then check only the four lines THROUGH it. For each
# direction walk outward both ways while the cells hold the same player and
# add the two counts plus one for the new mark. A run of at least `target`
# (3 by default) wins at once. Players take turns 1, 2, ..., k, 1, ...
# `target` is the winning length w (follow-up 1): the check costs O(w)
# per move, whatever the board size.
# Complexity: Time move is O(w) for win length w, Space O(n^2)
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n = n
        self.k = k
        self.target = target
        self.board = []
        for r in range(n):
            self.board.append([0] * n)              # 0 means an empty cell
        self.moves = 0
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        player = self.board[r][c]
        run = 1
        for sign in (1, -1):
```

```
            a = r + sign * dr
            b = c + sign * dc
            while self.inside(a, b) and self.board[a][b] == player:
                run += 1
                a += sign * dr
                b += sign * dc
        return run

    def move(self, player, r, c):
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c] != 0:
            raise ValueError("illegal cell")
        self.board[r][c] = player
```

```
        self.moves += 1
        for dr, dc in DIRECTIONS:
            if self.run_length(r, c, dr, dc) >= self.target:
                self.over = True
                return "player " + str(player) + " wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"
```

COMPLEXITY | Time O(1) per move (O(target) when generalised) | Space O(n^2)

176 Design Tic-Tac-Toe With K Players — MEDIUM

Template fit: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.

QUESTION

- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, and the two diagonals


# ---- main question: main function ----
# Idea: only the four lines through the NEW mark can create a win. After
# a move, walk outward in both directions of each line while the cells hold
# the same player; the run length is those two counts plus the new mark.
# Players take turns 1, 2, ..., k, 1, ... An invalid move (wrong player,
# wrong turn, outside the board, taken cell) changes nothing.
# `target` is how many in a row win (follow-up 1); cells are stored in a
# dictionary, so only taken cells use memory (follow-up 3: huge board).
# Complexity: Time move is O(w) for win length w (four directions), Space O(moves made)
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n = n
        self.k = k
        self.target = target
        self.cells = {}                               # (row, column) -> the player who took it
        self.history = []                             # the moves so far, for undo
        self.winner = None

    def turn(self):
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # the number of marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a = r + sign * dr
            b = c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
```

```
                a += sign * dr
                b += sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        if player < 1 or player > self.k or player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n) or (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
```

```
        self.history.append((r, c))
        for dr, dc in DIRECTIONS:
            if self.run_length(r, c, dr, dc, player) >= self.target:
                self.winner = player
                return str(player) + " won"
        return "Continue"

    # follow-up 2: take the last move back (this also reopens a finished game)
    def undo(self):
        if len(self.history) == 0:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True
```

COMPLEXITY | Time O(1) per move | Space O(k * n)

177 Infinite Deck Card Duel

MEDIUM

Template fit: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins

QUESTION

Design an InfiniteDeck that can be drawn from forever: each call to draw() returns a uniformly random integer card value in [1, 13] (cards are never exhausted). Make the deck seedable so games are reproducible. Then simulate a duel between two players. In each round, each player draws hand_size cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a draw (nobody scores). Play num_rounds rounds and tally round wins per player. Finally, declare a champion: the player is champion only if they lead the other in round wins by at least 2; otherwise there is no champion. Implement: InfiniteDeck.draw() returning the next card. round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning draw), based on the highest card in each hand. decide_champion(wins1, wins2) -> 1 , 2 , or 0 (0 meaning no champion), applying the "lead by at least 2" rule. play_game(...) tying it together (uses the deck for randomness).

QUESTION

Function signature

```
import random
class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...
    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...
def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...
def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...
def play_game(deck: InfiniteDeck, hand_size: int = 5, num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Follow-up (as reported): Extend to n players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round. Introduce multiple independent InfiniteDeck instances (or one shared deck with per-card metadata) to support probabilistic house rules.

EXAMPLES

Example 1	
Input	`round_winner([3, 7, 2], [7, 1, 9])`
Output	`2`
Why	`Player 2's highest card (9) beats player 1's highest (7).`

Example 2	
Input	`round_winner([5, 5], [5, 4])`
Output	`0`
Why	`Both players' highest card is 5, so the round is a draw.`

CONSTRAINTS

- 1 <= hand_size <= 20 , 1 <= num_rounds <= 100000 , card values in [1, 13] .

MAIN CODE | helper functions first, then the answer

```python
import random


# ---- main question: helper functions ----
class InfiniteDeck:
    # an endless deck: every draw is a random card from 1 to 13. The deck
    # owns its random generator, so a seed makes a game repeatable.
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)
```

```
def best_card(hand):
    best = hand[0]
    for card in hand:
        if card > best:
            best = card
    return best


# ---- main question: main functions ----
# The rules are plain functions of hands and win counts.
# A round: each player draws a hand; the player whose BEST card is higher
# wins the round; equal best cards give nobody a point.
# Complexity: Time O(hand size), Space O(1)
def round_winner(hand1, hand2):
    best1 = best_card(hand1)
    best2 = best_card(hand2)
    if best1 == best2:
        return 0
    if best1 > best2:
        return 1
    return 2


# The champion is the player who leads by at least 2 rounds, else nobody (0).
# Complexity: Time O(1), Space O(1)
def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    if wins2 - wins1 >= 2:
        return 2
    return 0
```

```
# Complexity: Time O(rounds * hand size), Space O(hand size)
def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for round_number in range(num_rounds):
        hand1 = []
        hand2 = []
        for i in range(hand_size):
            hand1.append(deck.draw())
        for i in range(hand_size):
            hand2.append(deck.draw())
        winner = round_winner(hand1, hand2)
        if winner != 0:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])
```

COMPLEXITY | Time O(rounds * hand) | Space O(1)

178 Implement iterator merging lists with filtering — MEDIUM

Template fit: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()

QUESTION

Design an iterator over several lists that takes one element from each list in turn (a zigzag), skipping any value in the blocked set and any value that was already returned. hasNext() says whether another value is available and next() returns it.

QUESTION

Function signature

class FilteredZigzag:
    def __init__(self, lists: list, blocked=())
    def hasNext(self) -> bool
    def next(self) -> int

EXAMPLES

Example 1	

Input	`it = FilteredZigzag([[1, 2, 3], [4, 5, 6, 7], [8, 9]], blocked=[2, 5])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 4, 8, 9, 3, 6, 7]`
Why	`Order of visit: 1, 4, 8, then 2 (blocked), 5 (blocked), 9, then 3, 6, then 7. The blocked values never appear.`

Example 2	

Input	`it = FilteredZigzag([[1, 1], [1, 2]])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 2]`
Why	`Repeated values are returned only once.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: take one item from each list in turn (list 0, list 1, ..., back
# to list 0), skipping ids that are blocked and ids that were already
# returned. Keep one item of LOOKAHEAD ready, so hasNext() only has to
# check whether there is one. advance() goes round and round until it
# finds the next good item, or gives up after a whole round in which every
# list was empty.
# Complexity: Time hasNext O(1), next O(1) amortized per item scanned, Space O(items already returned)
class FilteredZigzag:
    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.position = [0] * len(lists)            # how far we have read in each list
        self.turn = 0                               # whose turn it is
        self.seen = set()                           # the ids already returned
        self.ahead = None                           # the next item to return
        self.advance()

    def advance(self):
        self.ahead = None
        empty_in_a_row = 0
        while empty_in_a_row < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.position[i] >= len(self.lists[i]):
                empty_in_a_row += 1                 # this list has nothing left
                continue
            value = self.lists[i][self.position[i]]
            self.position[i] += 1
            empty_in_a_row = 0
```

```
            if value not in self.blocked and value not in self.seen:
                self.ahead = value
                return

    def hasNext(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value
```

COMPLEXITY | Time O(1) amortized per value | Space O(number of lists + distinct ids yielded)

179 Design an elevator control system — MEDIUM

Template fit: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

QUESTION

Design the control logic for a multi-elevator building. A Request(floor, direction) is a call from a floor (direction +1 for up, -1 for down). Each Elevator keeps its current floor, direction and a set of stops. cost(request) estimates how expensive it is for this car to take the request (the distance, plus a detour penalty when the car is heading away); the dispatcher gives the request to the cheapest car. step() moves the car one floor towards its stops and returns the floor it served, if any.

Function signature

class Elevator:
  def __init__(self, car_id, floor=0)
  def cost(self, req: Request) -> float
  def step(self) -> int | None

EXAMPLES

Example 1

Input	`e = Elevator(1, floor=0)`
	`e.stops.update({3, 5})`
	`[e.step() for _ in range(6)]`

Output	`[e.step() for _ in range(6)] -> [null, null, null, 3, null, 5]`
Why	The car climbs one floor per step and stops at floors 3 and 5 (served when it is on the floor at the start of a step).

Example 2

Input	`idle = Elevator(2, floor=7)`
	`busy = Elevator(3, floor=2)`
	`busy.direction = 1`
	`busy.stops.update({9})`
	`idle.cost(Request(4, -1))`
	`busy.cost(Request(4, 1))`
	`busy.cost(Request(4, -1))`

Output	`idle.cost(Request(4, -1)) -> 3`
	`busy.cost(Request(4, 1)) -> 2`
	`busy.cost(Request(4, -1)) -> 14`

Why	An idle car costs its distance (3). A car already going up that will pass floor 4 going the same way costs its distance (2); going the other way it must finish its run first, so it costs much more.

MAIN CODE | helper functions first, then the answer

```python
INF = float("inf")


# ---- main question: helper functions ----
class Request:
    def __init__(self, floor, direction):
        self.floor = floor
        self.direction = direction            # +1 for a call going up, -1 for going down


# ---- main question: main function ----
# Idea: every car is a small state machine. It keeps a set of floors to
# stop at and a direction (SCAN, like a real elevator: keep going the same
# way while there are stops ahead, then reverse). step() is one tick: serve
# a stop on the current floor, then move one floor. cost(req) says how
# bad it would be to give this car a new call; a Controller (below) gives
# each call to the cheapest car.
# Complexity: Time step and cost are O(stops), Space O(stops)
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id = car_id
        self.floor = floor
        self.direction = 0                    # 0 = idle, +1 = going up, -1 = going down
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        if not self.in_service:
            return INF                        # a broken car never gets calls
        distance = abs(self.floor - req.floor)
        if self.direction == 0:
            return distance                   # idle: just the distance
        gap = req.floor - self.floor
        on_the_way = gap * self.direction >= 0        # the car is moving toward the floor ...
        if on_the_way and req.direction == self.direction:
            return distance                           # ... and the call goes the same way
        highest = max(self.stops)                     # otherwise the car finishes its sweep first
        lowest = min(self.stops)
        return distance + 2 * (highest - lowest + 1) + 10

    def step(self):
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
```

```python
        if len(self.stops) == 0:
            self.direction = 0
            return served
        stops_above = False
        stops_below = False
        for stop in self.stops:
            if stop > self.floor:
                stops_above = True
            if stop < self.floor:
                stops_below = True
        if (self.direction >= 0 and stops_above) or (self.direction < 0 and not stops_below):
            self.direction = 1                # keep the direction while there are stops ahead
        else:
            self.direction = -1
        self.floor += self.direction
        return served
```

COMPLEXITY | Time O(cars) per call, O(1) per car per step | Space O(stops)

180 Simulate In-Place Cellular State Updates

HARD

Template fit: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

QUESTION

Conway's Game of Life on a grid of 0 (dead) and 1 (alive) cells. A live cell with 2 or 3 live neighbours (of its 8 neighbours) stays alive, a dead cell with exactly 3 live neighbours becomes alive, and every other cell is dead in the next generation. All cells change at the same time. Do it in place with O(1) extra space; with wrap=True the grid wraps around its edges.

Function signature

def step_in_place(board: list, wrap=False) -> list

EXAMPLES

Example 1	
Input	`[[0, 1, 0], [0, 1, 0], [0, 1, 0]]`
Output	`[[0, 0, 0], [1, 1, 1], [0, 0, 0]]`
Why	The vertical blinker turns into a horizontal one.

Example 2	
Input	`[[1, 1], [1, 1]]`
Output	`[[1, 1], [1, 1]]`
Why	A 2x2 block is stable: every cell has 3 live neighbours.

Example 3	
Input	`[[0, 0, 0], [0, 1, 0], [0, 0, 0]]`
Output	`[[0, 0, 0], [0, 0, 0], [0, 0, 0]]`
Why	A lonely cell dies of isolation.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
AROUND = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]     # the 8 neighbours


def live_neighbors(board, r, c, wrap=False):
    rows = len(board)
    cols = len(board[0])
    count = 0
    for dr, dc in AROUND:
        a = r + dr
        b = c + dc
        if wrap:
            a = a % rows                                # a torus: the edges are glued together
            b = b % cols
        elif not (0 <= a < rows and 0 <= b < cols):
            continue
        count += board[a][b] & 1                        # bit 0 is the OLD state, even for cells already update
d
    return count


# ---- main question: main function ----
# Idea (Game of Life without a second board): store TWO bits in every
# cell. Bit 0 is the current state, bit 1 will hold the NEXT state.
# Pass 1: count the live neighbours of each cell from bit 0 only
# (value & 1), apply the rules (3 neighbours -> alive; a live cell with 2
# -> stays alive) and write the result into bit 1. Pass 2: shift every cell
# right by one so the new state becomes bit 0.
# Complexity: Time O(m * n), Space O(1) extra (the second bit lives in the board)
```

```python
def step_in_place(board, wrap=False):
    rows = len(board)
    cols = len(board[0])
    for r in range(rows):
        for c in range(cols):
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            if live == 3 or (alive == 1 and live == 2):
                board[r][c] |= 2                    # bit 1 stores the NEW state
    for r in range(rows):
        for c in range(cols):
            board[r][c] >>= 1                       # the new state moves into bit 0
    return board
```

COMPLEXITY | Time O(m * n) | Space O(1)

Top-K by frequency: Counter plus nsmallest

181 Top-K Frequent in Large Logs

MEDIUM

Template fit: count the first word of every line (the speaker); return the names of the k most frequent

QUESTION

A chat log is a list of lines, each starting with the speaker's name ('A says hi'). Return the k speakers who appear most often; ties are broken alphabetically by name.

Function signature

def top_k_speakers(chat_log: list, k: int) -> list

EXAMPLES

Example 1	
Input	['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 2
Output	['A', 'B']
Why	A spoke 3 times, B and C once each; B comes before C alphabetically.

Example 2	
Input	['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 3
Output	['A', 'B', 'C']
Why	Asking for 3 returns everyone, most frequent first.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: helper functions ----
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def count_speakers(chat_log):
    counts = {}
    for line in chat_log:
        name = speaker(line)
        counts[name] = counts.get(name, 0) + 1
    return counts


def rank_of(pair):
    # pair = (name, count): more messages first, ties by name
    return (-pair[1], pair[0])


# ---- main question: main function ----
# Idea: take the speaker out of each line and count them in a dictionary.
# heapq.nsmallest keeps a heap of only k entries, so it never sorts all d
# distinct users: O(n + d log k). Ties are broken by name so the output is
# always the same.
# Complexity: Time O(n + u log k) for n lines and u speakers, Space O(u)
def top_k_speakers(chat_log, k):
    counts = count_speakers(chat_log)
    best = heapq.nsmallest(k, counts.items(), key=rank_of)
```

```
    names = []
    for name, count in best:
        names.append(name)
    return names
```

COMPLEXITY | Count, then keep a size-K heap: Time O(n + d log k) | Space O(d)
Quickselect on the counts (average O(d)): Time O(n + d) average, O(d^2) worst case | Space O(d)

182 Rearrange Tasks With Cooldown

MEDIUM

*Template fit: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)*

QUESTION

You are given a list of tasks (letters; the same letter means the same kind of task). Every task takes one unit of time, and two tasks of the same kind must have at least n other tasks or idle units between them (the cooldown). The CPU may stay idle. Return the least number of time units needed to finish all the tasks.

Function signature

def least_interval(tasks: list, n: int) -> int

EXAMPLES

Example 1	
Input	['A', 'A', 'A', 'B', 'B', 'B'], 2
Output	8
Why	A B idle A B idle A B: three A's and three B's with a cooldown of 2 take 8 units.

Example 2	
Input	['A', 'A', 'A', 'B', 'B', 'B'], 0
Output	6
Why	No cooldown: just run the 6 tasks back to back.

Example 3	
Input	['A', 'B', 'C'], 5
Output	3
Why	All tasks differ, so no cooldown is ever needed.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def count_tasks(tasks):
    counts = {}
    for task in tasks:
        counts[task] = counts.get(task, 0) + 1
    return counts


# ---- main question: main function ----
# The same task must be at least n slots apart. How many time slots
# (including idle ones) are needed?
# Idea (closed form): let `most` be the highest frequency and `tied` the
# number of tasks that have that frequency. The most frequent task needs
# (most - 1) full frames of n + 1 slots plus one last row holding every
# tied task: (most - 1) * (n + 1) + tied. If there are so many different
# tasks that no idle slot is needed, the answer is just the number of tasks.
# Complexity: Time O(n), Space O(1) (at most 26 task types)
def least_interval(tasks, n):
    if not tasks:
```

```
        return 0
    counts = count_tasks(tasks)
    most = 0
    for count in counts.values():
        if count > most:
            most = count
    tied = 0
    for count in counts.values():
        if count == most:
            tied += 1
    frame_length = (most - 1) * (n + 1) + tied
    if frame_length > len(tasks):
        return frame_length
    return len(tasks)
```

COMPLEXITY | Time O(len(tasks)) | Space O(1) for a fixed alphabet

183 Implement frequency + distance top■K queries

MEDIUM

Template fit: count the point tuples; the tie-break is distance from the origin, then x, then y

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **`p`** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **`k` distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **`x`** ascending, then **`y`** ascending

Input

- **`points`**: list of length **`n`**, each element **`[x, y]`**
- **`k`**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **`k`** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

QUESTION

Example

If for key **"a"** you stored timestamps **[2, 10, 15]** with values **["x", "y", "z"]**:

- **getClosest("a", 11)** returns **"y"** (10 is closest)
- **getClosest("a", 14)** returns **"z"** (15 is closest)
- **getClosest("a", 12)** returns **"y"** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **2 * 10^5** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1] appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each appear twice. Their squared distances are 4, 5, and
    5, so [0,2] comes first. Between [1,2] and [2,1], x is smaller for [1,2].
```

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like **(-frequency, distance, x, y)**.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either **['set', key, value, timestamp]** or **['getClosest', key, timestamp]**. For every **getClosest** operation, return the stored value whose timestamp is closest to the query tim

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: helper functions ----
def count_points(points):
    # {(x, y): how many times the point appears}
    counts = {}
    for x, y in points:
        counts[(x, y)] = counts.get((x, y), 0) + 1
    return counts


def ranked_points(counts):
    # one row per point: (minus its count, squared distance from the origin, x, y).
    # A smaller row is better: more frequent first, then closer to the origin, then by x, y.
    # (Squared distances need no square root and are exact for whole numbers.)
    rows = []
    for (x, y), count in counts.items():
        rows.append((-count, x * x + y * y, x, y))
    return rows
```

```
# ---- main question: main functions ----
# Q1: the k best points: most frequent, then closest to the origin.
# Idea: count the points, make one ranking row per point, and keep a heap
# of the k smallest rows (heapq.nsmallest).
# Complexity: Time O(n + u log k) for n points and u distinct points, Space O(u)
def top_points(points, k):
    best = heapq.nsmallest(k, ranked_points(count_points(points)))
    result = []
    for minus_count, distance, x, y in best:
        result.append([x, y])
    return result


# Q2: a store of values by key and time.
# Complexity: Time O(log n), Space O(1)
def first_not_before(items, ts):
    # binary search: the first index whose timestamp is >= ts
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle][0] < ts:
            low = middle + 1
```

```
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_after(items, ts):
    # binary search: the first index whose timestamp is > ts
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle][0] <= ts:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time set O(n) (a sorted insert), get O(log n), Space O(n)
class TimeStore:
    # set(key, value, ts) stores a value; get_closest(key, ts) returns the value
    # stored at the time closest to ts (the earlier time if two are equally
    # close). Each key keeps its (timestamp, value) pairs in order of time,
    # even when they arrive out of order (follow-up 2).
    def __init__(self):
        self.data = {}

    def set(self, key, value, ts):
        if key not in self.data:
            self.data[key] = []
        items = self.data[key]
        position = 0
        while position < len(items) and items[position] <= (ts, value):
            position += 1                          # insert so the list stays sorted
        items.insert(position, (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
        if not items:
```

```
            return ""
        i = first_not_before(items, ts)           # the first time at or after ts; the one before it is i -
1
        options = []
        if i > 0:
            options.append(items[i - 1])
```

```
        if i < len(items):
            options.append(items[i])
        best = options[0]
        for item in options:
            gap = item[0] - ts
            if gap < 0:
                gap = -gap
            best_gap = best[0] - ts
            if best_gap < 0:
                best_gap = -best_gap
            if gap < best_gap or (gap == best_gap and item[0] < best[0]):
                best = item
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value stored at a time not after ts
        items = self.data.get(key, [])
        i = first_after(items, ts) - 1
        if i >= 0:
            return items[i][1]
        return ""
```

COMPLEXITY | Q1: heap with the full key: Time O(n + d log k) | Space O(d)
Q2: parallel sorted lists and bisect: Time O(1) per set, O(log m) per getClosest | Space O(total entries)

184 Solve chat, grid paths, and car rentals — MEDIUM

Template fit: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.
Input:

- An array **`messages`** of length **n**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **k** (1 <= k <= number of distinct users).

Output:

- Return a list of **k** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **k** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **k** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:

QUESTION

- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

Tasks: 1.

Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.

Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

- An integer **`C`** = minimal number of cars needed.
- A mapping from each **`rental_id`** to a **`car_id`** in the range **`1..C`**, such that no two rentals assigned to the same **`car_id`** overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals **`[s1, e1)`** and **`[s2, e2)`** overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if **`e1 == s2`** they do **not** over

EXAMPLES

QUESTION

Example 1	
Input	['ann', 'bob', 'ann', 'cy', 'bob', 'ann'], 2
Output	['ann', 'bob']
Why	Task 1: ann sent 3 messages, bob 2, cy 1; the top 2 are ann and bob.

Example 2	
Input	3, 3
Output	2
Why	Task 2: paths over 3 columns of a 3-row grid that start and end in the bottom row, moving up-right, right or down-right.

Example 3	
Input	[(1, 0, 5), (2, 1, 4), (3, 5, 8)]
Output	(2, {1: 0, 2: 1, 3: 1})
Why	Task 3: rentals are (id, pickup, return). Rental 3 starts at 5, after rental 2 returned its car at 4, so it reuses that car: 2 cars in total.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: helper functions ----
def count_items(items):
    # how many times each item appears: {item: count}
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts


def rank_of(pair):
    # pair = (user, count): more messages first, ties by the smaller user id
    return (-pair[1], pair[0])


# ---- main question: main functions ----
# Q1: the k most active users in a chat log.
# Idea: count the messages of every user in a dictionary, then keep a heap
# of only the best k (heapq.nsmallest with a ranking key). That beats
# sorting all d users when k is small: O(n + d log k). Ties are broken by
# the smaller user id so the answer is always the same.
# Complexity: Time O(n + u log k) for n messages and u users, Space O(u)
def top_k_users(messages, k):
    counts = count_items(messages)
    best = heapq.nsmallest(k, counts.items(), key=rank_of)
    users = []
    for user, count in best:
        users.append(user)
    return users


# the second approach for Q1: sort every user, O(d log d)
# Complexity: Time O(n + u log u), Space O(u)
def top_k_users_sorted(messages, k):
    counts = count_items(messages)
    pairs = []
    for user in counts:
```

```
        pairs.append((-counts[user], user))
    pairs.sort()
    users = []
    for minus_count, user in pairs[:k]:
        users.append(user)
    return users
```

```
# Q2: count the paths through a grid with m rows and n columns. Start
# at the bottom-left cell (m - 1, 0), end at the bottom-right cell
# (m - 1, n - 1); each move goes right, right-and-up or right-and-down.
# Cells in `blocked` cannot be used (follow-up 4); mod keeps the numbers small.
# Idea: ways[r] = the number of paths in row r of the current column. One
# column is enough: O(m) memory.
# Complexity: Time O(m * n), Space O(m)
def count_paths(m, n, blocked=(), mod=None):
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
    for c in range(1, n):
        following = [0] * m
        for r in range(m):
            if (r, c) in wall:
                continue                        # a blocked cell contributes no paths
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            if mod:
                total = total % mod
            following[r] = total
        ways = following
    return ways[m - 1]


# Q3: rentals are (pickup, return) times. How many cars are needed?
# Idea: sort the rentals by pickup time and keep a min-heap of the return
# times of the cars that are out. If the earliest car is back by the
# pickup, reuse it; otherwise a new car is needed.
# Complexity: Time O(n log n), Space O(n)
def min_cars(rentals):
    out = []
    for pickup, back in sorted(rentals):
        if out and out[0] <= pickup:
            heapq.heapreplace(out, back)        # a car came back in time: reuse it
        else:
            heapq.heappush(out, back)
    return len(out)
```

COMPLEXITY | Q1: top-K active users with a size-K heap: Time O(n + d log k) for d distinct users | Space O(d)
Q2: count constrained paths with a rolling column: Time O(m * n) | Space O(m)
Q3: minimum cars and one valid assignment: Time O(R log R) | Space O(R)

185 Top-K Chatters by Word Count

MEDIUM

Template fit: count len(body.split()) per speaker instead of lines; ties by name

QUESTION

You are given a chat transcript as a list of message lines. Each line has the form "Name: message text" , where everything before the first ": " is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non- whitespace characters in the body). A person may appear on many lines; sum their words. Return the k people who typed the most words, as a list of (name, word_count) tuples sorted by word_count descending. Break ties by name in ascending (lexicographic) order.

Function signature

def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:

...

Follow-up (as reported): The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a ChatAnalyzer class that ingests lines incrementally via add_line(line) and answers top_k(k) on demand.

EXAMPLES

Example 1	
Input	`lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1`
Output	`[("Alice", 6)]`
Why	`Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.`

Example 2	
Input	`lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2`
Output	`[("Ann", 2), ("Bob", 2)]`
Why	`Ann and Bob both have 2 words; tie broken alphabetically, so Ann precedes Bob. Cara (1 word) is excluded.`

Example 3	
Input	`lines = [], k = 3`
Output	`[]`

CONSTRAINTS

- 0 <= len(lines) <= 10^5
- Every line contains exactly one ": " separating name from body; the name is non-empty and contains no ": " .
- The body may be empty (0 words).
- 1 <= k ; if k exceeds the number of distinct people, return all of them.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: helper functions ----
def rank_of(pair):
    # pair = (name, word count): more words first, ties by name
    return (-pair[1], pair[0])


def words_in(line):
    # "Alice: hello there friend" -> ("Alice", 3)
    name, body = line.split(": ", 1)
    return name, len(body.split())
```

```python
# ---- main question: main function ----
# Idea: add up the words written by every author in a dictionary, then
# pick the k best with a heap ranked by (-words, name).
# Complexity: Time O(total words + u log k), Space O(u)
def top_k_chatters(lines, k):
    counts = {}
    for line in lines:
        name, words = words_in(line)
        counts[name] = counts.get(name, 0) + words
    return heapq.nsmallest(k, counts.items(), key=rank_of)
```

COMPLEXITY | Time O(n + p log k) | Space O(p)

186 Find top/bottom-k words in list or stream

MEDIUM

Template fit: the same Counter; run nsmallest once with -count and once with +count for the bottom-k

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
Expected Output: [['i', 'love'], ['coding', 'leetcode']]
Explanation: `i` and `love` both appear twice, so lexicographical order gives `['i', 'love']`. `coding`
    and `leetcode` both appear once, so lexicographical order gives `['coding', 'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

QUESTION

Hints

Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).

Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.

Constraints

1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested

Examples

Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]

Expected Output: [['apple', 'banana'], ['banana', 'apple']]

Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]

Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]

Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.

Hints

Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: helper functions ----
def count_words(words):
    counts = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts


def names_of(pairs):
    names = []
    for first, second in pairs:
        names.append(second)
    return names


# ---- main question: main function ----
# Idea: one pass counts the words. A heap of size k then picks the k most
# frequent (rows (-count, word): the smallest rows win) and the k least
# frequent (rows (count, word)), each in O(d log k) without sorting
# everything. Ties are broken alphabetically.
# Complexity: Time O(n + u log k), Space O(u)
def top_bottom(words, k):
    counts = count_words(words)
    top_rows = []
    bottom_rows = []
    for word in counts:
        top_rows.append((-counts[word], word))
        bottom_rows.append((counts[word], word))
    top = names_of(heapq.nsmallest(k, top_rows))
    bottom = names_of(heapq.nsmallest(k, bottom_rows))
```

```
    return top, bottom
```

COMPLEXITY | Time O(n + d log k) | Space O(d)

GROUP 24 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 1 E 3 M 2 H

Digits and characters: peel one at a time and carry

187 Caesar Cipher: Recover Key, Encrypt, and Decrypt — EASY

Template fit: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k

QUESTION

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount k (0 <= k < 26) with wraparound (z +1 -> a); all non-lowercase characters were left unchanged. You are given one example pair — a known plaintext and its corresponding ciphertext — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement: find_shift(plaintext, ciphertext) -> int : return the shift k (0..25) that maps plaintext to ciphertext . The two strings have equal length and contain at least one lowercase letter that pins down k . encrypt(text, k) -> str : shift each lowercase letter forward by k with wraparound; leave other characters unchanged. decrypt(text, k) -> str : the inverse of encrypt — shift each lowercase letter backward by k with wraparound.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...
def encrypt(text: str, k: int) -> str:
    ...
def decrypt(text: str, k: int) -> str:
    ...
```

Follow-up (as reported): Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.

EXAMPLES

Example 1	
Input	`find_shift("abc", "def")`
Output	`3`
Why	`'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.`

Example 2	
Input	`encrypt("hello world", 3)`
Output	`"khoor zruog"`
Why	`Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space is unchanged.`

Example 3	
Input	`decrypt("khoor zruog", 3)`
Output	`"hello world"`
Why	`decrypt inverts encrypt, so the original message is recovered.`

Example 4	
Input	`encrypt("xyz", 3)`
Output	`"abc"`
Why	`Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.`

CONSTRAINTS

- 1 <= len(text) <= 10^5 . text may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters a – z are shifted.

QUESTION

- For find_shift , len(plaintext) == len(ciphertext) and they align character by character.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main functions ----
# A Caesar cipher moves every lowercase letter k places along the
# alphabet (wrapping z to a). Other characters stay as they are.

# Idea: one letter pair is enough to recover the key: the key is how far
# the ciphertext letter is from the plaintext letter, modulo 26.
# Complexity: Time O(1) usually (the first letter), O(n) at worst, Space O(1)
def find_shift(plaintext, ciphertext):
    for i in range(len(plaintext)):
        if "a" <= plaintext[i] <= "z":
            return (ord(ciphertext[i]) - ord(plaintext[i])) % 26
    return 0


# The same helper shifts forward (encrypt) or backward (decrypt, shift by -k).
# Complexity: Time O(n), Space O(n)
def shift_text(text, k):
    result = ""
    for ch in text:
        if "a" <= ch <= "z":
            position = ord(ch) - ord("a")                    # 0 for "a" ... 25 for "z"
            ch = chr((position + k) % 26 + ord("a"))
        result += ch
    return result


# Complexity: Time O(n), Space O(n)
def encrypt(text, k):
    return shift_text(text, k)


# Complexity: Time O(n), Space O(n)
def decrypt(text, k):
    return shift_text(text, -k)
```

COMPLEXITY | Time O(n) | Space O(n)

188 Add two big integers from digit lists

MEDIUM

Template fit: the template as written

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

QUESTION

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7, 0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the numbers are lists of digits with the MOST significant digit
# first. Walk both lists from the right with indices i and j. At each
```

```
# step add a[i] (or 0 when a is used up), b[j] (or 0) and the carry,
# keep total % 10 as the digit, and carry total // 10. Continue while
# anything is left, including a final carry. Reverse at the end.
# Complexity: Time O(max(n, m)), Space O(max(n, m))
def add_digits(a, b):
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry > 0:
        total = carry
        if i >= 0:
            total += a[i]
        if j >= 0:
            total += b[j]
        digits.append(total % 10)
        carry = total // 10
        i -= 1
        j -= 1
    if len(digits) == 0:
        return [0]
    digits.reverse()
    return digits
```

COMPLEXITY | Time O(max(m, n)) | Space O(max(m, n))

189 Solve several coding interview problems — MEDIUM

Template fit: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps

QUESTION

Three small tasks. (1) add_strings(a, b): add two non-negative integers given as digit strings and return the sum as a string, without converting the whole numbers. (2) intersection(head_a, head_b): two singly linked lists may share a common tail; return the first shared node, or None. (3) overlap_depth(rects): rectangles (x1, y1, x2, y2) include their left and bottom edges but not their right and top edges; return the largest number of rectangles that cover a single point.

Function signature

def add_strings(a: str, b: str) -> str
def intersection(head_a, head_b)
def overlap_depth(rects: list) -> int

EXAMPLES

Example 1	
Input	'999', '1'
Output	'1000'
Why	A carry runs through every digit: 999 + 1 = 1000.

Example 2	
Input	'456', '77'
Output	'533'
Why	456 + 77 = 533.

QUESTION

Example 3	

Input	`class N:`
	`def __init__(self, val, next=None):`
	`self.val, self.next = val, next`
	`shared = N(8, N(9))`
	`a = N(1, N(2, shared))`
	`b = N(5, shared)`
	`intersection(a, b).val`

Output	`intersection(a, b).val -> 8`
Why	`List a is 1, 2, 8, 9 and list b is 5, 8, 9: they join at the node holding 8.`

Example 4	
Input	`[(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]`
Output	`3`
Why	`The point (3, 3) lies inside all three rectangles.`

Example 5	
Input	`[(0, 0, 2, 2), (2, 0, 4, 2)]`
Output	`1`
Why	`Rectangles that only share an edge do not overlap.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


# ---- main question: main functions ----
# PART 1: add two big numbers kept as digit strings.
# Idea: walk both strings from the right, add the two digits and the
# carry, keep total % 10 and carry total // 10; after the loop add a final
# carry digit if there is one; reverse the digits at the end.
# Complexity: Time O(max(n, m)), Space O(max(n, m))
def add_strings(a, b):
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry > 0:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        digits.append(str(total % 10))
        carry = total // 10
        i -= 1
        j -= 1
    digits.reverse()
    return "".join(digits)


# PART 2: where do two linked lists meet?
# Idea: each pointer walks its own list and then continues on the OTHER
# list. Both walk the same total length (m + n), so they arrive at the
```

```
# meeting node at the same moment (or both reach None together).
# Complexity: Time O(a + b), Space O(1)
def intersection(head_a, head_b):
    a = head_a
    b = head_b
    while a is not b:
        if a is None:
            a = head_b
        else:
            a = a.next
        if b is None:
            b = head_a
        else:
            b = b.next
    return a


# PART 3: the deepest overlap of rectangles. rects are (x1, y1, x2, y2),
# half-open: the largest number of rectangles that share one point.
# Idea: the best point can be a rectangle's lower-left corner, so test
# those corners and count the rectangles that contain each.
# Complexity: Time O(n^2), Space O(1)
def overlap_depth(rects):
    best = 0
    for x, y, x2, y2 in rects:
        inside = 0
        for r in rects:
            if r[0] <= x < r[2] and r[1] <= y < r[3]:
                inside += 1
        if inside > best:
            best = inside
    return best
```

COMPLEXITY | Big numbers: digit by digit from the right with a carry: Time O(max(m, n)) | Space O(max(m, n))
Chains that meet: two pointers that switch heads at the end: Time O(m + n) | Space O(1)
Overlap: sweep over sorted start and end events: Time O(n log n) | Space O(n)

190 Check Whether Digits Are Monotone Increasing

MEDIUM

Template fit: peel digits from the right; each digit must not exceed the previous one you saw

QUESTION

Return true when the decimal digits of a non-negative integer never decrease from left to right (repeated digits are allowed). Zero and single-digit numbers qualify.

Function signature

def is_monotone_digits(n: int) -> bool

EXAMPLES

Example 1	
Input	`1234`
Output	`true`
Why	`1, 2, 3, 4 only goes up.`

Example 2	
Input	`1223`
Output	`true`
Why	`Equal neighbours are allowed.`

QUESTION

Example 3	
Input	332
Output	false
Why	3, 3, 2 goes down at the end.

Example 4	
Input	0
Output	true
Why	Zero qualifies.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def is_monotone_string(n):
    # the same test on the text of the number (used to check the main function)
    text = str(n)
    for i in range(len(text) - 1):
        if text[i] > text[i + 1]:
            return False
    return True


# ---- main question: main function ----
# Idea: take the last digit with n % 10, then drop it with n // 10, and so
# on. Moving from right to left, each digit just read must not be bigger
# than the digit read before it (the one to its right). Zero and a
# single digit are monotone.
# Complexity: Time O(d) for d digits, Space O(1)
def is_monotone_digits(n):
    previous = 9                            # nothing is bigger than 9
    while n > 0:
        digit = n % 10
        if digit > previous:
            return False                    # a digit bigger than the one to its right
        previous = digit
        n //= 10
    return True
```

COMPLEXITY | Time O(d) | Space O(1)

191 Check Digits and Combine Ranges

HARD

Template fit: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer **x**, return **true** if its decimal representation reads the same from left to right and from right to left; otherwise return **false**. Requirements and edge cases:

- Negative integers should return **false** because of the leading minus sign.
- You may assume **x** fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

QUESTION

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals **`intervals`**, where each interval is represented as **`[start, end]`**, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, **`[1, 4]`** and **`[4, 5]`** should be combined into **`[1, 5]`**.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

QUESTION

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.

Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.

Constraints

0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted

Examples

Input: [[1, 3], [2, 6], [8, 10], [15, 18]]

Expected Output: [[1, 6], [8, 10], [15, 18]]

Explanation: The first two intervals overlap and merge into [1, 6].

Input: [[1, 4], [4, 5]]

Expected Output: [[1, 5]]

Explanation: The intervals share the point 4, so they overlap.

Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main functions ----
# PART 1: is the number a palindrome, without turning it into text?
# Idea: negative numbers and positive numbers ending in 0 can never be
# palindromes. Build the REVERSED SECOND HALF of the number digit by digit
# and stop when it catches up with the first half. An odd number of digits
# has a middle digit, which is dropped from the reversed half.
# (base is the number base; follow-up 3 asks for another one.)
# Complexity: Time O(d) for d digits (half of them are processed), Space O(1)
def is_palindrome(x, base=10):
    if x < 0 or (x % base == 0 and x != 0):
        return False
    reversed_half = 0
    while x > reversed_half:
        reversed_half = reversed_half * base + x % base
        x //= base
    return x == reversed_half or x == reversed_half // base


# PART 2: combine overlapping ranges.
# Idea: sort by start so overlapping ranges are neighbours. A range that
# starts before (or exactly where) the last merged one ends is absorbed.
# With adjacent=True, ranges that merely touch ([1, 2] and [3, 4])
# are merged too (follow-up 1).
# Complexity: Time O(n log n), Space O(n)
def merge_ranges(intervals, adjacent=False):
    gap = 0
    if adjacent:
        gap = 1
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1] + gap:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged
```

COMPLEXITY | Task 1: reverse only the second half arithmetically: Time O(d) | Space O(1)
Task 2: sort by start, then one sweep: Time O(n log n) | Space O(n)

192 Permutation Sequence — HARD

Template fit: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

QUESTION

The set [1, 2, ..., n] has n! distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from 1 , return the k -th permutation as a string. Aim for an O(n) -ish construction using the factorial number system rather than generating all permutations.

Function signature

def get_permutation(n: int, k: int) -> str:

    ...

Follow-up (as reported): A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids O(n!) .

EXAMPLES

Example 1	
Input	`n = 3, k = 3`
Output	`"213"`
Why	`Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".`

Example 2	
Input	`n = 4, k = 9`
Output	`"2314"`
Why	`The 9th permutation of 1234 in lexicographic order is "2314".`

Example 3	
Input	`n = 3, k = 1`
Output	`"123"`
Why	`The first permutation is the sorted sequence.`

CONSTRAINTS

- 1 <= n <= 9
- 1 <= k <= n!

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def factorial(n):
    # 1 * 2 * ... * n  (the number of orderings of n different things)
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


# ---- main question: main function ----
# Idea (the factorial number system): the permutations of 1..n are in
# blocks. The first digit decides a block of (n - 1)! permutations, so
# k // (n - 1)! says which unused digit comes first; then k % (n - 1)!
# is the position inside that block, and the same step repeats on the
# remaining digits. k starts at 1, so subtract 1 first.
# Complexity: Time O(n^2) (removing from the digit list), Space O(n)
def get_permutation(n, k):
```

```
    digits = []
    for d in range(1, n + 1):
        digits.append(d)
    k -= 1
    answer = ""
    for left in range(n, 0, -1):
        block = factorial(left - 1)
        chosen = digits.pop(k // block)
        answer += str(chosen)
        k %= block
    return answer
```

COMPLEXITY | Time O(n^2) | Space O(n)

GROUP 25 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 4 M 2 H

Look at the input once and keep only a small summary (and count your calls)

193 Two-Day Log Intersection under Memory Limit

MEDIUM

Template fit: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id).
Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients {c1, c2}.
Output: [X]
```

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea (everything fits in memory): for each day build a dictionary
# object -> set of clients. Keep the objects that appear on BOTH days and
# whose clients, over the two days together, number at least 2.
# A row is (timestamp, object id, client id).
# Complexity: Time O(n + m) rows, Space O(distinct objects + clients)
def log_objects(day1, day2):
    clients1 = {}
    clients2 = {}
    for timestamp, obj, client in day1:
        if obj not in clients1:
            clients1[obj] = set()
        clients1[obj].add(client)
    for timestamp, obj, client in day2:
        if obj not in clients2:
            clients2[obj] = set()
        clients2[obj].add(client)
    result = []
    for obj in clients1:
        if obj in clients2 and len(clients1[obj] | clients2[obj]) >= 2:
            result.append(obj)
    return sorted(result)
```

COMPLEXITY | Hash map of client sets (everything fits in memory): Time O(n + m) | Space O(n + m)
Sorted merge with two pointers (memory limited): Time O(n log n + m log m) to sort, O(n + m) to merge | Space O(clients of one object)

194 Find shared objects across two log files — MEDIUM

Template fit: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']], [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def object_of(row):
    return row[1]                                   # a row is [timestamp, object id, client id]


# ---- main question: main function ----
# Idea: an object is "shared" when it appears on BOTH days and was seen by
# at least 2 different clients over the two days. Go through every row once
# and remember, for each object, which days it appeared on and which
# clients saw it. Each line is read once; memory depends on the number of
# distinct objects.
# Complexity: Time O(n + m) rows, Space O(distinct objects + clients)
def shared_objects(day1, day2):
    days = {}                                       # object -> the set of days it appeared on
    clients = {}                                    # object -> the set of clients that saw it
    for number, rows in ((1, day1), (2, day2)):
        for timestamp, obj, client in rows:
            if obj not in days:
                days[obj] = set()
                clients[obj] = set()
            days[obj].add(number)
```

```
            clients[obj].add(client)
    result = []
    for obj in days:
        if len(days[obj]) == 2 and len(clients[obj]) >= 2:
            result.append(obj)
    return sorted(result)


# the other way: sort both days by object, then merge them like two sorted
# lists, holding the clients of only ONE object at a time
# Complexity: Time O((n + m) log(n + m)) (the sorts), Space O(n + m) (O(1) extra per object when the files ar
e already sorted)
def shared_objects_sorted(day1, day2):
    a = sorted(day1, key=object_of)
    b = sorted(day2, key=object_of)
    i = 0
    j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
        else:
            obj = a[i][1]
```

```
            clients = set()
            while i < len(a) and a[i][1] == obj:
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                result.append(obj)
    return result
```

COMPLEXITY | Time O(n + m) | Space O(distinct ids of the smaller file)

195 Streaming Points: Emit K Within Distance — MEDIUM

Template fit: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked

QUESTION

Positions (floats) arrive one at a time. Whenever k stored points fit inside a span of d (largest minus smallest is at most d), output those k points and remove them. add(x) returns the groups emitted because of this arrival (a list of lists).

Function signature

class PointStream:
    def __init__(self, k: int, d: float)
    def add(self, x: float) -> list

EXAMPLES

QUESTION

Example 1

Input	`s = PointStream(3, 2.0)`
	`s.add(1.0)`
	`s.add(5.0)`
	`s.add(2.0)`
	`s.add(1.5)`

Output	`s.add(1.0) -> []`
	`s.add(5.0) -> []`
	`s.add(2.0) -> []`
	`s.add(1.5) -> [[1, 1.5, 2]]`

Why	After 1, 5, 2 no three points fit in a span of 2 (1 and 5 are too far apart); adding 1.5 gives 1, 1.5, 2 with span 1, which is emitted and removed.

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left, insort


# ---- main question: main function ----
# Idea: points arrive one at a time. As soon as there are k waiting points
# that fit inside a stretch of length d (the span of the group =
# largest - smallest <= d), emit them as a group and forget them. Keep
# the waiting points in a SORTED list (insort = binary search plus insert).
# Before the new point no such group existed, so a new group must contain the
# new point: only the (at most k) windows of k neighbours that include its
# position need to be checked.
# Complexity: Time add is O(n) (a sorted list insert) plus O(k) per window check, Space O(n) waiting points
class PointStream:
    def __init__(self, k, d):
        self.k = k
        self.d = d
        self.points = []                        # the waiting points, sorted

    def window_start(self, pos):
        # the first window of k neighbouring points that includes position
        # pos and fits in a span of d, or None
        first = pos - self.k + 1
        if first < 0:
            first = 0
        last = len(self.points) - self.k
        if pos < last:
            last = pos
        for i in range(first, last + 1):
            span = self.points[i + self.k - 1] - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        insort(self.points, x)
        groups = []
        pos = bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
            if i is None:
                return groups
            groups.append(self.points[i:i + self.k])      # emit the group ...
            del self.points[i:i + self.k]                 # ... and remove it so it is emitted only once
            pos = i
```

COMPLEXITY | Time O(log n + K) per check, plus list shifting | Space O(n)

196 Minimize calls to find all bad test pairs

MEDIUM

Template fit: the same group testing: put a whole group in one call and split only the groups that fail

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

QUESTION

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# run_test(set_of_tests) is the expensive oracle: it returns True when
# the tests in the set pass TOGETHER (no bad pair inside it).

def shortest_failing_prefix(items, extra, run_test):
    # binary search the smallest prefix of `items` that fails together
    # with the tests in `extra`; its LAST item is a partner of something
    low = 1
    high = len(items)
    while low < high:
        mid = (low + high) // 2
        if run_test(set(items[:mid]) | extra):
            low = mid + 1                       # still passes: the culprit is later
        else:
            high = mid
    return items[low - 1]


def partners(group, t, run_test):
    # every member of the group that is bad together with t
    rest = list(group)
    found = []
    while len(rest) > 0 and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)                          # take it out and test the others again
    return found


# ---- main question: main function ----
# Idea: keep GROUPS of tests that pass together. For each new test t: ask
# each group for its partners with t (one call tells us when there are
# none; otherwise a binary search finds each partner in about log n
# calls). t joins the first group in which it has no partner, or starts a
# new group. Returns the set of bad pairs (smaller test, bigger test).
# Complexity: Time O((n + p log n) * cost of runTest) calls: about n + p log n runTest calls for p bad pairs,
 Space O(n)
def find_bad_pairs(tests, run_test):
    groups = []
    pairs = set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                if x < t:
```

```
                    pairs.add((x, t))
                else:
                    pairs.add((t, x))
            if len(found) == 0 and not placed:
                group.append(t)
                placed = True
        if not placed:
```

```
            groups.append([t])
    return pairs
```

COMPLEXITY | Time O(n * g + p log n) calls (g = number of groups, p = number of bad pairs) | Space O(n)

197 Find All Bad Pairs Using runTest Oracle — HARD

Template fit: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair containing items from this set.
... binary search to isolate.
```

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# run_test(set_of_items) is the expensive oracle: it returns True when the
# items in the set pass TOGETHER (no bad pair inside it).

def first_partner(group, t, run_test):
    # binary search the shortest prefix of the group that fails together
    # with t; its LAST member is a partner of t
    low = 1
    high = len(group)
    while low < high:
        mid = (low + high) // 2
        if run_test(set(group[:mid]) | {t}):
            low = mid + 1
        else:
            high = mid
    return group[low - 1]


def partners(group, t, run_test):
    # all members of the group that are bad together with t
    rest = list(group)
    found = []
    while len(rest) > 0 and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        rest.remove(x)                              # take it out and test the others again
    return found


# ---- main question: main function ----
# Idea: keep groups of items that pass together (no bad pair inside a
# group). A new item t is checked against every group: one call says
# "no partner here", otherwise a binary search finds each partner. t joins
# the first group in which it has no partner, or starts a new group.
# Returns the set of bad pairs (smaller item, bigger item).
# Complexity: About n + p log n runTest calls for p bad pairs (best case about n, worst case about n^2 / 2),
Space O(n)
```

```
def find_bad_pairs(items, run_test):
    groups = []
    pairs = set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                if x < t:
                    pairs.add((x, t))
                else:
                    pairs.add((t, x))
            if len(found) == 0 and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs
```

COMPLEXITY | Time about O(K log N + N * G) calls (K bad pairs, G groups) | Space O(N)

198 Find maximum value in huge text file — HARD

Template fit: the template as written; the file is never loaded as a whole

QUESTION

A text file is too large to load into memory (tens or hundreds of gigabytes). Each line holds one signed 64-bit integer. Return the largest value by reading the file sequentially, holding only a small amount in memory.

Function signature

def max_in_file(path: str) -> int

EXAMPLES

Example 1

Input	**nums.txt contains the lines:** **5** **-3** **12** **7**
Output	**max_in_file(path) -> 12**
Why	**Only the current line and the best value so far are in memory, so the file size does not matter.**

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: never load the file. Read it line by line (the file object reads
# in buffered chunks) and keep ONE variable: the maximum so far. It
# starts as None, not 0, so that a file of only negative numbers, or of one
# line, gives the right answer. Blank lines are ignored (state this
# contract). An empty file has no maximum: None.
# Complexity: Time O(n) for n lines (one pass), Space O(1)
def max_in_file(path):
    best = None
    with open(path) as f:
        for line in f:
            text = line.strip()
            if text == "":
                continue
            value = int(text)
```

```
            if best is None or value > best:
                best = value
    return best
```

COMPLEXITY | Single machine: stream the file, keep one running maximum: Time O(n) | Space O(1) plus the read buffer
Parallel: split byte ranges on line boundaries (map), max of the maxima (reduce): Time O(n / workers) plus I/O limits | Space O(workers)

GROUP 26 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Grid DP: each cell from its neighbours above and to the left

199 Grid Path Count with Diagonal Moves and Checkpoints — MEDIUM

Template fit: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint

QUESTION

Count the paths in an n x m grid from the bottom-left cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1). From (r, c) you may move up-right (r - 1, c + 1), right (r, c + 1) or down-right (r + 1, c + 1) and must stay inside the grid. Optionally you are given checkpoint cells that every path has to pass through; with ordered=True they must be visited in the order given.

Function signature

def count_paths(n: int, m: int, checkpoints=(), ordered=False) -> int

EXAMPLES

Example 1	
Input	3, 3
Output	2
Why	Two paths: through (1, 1) and then down-right, or through (2, 1) and then right.

Example 2	
Input	3, 3, checkpoints = [(1, 1)]
Output	1
Why	The path must also pass through (1, 1), the middle cell.

Example 3	
Input	3, 4, checkpoints = [(1, 1), (2, 2)], ordered = true
Output	1
Why	Two checkpoints in different columns, visited in column order.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def step(ways):
    # the next column from the current column only: a path in row r came
    # from row r - 1, r or r + 1 of the column before (up-right, right, down-right)
    n = len(ways)
    following = []
    for r in range(n):
        total = ways[r]
        if r > 0:
            total += ways[r - 1]
        if r + 1 < n:
            total += ways[r + 1]
        following.append(total)
    return following


def keep_row(ways, row):
    # a checkpoint forces the path through one row of this column
    kept = []
    for r in range(len(ways)):
        if r == row:
            kept.append(ways[r])
```

```
        else:
            kept.append(0)
    return kept


def checkpoint_rows(checkpoints, ordered):
    # column -> the row the path must be in; None if that is impossible
    rows = {}
    for r, c in checkpoints:
        if c in rows and rows[c] != r:
            return None                          # two different checkpoints in one column: no path can visit
 both
        rows[c] = r
    if len(rows) != len(checkpoints):
        return None                              # the same checkpoint listed twice
    if ordered:
        for i in range(len(checkpoints) - 1):
            if checkpoints[i][1] > checkpoints[i + 1][1]:
                return None                      # ordered checkpoints must be listed left to right
    return rows


# ---- main question: main function ----
# A grid with n rows and m columns. A path starts at the bottom-left
```

```
# cell, ends at the bottom-right cell and each step moves to the next
# column in the same row or one row up or down (a diagonal). Count the
# paths that pass through all the checkpoints (row, column).
# Idea: ways[r] = the number of paths that are in row r at the current
# column. The next column is ways'[r] = ways[r - 1] + ways[r] + ways[r + 1].
# At a checkpoint column keep only the checkpoint's row. One rolling
# column is enough: O(n) memory. (Two checkpoints in one column: the
# answer is 0, since a path is in exactly one row per column: follow-up 1.)
# Complexity: Time O(n * m + checkpoints), Space O(n)
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]
```

COMPLEXITY | Time O(n * m) | Space O(n)

200 Longest Same-Color Grid Sequence — MEDIUM

Template fit: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest

QUESTION

Given a grid of colours (letters such as R, G, B), return the length of the longest run of one colour along a straight line: horizontal, vertical, or diagonal (in either diagonal direction).

Function signature

def longest_same_color_line(g: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	['RRRG', 'RRBB', 'GRRR', 'BGBR']
Output	4
Why	The main diagonal is R, R, R, R from corner to corner: 4. No other line is longer.

Example 2	
Input	['RGB', 'BGR', 'RGB']
Output	3
Why	The longest line is the vertical run of three G in the middle column.

Example 3	
Input	['RR', 'RR']
Output	2
Why	A single colour fills the grid: the longest line is a full row or column.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the longest straight line of one colour (across, down, or along a
# diagonal). Keep, for the PREVIOUS row, the length of the run that ends
# at each column going straight down, down-left and down-right, and a
# running number for the run across the current row. A cell extends the
# run of the cell above it (or above-left / above-right) if the colour
# matches, otherwise it starts a new run of 1.
# Complexity: Time O(m * n), Space O(n) (one row at a time)
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n = len(g[0])
    best = 0
    down = [0] * n                          # runs ending in the previous row ...
    down_left = [0] * n                     # (coming from the above-right cell)
    down_right = [0] * n                    # (coming from the above-left cell)
    for i in range(len(g)):
        row = g[i]
        down_now = [1] * n
        left_now = [1] * n
        right_now = [1] * n
        across = 0
        for j in range(n):
            color = row[j]
            if j > 0 and row[j - 1] == color:
                across += 1                 # continue the run across this row
            else:
                across = 1
            if i > 0:
                if g[i - 1][j] == color:
                    down_now[j] = down[j] + 1
                if j > 0 and g[i - 1][j - 1] == color:
                    right_now[j] = down_right[j - 1] + 1
                if j + 1 < n and g[i - 1][j + 1] == color:
                    left_now[j] = down_left[j + 1] + 1
            for length in (across, down_now[j], left_now[j], right_now[j]):
                if length > best:
                    best = length
        down = down_now
        down_left = left_now
        down_right = right_now
```

```
    return best
```

COMPLEXITY | Time O(m * n) | Space O(n)

201 Count Same-Color Squares in Matrix

MEDIUM

Template fit: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.
Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).
DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.
Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def square_sides(g):
    # side[i][j] = the side of the biggest same-colour square whose bottom-right corner is (i, j)
    sides = []
    for i in range(len(g)):
        row_sides = [1] * len(g[i])
        for j in range(len(g[i])):
            c = g[i][j]
            if i > 0 and j > 0 and g[i - 1][j] == c and g[i][j - 1] == c and g[i - 1][j - 1] == c:
                smallest = sides[i - 1][j]                # all three neighbours must match this cell
                if row_sides[j - 1] < smallest:
                    smallest = row_sides[j - 1]
                if sides[i - 1][j - 1] < smallest:
                    smallest = sides[i - 1][j - 1]
                row_sides[j] = 1 + smallest
        sides.append(row_sides)
    return sides


# ---- main question: main function ----
# Idea: a cell's square side is 1 + the smallest side of its top, left
# and top-left neighbours when all three have its colour, otherwise 1.
# Every cell is the bottom-right corner of one square per size up to its side.
# Complexity: Time O(m * n), Space O(m * n)
def count_same_color_squares(g):
    total = 0
    for row_sides in square_sides(g):
        for side in row_sides:
            total += side
    return total
```

COMPLEXITY | Time O(m * n) | Space O(n)

202 Find the Longest Straight Run of One Color

HARD

Template fit: the same four run lengths; extend a run when the neighbour in that direction has the same colour

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **`R`** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

EXAMPLES

Example 1	
Input	`['RRRG', 'RRBB', 'GRRR', 'BGBR']`
Output	`4`
Why	`Four R cells lie on the main diagonal.`

Example 2	
Input	`['RGB', 'BGR', 'RGB']`
Output	`3`
Why	`Three G cells in the middle column.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, down-right, down-left


def all_runs(grid):
    # every maximal straight run of one colour as (length, start cell, direction).
    # A run is walked only from its FIRST cell (the cell before it in that
```

```
    # direction has another colour or does not exist), so the total work is
    # linear in the number of cells.
    rows = len(grid)
    cols = len(grid[0])
    found = []
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                before_r = r - dr
                before_c = c - dc
                if 0 <= before_r < rows and 0 <= before_c < cols:
                    if grid[before_r][before_c] == grid[r][c]:
                        continue                        # not the first cell of its run
                length = 0
                a = r
                b = c
                while 0 <= a < rows and 0 <= b < cols and grid[a][b] == grid[r][c]:
                    length += 1
                    a += dr
                    b += dc
                found.append((length, (r, c), (dr, dc)))
    return found


# ---- main question: main function ----
# Idea: look for the longest straight line (across, down, or along
# either diagonal) made of one colour. Walk each run once from its first
# cell and keep the longest.
# Complexity: Time O(m * n), Space O(m * n)
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    best = 0
    for length, start, direction in all_runs(grid):
        if length > best:
            best = length
    return best
```

COMPLEXITY | Time O(m * n) | Space O(n)

203 Count same-color squares in a character grid — HARD

Template fit: exactly the same recurrence on characters

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **`n`** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).

QUESTION

- The 2×2 square in the top-left corner formed by `'a'`. Your function should return the total count of such monochromatic squares for a given input grid.

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.

Constraints

0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def square_table(grid):
    # side[i][j] = the side of the biggest same-colour square whose
    # BOTTOM-RIGHT corner is (i, j)
    sides = []
    for i in range(len(grid)):
        row_sides = [1] * len(grid[i])
        for j in range(len(grid[i])):
            ch = grid[i][j]
            if i > 0 and j > 0 and grid[i - 1][j] == ch and grid[i][j - 1] == ch and grid[i - 1][j - 1] == ch
:
                # the cells above, to the left and diagonally must match too
                smallest = sides[i - 1][j]
                if row_sides[j - 1] < smallest:
                    smallest = row_sides[j - 1]
                if sides[i - 1][j - 1] < smallest:
                    smallest = sides[i - 1][j - 1]
                row_sides[j] = 1 + smallest
        sides.append(row_sides)
    return sides


# ---- main question: main function ----
# Idea: if a cell matches its top, left and top-left neighbours its square
# side is 1 + the smallest of their sides; otherwise it is 1. A cell with
# side s is the corner of s squares (sizes 1..s), so adding up all the
# sides counts every square exactly once, at its bottom-right corner.
# Complexity: Time O(m * n), Space O(m * n)
def count_same_color_squares(grid):
    total = 0
    for row_sides in square_table(grid):
        for side in row_sides:
            total += side
    return total
```

COMPLEXITY | Time O(m * n) | Space O(n) with a rolling row

Nested structures: a stack of contexts, or recursion with a cache

204 Decode String — MEDIUM

Template fit: the template as written

QUESTION

An encoded string uses k[text] to mean that text is repeated k times; the brackets can be nested. Return the decoded string. (k is a positive integer, possibly with several digits; the input is always valid.)

Function signature

def decode(s: str) -> str

EXAMPLES

Example 1	
Input	'3[a]2[bc]'
Output	'aaabcbc'
Why	3 copies of a, then 2 copies of bc.

Example 2	
Input	'2[abc]3[cd]ef'
Output	'abcabccdcdcdef'
Why	Plain letters after the brackets are kept as they are.

Example 3	
Input	'3[a2[c]]'
Output	'accaccacc'
Why	Nested: the inner 2[c] first (cc), then 3 copies of acc.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: a stack of "frames", one for each "[" that is still open. On "["
# push the text built so far together with the repeat count just read, and
# start a fresh text for the inside. On "]" pop the frame and set
# text = saved text + (inside text repeated count times).
# Complexity: Time O(length of the decoded string), Space O(length of the decoded string)
def decode(s):
    stack = []
    current = ""
    number = 0
    for ch in s:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            stack.append((current, number))
            current = ""
            number = 0
        elif ch == "]":
            saved, times = stack.pop()
            current = saved + current * times
        else:
            current += ch
    return current
```

COMPLEXITY | Stack of (prefix, repeat) frames: Time O(output length) | Space O(output + nesting depth)
Encode (the reverse direction, greedy run-length): Time O(n) | Space O(n)

205 Template Variable Expansion with Follow-Ups

MEDIUM

Template fit: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string. Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values. Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def tokens(s):
    # split the text into ("text", character) and ("var", name) pieces:
    # "%%" is one literal percent sign, "%NAME%" is a variable
    pieces = []
    i = 0
    while i < len(s):
        if s[i] != "%":
            pieces.append(("text", s[i]))
            i += 1
        elif s[i:i + 2] == "%%":
            pieces.append(("text", "%"))
            i += 2
        else:
            end = s.find("%", i + 1)
            if end < 0:
                raise ValueError("unmatched %")
            pieces.append(("var", s[i + 1:end]))
            i = end + 1
    return pieces


# ---- main question: main function ----
# Idea: scan the text once: "%%" becomes "%", "%NAME%" becomes the
# resolved value of NAME. A value may itself hold placeholders, so resolve
# it with the same scan first, remember the result (memo), and keep a
# "visiting" set to catch a name that needs itself.
# Complexity: Time O(template + total length of the values used), Space the same
def expand_template(text, mapping):
    done = {}
    visiting = set()

    def resolve(name):
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
```

```
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
```

```
        result = ""
        for kind, piece in tokens(s):
            if kind == "var":
                result += resolve(piece)
            else:
                result += piece
        return result

    return scan(text)
```

COMPLEXITY | Time O(total length of the expanded text + keys) | Space O(keys)

206 Expand Recursive Key-Value Placeholders in a Template

MEDIUM

Template fit: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles

QUESTION

Expand %KEY% placeholders in a template. A placeholder is replaced by its value from the mapping, and placeholders inside the values are expanded too, recursively. An unknown key is left exactly as written. A cycle (A needs B needs A) raises an error.

Function signature

def expand_template(template: str, mapping: dict) -> str

EXAMPLES

Example 1	
Input	'%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING': 'Hello %NAME%'}
Output	'Hello Ada, welcome!'
Why	%GREETING% expands to 'Hello %NAME%', and %NAME% is expanded in turn.

Example 2	
Input	'Hi %WHO%', {}
Output	'Hi %WHO%'
Why	An unknown key is kept as written.

Example 3	
Input	'%A%', {'A': '%B%', 'B': '%A%'}
Output	raises ValueError: cycle at A
Why	A needs B and B needs A: a cycle is reported instead of looping forever.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def is_word(name):
    # a key is made of letters, digits and underscores (and is not empty)
    if name == "":
        return False
    for ch in name:
```

```
        if not (ch.isalnum() or ch == "_"):
            return False
    return True


# ---- main question: main function ----
# Idea: expand(key) replaces every placeholder in the key's value by the
# expansion of the key it names, remembers each result (a cache, so a
# key used many times is expanded once), and keeps a "visiting" set so
# that a key that needs itself (a cycle) raises an error. Unknown keys
# stay as written (follow-up 2 can also empty them or raise an error).
# "%%" is a literal percent sign (follow-up 1): output one "%" and skip
# both characters. `limit` stops an expansion that explodes (follow-up 3).
# Complexity: Time O(template + total length of the values used), each key is expanded once; Space the same
def expand_template(template, mapping, unknown="keep", limit=10 ** 6):
    cache = {}
    visiting = set()

    def expand_text(text):
        result = ""
        i = 0
        while i < len(text):
            if text[i] != "%":
                result += text[i]
                i += 1
            elif text[i + 1:i + 2] == "%":
                result += "%"                           # "%%" is a literal percent sign
                i += 2
            else:
                end = text.find("%", i + 1)
                name = ""
                if end != -1:
                    name = text[i + 1:end]
                if not is_word(name):
                    result += "%"                       # not a placeholder: keep the percent sign
                    i += 1
                elif name in mapping:
                    result += expand_key(name)
                    i = end + 1
```

```
                else:
                    if unknown == "error":
                        raise KeyError(name)
                    if unknown == "keep":
                        result += text[i:end + 1]       # leave it exactly as written
                    i = end + 1                         # (unknown == "empty": it simply disappears)
        if len(result) > limit:
            raise ValueError("expansion too large")
        return result

    def expand_key(key):
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError("cycle at " + key)
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)
```

COMPLEXITY | Time O(total size of all values + template) | Space O(number of keys)

207 Nested List Weight Sum (LC 339) with Follow-Ups

MEDIUM

*Template fit: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)*

QUESTION

A nested list holds integers and further lists. The weight of an integer is its depth: 1 at the top level, 2 inside one inner list, 3 inside two, and so on. Return the sum of every integer times its depth. A second version reads the nested list from a string like '[1,[4,[6]]]'.

Function signature

def weight_sum(nested: list) -> int

def weight_sum_string(s: str) -> int

EXAMPLES

Example 1	
Input	`[3, 8, [2, 14], [2, [91]]]`
Output	`320`
Why	`3 + 8 at depth 1, 2 + 14 and 2 at depth 2, and 91 at depth 3: 11 + 32 + 4 + 273 = 320.`

Example 2	
Input	`'[1,[4,[6]]]'`
Output	`27`
Why	`The string version of [1, [4, [6]]]: 1*1 + 4*2 + 6*3 = 27.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def with_depths(nested):
    # every integer together with its depth (an integer in the top-level list
    # has depth 1). A stack of (item, depth) is used instead of recursion, so
    # very deep input cannot overflow the call stack.
    pairs = []
    stack = [(nested, 0)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            pairs.append((item, depth))
    return pairs


# ---- main question: main functions ----
# Idea (LeetCode 339): every integer counts as value * depth. Collect the
# integers with their depths and add value * depth up.
# Complexity: Time O(n), Space O(n)
def weight_sum(nested):
    total = 0
    for value, depth in with_depths(nested):
        total += value * depth
    return total


# The same sum straight from the TEXT of the nested list, "[1,[4,[6]]]",
# in one pass with O(1) extra space. An integer is weighted by the number
# of "[" that are currently open (the depth). Digits are collected into a
# number, which is added when a non-digit character ends it. The extra "]"
# at the end flushes a number that finishes the text.
# Complexity: Time O(L) for text of length L, Space O(1)
def weight_sum_string(s):
```

```
    depth = 0
    total = 0
    number = 0
    sign = 1
    reading = False
    for ch in s + "]":
        if ch.isdigit():
            number = number * 10 + int(ch)
            reading = True
            continue
```

```
        if reading:
            total += sign * number * depth
            number = 0
            sign = 1
            reading = False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

COMPLEXITY | Single pass over the string with a depth counter: Time O(n) | Space O(1)
Parsed lists with BFS (no recursion depth limit): Time O(n) | Space O(width)

208 Nested Function Expression Evaluator — MEDIUM

Template fit: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

QUESTION

Evaluate an expression made of integers and nested function calls: add and mul (any number of arguments), sub and pow (two), div (integer division truncating toward zero), min and max (any number). Return the integer result.

Function signature

def evaluate(expr: str) -> int

EXAMPLES

Example 1	
Input	`'add(1, sub(1, 0))'`
Output	`2`
Why	`sub(1, 0) is 1, so add(1, 1) = 2.`

Example 2	
Input	`'add(2, mul(3, pow(4, 5)))'`
Output	`3074`
Why	`pow(4, 5) = 1024, times 3 = 3072, plus 2 = 3074.`

Example 3	
Input	`'max(3, div(-7, 2), min(5, 9))'`
Output	`5`
Why	`div(-7, 2) truncates toward zero to -3; max of 3, -3 and 5 is 5.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# The expression looks like  add(1, mul(2, x))  : a function name, then
# arguments (numbers, variables, or more calls) in brackets, separated by commas.

def add_all(*numbers):
    total = 0
    for x in numbers:
        total += x
    return total


def multiply_all(*numbers):
    total = 1
    for x in numbers:
        total *= x
    return total


def smallest(*numbers):
    best = numbers[0]
    for x in numbers:
        if x < best:
            best = x
    return best


def largest(*numbers):
    best = numbers[0]
    for x in numbers:
        if x > best:
            best = x
    return best


def subtract(a, b):
    return a - b


def power(a, b):
    return a ** b


def divide(a, b):
    # integer division that TRUNCATES toward zero (follow-up 3 explains why)
    if b == 0:
```

```
        raise ZeroDivisionError("division by zero")
    return int(a / b)


FUNCTIONS = {"add": add_all, "sub": subtract, "mul": multiply_all, "pow": power,
             "div": divide, "min": smallest, "max": largest}


def tokenize(expr):
    # split the text into numbers (a "-" right before digits is a sign),
    # names, and the characters ( ) ,
    tokens = []
    i = 0
    while i < len(expr):
        ch = expr[i]
        if ch == " ":
            i += 1
        elif ch in "(),":
            tokens.append(ch)
            i += 1
        elif ch.isdigit() or (ch == "-" and i + 1 < len(expr) and expr[i + 1].isdigit()):
```

```
            j = i + 1
            while j < len(expr) and expr[j].isdigit():
                j += 1
            tokens.append(expr[i:j])
            i = j
        elif ch.isalpha() or ch == "_":
            j = i + 1
            while j < len(expr) and (expr[j].isalnum() or expr[j] == "_"):
                j += 1
            tokens.append(expr[i:j])
            i = j
        else:
            raise ValueError("unexpected character " + ch)
    return tokens


def is_number(token):
    return token.lstrip("-").isdigit()


# ---- main question: main function ----
# Idea: tokenize, then "recursive descent": parse(i) reads ONE value that
# starts at token i and returns (value, index after it). A number is
# itself; a name not followed by "(" is a variable (looked up in env,
```

```
# follow-up 4); a name followed by "(" is a call: read the arguments one
# by one (each is again a value) until ")", then apply the function.
# Complexity: Time O(n) for n tokens, Space O(depth)
def evaluate(expr, env=None):
    if env is None:
        env = {}
    tokens = tokenize(expr)

    def parse(i):
        token = tokens[i]
        if is_number(token):
            return int(token), i + 1
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if token not in env:
                raise ValueError("undefined " + token)
            return env[token], i + 1
        if token not in FUNCTIONS:
            raise ValueError("unknown function " + token)
        i += 2                                      # skip the name and "("
        args = []
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return FUNCTIONS[token](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value
```

COMPLEXITY | Time O(n) | Space O(depth)

GROUP 28 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

209 Shortest Path Nodes and Escaping a Pursuer

MEDIUM

Template fit: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db

QUESTION

In an unweighted, undirected graph (an adjacency dict) Alice is at node a and wants to reach the target t. Return the length (number of edges) of the shortest path from a to t found with BFS. The follow-ups ask for every node that lies on some shortest path, and whether Alice can reach t before a pursuer Bob.

Function signature

def shortest_length(adj: dict, a, t) -> int

EXAMPLES

Example 1	
Input	{0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3
Output	2
Why	0 -> 1 -> 3 and 0 -> 2 -> 3 both take 2 edges.

Example 2	
Input	{0: []}, 0, 0
Output	0
Why	A start that is the target needs 0 edges.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
def bfs(adj, s):
    # distance from s to every node it can reach
    dist = {s: 0}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# ---- main question: main function ----
# Idea: BFS from Alice (a) gives the length of the shortest path to t.
# Complexity: Time O(V + E), Space O(V)
def shortest_length(adj, a, t):
    return bfs(adj, a).get(t)


# the nodes that lie on some shortest path a -> t
# Complexity: Time O(V + E), Space O(V)
def nodes_on_shortest_paths(adj, a, t):
    da = bfs(adj, a)
```

```
    dt = bfs(adj, t)
    result = set()
    if t not in da:
        return result
    for v in da:
        # on a shortest path exactly when the two distances add up to the full length
        if v in dt and da[v] + dt[v] == da[t]:
            result.add(v)
    return result
```

COMPLEXITY | Time O(V + E) | Space O(V)

210 Radio Broadcast Reachability — MEDIUM

Template fit: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached

QUESTION

You are given a list of radios placed on a 2D plane, each described by its (x, y) coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance radius (inclusive). A message can travel from one radio to another either directly (within radius) or through a chain of intermediate radios that each relay it to a next radio within radius . Given the radios, the broadcast radius , a source index, and a target index, determine whether a message originating at source can reach target through zero or more hops. Return True or False .

Function signature

```
from typing import List, Tuple
def can_reach(radios: List[Tuple[float, float]],
              radius: float,
              source: int,
              target: int) -> bool:
    ...
```

Follow-up (as reported): Model the radios as nodes and connect any two within radius , then run BFS (or DFS / union-find) from source . To avoid floating-point issues, compare squared distances against radius * radius .

EXAMPLES

Example 1	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 2`
Output	`True`
Why	`0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.`

Example 2	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 3`
Output	`False`
Why	`Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest; no chain of hops reaches it.`

Example 3	
Input	`radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1`
Output	`True`
Why	`The Euclidean distance is exactly 5.0, which is within radius.`

CONSTRAINTS

- 1 <= len(radios) <= 2000
- Coordinates are integers or floats with -10^4 <= x, y <= 10^4 . radius >= 0 .

QUESTION

- 0 <= source, target < len(radios) .
- Two radios a and b are directly connected when sqrt((ax-bx)^2 + (ay-by)^2) <= radius .
- If source == target , the answer is True .

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
def squared_distance(p, q):
    # compare squared distances so no square root is needed
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return dx * dx + dy * dy


# ---- main question: main function ----
# Idea: the radios are the nodes of an implicit graph: two radios are
# neighbours when they are within `radius`. BFS from the source radio;
# every radio enters the queue once.
# Complexity: Time O(n^2) distance checks, Space O(n)
def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
    queue = deque([source])
    while queue:
        u = queue.popleft()
        if u == target:
            return True
        for v in range(len(radios)):
            if not seen[v] and squared_distance(radios[u], radios[v]) <= limit:
                seen[v] = True
                queue.append(v)
    return False
```

COMPLEXITY | Time O(n^2) | Space O(n)

211 Minimum Direction Violations in a Directed Graph

MEDIUM

Template fit: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

QUESTION

A directed graph has n nodes. Walking along an edge in its own direction is free; walking against it costs 1 (a direction violation). Return the minimum total cost of a walk from start to end, or -1 if end cannot be reached at all.

Function signature

def min_violations(n: int, edges: list, start: int, end: int) -> int

EXAMPLES

Example 1	
Input	`5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4`
Output	`2`
Why	`Path 0 -> 1 (free), 1 -> 2 against edge 2 -> 1 (cost 1), 2 -> 3 (free), 3 -> 4 against edge 4 -> 3 (cost 1): total 2.`

QUESTION

Example 2	
Input	`3, [[0, 1]], 0, 2`
Output	`-1`
Why	`Node 2 has no edge at all, so it cannot be reached.`

MAIN CODE | helper functions first, then the answer

```
from collections import deque


INF = float("inf")


# ---- main question: helper functions ----
def build_graph(n, edges):
    # every edge u -> v is also usable backwards: forward costs 0, backward costs 1
    graph = []
    for i in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    return graph


# ---- main question: main function ----
# Idea: with every cost either 0 or 1 a deque is enough (0-1 BFS):
# a free step goes to the FRONT of the deque, a cost-1 step to the BACK,
# so cells are always taken out in order of distance.
# Complexity: Time O(V + E), Space O(V + E)
def min_violations(n, edges, start, end):
    graph = build_graph(n, edges)
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    if dist[end] == INF:
        return -1
    return dist[end]
```

COMPLEXITY | Time O(n + m) | Space O(n + m)

212 Compute shortest paths with blocked nodes — MEDIUM

Template fit: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

QUESTION

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3], False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def make_adjacency(n, edges, directed):
    # neighbour lists; an undirected edge is stored in both directions
    adj = []
    for i in range(n):
        adj.append([])
    for edge in edges:
        u, v = edge[0], edge[1]
        weight = 1
        if len(edge) == 3:
            weight = edge[2]
        adj[u].append((v, weight))
        if not directed:
            adj[v].append((u, weight))
    return adj


# ---- main question: main function ----
# Idea: a BFS that never enters a blocked node. If the source itself is
# blocked everything is -1; unreachable and blocked nodes also get -1.
# Complexity: Time O(V + E), Space O(V + E)
def blocked_distances(n, edges, source, blocked, directed=False):
    if source in blocked:
        return [-1] * n
    adj = make_adjacency(n, edges, directed)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, w in adj[u]:
            if v not in blocked and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

213 Shortest Directed Cycle Through a Node

MEDIUM

Template fit: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

QUESTION

Given a directed graph with n nodes and a list of edges (a, b), return the length (number of edges) of the shortest cycle that passes through node v, or -1 if v is on no cycle.

Function signature

def shortest_cycle_through(n: int, edges: list, v: int) -> int

EXAMPLES

Example 1	
Input	3, [(0, 1), (1, 2), (2, 0)], 0
Output	3
Why	0 -> 1 -> 2 -> 0 is a cycle of 3 edges.

Example 2	
Input	3, [(0, 1), (1, 2)], 0
Output	-1
Why	No edge leads back to 0.

Example 3	
Input	3, [(0, 1), (1, 0), (1, 2), (2, 0)], 0
Output	2
Why	A shorter cycle 0 -> 1 -> 0 exists next to the longer one.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
def make_graph(n, edges):
    # adjacency lists of (neighbour, cost); an edge may be (a, b) or (a, b, cost)
    adj = []
    for i in range(n):
        adj.append([])
    for edge in edges:
        a, b = edge[0], edge[1]
        cost = 1
        if len(edge) == 3:
            cost = edge[2]
        adj[a].append((b, cost))
    return adj


# ---- main question: main function ----
# Idea: a directed cycle through v is a path v -> u followed by an edge
# u -> v. BFS reaches nodes in order of distance, so the first edge we
# see that leads back to v closes the shortest cycle: dist[u] + 1.
# Complexity: Time O(V + E), Space O(V + E)
def shortest_cycle_through(n, edges, v):
    adj = make_graph(n, edges)
    dist = {v: 0}
    queue = deque([v])
    while queue:
        u = queue.popleft()
        for w, cost in adj[u]:
```

```
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1
```

COMPLEXITY | Time O(V + E) | Space O(V)

GROUP 29 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Topological sort (Kahn) and cycle detection

214 Course Scheduling and Dependency Variants

MEDIUM

Template fit: build the vertex set from the edges too; return the order or None

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1: contradiction)
```

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def edges_from_sequences(sequences):
    # each neighbouring pair in a sequence is a rule: the first comes before the second
    edges = []
    for seq in sequences:
        for i in range(len(seq) - 1):
            edges.append((seq[i], seq[i + 1]))
    return edges


# ---- main question: main function ----
# Idea (Kahn's algorithm): an edge (u, v) means u must come before v.
# Count how many edges point into each vertex (indegree), keep a queue
# of vertices with indegree 0, take one, add it to the order and lower
# the indegree of the vertices after it. If not every vertex was taken
# there is a cycle and no order exists (None).
```

```python
# Complexity: Time O(V + E), Space O(V + E)
def topological_order(vertices, edges):
    indegree = {}
    after = {}
    for v in vertices:
        indegree[v] = 0
        after[v] = []
    for u, v in edges:
        for x in (u, v):
            if x not in indegree:
                indegree[x] = 0                 # a vertex that appears only in an edge
                after[x] = []
        after[u].append(v)
        indegree[v] += 1
    queue = deque()
    for v in indegree:
        if indegree[v] == 0:
            queue.append(v)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in after[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
```

```python
    if len(order) == len(indegree):
        return order
    return None


# the same idea merges two sequences: shared values are the same vertex
# Complexity: Time O(total length of both sequences), Space the same
def merge_two_orders(a, b):
    vertices = set(a) | set(b)
    return topological_order(vertices, edges_from_sequences([a, b]))


# Complexity: Time O(total length of both sequences), Space the same
def can_merge(a, b):
    # is there an order that keeps both sequences in order?
    return merge_two_orders(a, b) is not None
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

215 Validate Race Results (Cycle Detection) — MEDIUM

Template fit: edge = winner -> loser; the results are consistent exactly when there is no cycle

QUESTION

You are given a list of pairwise race results as (winner, loser) string pairs, meaning winner beat loser . The results are consistent if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge winner -> loser) contains no cycle. Return True if the results are valid (acyclic) and False otherwise.

Function signature

def results_are_valid(results: list[tuple[str, str]]) -> bool:

...

Follow-up (as reported): Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

EXAMPLES

QUESTION

Example 1	
Input	`results = [("A","B"), ("B","C")]`
Output	`True`
Why	`A beat B beat C — a consistent ranking A > B > C, no cycle.`

Example 2	
Input	`results = [("A","B"), ("B","C"), ("C","A")]`
Output	`False`
Why	`A > B > C > A is a contradiction (a cycle).`

Example 3	
Input	`results = []`
Output	`True`
Why	`No results impose no contradiction.`

CONSTRAINTS

- 0 <= len(results) <= 10^5 .
- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (True).

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
def build_graph(results):
    # results are pairs (winner, loser); graph[winner] = the people they beat
    graph = {}
    for winner, loser in results:
        if winner not in graph:
            graph[winner] = []
        if loser not in graph:
            graph[loser] = []
        graph[winner].append(loser)
    return graph


# ---- main question: main function ----
# Idea (Kahn's algorithm): results are consistent when nobody beats
# somebody who (through a chain) beat them, i.e. there is no cycle.
# Repeatedly take people nobody is left above; everybody gets taken
# exactly when there is no cycle.
# Complexity: Time O(V + E), Space O(V + E)
def results_are_valid(results):
    graph = build_graph(set(results))              # a repeated result counts once
    beaten_by = {}
    for person in graph:
        beaten_by[person] = 0
    for winner in graph:
        for loser in graph[winner]:
            beaten_by[loser] += 1
    queue = deque()
    for person in graph:
        if beaten_by[person] == 0:
            queue.append(person)
    removed = 0
```

```
    while queue:
        u = queue.popleft()
        removed += 1
        for v in graph[u]:
            beaten_by[v] -= 1
            if beaten_by[v] == 0:
                queue.append(v)
    return removed == len(graph)
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

216 Solve tree, graph, sliding-window problems — MEDIUM

Template fit: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters.
https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/
https://leetcode.com/problems/course-schedule/description/
https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

QUESTION

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(values):
    # level-order list, None for a missing child: [3, 9, 20, None, None, 15, 7]
    if not values:
        return None
    root = TreeNode(values[0])
```

```
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
    return root


# ---- main question: main functions ----
# PART 1: zigzag level order of a binary tree.
# Idea: go level by level with a queue; reverse every second level.
# Complexity: Time O(n), Space O(w) for the widest level w (O(n) at worst)
def zigzag(root):
    result = []
    if root is None:
        return result
    queue = deque([root])
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):              # exactly the nodes of this level
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result


# PART 2: can all courses be finished? prereqs are pairs (a, b): take b before a.
# Idea (Kahn's algorithm): count the unmet prerequisites of each course;
# take a course with none, lower the count of the courses that needed it,
# and repeat. If every course gets taken there is no cycle.
# Complexity: Time O(V + E), Space O(V + E)
def can_finish(n, prereqs):
    needs = []
    for i in range(n):
        needs.append([])
    unmet = [0] * n
    for a, b in prereqs:
        needs[b].append(a)                       # b unlocks a
        unmet[a] += 1
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    taken = 0
    while queue:
        c = queue.popleft()
        taken += 1
        for nxt in needs[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
```

```
                queue.append(nxt)
    return taken == n


# PART 3: longest substring with at most k distinct letters.
# Idea: sliding window with a count of each letter inside it; when
# there are too many distinct letters, shrink from the left.
# Complexity: Time O(n), Space O(k) distinct letters
def longest_at_most_k(s, k):
```

```
    count = {}
    left = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best
```

COMPLEXITY | Course schedule: Kahn's topological sort: Time O(V + E) | Space O(V + E)
Longest window: sliding window with a count map: Time O(n) | Space O(k)
Zigzag: fill each level from the correct end: Time O(n) | Space O(width)

217 Validate course catalog dependencies — MEDIUM

Template fit: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle

QUESTION

You are given a set of course ids and prerequisite pairs (u, v), meaning course v requires course u. The catalog is valid when no prerequisite mentions a course that does not exist and there is no cycle. Return (True, [], order) with a possible study order when valid; (False, missing, None) listing the unknown course ids; or (False, [], cycle) with one cycle that was found.

Function signature

def validate_catalog(courses: set, prereqs: list) -> tuple

EXAMPLES

Example 1	
Input	{1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]
Output	(true, [], [1, 4, 2, 3])
Why	Course 1 comes first, 4 is unrelated, then 2 and 3.

Example 2	
Input	{1, 2}, [(1, 9)]
Output	(false, [9], null)
Why	Course 9 is not in the catalog.

QUESTION

Example 3	
Input	{1, 2}, [(1, 2), (2, 1)]
Output	(false, [], [1, 2, 1])
Why	1 requires 2 and 2 requires 1: a cycle.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
def find_cycle(known, prereqs, unmet):
    # `unmet` are the counts left after Kahn's algorithm. Every leftover
    # course still has a leftover prerequisite, so walking backwards along
    # prerequisites must eventually repeat a course: that repeat closes a cycle.
    left = set()
    for course in known:
        if unmet[course] > 0:
            left.add(course)
    before = {}                                 # course -> its prerequisites that are leftovers
    for u, v in prereqs:
        if u in left and v in left:
            if v not in before:
                before[v] = []
            before[v].append(u)
    x = min(left, key=str)
    walk = []
    position = {}
    while x not in position:
        position[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[position[x]:] + [x]
    cycle.reverse()                             # we walked against the edges: list prerequisites first
    return cycle


# ---- main question: main function ----
# Idea: 1) report ids that are used but not in the course set. 2) Run
# Kahn's algorithm: repeatedly take courses with no remaining
# prerequisite, building an order. 3) If courses are left over there is
# a cycle: find one in the leftovers.
# prereqs are pairs (u, v): course v requires course u. Returns
# (ok, missing ids, order or one cycle).
# Complexity: Time O(V + E + V log V) (the sorts), Space O(V + E)
def validate_catalog(courses, prereqs):
    known = set(courses)
    missing = set()
    for u, v in prereqs:
        if u not in known:
            missing.add(u)
        if v not in known:
```

```
            missing.add(v)
    if missing:
        return False, sorted(missing, key=str), None
    unlocks = {}
    unmet = {}
    for c in known:
        unlocks[c] = []
        unmet[c] = 0
    for u, v in prereqs:
```

```
            unlocks[u].append(v)
            unmet[v] += 1
    ready = []
    for c in known:
        if unmet[c] == 0:
            ready.append(c)
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, unmet)
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

218 Course Schedule — MEDIUM

Template fit: edge = prerequisite -> course; return the order, or an empty list when there is a cycle

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

EXAMPLES

Example 1	
Input	`2, [[1, 0]]`
Output	`true`
Why	`Course 1 requires course 0: finishing both is possible.`

Example 2	
Input	`2, [[1, 0], [0, 1]]`
Output	`false`
Why	`Course 1 requires 0 and course 0 requires 1: a cycle.`

Example 3	
Input	`2, [[1, 0]]`
Output	`[0, 1]`
Why	`A valid order: take 0 before 1.`

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
def build(n, prereqs):
    # prereqs (a, b): course b must be taken before course a.
    # unlocks[b] = the courses that need b; unmet[a] = how many prerequisites a still waits for
    unlocks = []
    for i in range(n):
        unlocks.append([])
    unmet = [0] * n
    for a, b in prereqs:
        unlocks[b].append(a)
        unmet[a] += 1
    return unlocks, unmet


# ---- main question: main function ----
# Idea (Kahn's algorithm): queue every course with no unmet
# prerequisite. Take one, lower the count of the courses that needed
# it, and queue those that reach zero. If all n courses get taken
# there is no cycle.
# Complexity: Time O(V + E), Space O(V + E)
def can_finish(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    taken = 0
    while queue:
        c = queue.popleft()
        taken += 1
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    return taken == n
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

GROUP 30 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Counting pairs by looking up each partner in a Counter or a bucket

219 Progressive Array Partitioning — MEDIUM

Template fit: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)

QUESTION

The problem is given in stages; stage 1 is: group the equal values of an array together. Return true when every group has a different size (no two values occur the same number of times). The function takes the occurrence count of each value as a dict, so counting stays separate from the rule and later stages can change only the rule.

Function signature

def stage1(counts: dict) -> bool

counts maps value -> how

many times it occurs

EXAMPLES

Example 1	

Input	`from collections import Counter`
	`nums = [1, 2, 3, 4, 2, 3, 3, 4, 4, 4]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> true`
Why	`The groups have sizes 1, 2, 3, 4: all different, so true.`

Example 2	

Input	`from collections import Counter`
	`nums = [1, 2, 3]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> false`
Why	`Every value occurs once, so all three groups have size 1: false.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def count_values(nums):
    # {value: how many times it occurs}
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    return counts


# ---- main question: main function ----
# Stage 1. `counts` maps value -> how many times it occurs. The array can be
# split as the stage asks exactly when EVERY value occurs a different number
# of times (no two values share a count).
# Idea: the counts have no repeats exactly when the SET of counts has as many
# elements as there are distinct values. One pass to count, one over the counts.
# Complexity: Time O(u) for u distinct values, Space O(u)
def stage1(counts):
    return len(set(counts.values())) == len(counts)
```

COMPLEXITY | Time O(n) | Space O(d)

220 K-diff Pairs in an Array

MEDIUM

Template fit: the template as written: partner = v + k; for k == 0 count values that appear twice

QUESTION

Given an integer array nums and an integer k , return the number of unique k-diff pairs in the array. A k-diff pair is a pair of integers (nums[i], nums[j]) where: i != j , and abs(nums[i] - nums[j]) == k . Two k-diff pairs (a, b) and (c, d) are considered the same (counted once) if they consist of the same pair of values. Count each distinct value pair only once.

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [3, 1, 4, 1, 5], k = 2`
Output	`2`
Why	`The unique 2-diff value pairs are (1, 3) and (3, 5).`

Example 2	
Input	`nums = [1, 2, 3, 4, 5], k = 1`
Output	`4`
Why	`(1,2), (2,3), (3,4), (4,5).`

Example 3	
Input	`nums = [1, 3, 1, 5, 4], k = 0`
Output	`1`
Why	`With k = 0 count values that appear at least twice; only 1 does.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -10^7 <= nums[i] <= 10^7
- 0 <= k <= 10^7

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Count the DISTINCT pairs of values (x, x + k) that both appear in the array.
# Idea: count the values in a dictionary. A value v starts a pair when
# v + k is also present; for k = 0 the same value must appear at least twice.
# Complexity: Time O(n), Space O(n)
def count_k_diff_pairs(nums, k):
    if k < 0:
        return 0
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    pairs = 0
    for v in counts:
        if k == 0:
            if counts[v] >= 2:
                pairs += 1
        elif v + k in counts:
            pairs += 1
    return pairs
```

COMPLEXITY | Time O(n) | Space O(n)

221 Count Axis-Aligned Squares Among Grid Points — MEDIUM

Template fit: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2

QUESTION

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice points in the plane. Count the number of axis-aligned squares whose four corners all appear in points . A square is axis-aligned when its sides are parallel to the x- and y-axes. Return the count of such squares.

Function signature

```
from typing import List, Tuple
def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...
```

Follow-up (as reported): A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

EXAMPLES

Example 1	
Input	`points = [(0, 0), (0, 1), (1, 0), (1, 1)]`
Output	`1`
Why	`The four points form exactly one unit square.`

Example 2	
Input	`points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]`
Output	`5`
Why	`A 3x3 grid of points has four 1x1 squares and one 2x2 square.`

Example 3	
Input	`points = [(0, 0), (2, 0), (0, 1), (2, 1)]`
Output	`0`
Why	`These form a 2x1 rectangle, which is not a square.`

CONSTRAINTS

- 0 <= len(points) <= 2000 ; all points are distinct.
- Each point is (x, y) with -10^9 <= x, y <= 10^9 .
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Count the squares with sides parallel to the axes whose four corners are
# all among the points.
# Idea: two OPPOSITE corners decide the other two, so one set lookup per pair
# is enough: for points (x1, y1) and (x2, y2) with |x1 - x2| == |y1 - y2| != 0,
# the other corners are (x1, y2) and (x2, y1). Every square has two diagonals,
# so it is found twice: halve the total.
# Complexity: Time O(n^2), Space O(n)
def count_axis_aligned_squares(points):
    present = set(points)
```

```
    total = 0
    for i in range(len(points)):
        x1, y1 = points[i]
        for j in range(i + 1, len(points)):
            x2, y2 = points[j]
            if abs(x1 - x2) == abs(y1 - y2) and x1 != x2:
                if (x1, y2) in present and (x2, y1) in present:
                    total += 1
    return total // 2
```

COMPLEXITY | Time O(n^2) | Space O(n)

222 Sum of Two Squares in Two Ways

MEDIUM

*Template fit: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice*

QUESTION

Given an integer n, find all integers s < n that can be written as a sum of two positive squares in exactly two different ways (the order of the two squares does not matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return them in increasing order.

Function signature

def two_squares_two_ways(n: int) -> list

EXAMPLES

Example 1	
Input	100
Output	[50, 65, 85]
Why	50 = 1 + 49 = 25 + 25 and 65 = 1 + 64 = 16 + 49 both have exactly two ways.

Example 2	
Input	50
Output	[]
Why	Nothing below 50 qualifies.

CONSTRAINTS

- 1 <= n <= 2^31 - 1

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Find every number s < n that can be written as a sum of two positive
# squares a^2 + b^2 (with a <= b) in EXACTLY two different ways.
# Idea: loop over every pair a <= b with a^2 + b^2 < n (about pi * n / 8
# pairs) and count how often each sum appears in a dictionary. The answer is
# the sorted list of the sums whose count is exactly 2.
# Follow-up 1: exactly k ways: compare with k instead of 2 (for k = 3 the
# smallest is 325). Follow-up 2: zero=True starts a at 0, so perfect squares
# like 25 = 0^2 + 5^2 gain a way: check the examples to see which convention the
# question uses (here: positive only).
# Complexity: Time O(n) (about n / 2 pairs a <= b with a^2 + b^2 < n), Space O(n)
def two_squares_two_ways(n, k=2, zero=False):
    ways = {}
    a = 1
    if zero:
        a = 0
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            total = a * a + b * b
            ways[total] = ways.get(total, 0) + 1
```

```
            b += 1
        a += 1
    sums = []
    for total in ways:
        if ways[total] == k and total > 0:
            sums.append(total)
    sums.sort()
    return sums
```

COMPLEXITY | Time O(n) | Space O(n)

223 Maximum-Area Rectangle from Planar Points — HARD

Template fit: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

EXAMPLES

Example 1	
Input	`[(1, 1), (1, 3), (3, 1), (3, 3)]`
Output	`4`
Why	`The four points form a 2 x 2 axis-aligned square.`

Example 2	
Input	`[(0, 1), (1, 0), (2, 1), (1, 2)]`
Output	`2`
Why	`A tilted rectangle: a square with diagonal 2 has area 2.`

Example 3	
Input	`[(0, 0), (1, 0), (0, 1)]`
Output	`0`
Why	`Three points never make a rectangle.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def cross(o, p, q):
    # twice the signed area of the triangle o, p, q
    ax = p[0] - o[0]
    ay = p[1] - o[1]
    bx = q[0] - o[0]
    by = q[1] - o[1]
    return ax * by - ay * bx


def all_rectangle_areas(points):
    # the area of every rectangle that the points form (the sides need not be
    # parallel to the axes). Idea: the two diagonals of a rectangle have the
```

```
    # SAME middle and the SAME length, so put every pair of points in a bucket
    # keyed by (sum of x, sum of y, squared length); any two pairs in one bucket
    # are the diagonals of a rectangle.
    unique = []
    seen = set()
    for x, y in points:
        if (x, y) not in seen:                          # duplicates cannot help
            seen.add((x, y))
            unique.append((x, y))
    buckets = {}
    for i in range(len(unique)):
        for j in range(i + 1, len(unique)):
            p = unique[i]
            q = unique[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            if key not in buckets:
                buckets[key] = []
            buckets[key].append((p, q))
    areas = []
    for pairs in buckets.values():
        for a in range(len(pairs)):
            for b in range(a + 1, len(pairs)):
                p, q = pairs[a]
                r, s = pairs[b]
                area = cross(p, r, s)                   # the two sides leaving p are p-r and p-s: their cross
 product is the rectangle's area
                if area < 0:
                    area = -area
                areas.append(area)
    return areas


# ---- main question: main function ----
```

```
# The biggest rectangle (any angle) with corners among the points, or 0.
# Complexity: Time O(n^2), Space O(n)
def max_rect_area(points):
    best = 0
    for area in all_rectangle_areas(points):
        if area > best:
            best = area
    return best


# the related variant: the SMALLEST area (LeetCode 963)
# Complexity: Time O(n^2), Space O(n)
def min_rect_area(points):
    smallest = 0
    for area in all_rectangle_areas(points):
        if smallest == 0 or area < smallest:
            smallest = area
    return smallest
```

COMPLEXITY | Time O(n^2) pairs (worst case O(n^3) if every pair shares a bucket) | Space O(n^2)

GROUP 31 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Tree DP: build a parent order once, then add up from the leaves

224 Compute File System Total Size — MEDIUM

Template fit: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)

QUESTION

A file system is a tree: File(size) is a file and Dir(*children) is a directory holding files and directories. Return the total size of all files. A directory object can be shared (it appears under more than one parent); its files are counted once.

Function signature

def total_size(root) -> int

EXAMPLES

Example 1	
Input	`root = Dir(Dir(File(10), File(20)), File(5), Dir(File(30)))` `total_size(root)`
Output	`total_size(root) -> 65`
Why	`10 + 20 + 5 + 30 = 65.`

Example 2	

Input	`shared = Dir(File(7))`
	`root = Dir(shared, shared, File(1))`
	`total_size(root)`

Output	`total_size(root) -> 8`
Why	`The same directory appears twice but its 7 is counted once: 7 + 1 = 8.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class File:
    def __init__(self, size):
        self.size = size
        self.parent = None


class Dir:
    def __init__(self, *children):
        self.children = list(children)
        self.parent = None
        self.cached = None                          # the total size; None means "needs recomputing"
        for child in children:
            child.parent = self


# ---- main question: main function ----
# Idea: an explicit stack removes the recursion limit, and a set of the
# nodes already seen makes each one count once, so a directory that is
# shared (hard links, follow-up 2) is not counted twice and a cycle
# cannot loop forever. count_once=False counts a shared file once per path.
# Complexity: Time O(n) for n files and folders, Space O(n)
def total_size(root, count_once=True):
    total = 0
    seen = set()
    stack = [root]
    while stack:
        node = stack.pop()
```

```
        if count_once:
            if node in seen:
                continue
            seen.add(node)
        if isinstance(node, File):
            total += node.size
        else:
            for child in node.children:
                stack.append(child)
    return total
```

COMPLEXITY | Time O(n) | Space O(n)

225 Minimum-Cost Tree Disconnection — MEDIUM

Template fit: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free

QUESTION

A rooted tree is given as directed edges (parent, child, cost) with node 0 as the root. Cut edges so that no path from the root to any original leaf remains. The price of a cut set is the sum of the costs of the edges cut (costs may be negative). Return the minimum total price.

Function signature

def disconnect_cost(n: int, edges: list) -> int

EXAMPLES

Example 1	
Input	3, [(0, 1, 3), (0, 2, 2)]
Output	5
Why	Two leaves hang off the root: both edges must be cut, 3 + 2 = 5.

Example 2	
Input	3, [(0, 1, 4), (1, 2, 1)]
Output	1
Why	A chain 0 - 1 - 2: cutting the cheaper edge 1 -> 2 (cost 1) already separates the leaf.

Example 3	
Input	3, [(0, 1, -1), (1, 2, -1)]
Output	-2
Why	Negative costs: cutting both edges costs -2, which beats cutting only one.

MAIN CODE | helper functions first, then the answer

```
INF = float("inf")


# ---- main question: helper functions ----
def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root.
    # best[v] = the cheapest set of cuts that separates v from every leaf below it
    # free[v] = the sum of all NEGATIVE costs below v (cutting those too only helps)
    kids = []
    for i in range(n):
        kids.append([])
    for u, v, w in edges:
        kids[u].append((v, w))
```

```
    order = [0]                                     # parents before children (breadth first)
    for u in order:                                 # (a loop, not recursion: safe on a long chain)
        for v, w in kids[u]:
            order.append(v)
    best = [0] * n
    free = [0] * n
    for i in range(len(order) - 1, -1, -1):         # children before parents
        v = order[i]
        for c, w in kids[v]:
            cut = w + free[c]                       # option 1: cut this edge (and the negatives below)
            if kids[c]:
                keep = best[c]                      # option 2: keep it and cut deeper
            else:
                keep = INF                          # a leaf has nothing deeper to cut
            if cut < keep:
                best[v] += cut
            else:
                best[v] += keep
            if w < 0:
                free[v] += w
            free[v] += free[c]
    return kids, best, free


# ---- main question: main function ----
# Idea: for a child c reached by an edge of weight w there are two
# options: cut that edge (pay w, plus the negative edges you can still
# cut below c) or keep it and cut deeper (best(c)), which is impossible
# if c is a leaf. Add up the cheaper option over all children of the root.
# For non-negative costs this is simply: a leaf child costs w, any other
# child costs min(w, best(child)).
# Complexity: Time O(n), Space O(n)
```

```
def disconnect_cost(n, edges):
    kids, best, free = tree_costs(n, edges)
    return best[0]
```

COMPLEXITY | Time O(n) | Space O(h)

226 Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

MEDIUM

Template fit: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root

QUESTION

Information starts at the root of a tree. In every iteration a node that already has the information can pass it to one of its children (the tree maps each node to its children as (child, cost) pairs; in the basic problem every cost is 1 iteration). Return the minimum number of iterations until every node has the information.

Function signature

def min_rounds(tree: dict, root) -> int

EXAMPLES

Example 1	
Input	`{'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]}, 'r'`
Output	`3`
Why	`The root informs a, b and c in iterations 1, 2, 3, while a informs d in iteration 2: everyone knows after 3 iterations.`

QUESTION

Example 2	
Input	`{'r': [('a', 1)], 'a': [('b', 1)]}, 'r'`
Output	2
Why	A path r - a - b needs one iteration per edge.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def unit_tree(children):
    # turn {"r": ["a", "b"]} into the form the question uses: every call costs 1 round
    tree = {}
    for node in children:
        tree[node] = []
        for child in children[node]:
            tree[node].append((child, 1))
    return tree


def finish_times(tree, root):
    # time[v] = rounds until v and everything below it has the news, once v has it.
    # v calls its children one after another; the call to the i-th child costs the
    # calls made before it, and that child then needs time[child] more rounds.
    # Best order: the child that needs the LONGEST first.
    order = []
    stack = [root]
    while stack:                                # parents before children
        node = stack.pop()
        order.append(node)
        for child, cost in tree.get(node, []):
            stack.append(child)
    time = {}
    plan = {}
    for i in range(len(order) - 1, -1, -1):     # children before parents
        node = order[i]
        kids = []
        for index, pair in enumerate(tree.get(node, [])):
            child, cost = pair
            kids.append((-time[child], index, child, cost))   # sorts the slowest child first
        kids.sort()
        spent = 0
        worst = 0
        order_of_calls = []
        for slowness, index, child, cost in kids:
            spent += cost                       # the call to this child is finished at `spent`
            if spent + time[child] > worst:
                worst = spent + time[child]
            order_of_calls.append(child)
        time[node] = worst
        plan[node] = order_of_calls
    return time, plan
```

```
# ---- main question: main function ----
# Idea: let T(v) be the rounds needed to inform the subtree of v once v
# knows. v informs its children one per round, so serve the child with
# the BIGGEST T first (the biggest tail should start earliest) and
# T(v) = the latest finish over its children. Answer: T(root).
# Complexity: Time O(n log n) (each node sorts its children's times), Space O(n)
def min_rounds(tree, root):
    time, plan = finish_times(tree, root)
    return time[root]
```

COMPLEXITY | Time O(n log n) | Space O(n)

227 Compute distance-sum from every tree node — HARD

Template fit: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child])

QUESTION

A tree has n nodes numbered 0 to n - 1 and n - 1 edges (u, v). For every node, return the sum of the distances from that node to all the other nodes.

Function signature

def distance_sums(n: int, edges: list) -> list

EXAMPLES

Example 1	
Input	`4, [(0, 1), (0, 2), (2, 3)]`
Output	`[4, 6, 4, 6]`
Why	`Node 0: 1 + 1 + 2 = 4. Node 1 is a leaf: 1 + 2 + 3 = 6. Node 2: 1 + 2 + 1 = 4. Node 3: 1 + 2 + 3 = 6.`

Example 2	
Input	`3, [(0, 1), (1, 2)]`
Output	`[3, 2, 3]`
Why	`A path of three nodes: the middle one is closest to the others.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def build_adjacency(n, edges):
    # an edge is [u, v] or [u, v, weight]; neighbours are (node, weight)
    adj = []
    for i in range(n):
        adj.append([])
    for e in edges:
        u, v = e[0], e[1]
        w = 1
        if len(e) > 2:
            w = e[2]
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (in a tree one walk is enough)
    dist = {s: 0}
    stack = [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# ---- main question: main function ----
# Idea (re-rooting): root the tree at 0 and compute size[v] (the number
# of nodes below v) and the answer for the root: the sum of its
# distances. Moving the "root" from a parent p across an edge of weight w
# to its child u makes the size[u] nodes below u closer by w and the
```

```
# other n - size[u] nodes farther by w:
#   answer[u] = answer[p] + w * (n - 2 * size[u]).
# Edges may carry a weight (follow-up 2); an edge without one costs 1.
# Complexity: Time O(n), Space O(n)
def distance_sums(n, edges):
    adj = build_adjacency(n, edges)
    parent = [-1] * n
    up = [0] * n                                    # weight of the edge to the parent
    order = [0]                                     # parents before children
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v] = u
                up[v] = w
                order.append(v)
    size = [1] * n
    for i in range(n - 1, -1, -1):                  # children before parents
        u = order[i]
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    answer = [0] * n
    for d in distances_from(adj, 0).values():
        answer[0] += d                              # the root's answer is a plain sum of distances
    for u in order[1:]:
        answer[u] = answer[parent[u]] + up[u] * (n - 2 * size[u])
    return answer
```

COMPLEXITY | Time O(n) | Space O(n)

228 Tree Distance Sum (Re-root DP) — HARD

Template fit: exactly the same two sweeps (this is the re-rooting version of the question)

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def build_adjacency(n, edges):
    adj = []
    for i in range(n):
        adj.append([])
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj


# ---- main question: main function ----
# Idea: two passes. Root the tree at 0 and list the nodes parents-first
# (a loop, no recursion). Going through that list backwards compute
```

```python
# size[u] (nodes in u's subtree) and inside[u] (the sum of distances
# from u to the nodes of its own subtree). For the root that is already
# the answer. Going forwards, moving the root from p to its child u makes
# the size[u] nodes below 1 closer and the other n - size[u] nodes 1
# farther: answer[u] = answer[p] + n - 2 * size[u].
# Complexity: Time O(n), Space O(n)
def tree_distance_sums(n, edges):
    if n == 0:
        return []
    adj = build_adjacency(n, edges)
    parent = [-1] * n
    order = [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    size = [1] * n
    inside = [0] * n
    for i in range(n - 1, -1, -1):                # children before parents
        u = order[i]
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]      # every node below u is one step farther from p
    answer = [0] * n
    answer[0] = inside[0]
    for u in order[1:]:
        answer[u] = answer[parent[u]] + n - 2 * size[u]
    return answer
```

COMPLEXITY | Time O(n) | Space O(n)

Counting or reaching an amount: the knapsack loop

229 Maximize Shelf Space (Unbounded Knapsack) MEDIUM

Template fit: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity

QUESTION

A shelf has a fixed integer capacity capacity . You have boxes of various integer sizes given in sizes , and you may use any number of each box, including duplicates (each size is available infinitely). Choose boxes so their total size does not exceed capacity , maximizing the total size used. Return that maximum total.

Function signature

def max_shelf_space(capacity: int, sizes: list[int]) -> int:
    ...

EXAMPLES

Example 1	
Input	capacity = 10, sizes = [3, 5, 7]
Output	10
Why	3 + 7 = 10 exactly fills the shelf.

Example 2	
Input	capacity = 11, sizes = [5, 3]
Output	11
Why	5 + 3 + 3 = 11.

Example 3	
Input	capacity = 7, sizes = [4, 5]
Output	5
Why	Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.

CONSTRAINTS

- 0 <= capacity <= 10**4 .
- 1 <= len(sizes) <= 100 , each size is a positive integer.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: reachable[c] says "the boxes can fill exactly c units of shelf".
# 0 is reachable, and c is reachable if taking away one box (size s <= c)
# leaves a reachable amount. A box size may be used many times (unbounded).
# The answer is the biggest reachable amount up to the capacity.
# Complexity: Time O(capacity * number of sizes), Space O(capacity)
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
    for c in range(1, capacity + 1):
        for s in sizes:
            if s <= c and reachable[c - s]:
                reachable[c] = True
    best = 0
    for c in range(capacity + 1):
        if reachable[c]:
            best = c
    return best
```

COMPLEXITY | Time O(capacity * boxes) | Space O(capacity)

230 Coin Change

MEDIUM

Template fit: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

EXAMPLES

Example 1	
Input	[1, 2, 5], 11
Output	3
Why	5 + 5 + 1 = 11: 3 coins.

Example 2	
Input	[2], 3
Output	-1
Why	A coin of 2 can never make 3.

Example 3	
Input	[1], 0
Output	0
Why	Zero needs no coins.

MAIN CODE | helper functions first, then the answer

```python
INF = float("inf")


# ---- main question: main function ----
# Idea (bottom-up table): best[a] = the fewest coins that make exactly a.
# best[0] = 0, and for a = 1, 2, ... try every coin c <= a: best[a] =
# 1 + best[a - c] when that is better. Every amount is solved once.
# Answer best[amount], or -1 if it stayed infinite.
# Complexity: Time O(amount * number of coins), Space O(amount)
def coin_change(coins, amount):
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
```

```
    if best[amount] == INF:
        return -1
    return best[amount]
```

COMPLEXITY | Time O(amount * coins) | Space O(amount)

231 Partition an array into two sub-arrays with equal sum. MEDIUM

Template fit: subset sum to total / 2, written with a bitset: reach |= reach << x

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

EXAMPLES

Example 1	
Input	`[1, 5, 11, 5]`
Output	`true`
Why	`[1, 5, 5] and [11] both add up to 11.`

Example 2	
Input	`[1, 2, 3, 5]`
Output	`false`
Why	`The total is 11, which is odd.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the array can be split into two parts with the same sum exactly
# when some subset adds up to half of the total. reachable[s] means "some
# subset adds up to s"; for each number x go through s from high to low
# (so x is used once) and mark reachable[s] when reachable[s - x] is true.
# Complexity: Time O(n * sum / 2), Space O(sum)
def can_partition(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return False                                # an odd total cannot be split evenly
    half = total // 2
    reachable = [True] + [False] * half
    for x in nums:
        for s in range(half, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    return reachable[half]
```

COMPLEXITY | Time O(n * sum / 64) with big-integer shifts (O(n * sum) as a boolean table) | Space O(sum)

232 Explain how to find a target sum in an array. MEDIUM

Template fit: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

EXAMPLES

Example 1	
Input	`[1, 1, 1, 1, 1], 3`
Output	`5`
Why	`Put + or - in front of every number; five ways to reach 3 with five 1s.`

Example 2	
Input	`[1], 1`
Output	`1`
Why	`+1 is the only way.`

Example 3	
Input	`[1, 2], 5`
Output	`-1`
Why	`No sign choice gives 5.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Put a + or - sign in front of every number so that they add up to
# `target`; count the ways (-1 if the target cannot be reached at all).
# Idea: the numbers with a PLUS sign add up to P, the others to total - P,
# so P - (total - P) = target, i.e. P = (total + target) / 2. The question
# becomes "how many subsets add up to P?". ways[s] = the number of subsets
# with sum s; for each number x update s from high to low (so each number is
# used once): ways[s] += ways[s - x].
# Complexity: Time O(n * S) for S = (sum + target) / 2, Space O(S)
def count_signs(nums, target):
    total = 0
    for x in nums:
        total += x
    if abs(target) > total or (total + target) % 2 == 1:
        return -1
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
```

```
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]
```

COMPLEXITY | Time O(n * sum) | Space O(sum)

233 Reverse Coin Change

MEDIUM

Template fit: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

QUESTION

In the coin change counting problem, dp[i] is the number of ways to make amount i from a set of coins (dp[0] = 1). In the reverse problem you are given the dp array and must find a set of coins that produces it, or None if no coin set does.

Function signature

def coins_from_dp(dp: list) -> list | None

EXAMPLES

Example 1	
Input	`[1, 1, 2, 2, 3, 3]`
Output	`[1, 2]`
Why	`Coins 1 and 2 make 1, 1, 2, 2, 3, 3 ways for the amounts 0 to 5.`

Example 2	
Input	`[1, 0, 1, 0, 1]`
Output	`[2]`
Why	`Only the coin 2 can make 2 and 4 but not 1 and 3.`

Example 3	
Input	`[1, 2, 3]`
Output	`null`
Why	`No coin set can produce these counts.`

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def ways_table(coins, n):
    # the FORWARD direction: dp[a] = the number of combinations of the coins that make a.
    # (coins in the outer loop: each combination counts once, not once per order)
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


# ---- main question: main function ----
# The usual coin-change table dp[a] counts the combinations of coins that
# make amount a. Here you get the table and must find the coins.
# Idea: scan the amounts upward and keep the table built from the coins
# found so far (`built`). At the first amount where the real dp is
# exactly ONE more than what we built, there must be a new coin of that
# value (it adds the single way "just that coin"). Any other difference is
```

```
# impossible. At the end the rebuilt table must equal the input (this is
# also follow-up 1: the array might not come from any coins).
# Complexity: Time O(n * k) for k coins found (k <= n, so O(n^2) at worst), Space O(n)
def coins_from_dp(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None                                 # dp[0] must be 1: one way to make 0
    n = len(dp) - 1
    built = [1] + [0] * n
    coins = []
    for a in range(1, n + 1):
        if dp[a] == built[a]:
            continue                                # the coins found so far already explain amount a
        if dp[a] != built[a] + 1:
            return None                             # needs more than one new way: impossible
        coins.append(a)
        for x in range(a, n + 1):
            built[x] += built[x - a]                # add the new coin to the table
    if built == list(dp):
        return coins
    return None
```

COMPLEXITY | Time O(n * coins) = O(n^2) worst case | Space O(n)

GROUP 33 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Prefix sums, Tier 2: running total plus a dict of earlier totals

234 Maximum-Sum Subarray With Equal Endpoints — MEDIUM

Template fit: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total

QUESTION

Given an integer array nums , choose a (contiguous) subarray whose first and last elements are equal and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Function signature

def max_sum_equal_endpoints(nums: list[int]) -> list[int]:

    ...

Follow-up (as reported): Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

EXAMPLES

Example 1	
Input	`nums = [1, 3, -2, 4, 1]`
Output	`[1, 3, -2, 4, 1]`
Why	`The value 1 appears at indices 0 and 4; that whole window sums to 7, beating any single element.`

Example 2	
Input	`nums = [5, -10, 5]`
Output	`[5]`
Why	`The window from the first 5 to the last 5 sums to 0, which is worse than taking a single 5 (sum 5).`

Example 3	
Input	`nums = [-3, -1, -2]`
Output	`[-1]`
Why	`All values are distinct, so only single elements qualify; -1 is the largest.`

CONSTRAINTS

- 1 <= len(nums) <= 10^5 .
- -10^4 <= nums[i] <= 10^4 .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Find the subarray with the biggest sum whose first and last elements are
# equal. Ties: the smaller start, then the shorter one. Return the subarray.
# Idea: with prefix sums, the sum of nums[i..j] is prefix[j+1] - prefix[i].
# For an end j with value x, the best start is an earlier index with the
# SAME value x and the smallest prefix[i] (a start at j itself is allowed:
# a single element). Remember, per value, the best start so far.
# Complexity: Time O(n), Space O(n)
def max_sum_equal_endpoints(nums):
    prefix = [0]
    for x in nums:
```

```
        prefix.append(prefix[len(prefix) - 1] + x)
    best_start = {}                                    # value -> the start index with the smallest prefix
    best = None                                        # (minus the sum, start, length): smaller is better
    for j in range(len(nums)):
        x = nums[j]
        if x not in best_start or prefix[j] < prefix[best_start[x]]:
            best_start[x] = j
        i = best_start[x]
        key = (-(prefix[j + 1] - prefix[i]), i, j - i)
        if best is None or key < best:
            best = key
    if best is None:
        return []
    return nums[best[1]:best[1] + best[2] + 1]
```

COMPLEXITY | Time O(n) | Space O(n)

235 Maximum Subarray Sum With a Minimum Length Constraint — MEDIUM

Template fit: the smallest earlier total is only allowed from min_len positions back, so update it one step late

QUESTION

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint"). Given an integer array nums and an integer min_len , find the contiguous subarray (containing at least min_len elements) that has the largest sum, and return that sum.

Function signature

def max_subarray_min_len(nums: list[int], min_len: int) -> int:

"""Return the maximum sum over all contiguous subarrays of length >= min_len."""

...

Follow-up (as reported): The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range [min_len, max_len] . Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

EXAMPLES

Example 1	
Input	`nums = [1, -2, 3, 4, -1], min_len = 2`
Output	`7`
Why	`The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays of length >= 2.`

Example 2	
Input	`nums = [-1, -2, -3], min_len = 2`
Output	`-3`
Why	`Every element is negative. The best length->=2 subarray is [-1, -2] with sum -3.`

Example 3	
Input	`nums = [5, 4, -1, 7, 8], min_len = 1`
Output	`23`
Why	`With min_len = 1 this reduces to ordinary Kadane; the whole array sums to 23.`

CONSTRAINTS

- 1 <= min_len <= len(nums) <= 10^5
- -10^4 <= nums[i] <= 10^4
- The array is guaranteed to be non-empty and min_len never exceeds len(nums) , so a valid subarray always exists.

QUESTION

- Values may be negative; the answer can therefore be negative.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def prefix_sums(nums):
    # prefix[j] = the sum of the first j numbers
    prefix = [0]
    for x in nums:
        prefix.append(prefix[len(prefix) - 1] + x)
    return prefix


# ---- main question: main function ----
# Idea: the sum of nums[i..end-1] is prefix[end] - prefix[i]. For each end,
# the best start is the SMALLEST prefix sum among the starts that leave at
# least min_len elements (i <= end - min_len). Sweep the end forward and
# keep that running minimum.
# Complexity: Time O(n), Space O(n)
def max_subarray_min_len(nums, min_len):
    prefix = prefix_sums(nums)
    best = None
    smallest = None
    for end in range(min_len, len(nums) + 1):
        allowed_start = prefix[end - min_len]            # the newest start that is allowed now
        if smallest is None or allowed_start < smallest:
            smallest = allowed_start
        candidate = prefix[end] - smallest
        if best is None or candidate > best:
            best = candidate
    return best
```

COMPLEXITY | Time O(n) | Space O(n)

236 Solve array-sum and city-community problems

MEDIUM

Template fit: count subarrays summing to k with the template as written; the second task is union-find over building cells

QUESTION

Two tasks. (1) count_subarrays(nums, k): count the contiguous subarrays whose elements add up to k. (2) communities(grid): a grid has 1 for a building and 0 for empty land; buildings that touch up, down, left or right belong to the same community. Return the number of communities.

Function signature

def count_subarrays(nums: list, k: int) -> int
def communities(grid: list) -> int

EXAMPLES

Example 1	
Input	`[1, 1, 1], 2`
Output	`2`
Why	`[1, 1] appears twice (positions 0-1 and 1-2).`

Example 2	
Input	`[1, 2, 3], 3`
Output	`2`
Why	`[1, 2] and [3] both add up to 3.`

QUESTION

Example 3	
Input	`[[1, 1, 0], [0, 0, 1], [1, 0, 1]]`
Output	3
Why	Top-left pair, the single building at the right of the middle row joined to the one below it, and the bottom-left: 3 communities.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# ---- main question: main functions ----
# PART 1: count the subarrays whose sum is exactly k.
# Idea: walk along the array with the running (prefix) sum. A subarray
# that ENDS here has sum k when some EARLIER prefix sum equals
# (prefix - k). Keep a dictionary: prefix sum -> how many times it occurred.
# Complexity: Time O(n), Space O(n)
def count_subarrays(nums, k):
    seen = {0: 1}                          # the empty prefix
    prefix = 0
    count = 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# PART 2: count the communities: groups of neighbouring buildings (1) in a grid.
# Idea: scan the grid; an unseen building starts a new community, and a
# flood fill marks all the buildings joined to it.
# Complexity: Time O(m * n), Space O(m * n)
def communities(grid):
    seen = set()
    count = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a = x + dx
                    b = y + dy
                    if a < 0 or a >= len(grid) or b < 0 or b >= len(grid[0]):
                        continue
                    if grid[a][b] == 1 and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
```

```
    return count
```

COMPLEXITY | Task 1: prefix sums in a dictionary: Time O(n) | Space O(n)
Task 2: Union-Find over building cells: Time O(m * n * alpha(m * n)) | Space O(m * n)

237 Find the maximum subarray sum. MEDIUM

Template fit: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

EXAMPLES

Example 1	
Input	`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`
Output	`6`
Why	`[4, -1, 2, 1] has the largest sum, 6.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single element.`

Example 3	
Input	`[5, 4, -1, 7, 8]`
Output	`23`
Why	`All numbers are positive: take them all.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea (Kadane): `current` is the best sum of a subarray that ENDS at the
# current element: either the element alone, or the element added to the
# previous `current`. `best` is the biggest `current` seen.
# Complexity: Time O(n), Space O(1)
def max_subarray(nums):
    current = nums[0]
    best = nums[0]
    for i in range(1, len(nums)):
        current = bigger(nums[i], current + nums[i])
        best = bigger(best, current)
    return best
```

COMPLEXITY | Time O(n) | Space O(1)

238 Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. HARD

Template fit: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

EXAMPLES

Example 1	
Input	`[23, 2, 4, 6, 7], 6`
Output	`true`
Why	`[2, 4] adds up to 6, a multiple of 6.`

Example 2	
Input	`[23, 2, 6, 4, 7], 13`
Output	`false`
Why	`No subarray of at least two elements adds up to a multiple of 13.`

Example 3	
Input	`[23, 2, 6, 4, 7], 6`
Output	`true`
Why	`[23, 2, 6, 4, 7] adds up to 42, a multiple of 6.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def find_good_subarray(nums, k):
    # the (start, end) indexes of a subarray of at least 2 elements whose
    # sum is a multiple of k, or None.
    # Idea: prefix sums MODULO k. Two equal remainders at indexes at least 2
    # apart mean the elements between them add up to a multiple of k. Keep
    # only the FIRST index of each remainder: a later one only gives a
    # shorter block. (k == 0 means "equal to zero": compare the sums themselves.)
    first = {0: -1}                        # remainder -> the first index where it occurred
    total = 0
    for i in range(len(nums)):
        total += nums[i]
        if k != 0:
            key = total % k
        else:
            key = total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return first[key] + 1, i
    return None


# ---- main question: main function ----
```

```
# Complexity: Time O(n), Space O(min(n, k))
def has_good_subarray(nums, k):
    return find_good_subarray(nums, k) is not None
```

COMPLEXITY | Time O(n) | Space O(min(n, k))

Group by a canonical key

239 **Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.** **EASY**

Template fit: key = the keyboard row of each letter; a word qualifies when its letters map to one row

QUESTION

Given an array of words, return those that can be typed using letters from a single row of an American keyboard (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore upper and lower case and keep the input order.

Function signature

def findWords(words: list) -> list

EXAMPLES

Example 1	
Input	`['Hello', 'Alaska', 'Dad', 'Peace']`
Output	`['Alaska', 'Dad']`
Why	`'Alaska' and 'Dad' use only the middle row; 'Hello' and 'Peace' mix rows.`

Example 2	
Input	`['omk']`
Output	`[]`
Why	`'omk' mixes the top and bottom rows.`

Example 3	
Input	`['adsdf', 'sfd']`
Output	`['adsdf', 'sfd']`
Why	`Both words stay on one row.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]


def row_map(rows):
    # letter -> the number of its row; built once and reused. Change `rows`
    # for another keyboard layout (follow-up 1).
    row_of = {}
    for i in range(len(rows)):
        for ch in rows[i]:
            row_of[ch] = i
    return row_of


# ---- main question: main function ----
# Which words can be typed with the letters of ONE keyboard row?
# Idea: build a dictionary from letter to row number (one entry per letter).
# A word qualifies if all of its (lowercased, so "Alaska" works) letters are
# on the same row. A letter that is on no row makes the word invalid.
# Complexity: Time O(total length of all words), Space O(1) (26 letters)
def findWords(words, rows=QWERTY):
    row_of = row_map(rows)
    result = []
```

```
    for word in words:
        letters = word.lower()
        valid = True
        used_rows = set()
        for ch in letters:
            if ch not in row_of:
                valid = False                          # a letter on no row
                break
            used_rows.add(row_of[ch])
        if valid and len(used_rows) <= 1:
            result.append(word)
    return result
```

COMPLEXITY | Time O(total characters) | Space O(1)

240 Check if all substrings are anagrams of words — MEDIUM

Template fit: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - “Can be rearranged into a word” means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

EXAMPLES

Example 1	
Input	`'abca', ['abc', 'cab', 'bca', 'abca']`
Output	`true`
Why	`Every substring of length 3 or more of 'abca' is an anagram of a dictionary word.`

Example 2	
Input	`'abca', ['abc']`
Output	`false`
Why	`'bca' is an anagram of 'abc' but 'abca' has no match.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


def count_vector(text):
    # the number of times each letter a..z occurs, as a tuple (the "signature")
    counts = [0] * len(ALPHABET)
    for ch in text:
        counts[ord(ch) - ord("a")] += 1
    return tuple(counts)


# ---- main question: main function ----
# Is EVERY substring of s (of length at least min_len) an anagram of some
# dictionary word?
# Idea: precompute the set of letter-count signatures of the dictionary
```

```
# words. For each start index grow the substring one letter at a time and
# update its count vector in O(1); once the length is at least min_len, look
# its signature up in the set.
# Complexity: Time O(n^2 * A) for a string of length n (the 26 counts are compared for each substring), Space
 O(words * A)
def all_anagram_words(s, words, min_len=3):
    signatures = set()
    for word in words:
        signatures.add(count_vector(word))
    for i in range(len(s)):
        counts = [0] * len(ALPHABET)
        for j in range(i, len(s)):
            counts[ord(s[j]) - ord("a")] += 1
            if j - i + 1 >= min_len and tuple(counts) not in signatures:
                return False
    return True
```

COMPLEXITY | Time O(n^2 * A) for alphabet size A | Space O(dictionary size)

241 Group anagrams — MEDIUM

Template fit: key = the sorted letters (or a tuple of 26 letter counts)

QUESTION

Given a list of words, group together the words that are anagrams of each other (the same letters in a different order). Return the groups in any order.

Function signature

def group_by_sorting(words: list) -> list

EXAMPLES

Example 1	
Input	['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
Output	[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
Why	eat, tea and ate use the same letters; tan and nat; bat is alone.

Example 2	
Input	['']
Output	[['']]
Why	An empty string is its own group.

Example 3	
Input	['a']
Output	[['a']]
Why	One word.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: anagrams have the SAME letters in sorted order, so sort the letters
# of each word and use that string as the key of a dictionary; words with the
# same key are anagrams. One pass, one dictionary (groups come out in the
# order their first word appears).
# Complexity: Time O(n * L log L) for n words of length L, Space O(n * L)
def group_by_sorting(words):
    groups = {}
    for word in words:
```

```
        key = "".join(sorted(word))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())
```

COMPLEXITY | Time O(n * L log L) | Space O(n * L)

242 Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same. MEDIUM

Template fit: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)

QUESTION

Two strings are buddy strings when they have the same length and the distance between each pair of neighbouring letters is the same (the alphabet wraps around after z), so one is the other shifted by a constant. Group the strings into buddy groups and return the groups in any order.

Function signature

def group_buddies(words: list) -> list

EXAMPLES

Example 1	
Input	`['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']`
Output	`[['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a', 'z']]`
Why	`'abc', 'bcd' and 'xyz' all step +1, +1; 'az' and 'ba' both step +25 (a to z wraps); single letters 'a' and 'z' form a group.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to a (mod 26).
    # Words that are shifts of each other share this key.
    gaps = []
    for i in range(len(word) - 1):
        gaps.append((ord(word[i + 1]) - ord(word[i])) % 26)
    return tuple(gaps)


def canonical(groups):
    # the groups sorted, so two answers can be compared
    result = []
    for group in groups:
        result.append(sorted(group))
    result.sort()
    return result


# ---- main question: main function ----
# Group the strings that are "buddies": the same length, and every letter
# is shifted by the same amount (a -> b, b -> c, ...; z wraps to a).
# Idea: two strings are buddies exactly when the GAPS between neighbouring
# letters (mod 26) are the same, so use the gaps (and the length) as the key of a
# dictionary and put the words with the same key together.
# Complexity: Time O(total length of all words), Space O(total length)
def group_buddies(words, key=shift_key):
    groups = {}
    for word in words:
        word_key = (len(word), key(word))          # words of different length never share a group
```

```
        if word_key not in groups:
            groups[word_key] = []
        groups[word_key].append(word)
    return list(groups.values())
```

COMPLEXITY | Time O(total characters) | Space O(total characters)

243 Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? HARD

Template fit: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values

QUESTION

Two strings are 'close' if you can turn one into the other with these operations, used any number of times: (1) swap the positions of any two letters; (2) take two letters that occur in the string and swap all of their occurrences (every a becomes b and every b becomes a). Return whether the first string can be transformed into the second.

Function signature

def close_strings(s: str, t: str) -> bool

EXAMPLES

Example 1	
Input	'abc', 'bca'
Output	true
Why	Swapping positions turns 'abc' into 'bca'.

Example 2	
Input	'a', 'aa'
Output	false
Why	The lengths differ.

Example 3	
Input	'cabbba', 'abbccc'
Output	true
Why	Use operation 1 to reorder, then operation 2 to swap the letters a and b, and c and ... until the counts match.

Example 4	
Input	'uau', 'ssx'
Output	false
Why	'u' appears in the first string but not the second, so they cannot be close.

MAIN CODE | helper functions first, then the answer

```
ALPHABET = 26


# ---- main question: helper functions ----
def count_letters(text):
    counts = {}
    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1
    return counts
```

```
def can_group(have, want):
    # can the counts in `have` be grouped (each used once) so that the group
    # sums are exactly the counts in `want`? Letters can merge, never split.
    # Backtracking: put each count (biggest first) into a bin of `want`.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
    room = list(want)                                  # how much each bin can still take

    def place(i):
        if i == len(have):
            for r in room:
                if r != 0:
                    return False
            return True
        tried = set()
        for j in range(len(room)):
            if room[j] >= have[i] and room[j] not in tried:    # equal bins behave the same: try each size on
ce
                tried.add(room[j])
                room[j] -= have[i]
                if place(i + 1):
                    return True
                room[j] += have[i]
        return False

    return place(0)


# ---- main question: main function ----
# Two operations: swap the positions of two letters, and convert ALL
# occurrences of one letter into another letter. Can s become t?
# Idea: swaps make the positions irrelevant, so only the letter COUNTS
# matter. Converting a letter can only rename it or merge it into another one. So
# if the counts already match, yes; otherwise the counts of s must be
```

```
# groupable into the counts of t (can_group). If both strings already use all
# 26 letters there is no spare letter left to rename safely.
# Complexity: Time O(n + A log A) for strings of length n and alphabet size A, Space O(A)
def close_strings(s, t, alphabet=ALPHABET):
    if len(s) != len(t):
        return False
    count_s = count_letters(s)
    count_t = count_letters(t)
    if count_s == count_t:
        return True
    if not can_group(list(count_s.values()), list(count_t.values())):
        return False
    if len(count_s) == len(count_t) == alphabet:
        return False
    return True
```

COMPLEXITY | Reading 1 (LeetCode 1657): same letter set and same multiset of frequencies: Time O(n) | Space O(1) for a fixed alphabet
Reading 2 (LeetCode 1153): one-directional conversion, no swaps: Time O(n) | Space O(1)

GROUP 35 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 2 M 2 H

K-way merge: a heap of the next item from each source

244 Find k pairs with smallest sums — EASY

Template fit: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **`k`** pairs **`(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17
  The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

EXAMPLES

Example 1	
Input	`[1, 7, 11], [2, 4, 6], 3`
Output	`[(1, 2), (1, 4), (1, 6)]`
Why	`Pairs (one from each array) in order of their sum.`

Example 2	
Input	`[1, 1, 2], [1, 2, 3], 2`
Output	`[(1, 1), (1, 1)]`
Why	`Equal numbers give equal pairs.`

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: main function ----
# a and b are sorted. Find the k pairs (x from a, y from b) with the
# smallest sums x + y.
# Idea: think of a table with a[i] + b[j]; every row i is sorted. A
# min-heap holds the best UNUSED pair of every row. Start with the first
# pair of each row (only the first k rows can matter). Take the smallest
```

```
# out, record the pair, and offer the next pair of that row, (i, j + 1).
# After k pops you have the k smallest sums.
# Complexity: Time O(k log k), Space O(k)
def k_smallest_pairs(a, b, k):
    if not a or not b or k <= 0:
        return []
    heap = []
    for i in range(min(k, len(a))):
        heap.append((a[i] + b[0], i, 0))
    heapq.heapify(heap)
    pairs = []
    while heap and len(pairs) < k:
        total, i, j = heapq.heappop(heap)
        pairs.append((a[i], b[j]))
        if j + 1 < len(b):
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    return pairs
```

COMPLEXITY | Time O(k log min(k, m)) | Space O(min(k, m))

245 K Largest Elements Across Sorted Lists — MEDIUM

Template fit: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result

QUESTION

You are given a list of inner lists, where each inner list is sorted in ascending order. Return the k largest elements taken from the union of all the inner lists, and return them sorted in ascending order. Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Function signature

```
from typing import List
def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Follow-up (as reported): Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop k times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4`
Output	`[6, 7, 8, 9]`
Why	`The four largest values overall are 9, 8, 7, 6, returned ascending.`

Example 2	
Input	`lists = [[10, 20, 30]], k = 2`
Output	`[20, 30]`

Example 3	
Input	`lists = [[5], [1, 3]], k = 5`
Output	`[1, 3, 5]`
Why	`k exceeds the total count, so all elements are returned sorted.`

QUESTION

Example 4	
Input	`lists = [[1, 2], [3, 4]], k = 0`
Output	`[]`

CONSTRAINTS

- Let m be the number of inner lists and T the total number of elements across all of them.
- 0 <= m <= 10^4 , 0 <= T <= 10^6 .
- Each element is an integer in [-10^9, 10^9] .
- 0 <= k . If k == 0 return [] . If k >= T , return every element sorted ascending.
- Some inner lists may be empty.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main function ----
# Each list is sorted (smallest first). Return the k largest numbers of
# all lists together, in increasing order.
# Idea: the next biggest number is always at the END of one of the lists.
# A max-heap (push negatives) holds the last unused element of every list.
# Take the biggest out, then offer the element just before it in its list.
# Complexity: Time O(m + k log m) for m lists, Space O(m + k)
def k_largest_sorted(lists, k):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            last = len(lists[i]) - 1
            heap.append((-lists[i][last], i, last))
    heapq.heapify(heap)
    biggest_first = []
    while heap and len(biggest_first) < k:
        negative, i, j = heapq.heappop(heap)
        biggest_first.append(-negative)
        if j > 0:
            heapq.heappush(heap, (-lists[i][j - 1], i, j - 1))
    biggest_first.reverse()
    return biggest_first
```

COMPLEXITY | Time O(m + k log m) | Space O(m)

246 Sort a nearly sorted (or K sorted) array. MEDIUM

Template fit: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)

QUESTION

In a k-sorted array every element is at most k positions away from where it would be in the sorted array. Sort such an array in O(n log k) time (better than a full sort when k is small).

Function signature

def sort_k_sorted(arr: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[6, 5, 3, 2, 8, 10, 9], 3`
Output	`[2, 3, 5, 6, 8, 9, 10]`
Why	`Every number is within 3 positions of its place in the sorted order.`

QUESTION

Example 2	
Input	`[2, 1, 4, 3], 1`
Output	`[1, 2, 3, 4]`
Why	`k = 1: neighbours may be swapped.`

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: main function ----
# Every element is at most k places away from where it belongs when the
# array is sorted. Idea: the smallest of the next k + 1 elements must be
# the next one of the answer. Keep a min-heap of k elements; for every new
# element push it and take out the smallest (heappushpop does both). At the
# end empty the heap. O(n log k).
# Complexity: Time O(n log k), Space O(k)
def sort_k_sorted(arr, k):
    heap = arr[:k]
    heapq.heapify(heap)
    result = []
    for x in arr[k:]:
        result.append(heapq.heappushpop(heap, x))      # push x, then pop the smallest
    while heap:
        result.append(heapq.heappop(heap))
    return result
```

COMPLEXITY | Time O(n log K) | Space O(K)

247 Merge k sorted linked lists. HARD

Template fit: seed with each list head (value, list number, node); the successor of a node is node.next

QUESTION

You are given k linked lists, each sorted in ascending order. Merge them into one sorted linked list and return its head. (In the examples the linked lists are written as arrays.)

Function signature

def merge_k_heap(lists: list) -> ListNode

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 5], [1, 3, 4], [2, 6]]`
Output	`_to_list(merge_k_heap([mk([1, 4, 5]), mk([1, 3, 4]), mk([2, 6])])) -> [1, 1, 2, 3, 4, 4, 5, 6]`
Why	`The three sorted lists merge into one sorted list.`

Example 2	
Input	`lists = []`
Output	`_to_list(merge_k_heap([])) -> []`
Why	`No lists: an empty result.`

MAIN CODE | helper functions first, then the answer

```
import heapq
```

```python
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make(values):
    # a linked list from a Python list
    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


def merge_two(a, b):
    # merge two sorted linked lists into one
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    if a:
        tail.next = a
    else:
        tail.next = b
    return dummy.next
```

```python
# ---- main question: main function ----
# Idea: a min-heap holds the CURRENT head of every list as (value, list
# number, node). Take the smallest out, add it to the result and put
# its successor in the heap. The list number breaks ties so two nodes are
# never compared. O(N log k) for N nodes in k lists.
# Complexity: Time O(N log k) for N nodes in total, Space O(k)
def merge_k_heap(lists):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            heap.append((lists[i].val, i, lists[i]))
    heapq.heapify(heap)
    dummy = ListNode()
    tail = dummy
    while heap:
        value, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
```

```
    return dummy.next
```

COMPLEXITY | Time O(N log k) | Space O(k)

248 Return top-k subset sums in descending order — HARD

Template fit: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next

QUESTION

Given a list of integers (possibly negative), return the k largest subset sums in descending order. The empty subset counts and has sum 0; different subsets with the same sum are listed separately.

Function signature

def top_k_sums(nums: list, k: int) -> list

EXAMPLES

Example 1	
Input	[1, 2, 3], 4
Output	[6, 5, 4, 3]
Why	The eight subset sums are 6, 5, 4, 3, 3, 2, 1, 0; the top four are 6, 5, 4, 3.

Example 2	
Input	[3, -2, 1], 3
Output	[4, 3, 2]
Why	With a negative number: 3 + 1 = 4, then 3, then 3 + (-2) + 1 = 2.

MAIN CODE | helper functions first, then the answer

```
import heapq


# ---- main question: helper functions ----
def sorted_costs(nums):
    # |x| of every number, smallest first
    costs = []
    for x in nums:
        if x < 0:
            costs.append(-x)
        else:
            costs.append(x)
    costs.sort()
    return costs


# ---- main question: main function ----
# Return the k biggest sums of subsets of nums (the empty subset counts).
# Idea: the best subset takes every positive number. Any other subset
# is the best one with some numbers "flipped" (a positive one left out or a
# negative one added), and flipping x costs |x|. So the k best subsets are
# the k CHEAPEST sets of flips. Generate the flip sets in order of cost with a
# heap over the sorted costs: from a set whose last flipped item is i, the
# two successors are (a) add item i + 1, (b) replace item i by item i + 1.
# Neither is ever cheaper than the set itself, so the heap yields them in order.
# Complexity: Time O(n log n + k log k), Space O(n + k)
def top_k_sums(nums, k):
    base = 0
    for x in nums:
        if x > 0:
            base += x
```

```
    costs = sorted_costs(nums)
    flip_costs = [0]                                    # the best subset costs 0 flips
    heap = []
    if costs:
        heap.append((costs[0], 0))                      # (total cost, index of the last flipped item)
    while heap and len(flip_costs) < k:
        cost, i = heapq.heappop(heap)
        flip_costs.append(cost)
        if i + 1 < len(costs):
            heapq.heappush(heap, (cost + costs[i + 1], i + 1))                  # add the next item
            heapq.heappush(heap, (cost - costs[i] + costs[i + 1], i + 1))       # move the last item up by o
ne
    sums = []
    for cost in flip_costs[:k]:
        sums.append(base - cost)
```

```
    return sums
```

COMPLEXITY | Time O(n log n + k log k) | Space O(n + k)

GROUP 36 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 4 M

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

249 Next Birthday (BST In-Order Successor with Wraparound) EASY

Template fit: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node

QUESTION

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur. Each birthday is stored as an integer day-of-year in [1, 365] , and the tree is a valid BST keyed on that value. Given the root of the BST and an integer today , return the next upcoming birthday: the smallest key that is strictly greater than today . If no key is strictly greater than today , the calendar wraps around, so return the smallest key in the whole tree. If the tree is empty, return None .

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def next_birthday(root: Optional[TreeNode], today: int) -> Optional[int]:
    """Return the smallest key strictly greater than today, wrapping around
    to the smallest key if none is greater. Return None if the tree is empty."""
    ...
```

Follow-up (as reported): If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: bisect_right .

EXAMPLES

Example 1	
Input	root = above, today = 25
Output	30
Why	Smallest key strictly greater than 25 is 30.

Example 2	
Input	root = above, today = 40
Output	10
Why	No key > 40, so wrap around to the smallest key, 10.

Example 3	
Input	root = above, today = 5
Output	10
Why	Smallest key greater than 5 is 10.

Example 4	
Input	root = None, today = 100
Output	None

CONSTRAINTS

QUESTION

- 0 <= number of nodes <= 10^5 ; all node values are distinct and in [1, 365] .
- 1 <= today <= 365 .
- Aim for O(h) time where h is the tree height (do not sort all nodes if you can avoid it).

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: the birthdays are in a binary SEARCH tree. Walk down once: a
# value above today is a candidate (the answer might be even closer in
# its left subtree), a value at or below today sends us right. If no
# value is above today, wrap around to the smallest value (the leftmost node).
# Complexity: Time O(h) for the height h of the tree, Space O(1)
def next_birthday(root, today):
    if root is None:
        return None
    best = None
    node = root
    while node:
        if node.val > today:
            best = node.val
            node = node.left
        else:
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val
```

COMPLEXITY | Time O(h) | Space O(1)

250 Lowest Common Ancestor

MEDIUM

Template fit: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor

QUESTION

Given the root of a binary tree and two of its nodes p and q, find their lowest common ancestor: the lowest node that has both p and q as descendants (a node can be a descendant of itself).

Function signature

def lowestCommonAncestor_tree(root: BNode, p: BNode, q: BNode) -> BNode

EXAMPLES

Example 1	
Input	root = [3, [5, [6], [2]], [1]] (value, left, right) first query: p = 5, q = 1 second query: p = 5, q = 6
Output	lowestCommonAncestor_tree(root, p, q).val -> 3 lowestCommonAncestor_tree(root, p, p.left).val -> 5
Why	5 and 1 are in different subtrees of 3, so 3 is their lowest common ancestor. 5 is an ancestor of 6, and a node may be its own ancestor, so the answer is 5.

QUESTION

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Node:
    def __init__(self, val, children=None):
        # two children for a binary tree, any number for an n-ary tree
        self.val = val
        self.children = []
        if children:
            self.children = children


# ---- main question: main function ----
# Idea: one recursive pass. find(node) answers "where did I meet p or q
# in this subtree?": the node itself if it is p or q, otherwise what the
# children report. If TWO children each found something, this node is
# the lowest common ancestor; if only one did, pass that result up.
# The same code works for an n-ary tree (follow-up 1): it just loops
# over every child.
# Complexity: Time O(n), Space O(h) for the height h of the tree
def lca(root, p, q):
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        hits = []
        for child in node.children:
            found = find(child)
            if found is not None:
                hits.append(found)
        if len(hits) >= 2:
            return node
        if len(hits) == 1:
            return hits[0]
        return None

    return find(root)
```

COMPLEXITY | Time O(n) | Space O(h)

251 Determine if a given binary tree is a binary search tree (BST). MEDIUM

Template fit: check(): pass lo and hi bounds down; fail when a value leaves its allowed window

QUESTION

Given the root of a binary tree, determine whether it is a valid binary search tree: for every node, all values in its left subtree are smaller and all values in its right subtree are larger (strictly).

Function signature

def isValidBST(root: TreeNode) -> bool

EXAMPLES

QUESTION

Example 1	
Input	`[2, 1, 3]`
Output	`true`
Why	`2 with 1 on the left and 3 on the right.`

Example 2	
Input	`[5, 1, 4, null, null, 3, 6]`
Output	`false`
Why	`The 3 sits in the right subtree of 5 but is smaller than 5.`

Example 3	
Input	`[2, 2, 2]`
Output	`false`
Why	`A node equal to its parent breaks the strict rule.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def value_fits(val, low, high, dup):
    # may `val` sit between the bounds from its ancestors (None = no bound)?
    # dup says where an EQUAL value is allowed: "none", "left" or "right"
    if low is not None:
        if val < low:
            return False
        if val == low and dup != "right":
            return False
    if high is not None:
        if val > high:
            return False
        if val == high and dup != "left":
            return False
    return True


# ---- main question: main function ----
# Idea: pass the allowed range (low, high) DOWN. The left child inherits
# (low, node.val) and the right child (node.val, high); the root starts
# with no bounds. Every node must lie strictly between its bounds.
# (dup is for follow-up 1; leave it as "none" for a normal BST.)
# Complexity: Time O(n), Space O(h)
def isValidBST(root, dup="none"):
    stack = [(root, None, None)]
    while stack:
        node, low, high = stack.pop()
        if node is None:
            continue
        if not value_fits(node.val, low, high, dup):
            return False
        stack.append((node.left, low, node.val))
        stack.append((node.right, node.val, high))
    return True
```

COMPLEXITY | Time O(n) | Space O(h)

252 Find mode in a trinary search tree

MEDIUM

Template fit: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all v
alues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

QUESTION

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.

Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.

Constraints

0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer

Examples

Input: ([], [], [], [], -1)

Expected Output: []

Explanation: The tree is empty, so there are no modes.

Input: ([-2147483648], [-1], [-1], [-1], 0)

Expected Output: [-2147483648]

Explanation: A single-node tree has that node's value as the only mode.

Hints

Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TNode:
    # a trinary search tree node: everything in `left` is smaller than
    # val, `middle` holds a copy of val, everything in `right` is bigger
    def __init__(self, val, left=None, middle=None, right=None):
        self.val = val
        self.left = left
        self.middle = middle
        self.right = right


class ModeTracker:
    # values arrive in sorted order, so equal values come one after the
    # other and a running count is enough (no dictionary)
    def __init__(self):
        self.best = 0              # the longest run so far
        self.modes = []            # the values that have that run length
        self.current = None
        self.run = 0

    def add(self, value):
        if value == self.current:
            self.run += 1
        else:
            self.run = 1
        self.current = value
        if self.run > self.best:
            self.best = self.run
            self.modes = [value]
        elif self.run == self.best and value not in self.modes:
            self.modes.append(value)
```

```
# ---- main question: main function ----
# Idea: an IN-ORDER walk (left, middle chain, right) gives the values in
# sorted order. Use an explicit stack: O(height) extra space.
# Complexity: Time O(n), Space O(h)
def find_modes(root):
    tracker = ModeTracker()
    stack = []
    node = root
    while stack or node:
        while node:                        # go as far left as possible
            stack.append(node)
            node = node.left
```

```
        node = stack.pop()
        tracker.add(node.val)
        middle = node.middle               # the middle chain repeats node.val once per node
        while middle:
            tracker.add(middle.val)
            middle = middle.middle
        node = node.right
    return sorted(tracker.modes)
```

COMPLEXITY | Time O(n) | Space O(h)

253 Next Upcoming Birthday in a Family Tree — MEDIUM

Template fit: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today

QUESTION

You are given a family tree rooted at "me". Each Person node has a name , a birthday given as (month, day) , and pointers to parent1 and parent2 (either may be None). Following parent pointers reaches the whole ancestry; treat the reachable set of Person nodes as the tree to search (each distinct person counted once, even if reachable by two paths). Given today's date (today_month, today_day) , implement alarm_next_birthday(me, today_month, today_day) that scans every person and returns the (month, day) of the soonest upcoming birthday, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (me is None), return the empty string "" . Use this ordering key to compare dates: key(m, d) = (m - 1) * 31 + d , and cyclic distance (key(person) - key(today)) mod 372 . Return the person with the smallest such distance; if several people share that soonest date, return that (month, day) .

Function signature

```
from typing import Optional, Union
class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2
def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

QUESTION

Follow-up (as reported): Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return "" .

EXAMPLES

Example 1	
Input	me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)
Output	(9, 25)
Why	keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10- 260) mod 372 = 122). Alice is soonest.

Example 2	
Input	me = None; today = (5, 5)
Output	""
Why	No people to check.

Example 3	
Input	me = Me(3,3) alone; today = (3, 3)
Output	(3, 3)
Why	The only birthday is today, distance 0.

CONSTRAINTS

- 1 <= month <= 12 , 1 <= day <= 31 .
- Up to 10^5 people; parent pointers form a DAG (no cycles).

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Person:
    def __init__(self, name, month, day, parent1=None, parent2=None):
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2


def day_number(month, day):
    # a number that grows through the year; 12 months of up to 31 days = 372
    return (month - 1) * 31 + day


def all_people(me):
    # `me` and every ancestor, each person once (a person can be reached
    # through both parents, so remember who was seen)
    people = []
    seen = set()
    stack = []
    if me:
        stack.append(me)
    while stack:
        person = stack.pop()
        if person in seen:
            continue
        seen.add(person)
        people.append(person)
        for parent in (person.parent1, person.parent2):
            if parent:
                stack.append(parent)
```

```
    return people


# ---- main question: main function ----
# Idea: look at every person once and keep the birthday that comes next:
# the number of days from today to the birthday, going round the year
# (the "% 372" wraps a birthday that already passed to next year).
# Complexity: Time O(n) for n people, Space O(n)
def alarm_next_birthday(me, today_month, today_day):
    if me is None:
        return ""
    today = day_number(today_month, today_day)
    best_distance = None
```

```
    answer = ""
    for person in all_people(me):
        distance = (day_number(person.month, person.day) - today) % 372
        if best_distance is None or distance < best_distance:
            best_distance = distance
            answer = (person.month, person.day)
    return answer
```

COMPLEXITY | Time O(p) | Space O(p)

GROUP 37 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Random pick without repeats: swap the chosen item with the last one and pop

254 Random Generator Without Repeats

MEDIUM

Template fit: the template as written: values 0..n-1, every call returns a new one; raise when empty

QUESTION

Design a class RandomGenerator initialized with a positive integer n . Each call to generate() returns one integer chosen uniformly at random from the values in [0, n-1] that have not yet been returned. After n calls every value in [0, n-1] has been returned exactly once; a further call must raise IndexError (the pool is empty). Each generate() call must run in O(1) time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Function signature

```
import random
class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...
    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Follow-up (as reported): Add a reset() that makes all n values available again in O(1) amortized.

EXAMPLES

Example 1	
Input	`g = RandomGenerator(3); [g.generate() for _ in range(3)]`
Output	`e.g. [2, 0, 1]`
Why	`The three calls return a permutation of {0, 1, 2} with no repeats; any of the 6 permutations is a valid output. A 4th call would raise IndexError.`

Example 2	
Input	`g = RandomGenerator(1); g.generate()`
Output	`0`
Why	`Only one value exists.`

CONSTRAINTS

- 1 <= n <= 10**6 .

MAIN CODE | helper functions first, then the answer

```python
import random


# ---- main question: main function ----
# Idea (partial Fisher-Yates): keep the numbers 0..n-1 in an array and
# an "active region" at its start that holds the numbers not returned
# yet. To generate: pick a random position in the active region, swap
# that number to the END of the region, shrink the region by one and
```

```python
# return the number that is now just outside it. O(1) per call.
# Complexity: Time generate O(1), Space O(n)
class RandomGenerator:
    def __init__(self, n, seed=None):
        self.n = n
        self.values = list(range(n))
        self.size = n                              # the active region is values[0:size]
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError("all values have been generated")
        i = self.rng.randrange(self.size)
        self.size -= 1
        self.values[i], self.values[self.size] = self.values[self.size], self.values[i]
        return self.values[self.size]
```

COMPLEXITY | Time O(1) per call | Space O(n)

255 Random Song Shuffler With a No-Repeat Window

MEDIUM

Template fit: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque

QUESTION

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the last k songs played may not be replayed. In other words, a song becomes eligible again only after k other (distinct) plays have occurred since it was last played. Implement the class so that each call to play() runs in O(1) time (no scanning of the playlist). The intended technique is a valid_songs list of currently eligible songs plus a song_to_index dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Function signature

```
import random
class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...
    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Follow-up (as reported): The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per play() .

EXAMPLES

Example 1

Input	`SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()`
Output	`e.g. "B", "A", "C", "B"`
Why	`k=1 forbids repeating the single most recent song. "B" then "A" is fine (A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is valid.`

QUESTION

Example 2	
Input	`SongShuffler(["X", "Y"], k=0); play(), play()`
Output	`e.g. "Y", "Y"`
Why	`With k=0 there is no restriction, so repeats are allowed.`

CONSTRAINTS

- 1 <= len(songs) <= 10**5 , song titles are distinct strings.
- 0 <= k < len(songs) (so at least one song is always eligible). play() returns any eligible song chosen uniformly at random; every returned song must not be among the previous k returned songs.

MAIN CODE | helper functions first, then the answer

```
from collections import deque
import random


# ---- main question: main function ----
# Idea: keep a list `valid` of the songs that may be played now, and a
# dictionary `index` with each song's place in that list. To play: pick a
# random index in `valid`. To REMOVE the played song in O(1), swap it
# with the LAST song of the list and pop (the swap-and-pop trick). The
# played song waits in a queue `recent` of the last k songs; when the
# queue is longer than k, the oldest song leaves it and is added back
# to `valid`. Every call is O(1).
# Complexity: Time play O(1), Space O(number of songs)
class SongShuffler:
    def __init__(self, songs, k, seed=None):
        self.valid = list(songs)                    # songs that may be played now
        self.index = {}                             # song -> its place in valid
        for i in range(len(self.valid)):
            self.index[self.valid[i]] = i
        self.recent = deque()                       # the last k songs played, oldest first
        self.k = k
        self.rng = random.Random(seed)

    def play(self):
        i = self.rng.randrange(len(self.valid))
        song = self.valid[i]
        last = self.valid[-1]
        self.valid[i] = last                         # swap-and-pop: the last song takes the hole
        self.index[last] = i
        self.valid.pop()
        del self.index[song]
        self.recent.append(song)
        if len(self.recent) > self.k:
            back = self.recent.popleft()             # this song may be played again
            self.index[back] = len(self.valid)
            self.valid.append(back)
        return song
```

COMPLEXITY | Time O(1) per play | Space O(n)

256 Design set with O(1) random access

MEDIUM

Template fit: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.
- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True, None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

MAIN CODE | helper functions first, then the answer

```python
import random


# ---- main question: main function ----
# Idea: keep the values in a LIST (so a random one can be picked by a
# random index) and a dictionary from value to its index in the list (so
# add and delete can find a value at once).
#   add: append to the list and record the index.
#   delete: look up the index, move the LAST value into that hole (and
#   update its index), remove the last list slot and the dictionary entry.
# Every operation is O(1) on average.
# Complexity: Time add, delete and getRandom are O(1) on average, Space O(n)
class FancySet:
    def __init__(self, seed=None):
        self.items = []
        self.position = {}                          # value -> its index in items
        self.rng = random.Random(seed)

    def add(self, x):
        if x in self.position:
            return False
        self.position[x] = len(self.items)
        self.items.append(x)
        return True

    def delete(self, x):
        if x not in self.position:
            return False
        i = self.position[x]
        last = self.items[-1]
        self.items[i] = last                         # the last value fills the hole
        self.position[last] = i
        self.items.pop()
        del self.position[x]                         # (when x was the last one this removes it again)
        return True

    # follow-up 2: on an empty set there is nothing to return, so raise an
    # IndexError (like random.choice does) instead of returning a value
    # that could be mistaken for an element
    def getRandom(self):
        if len(self.items) == 0:
            raise IndexError("empty set")
        return self.items[self.rng.randrange(len(self.items))]
```

COMPLEXITY | Time O(1) average per operation | Space O(n)

257 Unique Random Number Generator — MEDIUM

Template fit: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)

QUESTION

Design a class UniqueRandom initialized with an integer n . Each call to generate() returns a number drawn uniformly at random from [0, n-1] that has not been returned before, so across successive calls no value ever repeats. After all n distinct values have been produced, further calls return None . Every unused value must remain equally likely on each draw (no bias), and generate() must run in O(1) time per call.

QUESTION

Function signature

import random
class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...
    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...

Follow-up (as reported): Use a partial Fisher-Yates shuffle: keep a virtual array 0..n-1 , and on the i -th call swap a random unused slot into position i using a sparse dict of moved entries, giving O(1) time and O(number-of- draws) space.

EXAMPLES

Example 1	
Input	`g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]`
Output	`a permutation of [0, 1, 2] (e.g. [2, 0, 1])`
Why	`The three calls return all of 0,1,2 in some random order with no repeats.`

Example 2	
Input	`(continuing above) g.generate()`
Output	`None`
Why	`All 3 values already returned, so the pool is exhausted.`

Example 3	
Input	`g = UniqueRandom(1); [g.generate(), g.generate()]`
Output	`[0, None]`
Why	`The only value 0 is returned once, then the generator is exhausted.`

CONSTRAINTS

- 1 <= n <= 10^7 .
- Support an optional seed for reproducible tests.

MAIN CODE | helper functions first, then the answer

```python
import random


# ---- main question: main function ----
# Idea (a lazy Fisher-Yates shuffle): imagine the numbers 0..n-1 in an
# array and shuffle it as you go. Draw number `drawn` by picking a random
# slot j from drawn..n-1; the value in slot j is the answer. Then the
# value that sat in slot `drawn` takes slot j's place. Only the slots
# that were MOVED are stored (in a dictionary); an untouched slot
# j simply holds the value j. O(1) per call, and O(draws) memory.
# Complexity: Time generate O(1), Space O(number of draws so far)
class UniqueRandom:
    def __init__(self, n, seed=None):
        self.n = n
        self.drawn = 0
        self.moved = {}                              # slot -> the value now sitting there
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.n:
```

```
            return None                                # every number has been returned
        i = self.drawn
        j = self.rng.randrange(i, self.n)
        value = j
        if j in self.moved:
            value = self.moved[j]
        at_i = i
        if i in self.moved:
            at_i = self.moved[i]
        self.moved[j] = at_i                           # the value from slot i takes the place of the drawn one
        self.drawn += 1
        return value
```

COMPLEXITY | Time O(1) per call | Space O(draws)

258 Shuffle array using random integer API

HARD

Template fit: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

QUESTION

You have an array nums and no built-in shuffle, only rand_int(l, r), which returns a uniformly random integer in [l, r] (both ends included). Shuffle nums in place so that every one of the n! orders is equally likely.

Function signature

def shuffle(nums: list, rand_int) -> list

EXAMPLES

Example 1

Input	`rand_int = lambda l, r: l # a fake 'random' source that always returns the lowest allowed index`
	`nums = [1, 2, 3, 4]`
	`shuffle(nums, rand_int)`

Output	`shuffle(nums, rand_int) -> [2, 3, 4, 1]`
Why	`Fisher-Yates walks i from the end and swaps nums[i] with nums[rand_int(0, i)]. With the fake source the swaps are (3,0), (2,0), (1,0), giving a predictable result; with a real source each of the 24 orders has probability 1/24.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea (Fisher-Yates): go from the LAST position down to position 1. For
# each position i pick j uniformly from 0..i (both included) and swap
# nums[i] with nums[j]. Every one of the n! orders is equally likely,
# because each step picks uniformly among i + 1 choices.
# rand_int(low, high) gives a random whole number from low to high.
# Complexity: Time O(n), Space O(1)
def shuffle(nums, rand_int):
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

COMPLEXITY | Time O(n) | Space O(1) extra

GROUP 38 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Build and serialize: write a structure out, read it back with the same walk

259 Construct a binary tree given its preorder and inorder traversals. MEDIUM

Template fit: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7] Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]
Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

EXAMPLES

Example 1

Input	`preorder = [3, 9, 20, 15, 7]` `inorder = [9, 3, 15, 20, 7]`
Output	`values(build_pre_in([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])) -> [3, 9, 20, null, null, 15, 7]`
Why	`The first preorder value 3 is the root; in the inorder list 9 is left of it and 15, 20, 7 are right. The tree is shown level by level.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def preorder_of(root):
    # the values in pre-order (root, left, right), with a stack
    values = []
    stack = []
    if root:
        stack.append(root)
    while stack:
        node = stack.pop()
        values.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return values


def inorder_of(root):
    # the values in in-order (left, root, right), with a stack
    values = []
    stack = []
    node = root
```

```
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        values.append(node.val)
        node = node.right
    return values


def positions(values):
    # value -> its index (the values are all different)
    where = {}
    for i in range(len(values)):
        where[values[i]] = i
    return where
```

```
# ---- main question: main function ----
# Idea: preorder is "root, left part, right part", so its next unused
# value is always the next ROOT. That root's position in the inorder
# list splits the inorder list into the left part (before it) and the
# right part (after it). Build the left subtree first, then the right.
# Complexity: Time O(n) (a dictionary gives each value's inorder position), Space O(n)
def build_pre_in(preorder, inorder):
    where = positions(inorder)
    next_root = 0

    def build(low, high):
        # build the tree for inorder[low:high]
        nonlocal next_root
        if low >= high:
            return None
        root = TreeNode(preorder[next_root])
        next_root += 1
        middle = where[root.val]
        root.left = build(low, middle)
        root.right = build(middle + 1, high)
        return root

    return build(0, len(inorder))
```

COMPLEXITY | Time O(n) | Space O(n)

260 Write functions to serialize and deserialize a list of strings. MEDIUM

Template fit: write len + '#' + text for each string; read the length, then take exactly that many characters

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

QUESTION

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

EXAMPLES

Example 1	

Input	`items = ['hello', 'wor#ld', '']`
	`serialize(items)`
	`deserialize(serialize(items)) == items`

Output	`serialize(items) -> '5#hello6#wor#ld0#'` `deserialize(serialize(items)) == items -> true`
Why	`Each string is written as its length, a #, and its text, so a # inside a string cannot confuse the reader.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main functions ----
# Idea: write the LENGTH of every string before it: "<length>#<text>".
# The reader knows exactly how many characters to take, so the text may
# contain any character, even "#" and digits.
# Complexity: Time O(total characters), Space O(total characters)
def serialize(items):
    text = ""
    for s in items:
        text += str(len(s)) + "#" + s
    return text


# Complexity: Time O(total characters), Space O(total characters)
def deserialize(text):
    items = []
    i = 0
    while i < len(text):
        sep = text.index("#", i)               # the digits of the length end at the next "#"
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items
```

COMPLEXITY | Time O(total length) | Space O(total length)

261 Print a Directory Tree — MEDIUM

Template fit: build nested dicts from the path parts, then print them with a recursive walk that indents by depth

QUESTION

You are given a list of file/directory paths , each a '/' -separated string (for example "dir/subdir/file.txt"). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string. Rules: Each path component is a node. A component that appears under the same parent in several paths must be printed only once. Indent each node with 2 * depth spaces, where the top-level components have depth 0 . Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort). The output must not have a trailing newline.

Function signature

def print_directory_tree(paths: list[str]) -> str:

    ...

Follow-up (as reported): Add box-drawing connectors (, ,) like the real tree command. Hint: track, per level, whether the current node is the last child of its parent.

EXAMPLES

Example 1	
Input	`paths = ["a/b/c", "a/b/d", "a/e"]`
Output	`"a\n b\n c\n d\n e"`
Why	`Rendered: a b c d e`

Example 2	
Input	`paths = ["src/main.py", "src/util.py", "README.md"]`
Output	`"src\n main.py\n util.py\nREADME.md"`
Why	`"src" and its two files come first (first-seen order), then the top-level "README.md".`

Example 3	
Input	`paths = []`
Output	`""`

CONSTRAINTS

- 0 <= len(paths) <= 10^4
- Each path is non-empty, has no leading/trailing '/' , and no empty components.
- If paths is empty, return the empty string "" .

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def build_dirs(paths):
    # nested dictionaries: every folder or file name maps to its own contents
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            if part not in node:
                node[part] = {}
            node = node[part]
    return root


# ---- main question: main function ----
# Idea: put every path into a dictionary of dictionaries (a tree), then
# print it depth first with two spaces per level.
# Complexity: Time O(n) for n entries, Space O(depth) for the walk plus the output
def print_directory_tree(paths):
    root = build_dirs(paths)
    lines = []
```

```
    def walk(node, depth):
        for name in node:
            lines.append("  " * depth + name)
            walk(node[name], depth + 1)

    walk(root, 0)
    return "\n".join(lines)
```

COMPLEXITY | Time O(total length) | Space O(total length)

262 Implement a decode and encode function of a general N-ary tree. HARD

Template fit: write value and child count for each node (preorder); read the count to know how many children to build

QUESTION

A general tree has a value in every node and any number of children. Write encode(root), which turns the tree into a string, and decode(text), which rebuilds the identical tree from that string.

Function signature

def encode(root: Node) -> str
def decode(text: str) -> Node

EXAMPLES

Example 1

Input	`root = Node(1, [Node(3, [Node(5), Node(6)]), Node(2), Node(4)])`
	`text = encode(root)`
	`text`
	`encode(decode(text)) == text`

Output	`text -> '[1, 3, 3, 2, 5, 0, 6, 0, 2, 0, 4, 0]'` `encode(decode(text)) == text -> true`
Why	Each node is written as its value followed by its number of children, in pre-order: 1 has 3 children, 3 has 2, and so on. Decoding reads exactly that many children for each node, so encoding the decoded tree gives the same text.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Node:
    def __init__(self, val, children=None):
        self.val = val
        self.children = []
        if children:
            self.children = children


def preorder_pairs(root):
    # (value, number of children) for every node in pre-order. A stack
    # instead of recursion, so a very deep tree cannot overflow the call stack.
    pairs = []
    stack = []
    if root:
        stack.append(root)
    while stack:
        node = stack.pop()
        pairs.append((node.val, len(node.children)))
        for i in range(len(node.children) - 1, -1, -1):     # push the LAST child first so the first is poppe
d first
            stack.append(node.children[i])
    return pairs
```

```
def build_preorder(pairs):
    # the reverse: rebuild the tree from the same (value, child count) pairs
    if not pairs:
        return None
    root = Node(pairs[0][0])
    stack = [[root, pairs[0][1]]]              # [a node, how many children it still has to receive]
    position = 1
    while stack:
        top = stack[-1]
        if top[1] == 0:
            stack.pop()                        # this node is complete
            continue
        top[1] -= 1
        value, count = pairs[position]
        position += 1
        child = Node(value)
        top[0].children.append(child)
        stack.append([child, count])
    return root
```

```
def same(a, b):
    # do two trees have the same shape and values?
    return preorder_pairs(a) == preorder_pairs(b)


# ---- main question: main functions ----
# Idea: write every node as "value child_count" in PRE-ORDER (a node
# first, then its children in order). The counts say where each node's
# children end, so no brackets are needed. Values here are words without
# spaces (follow-up 1 handles any string).
# Complexity: Time O(n), Space O(n)
def encode(root):
    parts = []
    for value, count in preorder_pairs(root):
        parts.append(str(value))
        parts.append(str(count))
    return " ".join(parts)


# Complexity: Time O(n), Space O(n)
def decode(text):
    if text == "":
        return None
    tokens = text.split(" ")
    pairs = []
    for i in range(0, len(tokens), 2):
        pairs.append((tokens[i], int(tokens[i + 1])))
    return build_preorder(pairs)
```

COMPLEXITY | Time O(n) | Space O(n)

263 Serialize and Deserialize a Graph — HARD

Template fit: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking

QUESTION

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer val and a list of neighbors . The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original. You are given the reference node class below. Implement serialize (node -> string) and deserialize (string -> node). serialize(None) must round-trip to None via deserialize .

QUESTION

Function signature

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from node into a string."""
        ...
    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Follow-up (as reported): Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

EXAMPLES

Example 1	
Input	`adjList = [[]] # a single node with no neighbors`
Output	`A single node with val 1 and an empty neighbor list.`

Example 2	
Input	`node = None # empty graph`
Output	`codec.deserialize(codec.serialize(None)) is None`

CONSTRAINTS

- 0 <= number of nodes <= 100
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: helper functions ----
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = []
        if neighbors is not None:
            self.neighbors = neighbors


def build_graph(adj):
    # adj[i] lists the neighbours of node i + 1, e.g. [[2, 4], [1, 3], [2, 4], [1, 3]]
    nodes = []
    for i in range(len(adj)):
        nodes.append(Node(i + 1))
    for i in range(len(adj)):
        for j in adj[i]:
            nodes[i].neighbors.append(nodes[j - 1])
    if nodes:
        return nodes[0]
    return None
```

```python
# ---- main question: main function ----
# Idea: one entry "value:neighbour,neighbour" per node, found with a
# BFS from the given node, separated by ";". To read it back, create
# every node FIRST and link the neighbours afterwards, so a neighbour
# always exists before it is linked (a graph can have cycles).
# Complexity: Time encode and decode are O(V + E), Space O(V + E)
class Codec:
    def serialize(self, node):
        if node is None:
            return ""
        seen = {node.val}
        queue = deque([node])
        parts = []
        while queue:
            current = queue.popleft()
            names = []
            for neighbour in current.neighbors:
                names.append(str(neighbour.val))
                if neighbour.val not in seen:
                    seen.add(neighbour.val)
```

```python
                    queue.append(neighbour)
            parts.append(str(current.val) + ":" + ",".join(names))
        return ";".join(parts)

    def deserialize(self, data):
        if data == "":
            return None
        rows = []
        for part in data.split(";"):
            rows.append(part.split(":"))
        nodes = {}
        for value, neighbour_text in rows:            # pass 1: create every node
            nodes[int(value)] = Node(int(value))
        for value, neighbour_text in rows:            # pass 2: link the neighbours
            if neighbour_text != "":
                for name in neighbour_text.split(","):
                    nodes[int(value)].neighbors.append(nodes[int(name)])
        return nodes[int(rows[0][0])]
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

GROUP 39 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 M 4 H

Games: win or lose with a cache, and guessing with feedback

264 Choose optimal guesses for green-only Wordle — MEDIUM

Template fit: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups

QUESTION

You are guessing a hidden word from a candidate list. After each guess you only learn which positions are green, that is, where your letter equals the answer's letter. best_guess(words) picks the candidate whose answer pattern splits the remaining words most evenly: the one with the smallest expected size of the group left (sum of squared group sizes divided by the number of words); ties go to the alphabetically smaller word.

Function signature

def green(guess: str, answer: str) -> tuple
def best_guess(words: list) -> str

EXAMPLES

Example 1	
Input	`'cat', 'car'`
Output	`(true, true, false)`
Why	`c and a match, the last letters differ.`

Example 2	
Input	`['cat', 'car', 'cap', 'dog']`
Output	`'cap'`
Why	`'cap', 'car' and 'cat' split the other words equally well and 'cap' wins the alphabetical tie-break; 'dog' shares no letter positions with the others, so it tells less.`

Example 3	
Input	`['tea']`
Output	`'tea'`
Why	`With a single candidate the answer is that candidate.`

MAIN CODE | helper functions first, then the answer

```
from fractions import Fraction


# ---- main question: helper functions ----
def green(guess, answer):
    # green-only feedback: for every position, does the letter match exactly?
    feedback = []
    for i in range(len(guess)):
        feedback.append(guess[i] == answer[i])
    return tuple(feedback)


def partition(guess, words):
    # group the possible answers by the feedback they would give for this guess
    groups = {}
    for word in words:
        pattern = green(guess, word)
        if pattern not in groups:
```

```
            groups[pattern] = []
        groups[pattern].append(word)
    return list(groups.values())


def score(guess, words, weights=None):
    # expected number of answers left after this guess:
    # (sum of group size squared) / (number of answers). Lower is better.
    # weights (follow-up 2): word -> how likely it is the answer (default 1).
    def weight_of(word):
        if weights and word in weights:
            return weights[word]
        return 1

    total = 0
    for word in words:
        total += weight_of(word)
    spread = 0
    for group in partition(guess, words):
        group_weight = 0
        for word in group:
            group_weight += weight_of(word)
        spread += group_weight * group_weight
    return Fraction(spread, total)


# ---- main question: main function ----
```

```
# Idea: for every possible guess, split the remaining answers by the
# feedback that guess would give, and pick the guess that leaves the
# fewest answers on average. Ties go to the alphabetically first word,
# so the result is deterministic.
# Complexity: Time O(n^2 * L) for n words of length L, Space O(n)
def best_guess(words, weights=None):
    best_word = None
    best_score = None
    for guess in words:
        s = score(guess, words, weights)
        if best_score is None or s < best_score or (s == best_score and guess < best_word):
            best_score = s
            best_word = guess
    return best_word


# a faster way: one bitmask per (position, letter); every position splits
# each group in two with two AND operations
# Complexity: Time O(n * L * n / 64 + n^2 / 64) (the groups are bitmasks), Space O(L * 26 * n / 64)
def best_guess_bitsets(words):
    count = len(words)
    bits = []
    for pos in range(len(words[0])):
        bits.append({})
    for i in range(count):
        for pos in range(len(words[i])):
            letter = words[i][pos]
            bits[pos][letter] = bits[pos].get(letter, 0) | (1 << i)
    best_word = None
    best_score = None
    for guess in words:
        groups = [(1 << count) - 1]                # one group holding every word
        for pos in range(len(guess)):
            mask = bits[pos].get(guess[pos], 0)
            split = []
            for group in groups:
                matching = group & mask
                other = group & ~mask
```

```
                if matching:
                    split.append(matching)
                if other:
                    split.append(other)
            groups = split
        spread = 0
        for group in groups:
```

```
            size = bin(group).count("1")
            spread += size * size
        s = Fraction(spread, count)
        if best_score is None or s < best_score or (s == best_score and guess < best_word):
            best_score = s
            best_word = guess
    return best_word
```

COMPLEXITY | Time O(W * L * groups) with bit-parallel work per group | Space O(W * L)

265 Wordle-Style 5-Letter Minimum-Guess Strategy — HARD

Template fit: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest)

QUESTION

A hidden word has 5 lowercase letters. The only thing you can ask is the oracle query(pos, letter), which says whether the letter is at that position (positions 0 to 4). Find the word with as few oracle calls as possible. by_frequency tries letters in order of English frequency; with_dictionary also knows a list of candidate words and always asks the question that splits the candidates most evenly. Both return (word, number of calls).

Function signature

def by_frequency(query) -> (str, int)
def with_dictionary(query, dictionary: list) -> (str, int)

EXAMPLES

Example 1

Input	secret = 'crane' query = lambda pos, letter: secret[pos] == letter by_frequency(query) with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape'])
Output	by_frequency(query) -> ('crane', 31) with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape']) -> ('crane', 2)
Why	Without a dictionary every position is searched letter by letter; with five candidate words a handful of well-chosen questions are enough.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# letters from the most to the least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"


def split_score(words, i, c):
    # how evenly does the question "is letter c at position i?" split the words?
    # The bigger the smaller side, the better: a perfect split is half and half.
    yes = 0
    for word in words:
        if word[i] == c:
            yes += 1
    no = len(words) - yes
    if yes < no:
        return yes
```

```
    return no


# ---- main question: main function ----
# query(i, c) is the black box: is letter c at position i of the secret?
# Idea (no dictionary): for each position ask the letters from the most
# to the least common and stop at the first "yes"; if 25 letters said
# "no" the 26th letter is implied and needs no call.
# Complexity: Time O(1) (at most 5 * 25 = 125 calls), Space O(1)
def by_frequency(query):
    calls = 0
    word = []
    for i in range(5):
        for k in range(26):
            c = BY_FREQUENCY[k]
            if k == 25:
                word.append(c)              # the last letter left: no call needed
                break
            calls += 1
            if query(i, c):
                word.append(c)
                break
    return "".join(word), calls


# Idea (a dictionary is known): keep the words that still fit every
# answer, and always ask the question that cuts the list closest to
# half, so each call removes about half of the words.
# Complexity: Time O(rounds * n^2) for n dictionary words (about log2 n rounds / calls), Space O(n)
def with_dictionary(query, dictionary):
```

```
    words = list(dictionary)
    calls = 0
    while len(words) > 1:
        options = set()
        for word in words:
            for i in range(5):
                options.add((i, word[i]))
        best_option = None
        best_value = -1
        for option in sorted(options):
            value = split_score(words, option[0], option[1])
            if value > best_value:
                best_value = value
                best_option = option
        i, c = best_option
        calls += 1
        answer = query(i, c)
        remaining = []
        for word in words:
            if (word[i] == c) == answer:
                remaining.append(word)
        words = remaining
    return words[0], calls
```

COMPLEXITY | Time at most 5 x 25 calls without a dictionary; about log2(words) with one | Space O(words)

266 Determine winner in stack-merging game

HARD

Template fit: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap

QUESTION

A game starts with stacks of the given sizes. A move merges two stacks into one if their combined size is at most cap. The player who cannot move loses (in the misere variant, the player who cannot move wins). Return whether the first player wins with best play.

Function signature

def first_player_wins(stacks: list, cap: int, misere=False) -> bool

EXAMPLES

Example 1	
Input	`[1, 1], 2`
Output	`true`
Why	`Merge the two stacks of 1 into 2; the opponent has a single stack and cannot move.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`true`
Why	`Any merge makes a stack of 2 next to a stack of 1, whose sum 3 exceeds the cap: the opponent is stuck.`

Example 3	
Input	`[3], 5`
Output	`false`
Why	`A single stack: the first player cannot move and loses.`

Example 4	
Input	`[3], 5, misere = true`
Output	`true`
Why	`The same single stack in the misere variant: being unable to move wins.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def moves(state, cap):
    # every position one move away: merge two stacks if the new stack fits under the cap.
    # `state` is a sorted tuple of stack sizes; equal results count as one move.
    results = []
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] > cap:
                continue
            rest = []
            for k in range(len(state)):
                if k != i and k != j:
                    rest.append(state[k])
            rest.append(state[i] + state[j])
            new_state = tuple(sorted(rest))
            if new_state not in results:
                results.append(new_state)
    return results
```

```
# ---- main question: main function ----
# Idea: merging does not depend on the order, so a sorted tuple
# identifies a position. Solve each position once and remember it
# (a dictionary), because many move orders reach the same position.
# A position is WON for the player to move when some move leaves the
# opponent in a LOST position. The player who cannot move loses
# (misere play, follow-up 2: the player who cannot move wins).
# Complexity: Time O(S * M) for S reachable states and M moves from each (S can grow exponentially with the n
umber of stacks), Space O(S)
def first_player_wins(stacks, cap, misere=False):
    memo = {}

    def wins(state):
        if state in memo:
            return memo[state]
        options = moves(state, cap)
        if not options:
            answer = misere                 # no move: loses (or wins, in misere play)
        else:
            answer = False
            for next_state in options:
                if not wins(next_state):    # a winning move leaves the opponent lost
                    answer = True
                    break
        memo[state] = answer
        return answer
```

```
    return wins(tuple(sorted(stacks)))
```

COMPLEXITY | Time O(number of distinct sorted states * k^2) | Space O(number of states)

267 Find secret word with match-count feedback — HARD

Template fit: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- **`matchCount(guess) -> int`**: when you submit a word **`guess`** from the list, the API returns the number of positions **`i`** such that **`guess[i] == secret[i]`** (i.e., exact character match at the same index). You are allowed to call **`matchCount`**

at most 10 times.

Task

Design/implement a function (e.g., **`findSecretWord(words)`**) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- **`words`** contains between 1 and 100 words.
- All words have equal length **`L`** (commonly **`L = 6`**).
- Every guess must be a word from **`words`**.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once **`matchCount(guess) == L`** (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

QUESTION

You are given a list of unique lowercase words, and every word has the same length **L**. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- **`matchCount(guess) -> int`**
- It returns how many positions **`i`** satisfy **`guess[i] == secret[i]`** You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement **`solution(words, secret)`** and simulate the API inside your function. The parameter **`secret`** is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string **`""`**.

Constraints**`1 <= len(words) <= 1001 <= len(words[i]) <= 10`**All words are unique, lowercase, and have the same length **`Lsecret`** is guaranteed to be one of the words in **`words`**Your algorithm may use at most 10 simulated **`matchCount`** callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until **`acckzz`** is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns **k**, any valid remaining candidate must also have exactly **k** matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def match(a, b):
    # the number of positions that hold the same letter
    count = 0
    for i in range(len(a)):
        if a[i] == b[i]:
            count += 1
    return count


def make_words(alphabet, length):
    # every word of the given length over the alphabet
    words = [""]
    for position in range(length):
        longer = []
        for word in words:
            for letter in alphabet:
                longer.append(word + letter)
        words = longer
    return words


# ---- main question: main function ----
# Idea: the secret agrees with every earlier guess in exactly as many
# positions as the API reported, so after each guess only the words
# with that same match count against the guess can still be the secret.
# Guess the word that makes the BIGGEST remaining group as small as
# possible (minimax), so the candidates shrink fast.
# Complexity: Time O(n^2 * L) for n candidates of length L, Space O(L)
def best_guess(candidates):
```

```
    best_word = None
    best_worst = None
    for guess in candidates:
        group_sizes = {}
        for word in candidates:
            m = match(guess, word)
            group_sizes[m] = group_sizes.get(m, 0) + 1
        worst = 0
        for size in group_sizes.values():
            if size > worst:
                worst = size
        if best_worst is None or worst < best_worst or (worst == best_worst and guess < best_word):
            best_worst = worst
            best_word = guess
    return best_word
```

```
# Complexity: Time O(guesses * n^2 * L), Space O(n)
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API: how many positions match the secret
    candidates = list(words)
    guesses = 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        remaining = []
        for word in candidates:
            if match(guess, word) == score:
                remaining.append(word)
        candidates = remaining
    return None, guesses
```

COMPLEXITY | Time O(guesses * W^2 * L) | Space O(W)

268 Babylon Tile-Stack Merge Game — HARD

Template fit: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply

QUESTION

A two-player game on stacks of tiles. Each stack has a height and a colour (the colour of its top tile). A move places one stack on top of another when the two stacks have the same height or the same colour; the new stack has the combined height and the colour of the stack that was placed on top. The player who cannot move loses. Given the stacks as (height, colour) pairs, return whether the player to move wins with best play.

Function signature

def babylon_wins(stacks: list) -> bool

EXAMPLES

Example 1	
Input	`[(1, 'Y'), (1, 'W')]`
Output	`true`
Why	`Two stacks of height 1: the mover merges them, and the opponent has nothing left to move.`

QUESTION

Example 2	
Input	`[(1, 'Y')]`
Output	`false`
Why	A single stack: the mover has no move and loses.

Example 3	
Input	`[(1, 'Y'), (2, 'W')]`
Output	`false`
Why	Heights differ and colours differ: no move is possible, so the mover loses.

Example 4	
Input	`[(1, 'Y'), (1, 'Y'), (1, 'W'), (1, 'W')]`
Output	`true`
Why	Four height-1 stacks of two colours: the mover has a winning plan.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Babylon:
    # a stack is (height, colour of its top tile); players take turns putting
    # one stack on top of another, allowed when the two stacks have the same
    # height or the same colour on top. A player with no legal move loses.
    def __init__(self, stacks=None):
        if stacks is None:
            stacks = []
            for colour in "YWGB" * 3:               # the real start: 12 tiles, three of each colour
                stacks.append((1, colour))
        self.stacks = list(stacks)

    def legal_moves(self):
        # (i, j) means: put stack i on top of stack j
        moves = []
        for i in range(len(self.stacks)):
            for j in range(len(self.stacks)):
                if i == j:
                    continue
                height_i, colour_i = self.stacks[i]
                height_j, colour_j = self.stacks[j]
                if height_i == height_j or colour_i == colour_j:
                    moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move; the game itself is not changed
        height_i, colour_i = self.stacks[i]
        height_j, colour_j = self.stacks[j]
        rest = []
        for k in range(len(self.stacks)):
            if k != i and k != j:
                rest.append(self.stacks[k])
        rest.append((height_i + height_j, colour_i))      # heights add; the moved stack's colour is on top
        return rest

    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)
```

```
    def is_over(self):
        return not self.legal_moves()
```

```
# ---- main question: main function ----
# Idea: a position is WON for the player to move when some legal move
# leaves the opponent in a LOST position (and a position with no move is
# lost). Remember every solved position; sort the stacks first so equal
# positions share one memory entry.
# Complexity: Time O(S * M) for S reachable states and M moves from each, Space O(S)
def babylon_wins(stacks):
    memo = {}

    def solve(state):
        if state in memo:
            return memo[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            next_state = tuple(sorted(game.after(i, j)))
            if not solve(next_state):                    # a move to a position the opponent loses
                result = True
                break
        memo[state] = result
        return result

    return solve(tuple(sorted(stacks)))


# Complexity: Time O(M * S * M), Space O(S)
def best_move(game):
    # a move that leaves the opponent lost, if there is one
    for i, j in game.legal_moves():
        if not babylon_wins(game.after(i, j)):
            return i, j
    moves = game.legal_moves()
    if moves:
        return moves[0]
    return None
```

COMPLEXITY | Time O(states * moves^2) | Space O(states)

GROUP 40 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Sliding window: grow on the right, shrink on the left while it is too bad

269 Temperature Monitoring Data Structure — MEDIUM

Template fit: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum

QUESTION

Design a monitor over the last n temperature readings. insert(x) adds a reading (the oldest one drops out once more than n are stored), average() returns the mean of the stored readings, and maximum() returns the largest of them in O(1).

Function signature

class Monitor:
  def __init__(self, n: int)
  def insert(self, x: float) -> None
  def average(self) -> float
  def maximum(self) -> float

EXAMPLES

Example 1

Input	`m = Monitor(3)`
	`m.insert(20)`
	`m.insert(22)`
	`m.insert(19)`
	`m.average()`
	`m.insert(25)`
	`m.average()`
	`m.maximum()`
	`m.insert(10)`
	`m.maximum()`

Output	`m.average() -> 20.333333`
	`m.average() -> 22`
	`m.maximum() -> 25`
	`m.maximum() -> 25`

Why	`With n = 3 the window slides: after 25 arrives the window is 22, 19, 25 (average 22, maximum 25); after 10 it is 19, 25, 10, so the maximum is still 25.`

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# ---- main question: helper functions ----
# Complexity: Time push is O(1) amortized, best O(1), Space O(window)
class MonotoneQueue:
    # keeps the BEST value of the window at the front. keep_max=True keeps
    # the maximum, keep_max=False the minimum. A new value removes from the
    # back every value it beats: those can never be the best again, because
    # the new one stays in the window longer.
    def __init__(self, keep_max):
        self.items = deque()                    # (key, value) pairs, best first
        self.keep_max = keep_max

    def at_least_as_good(self, a, b):
        if self.keep_max:
            return a >= b
        return a <= b
```

```
    def push(self, key, value):
        while self.items and self.at_least_as_good(value, self.items[len(self.items) - 1][1]):
            self.items.pop()
        self.items.append((key, value))

    def expire(self, cutoff):
        # forget the entries whose key is at or below the cutoff (they left the window)
        while self.items and self.items[0][0] <= cutoff:
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# The shared window: every reading with a key (its number or its day), a running
# total for the average, and two monotone queues for the maximum and the
# minimum (follow-up 2). Median needs more (two heaps with lazy deletion or a
# sorted container, O(log N) per update) because it is not decomposable like
# a sum or a maximum.
# Complexity: Time add and expire are O(1) amortized, average, maximum and minimum O(1), Space O(window)
class SlidingStats:
    def __init__(self):
        self.items = deque()                        # (key, value), oldest first
        self.total = 0
        self.max_queue = MonotoneQueue(True)
        self.min_queue = MonotoneQueue(False)
```

```
    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.max_queue.push(key, value)
        self.min_queue.push(key, value)

    def expire(self, cutoff):
        # drop the readings with a key at or below the cutoff, everywhere
        while self.items and self.items[0][0] <= cutoff:
            self.total -= self.items.popleft()[1]
        self.max_queue.expire(cutoff)
        self.min_queue.expire(cutoff)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.max_queue.best()

    def minimum(self):
        return self.min_queue.best()


# ---- main question: main class ----
# insert(x) adds a reading; average() and maximum() describe the LAST n readings.
# Idea: keep the window in a deque with a running sum; on insert add the new
# value and drop the oldest when the window holds more than n. The maximum comes
# from a monotone queue, so every operation is O(1) amortised.
# Complexity: Time insert O(1) amortized, Space O(n) for the newest n readings
class Monitor(SlidingStats):
    def __init__(self, n):
        SlidingStats.__init__(self)
        self.n = n
        self.seen = 0

    def insert(self, x):
        self.add(self.seen, x)                      # the number of the reading is its key
        self.seen += 1
```

```
        self.expire(self.seen - self.n - 1)        # keep only the newest n readings
```

COMPLEXITY | Time O(1) amortized per operation | Space O(N)

270 Shortest Subarray with K Distinct Integers

MEDIUM

Template fit: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left

QUESTION

Given an array nums and an integer k, return the length of the shortest contiguous subarray that contains at least k different integers, or -1 if there is no such subarray.

Function signature

def shortest_k_distinct(nums: list, k: int) -> int

EXAMPLES

Example 1	
Input	[1, 2, 1, 3, 2], 3
Output	3
Why	[2, 1, 3] is the shortest window holding three different numbers.

Example 2	
Input	[1, 1, 1], 2
Output	-1
Why	Only one distinct number exists.

Example 3	
Input	[1, 2], 2
Output	2
Why	The whole array is needed.

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# The length of the shortest subarray with at least k distinct values (-1
# if none).
# Idea: a sliding window with a count of every value in it. Grow the right
# end; whenever the window holds at least k distinct values, record its length
# and shrink from the left as far as it stays valid (delete a value from the
# counts when its count reaches 0).
# Complexity: Time O(n), Space O(k)
def shortest_k_distinct(nums, k):
    counts = {}
    left = 0
    best = None
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) >= k:
            size = right - left + 1
            if best is None or size < best:
                best = size
```

```
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
    if best is None:
        return -1
    return best
```

COMPLEXITY | Time O(n) | Space O(n)

271 Find the minimum window substring. MEDIUM

Template fit: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink

QUESTION

Given strings s and t, return the shortest substring of s that contains every character of t (a character that appears twice in t must appear at least twice in the substring). Return the empty string if there is no such substring.

Function signature

def min_window(s: str, t: str) -> str

EXAMPLES

Example 1	
Input	'ADOBECODEBANC', 'ABC'
Output	'BANC'
Why	BANC is the shortest window that holds an A, a B and a C.

Example 2	
Input	'a', 'a'
Output	'a'
Why	The whole string is the only window.

Example 3	
Input	'a', 'aa'
Output	''
Why	t needs two a's but s has only one.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def valid_windows(s, t):
    # a sliding window: grow `right` until the window holds all of t, then
    # shrink `left` while it still does. Returns every (left, right) that is
    # valid at the moment it is seen. `need[c]` is how many more of the letter c
    # the window still requires; `missing` counts the letters of t still missing.
    need = {}
    for ch in t:
        need[ch] = need.get(ch, 0) + 1
    missing = len(t)
    left = 0
    windows = []
    for right in range(len(s)):
        ch = s[right]
        if need.get(ch, 0) > 0:
            missing -= 1
        need[ch] = need.get(ch, 0) - 1
        while missing == 0:
```

```
            windows.append((left, right))
            need[s[left]] += 1                      # drop the leftmost letter and see if it hurts
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return windows


# ---- main question: main function ----
# The shortest substring of s that contains every letter of t (with its
# multiplicity); the first one if there is a tie; "" if there is none.
# Complexity: Time O(n + m) for s of length n and t of length m, Space O(m + A)
def min_window(s, t):
    best = None
    width = 0
    for left, right in valid_windows(s, t):
        if best is None or right - left < width:
            best = (left, right)
            width = right - left
    if best is None:
        return ""
    return s[best[0]:best[1] + 1]
```

COMPLEXITY | Time O(n + m) | Space O(m)

272 Footpath Beauty After Repainting

MEDIUM

Template fit: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer

QUESTION

A footpath is built from n blocks laid in a line, painted with one of k colors. block_colors[i] is the color of block i (colors are integers in 1..k). The beauty value for a given favorite color c is the length of the longest run of consecutive blocks that are all color c . You are allowed to repaint at most m blocks (each to any color you choose). Return the maximum possible beauty value for favorite color c after repainting at most m blocks.

Function signature

def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:

...

Follow-up (as reported): (Reported) Answer q queries, each with a different favorite color, and return the beauty value (with m fixed) for each. Hint: a sliding window per query, or precompute runs. (Reported) The base version with m = 0 was asked first; the repaint version is the twist.

EXAMPLES

Example 1	
Input	`block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1`
Output	`4`
Why	`Repaint one of the 2's. The best window has at most one non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.`

Example 2	
Input	`block_colors = [3, 3, 3], c = 3, m = 0`
Output	`3`
Why	`All blocks are already color 3; no repaint needed.`

QUESTION

Example 3	
Input	`block_colors = [2, 2, 2], c = 1, m = 2`
Output	`2`
Why	`Only 2 repaints allowed, so at most 2 consecutive c blocks.`

CONSTRAINTS

- 1 <= n <= 10^5 , block_colors has length n
- 1 <= k , 1 <= c <= k , 1 <= block_colors[i] <= k
- 0 <= m <= n
- With m = 0 , the answer is simply the longest existing run of color c .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# A footpath is a row of blocks with colours. You may repaint at most m
# blocks. The beauty is the length of the longest run of blocks of the
# favourite colour c. What is the best beauty?
# Idea: the answer is the longest window that holds at most m blocks of
# another colour (those are the ones to repaint). Grow the right end; while the
# window needs more than m repaints, shrink it from the left.
# Complexity: Time O(n), Space O(1)
def max_beauty_after_repaint(block_colors, c, m):
    left = 0
    other = 0                                       # the blocks in the window that are not colour c
    best = 0
    for right in range(len(block_colors)):
        if block_colors[right] != c:
            other += 1
        while other > m:
            if block_colors[left] != c:
                other -= 1
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best
```

COMPLEXITY | Time O(n) | Space O(1)

273 Necklace Cut into Two Halves with Equal D/R Counts — HARD

Template fit: a fixed-size window of half the length slides around the doubled string, keeping a count of D

QUESTION

A necklace is a circular string of beads D and R. Cut it in two places into two arcs of equal length that contain the same number of D beads (and so the same number of R beads). Return the two cut positions and the two arcs, or None if no such cut exists.

Function signature

def necklace_two_cuts(s: str) -> tuple | None

(cut1, cut2, arc1, arc2)

EXAMPLES

Example 1	
Input	`'DRDR'`
Output	`(0, 2, 'DR', 'DR')`
Why	`Cut at 0 and 2: both arcs are DR.`

QUESTION

Example 2	
Input	`'DDRR'`
Output	`(1, 3, 'DR', 'RD')`
Why	Linear halves DD and RR are unbalanced, but rotating the cuts by one gives DR and RD.

Example 3	
Input	`'DDR'`
Output	`null`
Why	An odd number of beads cannot be split into equal arcs.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def count_letter(text, letter):
    count = 0
    for ch in text:
        if ch == letter:
            count += 1
    return count


# ---- main question: main function ----
# A circular necklace of D and R beads. Cut it in TWO places so that each of
# the two arcs holds half of the D's and half of the R's. Return
# (cut1, cut2, arc1, arc2) or None.
# Idea: if each arc has half of the D's and half of the R's, each arc has length
# n / 2 (and the total number of D's and of R's must both be even). So slide a
# window of length n / 2 once around the circle (work on s + s so the window may
# wrap) and keep its number of D's; when that equals half of all D's, the window
# and the rest of the circle are the two good arcs.
# Complexity: Time O(n), Space O(n) (the doubled string)
def necklace_two_cuts(s):
    n = len(s)
    d_total = count_letter(s, "D")
    r_total = count_letter(s, "R")
    if n % 2 == 1 or d_total % 2 == 1 or r_total % 2 == 1:
        return None                                    # each piece needs half of both letters
    half = n // 2
    target = d_total // 2
    doubled = s + s
    window = count_letter(doubled[:half], "D")         # the D's in the first window
    for i in range(n):
        if window == target:
            return i, (i + half) % n, doubled[i:i + half], doubled[i + half:i + n]
        if doubled[i + half] == "D":
            window += 1                                # move the window one step: add the new bead ...
        if doubled[i] == "D":
            window -= 1                                # ... and drop the old one
    return None


# the case of ONE cut (follow-up 1): the necklace is treated as a line
# Complexity: Time O(n), Space O(n)
def single_cut(s):
    if len(s) % 2 == 1:
        return None
    half = len(s) // 2
```

```
    if count_letter(s[:half], "D") * 2 == count_letter(s, "D"):
        return half                                    # a prefix with half of everything
```

```
    return None
```

COMPLEXITY | Time O(n) | Space O(n)

GROUP 41 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 4 M

Parent pointers: store who is above each node and climb

274 Tree Node. EASY

Template fit: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf

QUESTION

A binary tree is stored in a table tree_node_table(id, p_id), where p_id is the id of the parent node (NULL for the root). Write a query that labels every node: 'Root' if it has no parent, 'Inner' if it has a parent and at least one child, and 'Leaf' if it has a parent and no children. The output has the columns id and node_types, ordered by id.

Function signature

-- columns: id, node_types

EXAMPLES

Example 1

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL), (2, 1), (3, 1), (4, 2), (5, 2);`

Output	`1, Root`
	`2, Inner`
	`3, Leaf`
	`4, Leaf`
	`5, Leaf`

Why	`Node 1 has no parent, node 2 has a parent and children 4 and 5, and nodes 3, 4 and 5 have no children.`

Example 2

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL);`
Output	`1, Root`
Why	`A tree with one node: it has no parent, so it is the Root (not a Leaf).`

MAIN CODE | helper functions first, then the answer

```python
import sqlite3


# ---- main question: helper functions ----
# The table has one row per node; the root has p_id NULL.
SCHEMA = "CREATE TABLE tree_node_table (id INT, p_id INT)"
INSERT = "INSERT INTO tree_node_table VALUES (?, ?)"


# Complexity: Time O(n^2) for the correlated EXISTS check without an index (O(n log n) with an index on p_id)
, Space O(n)
def run(rows, sql):
    # run one query on a fresh in-memory database holding `rows`
    db = sqlite3.connect(":memory:")
    db.execute(SCHEMA)
    db.executemany(INSERT, rows)
    result = db.execute(sql).fetchall()
    db.close()
    return result


# ---- main question: main function ----
```

```
# Idea: one CASE expression, tested in order.
#   Root:  no parent.
#   Inner: has a parent AND somebody lists it as their parent.
#   Leaf:  has a parent and nobody lists it as their parent.
# The Root test comes first, so a one-node tree is a Root, not a Leaf.
TYPES_SQL = """
SELECT t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (SELECT 1 FROM tree_node_table c
                 WHERE c.p_id = t.id) THEN 'Inner'
    ELSE 'Leaf'
  END AS node_types
FROM tree_node_table t
ORDER BY t.id
"""
```

COMPLEXITY | Time O(n) with the set of parent ids built once | Space O(n)

275 Employee Hierarchy Add / Delete

MEDIUM

Template fit: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager

QUESTION

Maintain a company hierarchy. Org(ceo) starts with the CEO. add(emp, mgr) adds a new employee under an existing manager (False if emp already exists or mgr does not). delete(emp) removes an employee and their direct reports move up to the deleted person's manager. The CEO can only be deleted when exactly one person reports to them, who then becomes the CEO.

Function signature

class Org:
  def __init__(self, ceo)
  def add(self, emp, mgr) -> bool
  def delete(self, emp) -> bool

EXAMPLES

Example 1

Input	`o = Org('ceo')`
	`o.add('vp', 'ceo')`
	`o.add('dev', 'vp')`
	`o.add('qa', 'vp')`
	`o.add('dev', 'ceo')`
	`o.delete('vp')`
	`o.manager['dev']`
	`o.delete('ceo')`

Output	`o.add('vp', 'ceo') -> true`
	`o.add('dev', 'vp') -> true`
	`o.add('qa', 'vp') -> true`
	`o.add('dev', 'ceo') -> false`
	`o.delete('vp') -> true`
	`o.manager['dev'] -> 'ceo'`
	`o.delete('ceo') -> false`

Why	`'dev' already exists so the second add fails. Deleting 'vp' moves dev and qa up to the ceo. The ceo now has two reports, so the ceo cannot be deleted.`

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main class ----
# Idea: keep two dictionaries: manager[emp] (who is above) and reports[emp]
# (the set of people directly below). add() fills both; delete() hands
# each report of the leaving person to that person's manager and removes
# the person from the manager's reports.
# Complexity: Time add and reassign are O(1), delete is O(r) for r direct reports, Space O(n)
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        self.hired = {ceo: 0}          # hire order: a smaller number is more senior (follow-ups 2 and 3)
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False               # already here, or the manager does not exist
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
        self.clock += 1
        return True

    def reassign(self, emp, new_mgr):
        # move emp under new_mgr (take it out of the old manager's reports first)
        old = self.manager[emp]
        if old is not None:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    def delete(self, emp):
        # the CEO cannot be deleted here (follow-up 3 handles it)
        if emp not in self.manager or self.manager[emp] is None:
            return False
        boss = self.manager[emp]
        for report in list(self.reports[emp]):
            self.reassign(report, boss)        # the reports go to the grandparent
        self.reports[boss].discard(emp)
        del self.manager[emp]
        del self.reports[emp]
        return True
```

COMPLEXITY | Time O(r) per delete where r is the number of reports | Space O(n)

276 Validate parent array forms a tree — MEDIUM

Template fit: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

QUESTION

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints

1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are 0->2->1, 2->1, and 3->2->1, so every node reaches the
    same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def reason(parent):
    # None if parent[] describes a tree, else the first rule that fails.
    # parent[i] is the parent of node i; the root has parent -1.
    n = len(parent)
    roots = 0
    for p in parent:
        if p == -1:
            roots += 1
    if roots != 1:
        return str(roots) + " roots (need exactly 1)"
    for i in range(n):
        p = parent[i]
        if p != -1 and not (0 <= p < n):
            return "node " + str(i) + ": out-of-range parent " + str(p)
```

```
        if p == i:
            return "node " + str(i) + " is its own parent"
    # walk up from every node; 0 = unseen, 1 = on the current walk, 2 = already checked
    state = [0] * n
    for start in range(n):
        path = []
        x = start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        if x != -1 and state[x] == 1:
            return "cycle through node " + str(x)        # met a node twice on one walk
        for v in path:
            state[v] = 2
    return None


# ---- main question: main function ----
# Idea: exactly one root (-1), every other parent is a real node, and no
# walk upwards ever meets a node twice (that would be a cycle). Each
# node is coloured once, so the whole check is O(n).
# Complexity: Time O(n), Space O(n)
def validate_parent_array(parent):
    return reason(parent) is None
```

COMPLEXITY | Time O(n) | Space O(n)

277 Lowest Common Ancestor of a Binary Tree III — MEDIUM

Template fit: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

QUESTION

Given two nodes p and q of a binary tree, return their lowest common ancestor: the deepest node that has both of them as descendants (a node counts as a descendant of itself). Every node has a parent pointer, but you are not given the root.

Function signature

def lca_swap(p: PNode, q: PNode) -> PNode

EXAMPLES

Example 1

Input	`class PNode:`
	`def __init__(self, val):`
	`self.val, self.parent = val, None`
	`root, a, b, c = PNode(3), PNode(5), PNode(1), PNode(6)`
	`a.parent = b.parent = root`
	`c.parent = a`
	`lca_swap(c, b).val`
	`lca_swap(c, a).val`
	`lca_swap(a, b).val`

Output	`lca_swap(c, b).val -> 3`
	`lca_swap(c, a).val -> 5`
	`lca_swap(a, b).val -> 3`

Why	`Node 6 hangs under 5, which hangs under the root 3 next to node 1. The ancestor of 6 and 1 is the root 3; the ancestor of 6 and its own parent 5 is 5 itself.`

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique

QUESTION

- p != q
- p and q exist in the tree

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
class Node:
    # every node knows its parent (the root's parent is None)
    def __init__(self, val, parent=None):
        self.val = val
        self.parent = parent
        self.left = None
        self.right = None


def attach(parent, val, side):
    # create a child on the "left" or "right" of parent and link it both ways
    child = Node(val, parent)
    setattr(parent, side, child)
    return child


def lca_set(p, q):
    # the first idea: write down every ancestor of p (including p), then
    # climb from q until we meet one of them. O(h) time and space.
    seen = set()
    while p:
        seen.add(p)
        p = p.parent
    while q:
        if q in seen:
            return q
        q = q.parent
    return None


# ---- main question: main function ----
# Idea: two pointers climb towards the root, and when one falls off the
# top it restarts at the OTHER node. Both then walk the same total
# distance (the two path lengths added), so they meet exactly at the
# lowest common ancestor. O(h) time, O(1) space. If the nodes are in
# different trees both end up as None together.
# Complexity: Time O(h) for the height h of the tree, Space O(1)
def lca_swap(p, q):
    a = p
    b = q
    while a is not b:
        if a:
            a = a.parent
        else:
```

```
            a = q
        if b:
            b = b.parent
        else:
            b = p
    return a
```

COMPLEXITY | Time O(h) | Space O(1)

278 Root a Tree So It Becomes a Binary Tree

MEDIUM

Template fit: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2

QUESTION

You are given a connected, undirected, acyclic graph (i.e. a tree) with n nodes labeled 0 .. n-1 , described by an edge list. It is guaranteed that every node has at most 3 neighbors. Choose a node to be the root. When rooted at node r , one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a binary tree if every node has at most 2 children. Return the smallest label of a node that can serve as such a root. If no node works, return -1 . Key observation (the trick): a non-root node with degree d has d - 1 children, so it needs d <= 3 ; the root has degree children, so the root needs degree <= 2 . Since all nodes already have degree <= 3 , any node with degree <= 2 is a valid root — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for n >= 1 .

Function signature

```
from typing import List, Tuple
def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) -> int:
    """Return the smallest node label that can root the tree as a binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Follow-up (as reported): Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by depth % 2 from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

EXAMPLES

Example 1	
Input	`n = 4, edges = [(0, 1), (0, 2), (0, 3)]`
Output	`1`
Why	`Node 0 has degree 3, so as root it would have 3 children (invalid). Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.`

Example 2	
Input	`n = 3, edges = [(0, 1), (1, 2)]`
Output	`0`
Why	`A path; every node has degree <= 2, so the smallest label 0 works.`

Example 3	
Input	`n = 1, edges = []`
Output	`0`
Why	`A single node is trivially a binary tree.`

CONSTRAINTS

- 1 <= n <= 10^5 ; the input is a valid tree with n - 1 edges (for n == 1 , no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: in a tree where every node may have at most 2 children, only the
# root has as many children as its degree; every other node has one
# FEWER (one edge goes to its parent). So any node of degree 2 or less
```

```python

# can be the root, and the first one is a valid choice. -1 if there is none.
# Complexity: Time O(n), Space O(n)
def find_binary_tree_root(n, edges):
    degree = [0] * n
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    for node in range(n):
        if degree[node] <= 2:
            return node
    return -1
```

COMPLEXITY | Time O(n) | Space O(n)

Build or transform a grid with index arithmetic

279 Filled ASCII Diamond — EASY

*Template fit: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle*

QUESTION

Given a non-negative integer radius R , render a filled diamond on a square grid of side 2*R + 1 . The grid is indexed by (i, j) with 0 <= i, j <= 2*R , and its center is (R, R) . A cell belongs to the diamond (draw 'x') if its Manhattan distance from the center is at most R , i.e. abs(i - R) + abs(j - R) <= R . Every other cell is background (draw '_'). Return the picture as a list of 2*R + 1 strings, one per row (top to bottom), each of length 2*R + 1 .

Function signature

def render_diamond(R: int) -> list[str]:

    ...

Follow-up (as reported): The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1 times, padded with '_' .)

EXAMPLES

Example 1	
Input	`R = 1`
Output	`["_x_", "xxx", "_x_"]`
Why	`Center (1,1). Row 0 has distance 1 from center vertically, so only column 1 (distance 0) is inside.`

Example 2	
Input	`R = 2`
Output	`["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]`
Why	`Each row i allows columns within (R - abs(i-R)) of the center.`

Example 3	
Input	`R = 0`
Output	`["x"]`
Why	`A 1x1 grid whose only cell is the center.`

CONSTRAINTS

- 0 <= R <= 500 .
- For R = 0 the result is the single cell ["x"] .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the diamond has 2 * R + 1 rows. Row i is `pad` = |i - R| steps away
# from the middle row, so it has `pad` underscores on each side and
# 2 * (R - pad) + 1 letters "x" in the middle.
# Complexity: Time O(R^2), Space O(R^2) for the output
def render_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = i - R
        if pad < 0:
            pad = -pad                                  # the distance from the middle row
        rows.append("_" * pad + "x" * (2 * (R - pad) + 1) + "_" * pad)
```

```
    return rows
```

COMPLEXITY | Time O(R^2) characters | Space O(R^2)

280 Connected Crop Arrangement on a Grid

MEDIUM

Template fit: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip

QUESTION

A garden is an n x m grid. There are k crop types numbered 1..k, and crop i must fill exactly counts[i - 1] cells; the counts add up to n * m so every cell is used. Fill the grid so that the cells of each crop form one connected region (cells joined up, down, left or right). Return any valid grid.

Function signature

def arrange_crops(n: int, m: int, counts: list) -> list

EXAMPLES

Example 1	
Input	2, 3, [2, 3, 1]
Output	[[1, 1, 2], [3, 2, 2]]
Why	One valid answer: crop 3 is the single cell at the bottom left; crops 1 and 2 are each connected.

Example 2	
Input	1, 4, [1, 3]
Output	[[1, 2, 2, 2]]
Why	A single row: the crops simply sit side by side.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: helper functions ----
def snake_cells(n, m):
    # the cells of an n x m grid in "snake" order: row 0 left to right,
    # row 1 right to left, row 2 left to right, ... Consecutive cells in
    # this order always share an edge.
    path = []
    for i in range(n):
        for j in range(m):
            if i % 2 == 0:
                path.append((i, j))
            else:
                path.append((i, m - 1 - j))
    return path


def fill_along(grid, path, counts):
    # hand out the path: the first counts[0] cells to crop 1, the next
    # counts[1] cells to crop 2, and so on
    k = 0
    for crop in range(1, len(counts) + 1):
        for i in range(counts[crop - 1]):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# ---- main question: main function ----
# Idea: every crop must be one connected piece. Take the snake order of
```

```python
# the cells and give each crop a consecutive stretch of it. Neighbouring
# cells of the snake share an edge, so each stretch (one crop) is connected.
# counts[i] is how many cells crop i + 1 needs; together they fill the grid.
# Complexity: Time O(n * m), Space O(n * m)
def arrange_crops(n, m, counts):
    total = 0
    for count in counts:
        total += count
    if total != n * m:
        raise ValueError("the counts must add up to n * m")
    grid = []
    for r in range(n):
        grid.append([0] * m)
    return fill_along(grid, snake_cells(n, m), counts)
```

COMPLEXITY | Time O(n * m) | Space O(n * m)

281 Rotate Any m×n 2D Matrix

MEDIUM

*Template fit: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))*

QUESTION

Rotate an m x n matrix 90 degrees clockwise and return the resulting n x m matrix.

Function signature

def rotate_cw(matrix: list) -> list

EXAMPLES

Example 1	
Input	`[[1, 2, 3], [4, 5, 6]]`
Output	`[[4, 1], [5, 2], [6, 3]]`
Why	The last row becomes the first column: [4, 1] is the old first column read from the bottom.

Example 2	
Input	`[[1, 2], [3, 4]]`
Output	`[[3, 1], [4, 2]]`
Why	A square matrix.

Example 3	
Input	`[[1, 2, 3]]`
Output	`[[1], [2], [3]]`
Why	A single row becomes a single column.

MAIN CODE | helper functions first, then the answer

```python
# ---- main question: main function ----
# Idea: turning a matrix 90 degrees CLOCKWISE moves the item at row i,
# column j of the old m x n matrix to row j, column m - 1 - i of the
# new n x m matrix (the last row becomes the first column). Build the new
# matrix once with that rule. A non-square matrix changes its shape, which
# is why it cannot be rotated in place the way the square LeetCode 48 can.
# Complexity: Time O(m * n), Space O(m * n)
def rotate_cw(matrix):
    m = len(matrix)
    if m == 0:
        return []
```

```
    n = len(matrix[0])
    rotated = []
    for j in range(n):
        rotated.append([None] * m)
    for i in range(m):
        for j in range(n):
            rotated[j][m - 1 - i] = matrix[i][j]
    return rotated


# the other direction: item (i, j) goes to row n - 1 - j, column i
# Complexity: Time O(m * n), Space O(m * n)
def rotate_ccw(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    rotated = []
    for j in range(n):
        rotated.append([None] * m)
    for i in range(m):
        for j in range(n):
            rotated[n - 1 - j][i] = matrix[i][j]
    return rotated
```

COMPLEXITY | Rotate 90 degrees clockwise by reverse + transpose: Time O(mn) | Space O(mn)
Other angles, and the in-place square case: Time O(mn) | Space O(1) for the in-place square version

282 Solve matrix groups and recipe inventory

MEDIUM

Template fit: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort

QUESTION

Two tasks. (1) block_mins(matrix, k): cut the matrix into k x k blocks (blocks at the right and bottom edge may be smaller) and return the minimum of every block as a smaller matrix. (2) possible_recipes(recipes, supplies): recipes maps a recipe name to the ingredients it needs, and an ingredient may itself be another recipe; return the recipes that can be made from the supplies.

Function signature

def block_mins(matrix: list, k: int) -> list
def possible_recipes(recipes: dict, supplies: list) -> list

EXAMPLES

Example 1	
Input	`[[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2`
Output	`[[1, 1], [3, 5]]`
Why	`Top-left block (3, 1, 5, 9) has minimum 1, top-right (4, 1, 2, 6) has 1; the bottom row forms smaller blocks with minimum 3 and 5.`

Example 2	
Input	`{'bread': ['flour', 'yeast'], 'sandwich': ['bread', 'ham'], 'cake': ['flour', 'egg', 'sugar']}, ['flour', 'yeast', 'ham', 'egg']`
Output	`['bread', 'sandwich']`
Why	`bread needs flour and yeast (available); sandwich needs bread and ham; cake needs sugar, which is missing.`

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# ---- main question: main functions ----
# PART 1: the minimum of every k x k block of a matrix.
# Idea: cell (r, c) belongs to block (r // k, c // k). Go over the matrix
# once and keep the smallest value seen for each block. The blocks at
# the right and bottom edges may be smaller when k does not divide the
# size; r // k and c // k handle that by themselves.
# Complexity: Time O(m * n), Space O(m * n / k^2) for the blocks
def block_mins(matrix, k):
    rows = len(matrix)
    cols = len(matrix[0])
    block_rows = (rows + k - 1) // k
    block_cols = (cols + k - 1) // k
    mins = []
    for i in range(block_rows):
        mins.append([float("inf")] * block_cols)
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] < mins[r // k][c // k]:
                mins[r // k][c // k] = matrix[r][c]
    return mins


# PART 2: which recipes can be made? recipes maps a name to its list of
# ingredients, which may be supplies or other recipes.
# Idea (Kahn's algorithm): count for each recipe how many ingredients are
# still missing. A recipe with none missing is made; making it supplies
# an ingredient to every recipe that uses it, lowering their counts.
# Complexity: Time O(R + total ingredients), Space O(R + total ingredients)
def possible_recipes(recipes, supplies):
    have = set(supplies)
    missing = {}                                    # recipe -> how many different ingredients it still lack
s
    users = {}                                      # ingredient -> the recipes that need it
    for name in recipes:
        missing[name] = 0
        for item in set(recipes[name]):
            if item not in have:
                missing[name] += 1
                if item not in users:
                    users[item] = []
                users[item].append(name)
    queue = deque()
    for name in missing:
        if missing[name] == 0:
            queue.append(name)
    made = []
    while queue:
        name = queue.popleft()
        made.append(name)
        for other in users.get(name, []):           # a made recipe is a new ingredient for its users
            missing[other] -= 1
            if missing[other] == 0:
                queue.append(other)
    return made
```

COMPLEXITY | Blocks: one pass updating a minimum per block: Time O(m * n) | Space O(blocks)
Recipes: Kahn's algorithm on the ingredient graph: Time O(R + I) | Space O(R + I)

283 Range Sum Query 2D (Immutable Matrix) MEDIUM

Template fit: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups

QUESTION

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner (row1, col1) and lower-right corner (row2, col2) , inclusive. Implement a class NumMatrix that is initialized once with the matrix and can then answer many sum_region queries efficiently (each query in O(1) after preprocessing).

Function signature

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...
    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...
```

Follow-up (as reported): Follow-up constraint change from the report: what if the matrix is mutable (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

EXAMPLES

Example 1	
Input	`matrix = [[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]] sum_region(2, 1, 4, 3)`
Output	`8`
Why	`The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums to 8.`

Example 2	
Input	`same matrix; sum_region(1, 1, 2, 2)`
Output	`11`
Why	`Block [[6,3],[2,0]] sums to 11.`

Example 3	
Input	`same matrix; sum_region(1, 2, 2, 4)`
Output	`12`
Why	`Block [[3,2,1],[0,1,5]] sums to 12.`

CONSTRAINTS

- 1 <= rows, cols <= 200
- -10^5 <= matrix[i][j] <= 10^5
- 0 <= row1 <= row2 < rows , 0 <= col1 <= col2 < cols
- There may be up to 10^4 calls to sum_region .

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea (2D prefix sums): p[r][c] is the sum of the rectangle from the
# top-left corner (0, 0) up to (but not including) row r and column c.
# Building p: a cell's value plus the block above plus the block to the
# left, minus the corner block that was counted twice. A query then
# combines FOUR entries of p (inclusion-exclusion): big rectangle, minus
# the strip above, minus the strip on the left, plus the corner that was
```

```python
# subtracted twice. Each query is O(1).
# Complexity: Time build O(m * n), each query O(1), Space O(m * n)
class NumMatrix:
    def __init__(self, matrix):
        rows = len(matrix)
        cols = 0
        if rows > 0:
            cols = len(matrix[0])
        self.p = []
        for r in range(rows + 1):
            self.p.append([0] * (cols + 1))
        for r in range(rows):
            for c in range(cols):
                self.p[r + 1][c + 1] = matrix[r][c] + self.p[r][c + 1] + self.p[r + 1][c] - self.p[r][c]

    def sum_region(self, r1, c1, r2, c2):
        p = self.p
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1]
```

COMPLEXITY | Time O(R * C) build, O(1) per query | Space O(R * C)

GROUP 43 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 2 M 2 H

Split into pieces, fix each piece, join (and pack words into lines)

284 Implement string formatting and request tracking tasks

EASY

Template fit: split, fix each piece, join; the request-tracking part is a small dict

QUESTION

Four small tasks. (A) snake_to_camel(s): convert snake_case to camelCase, ignoring leading, trailing and repeated underscores. (B) format_thousands(n): write an integer with a comma between every group of three digits (keep the minus sign). (C) ocr_valid(t, rot): an OCR scanner may read a label upside down; given the rotation table rot (character -> character seen after turning 180 degrees), return true if t, or t turned 180 degrees (reversed and mapped through rot), is an integer from 1 to 200 without leading zeros. (D) Server: begin(rid, ts) and end(rid, ts) arrive in any order for each request id; drain(watermark) returns the completed requests as 'id begin=b end=e' lines in order of end time, only those with end at most the watermark (all of them when no watermark is given).

Function signature

```
def snake_to_camel(s: str) -> str
def format_thousands(n: int) -> str
def ocr_valid(t: str, rot: dict) -> bool
class Server:
    def begin(self, rid, ts) -> None
    def end(self, rid, ts) -> None
    def drain(self, watermark=None) -> list
```

EXAMPLES

Example 1	
Input	`'make__it_camel_'`
Output	`'makeItCamel'`
Why	`Repeated and trailing underscores are ignored.`

Example 2	
Input	`1234567`
Output	`'1,234,567'`
Why	`A comma before every third digit from the right.`

Example 3	
Input	`-1000`
Output	`'-1,000'`
Why	`The sign stays at the front.`

QUESTION

Example 4	
Input	rot = {'0': '0', '1': '1', '2': '2', '5': '5', '6': '9', '8': '8', '9': '6'} ocr_valid('186', rot) ocr_valid('981', rot) ocr_valid('500', rot)
Output	ocr_valid('186', rot) -> true ocr_valid('981', rot) -> true ocr_valid('500', rot) -> false
Why	981 is above 200, but turned upside down it reads 186, which is valid. 500 is too large and its rotation 005 has leading zeros.

Example 5	
Input	sv = Server() sv.begin('r2', 5) sv.end('r1', 4) sv.begin('r1', 1) sv.end('r2', 9) sv.drain(8) sv.drain()
Output	sv.drain(8) -> ['r1 begin=1 end=4'] sv.drain() -> ['r2 begin=5 end=9']
Why	r1 finished at 4 and r2 at 9. Draining with watermark 8 prints only r1; draining everything then prints r2.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# ---- main question: main functions ----
# TASK A: snake_case to camelCase, ignoring leading, trailing and
# repeated underscores.
# Idea: split on "_" and skip the empty parts (that handles every odd
# underscore in one move). The first word stays as it is; every later
# word gets a capital first letter.
# Complexity: Time O(n), Space O(n)
def snake_to_camel(s):
    words = []
    for part in s.split("_"):
        if part != "":
            words.append(part)
    if len(words) == 0:
        return ""
    result = words[0]
    for word in words[1:]:
        result += word.capitalize()
    return result


# TASK B: write an integer with a comma between every group of three
# digits (keep the minus sign).
# Idea: peel three digits at a time from the RIGHT end of the digit text.
# Complexity: Time O(d) for d digits, Space O(d)
def format_thousands(n):
    sign = ""
    if n < 0:
        sign = "-"
    digits = str(abs(n))
    groups = []
```

```
    while len(digits) > 3:
        groups.append(digits[-3:])
        digits = digits[:-3]
    groups.append(digits)
    groups.reverse()
    return sign + ",".join(groups)


# TASK C: an OCR scanner may read a label upside down. ocr_valid is True
# if the text, or the text turned 180 degrees (reversed, then every
# character mapped through the table `rot`), is an integer from 1 to
# 200 without leading zeros.
```

```
# Complexity: Time O(L), Space O(1)
def is_valid_number(text):
    # only digits, no leading zero, value 1 to 200
    if text == "" or text[0] == "0":
        return False
    for ch in text:
        if ch < "0" or ch > "9":
            return False
    return 1 <= int(text) <= 200


# Complexity: Time O(L), Space O(L)
def turn_around(text, rot):
    # the text as seen after turning it 180 degrees, or None if a character cannot be turned
    result = ""
    for i in range(len(text) - 1, -1, -1):
        if text[i] not in rot:
            return None
        result += rot[text[i]]
    return result


# Complexity: Time O(L), Space O(L)
def ocr_valid(t, rot):
    if is_valid_number(t):
        return True
    turned = turn_around(t, rot)
    return turned is not None and is_valid_number(turned)


# TASK D: requests arrive as begin(rid, ts) and end(rid, ts), in any
# order for each id. drain(watermark) returns the COMPLETED requests (both
# events seen) as "id begin=b end=e" lines, in order of end time, only
# those with end <= watermark (all of them when no watermark is given).
# Idea: remember the half that arrived first; when the other half arrives
# the request is complete and goes into a min-heap ordered by end time;
# drain() pops the heap while the end time is within the watermark.
# Complexity: Time begin and end are O(log n) when a request completes, drain is O(k log n) for k lines, Spac
e O(n)
class Server:
    def __init__(self):
        self.begins = {}                    # id -> begin time, waiting for its end
        self.ends = {}                      # id -> end time, waiting for its begin
        self.done = []                      # heap of (end time, id, begin time)

    def begin(self, rid, ts):
```

```
        if rid in self.begins:
            raise ValueError("duplicate begin")        # follow-up 2
        self.begins[rid] = ts
        self.check_complete(rid)

    def end(self, rid, ts):
```

```
        if rid in self.ends:
            raise ValueError("duplicate end")
        self.ends[rid] = ts
        self.check_complete(rid)

    def check_complete(self, rid):
        if rid in self.begins and rid in self.ends:
            begin_time = self.begins.pop(rid)
            end_time = self.ends.pop(rid)
            if end_time < begin_time:
                raise ValueError("ends before it begins")      # follow-up 2
            heapq.heappush(self.done, (end_time, rid, begin_time))

    def drain(self, watermark=None):
        lines = []
        while self.done and (watermark is None or self.done[0][0] <= watermark):
            end_time, rid, begin_time = heapq.heappop(self.done)
            lines.append(str(rid) + " begin=" + str(begin_time) + " end=" + str(end_time))
        return lines
```

COMPLEXITY | Task A: snake_case to camelCase: Time O(n) | Space O(n)
Task B: thousands separators: Time O(d) | Space O(d)
Task C: valid as-is or after a 180 degree rotation: Time O(L) | Space O(L)
Task D: request tracker that prints completed requests in end-time order: Time O(log n) per event | Space O(n)

285 Snake Case to Camel Case with Repository-Wide Refactor

MEDIUM

Template fit: the same function applied to every identifier found in the files

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

MAIN CODE | helper functions first, then the answer

```
import ast
import io
import tokenize


# ---- main question: helper functions ----
def snake_to_camel(name):
    # foo_bar -> fooBar (the input is a valid snake_case name)
    parts = name.split("_")
    result = parts[0]
    for part in parts[1:]:
        result += part[:1].upper() + part[1:]
    return result
```

```
def owned_names(source, keep=()):
    # the names this code defines itself (function names, arguments,
    # variables that are assigned): only these are safe to rename. The syntax
    # tree (ast) tells us what is defined, so imported or library names
    # stay untouched. Names in `keep` are never renamed.
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            for arg in node.args.args + node.args.kwonlyargs:
                names.add(arg.arg)
        elif isinstance(node, ast.Name) and isinstance(node.ctx, ast.Store):
            names.add(node.id)
    result = set()
    for name in names:
        is_snake = "_" in name.strip("_")             # a real snake_case name
        if is_snake and not name.startswith("_") and name not in keep:
            result.add(name)
    return result


# ---- main question: main function ----
# Idea: snake_to_camel capitalises each part after the first. To rename a
# whole file, find the names the code OWNS (owned_names) and rewrite only
# those, working on TOKENS so that strings and comments are never touched.
# A name right after a dot (an attribute) may belong to a library, so it
# stays as written.
# Complexity: Time O(L) for source of length L, Space O(L)
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
```

```
    tokens = []
    previous = None
    for token in tokenize.generate_tokens(io.StringIO(source).readline):
        after_dot = previous is not None and previous.string == "."
        if token.type == tokenize.NAME and token.string in owned and not after_dot:
            token = token._replace(string=snake_to_camel(token.string))
        tokens.append(token)
        previous = token
    return tokenize.untokenize(tokens)
```

COMPLEXITY | Time O(size of the source) | Space O(names)

286 Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) MEDIUM

Template fit: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
def split_long(word, width):
```

```
    # a word longer than the line is cut into pieces of `width` letters
    pieces = []
    for start in range(0, len(word), width):
        pieces.append(word[start:start + width])
    return pieces


def wrap_lines(text, width, break_long=False):
    # the wrapped lines themselves (follow-ups 1 and 2). A newline always
    # ends a line. Each line is built from a list of words and joined once.
    lines = []
    for paragraph in text.split("\n"):
        words = []
        for word in paragraph.split():
            if break_long:
                words += split_long(word, width)
            else:
                words.append(word)
        current = ""
        for word in words:
            if current == "":
                current = word
            elif len(current) + 1 + len(word) <= width:
                current += " " + word               # the word and one space still fit
            else:
                lines.append(current)
                current = word
        lines.append(current)                       # the last (or only, or empty) line of the paragraph
    return lines


# ---- main question: main function ----
# Idea: greedy. Split on newlines first (a newline always ends a line, and
# an empty paragraph is an empty line). Then for each word compare
# current length + 1 + length of the word with the width. Only the LENGTH
# of the current line is tracked, never the joined text.
# Complexity: Time O(total characters), Space O(1)
def wrap_count(text, width):
    lines = 0
    for paragraph in text.split("\n"):
        lines += 1                                  # every paragraph has at least one line
        current_length = 0                          # 0 means: nothing on the line yet
        for word in paragraph.split():
            if current_length == 0:
                current_length = len(word)
            elif current_length + 1 + len(word) <= width:
                current_length += 1 + len(word)
            else:
                lines += 1
                current_length = len(word)
    return lines
```

COMPLEXITY | Time O(n) | Space O(1) extra beyond the output

287 Implement a function to justify a list of words within a given width. HARD

Template fit: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned

QUESTION

Given a list of words and a line width, pack the words into lines of exactly that width (full justification): put as many words on a line as fit (with at least one space between them), then spread the extra spaces between the words as evenly as possible, giving the leftmost gaps the extra ones. The last line, and any line with a single word, is left-justified and padded with spaces on the right. Return the list of lines.

Function signature

def fullJustify(words: list, max_width: int) -> list

EXAMPLES

Example 1	
Input	`['This', 'is', 'an', 'example', 'of', 'text', 'justification.'], 16`
Output	`['This is an', 'example of text', 'justification. ']`
Why	Three lines of width 16: the first two are spread out, the last is left-aligned and padded.

Example 2	
Input	`['a', 'b'], 5`
Output	`['a b ']`
Why	Both words fit on one line, which is the last line: left-aligned with single spaces and padded on the right.

MAIN CODE | helper functions first, then the answer

```
# ---- main question: helper functions ----
# `measure(text)` is how wide a piece of text is. It is len() for plain
# characters; follow-ups 2 and 4 pass another function (wide characters,
# pixel widths) and nothing else has to change.
def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line (one space between words)
    lines = []
    line = []
    used = 0                                    # width of the line so far, spaces included
    for word in words:
        size = measure(word)
        if line and used + 1 + size > width:
            lines.append(line)                  # the word does not fit: close the line
            line = []
            used = 0
        if line:
            used += 1
        used += size
        line.append(word)
    if line:
        lines.append(line)
    return lines


def spread(line, width, measure=len):
    # fully justified: the spare space is shared over the gaps, and the EXTRA
    # spaces (when it does not divide evenly) go to the LEFT gaps first.
    # A line with one word is left-aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    letters = 0
    for word in line:
```

```
        letters += measure(word)
    spaces = width - letters
    gaps = len(line) - 1
    each = spaces // gaps
    extra = spaces % gaps
    text = ""
    for i in range(gaps):
        text += line[i] + " " * each
        if i < extra:
            text += " "
    return text + line[-1]
```

```
def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))


def center(line, width, measure=len):
    # the spare spaces are split on both sides, the odd one goes right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)


# ---- main question: main function ----
# Idea: greedy line packing. Put as many words as fit on a line (counting
# one space between words). Spread the spare spaces over the gaps, giving
# the extra ones to the leftmost gaps. The LAST line is left-aligned.
# Complexity: Time O(total characters), Space O(total characters)
def fullJustify(words, max_width):
    lines = pack_greedy(words, max_width)
    result = []
    for i in range(len(lines)):
        if i < len(lines) - 1:
            result.append(spread(lines[i], max_width))
        else:
            result.append(left_align(lines[i], max_width))
    return result
```

COMPLEXITY | Time O(total characters) | Space O(total characters)

288 Convert Valid Snake Case to Lower Camel Case — HARD

Template fit: reformat(): split on '_', keep the first piece, capitalise the others

QUESTION

Given a valid snake_case identifier (one or more lowercase words separated by single underscores, with no leading, trailing or doubled underscores), convert it to lowerCamelCase: keep the first word as it is, remove the underscores and capitalise the first letter of every later word.

Function signature

def snake_to_camel(s: str) -> str

EXAMPLES

Example 1	
Input	`'get_http_response_code'`
Output	`'getHttpResponseCode'`
Why	`Words are joined and every word after the first is capitalised.`

QUESTION

Example 2	
Input	`'user_id'`
Output	`'userId'`
Why	`Two words.`

Example 3	
Input	`'name'`
Output	`'name'`
Why	`A single word has nothing to change.`

MAIN CODE | helper functions first, then the answer

```
# ---- main question: main function ----
# Idea: the input is valid, so there are no empty parts. Split on "_",
# keep the first word as it is, and give every other word a capital first
# letter.
# Complexity: Time O(n), Space O(n)
def snake_to_camel(s):
    parts = s.split("_")
    result = parts[0]
    for part in parts[1:]:
        result += part[:1].upper() + part[1:]
    return result
```

COMPLEXITY | Time O(n) | Space O(n)